INSTRUCTOR KIT · PHP AND WORDPRESS DEVELOPMENT · COMPLETE ANSWER KEY

PHP and WordPress Development

Every self-check item answered and explained, plus reference notes for each lab, for all 18 sessions. Use for grading and reteaching. Confidential instructor material.

18 sessions | 216 self-check items + lab solutions | Confidential · do not distribute

CONTENTS

PART 1: THEMES · SESSIONS 1 TO 8

S01 **WordPress for Developers and Your Local Environment**

- 1.1 WordPress for Developers
- 1.2 Your Local Development Environment

S02 **PHP the WordPress Way and the Content Model**

- 1.3 PHP the WordPress Way
- 2.1 The Content Model

S03 **Core Architecture, the Database and Hooks**

- 2.2 Core Architecture & the Database
- 2.3 The Hooks System

S04 **Theme Anatomy and Building a Block Theme · Checkpoint 1**

- 3.1 Theme Anatomy & the Template Hierarchy
- 3.2 Building a Block Theme

S05 **Patterns, Style Variations and Child Themes**

- 3.3 Patterns, Style Variations & the Site Editor
- 3.4 Child Themes & Customizing Existing Themes

S06 **The Loop and Custom Queries**

- 4.1 The Loop & Template Tags
- 4.2 Custom Queries

S07 **functions.php: Supports, Assets, Menus & Widgets**

- 4.3 functions.php: Supports, Assets, Menus & Widgets

S08 **Part 1 Project and Assessment**

PART 2: PLUGINS TO LAUNCH · SESSIONS 9 TO 18

S09 **Your First Plugin and Custom Post Types**

- 5.1 Your First Plugin
- 5.2 Custom Post Types, Taxonomies & Meta

S10 **Security and the Settings API**

- 5.3 Security in WordPress
- 5.4 Admin Pages, Settings API & Options

S11 **Custom Tables, Shortcodes and Block Setup**

- 5.5 Shortcodes, $wpdb & Custom Tables
- 6.1 Block Development Setup

S12 **Static and Dynamic Blocks**

- 6.2 Static Blocks: Attributes, Controls & Supports
- 6.3 Dynamic Blocks & Block Bindings

S13 **The Interactivity API and Using the REST API · Checkpoint 2**

- 6.4 The Interactivity API
- 7.1 Using the REST API

S14 **Custom Endpoints, External APIs and Cron**

- 7.2 Custom REST Endpoints
- 7.3 External APIs, Transients & WP-Cron

S15 **Debugging, Testing and Performance**

- 8.1 Debugging & Testing
- 8.2 Performance & Caching

S16 **Distribution and Deployment**

- 8.3 Internationalization, Accessibility & Distribution
- 8.4 Deployment & Maintenance

S17 **Final Project Kickoff and Studio I**

- 9.1 Final Project: Launch the Pages & Co. Website

S18 **Final Project Studio II, Showcase and Next Steps**

- 9.1 Final Project: Launch the Pages & Co. Website

9.2 Next Steps: Headless, AI in Core & Your Career

PHP AND WORDPRESS DEVELOPMENT · ANSWER KEY

Part 1: Themes

Sessions 1 to 8. Modules 1 to 4: set up a WordPress 7.1 development site with Docker, WP-CLI, debugging and Mailpit, write PHP the WordPress way, learn the content model, the database and hooks, then build the Pages & Co. block theme with theme.json, templates, patterns and style variations, a child theme, and a classic/hybrid theme with the Loop, custom queries and functions.php. Runs on its own as a short course and ends with the Part 1 project and assessment.

INSTRUCTOR KIT · ANSWER KEY

Session 1: WordPress for Developers and Your Local Environment

Answers and reasoning for every Session 1 item: the diagnostic, both self-checks, the predict-then-run items, the lab reference outputs, the lesson pages' own quizzes and the exit ticket.

Session 1 | 12 self-check items | Confidential

START HERE · DIAGNOSTIC

Not graded. Each reason also says where the skill comes back in this course. PHP's own answers to D1, D2 and D6 are in the diagnostic lab's output below.

D1

What does this PHP print?

```php
$books = ['Dune' => 14.99, '1984' => 9.99, 'Emma' => 7.99];
echo count($books), ' ', array_key_first($books);
```

A) 3 Dune

B) 3 Emma

C) 2 Dune

D) 3 14.99

> **Why:** count() counts the three entries; array_key_first() returns the first KEY, 'Dune'. WordPress passes arrays of arguments almost everywhere (Lesson 1.3). **Skill:** PHP arrays. **Where it comes back:** PHP Foundations Module 3; every WordPress function that takes an array of arguments (Lesson 1.3).

D2

A visitor typed Maya into a form. What reaches the browser from this line?

```
echo 'Hello, ', htmlspecialchars('<b>Maya</b>');
```

A) Hello, Maya (in bold)

B) `Hello, <b>Maya</b>`

C) `Hello, <b>Maya</b>`

D) Hello, (the tags are removed)

> **Why:** htmlspecialchars() turns < and > into entities, so the browser shows the tags as text instead of running them. WordPress's esc_html() does the same job. **Skill:** escaping output. **Where it comes back:** PHP Foundations Lesson 4.2; esc_html() and friends in Lessons 1.3 and 5.3.

D3

Which query lists the titles of the books under $10, cheapest first?

A) `SELECT title FROM books ORDER BY price < 10;`

B) `SELECT title WHERE price < 10 FROM books;`

C) `SELECT title FROM books WHERE price < 10 ORDER BY price;`

D) `SELECT * FROM books HAVING price < 10 DESC;`

> **Why:** FROM names the table, WHERE keeps the rows that match, ORDER BY sorts (ascending unless you say DESC). Lesson 2.2 runs queries like this on WordPress's own tables. **Skill:** SQL. **Where it comes back:** MySQL Foundations; WordPress's tables in Lesson 2.2 and $wpdb in Lesson 5.5.

D4

Your terminal is in your home folder; the course project is ~/pagesco-dev. Which starts the site?

A) `docker compose up -d`

B) `docker compose up -d pagesco-dev`

C) `sudo docker compose up -d`

D) `cd pagesco-dev`, then `docker compose up -d`

> **Why:** Docker Compose reads the compose.yaml of the folder you are in (or a folder above it). From your home folder there is none, so every course command starts with being in the project folder. **Skill:** folders and the terminal. **Where it comes back:** Lesson 1.2 (every docker compose command) and every lab.

D5

A browser asks a server for a page that does not exist. Which status code should come back?

A) 404 Not Found

B) 200 OK

C) 301 Moved Permanently

D) 500 Internal Server Error

> **Why:** 404 means "no such page". 200 is success, 301 a redirect, 500 a crash on the server. WordPress answers 404 when the main query finds nothing, and picks the theme's 404 template. **Skill:** HTTP status codes. **Where it comes back:** PHP Foundations Lesson 1.1; rewrite rules and 404 templates in Lessons 2.2 and 3.1.

D6

What does this PHP print?

```
function shout($t) { return strtoupper($t) . '!'; }
echo implode(' ', array_map('shout', ['Dune', 'Emma']));
```

A) Dune! Emma!

B) DUNE! EMMA!

C) shout shout

D) An error: a function name cannot be a string

> **Why:** array_map() calls the function whose NAME you pass, once per item. That is a callback. In WordPress you hand function names to add_action() and add_filter(), and core calls them at the right moment (Lesson 2.3). **Skill:** callbacks. **Where it comes back:** PHP Foundations functions; hooks in Lesson 2.3 work exactly like this.

1.1 · WORDPRESS FOR DEVELOPERS: SELF-CHECK

Q1

Pages & Co. updates WordPress from 7.1.3 to 7.1.4. Which of these folders does the update REPLACE?

A) `wp-content/plugins/pagesco-books/`

B) `wp-content/themes/pagesco/`

C) `wp-includes/`

D) `wp-content/uploads/`

> **Why:** `wp-includes/`, `wp-admin/` and the `wp-*.php` files in the root are core: every update overwrites them, which is why a change there disappears (Lesson 1.1, "Never edit core files"). `wp-content/` belongs to the site.
> **A** is wrong: Your own plugin is never touched by a core update.
> **B** is wrong: Your own theme is never touched by a core update (a directory theme such as Twenty Twenty-Five is replaced by its own updates).
> **D** is wrong: Uploads are the site's media; no update touches them.

Q2

From the WP-CLI container, on a fresh course site: what does this print?

```
curl -s http://localhost:8080/ | grep -o '<title>.*</title>'
```

`<title>Pages & Co.</title>`

Why: The home page shows the latest posts, so WordPress titles it with the site title alone; a single post adds its own title and the separator (`Hello world! – Pages & Co.`). The title comes from the database, stored escaped: `&` is the ampersand.

Q3

A front-end request travels through core. Which of these happens FIRST?

A) template-loader.php picks a template from the theme

B) wp() parses the URL and runs the main query

C) the init action fires, after plugins and the theme are loaded

D) the theme's template prints the HTML

Why: wp-settings.php loads core, must-use plugins, plugins and the theme and fires `init`; only then does `wp()` work out the request and run the main query, and template-loader.php draws the page. Load, then query, then template.

A is wrong: Choosing a template needs the main query's result, so it comes last but one.

B is wrong: wp() runs after wp-settings.php has finished, init included.

D is wrong: That is the very end of the trip.

Q4

`wp core verify-checksums` prints `Warning: File doesn't verify against checksum: wp-includes/functions.php`. What happened, and what do you do?

A) The download was damaged: reinstall WordPress from scratch

B) WordPress 7.1 changed functions.php: nothing to do

C) A plugin is outdated: update all plugins

D) Someone edited a core file: restore it and put the change in a plugin or theme, on a hook

> **Why:** WordPress knows what every core file must contain. A file that fails the check was changed after installation, here by a "quick fix". The next update would silently undo it; security scanners flag it as a possible hack.
> **A** is wrong: A reinstall would also wipe the evidence of what was changed; restore the one file.
> **B** is wrong: Core files match their checksums in every release; a mismatch is never normal.
> **C** is wrong: Plugins live in wp-content/ and are not part of the core checksums.

Q5

With pretty permalinks, which file does the web server hand `/books/dune/`, `/about/` and almost every other front-end URL?

A) `wp-config.php`

B) `index.php` in the WordPress root

C) `wp-settings.php`

D) A file named after the URL, such as `books/dune.php`

> **Why:** Any request that does not match a real file goes to index.php; from there WordPress loads itself, matches the URL against its rewrite rules and runs the main query. That replaces the one-file-per-page router of PHP Foundations.
> **A** is wrong: wp-config.php only holds settings; it is loaded on the way, by wp-load.php.
> **C** is wrong: wp-settings.php is loaded later in the chain, never requested directly.
> **D** is wrong: No such files exist: the address is matched to content in the database.

Q6

In PHP Foundations you printed a visitor's input with `htmlspecialchars()`. Which WordPress function do you use instead when printing text into HTML?

`esc_html()`

> **Why:** It is Lesson 1.1's bridge table: `esc_html()` for text, `esc_attr()` inside attributes, `esc_url()` for links, `wp_kses_post()` for HTML that may keep safe tags. Lesson 1.3 practices the family; the fixed debug demo of Lesson 1.2 already used it.

1.1 · WORDPRESS FOR DEVELOPERS: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
curl -s http://localhost:8080/sample-page/ | grep -o '<title>.*</title>'
```

```
<title>Sample Page &#8211; Pages &amp; Co.</title>
```

> **Why:** A single page is titled with its own title, the separator (an en dash written as `–`) and the site title, which is stored escaped (`&`). Compare self-check Q2: the home page has the site title alone.

P2

Predict the output, then run it.

```
wp eval 'echo did_action( "init" ), " ", did_action( "template_redirect" ), "\n";'
```

```
1 0
```

> **Why:** WP-CLI loads all of WordPress, so `init` has fired once (`did_action()` counts). But no front-end request was made: `wp()` and the template loader never ran, so `template_redirect` fired 0 times. That is the line between loading and drawing a page (Lesson 1.1).

1.2 · YOUR LOCAL DEVELOPMENT ENVIRONMENT: SELF-CHECK

Q7

You want to throw your site away and start over with a blank WordPress. Which command deletes the database?

A) `docker compose stop`

B) `docker compose down`

C) `docker compose down -v`

D) `docker compose restart`

> **Why:** The database lives in the `db_data` volume, and only `-v` ("and the volumes") deletes it. Afterwards delete wordpress/ too and run the install steps again (Lesson 1.2, Everyday Commands).
>
> **A** is wrong: stop pauses the containers; everything stays.
>
> **B** is wrong: down removes the containers but keeps the volume: Exercise 1 proves the tagline survives it.
>
> **D** is wrong: restart stops and starts the same containers.

Q8

What does this print on the course site, exactly?

```
wp option get blogname
```

`Pages & Co.`

> **Why:** WordPress stores the site title HTML-escaped (`--title="Pages & Co."` is saved with `&`). Browsers show a plain &, but code that reads the option gets the escaped form: it matters when you compare strings or put the title in an email.

Q9

wp-content/debug.log says `PHP Warning: Undefined variable $pagesco_name in /var/www/html/wp-content/mu-plugins/pagesco-debug-demo.php on line 8`. Where is that file on YOUR computer?

A) `/var/www/html/wp-content/...` on your computer's disk

B) `wordpress/wp-content/mu-plugins/pagesco-debug-demo.php` in your project folder

C) Inside the db container

D) Nowhere: it only exists while the container runs

> **Why:** `./wordpress:/var/www/html` in compose.yaml is a bind mount: your wordpress/ folder IS the WordPress root inside the containers. Every path in an error message translates by swapping `/var/www/html` for your `wordpress` folder.
> **A** is wrong: /var/www/html exists only inside the containers.
> **C** is wrong: The database container holds the database, not WordPress's files.
> **D** is wrong: A bind-mounted folder is an ordinary folder on your disk; it outlives every container.

Q10

In a new terminal you type `wp core version` and get `no configuration file provided: not found`. What is wrong?

A) You are not in the project folder: Docker Compose found no compose.yaml here or above

B) WordPress is not installed yet

C) Docker is not running

D) The wp shortcut is missing

> **Why:** Docker Compose looks for compose.yaml in the current folder and every folder above it. `cd ~/pagesco-dev` (or any folder inside it) and try again (Lesson 1.2, Common Mistakes 5).
> **B** is wrong: An uninstalled site gives a WP-CLI error about the database tables, not this.
> **C** is wrong: With Docker stopped the message is about the Docker daemon.
> **D** is wrong: Without the shortcut the shell says wp: command not found.

Q11

Most Docker tutorials map `8080:80`. Why does the course project use `8080:8080` (Apache listens on 8080 inside the container too)?

A) Port 80 is reserved for MySQL

B) Docker Desktop cannot publish port 80

C) It makes the site faster

D) WordPress calls its own address, http://localhost:8080, from inside the container (WP-Cron, Site Health), and that must reach Apache

> **Why:** With `8080:80` the site works in your browser, but WordPress's requests to itself at http://localhost:8080 reach nothing inside the container, and `wp cron test` fails. config/apache-8080.conf adds the second port.
> **A** is wrong: MySQL uses 3306, and only on the private network.
> **B** is wrong: It can; the choice is about WordPress's own requests.
> **C** is wrong: Speed is the same; correctness is the point.

Q12

What does this print on the course site? Name one thing that value changes.

```
wp eval 'echo wp_get_environment_type(), "\n";'
```

`local`: for example, Mailpit's mu-plugin only reroutes email when the environment is local; application passwords work over plain HTTP; WordPress 7.1 turns pings off

> **Why:** `WP_ENVIRONMENT_TYPE` comes from WORDPRESS_CONFIG_EXTRA in compose.yaml. Code (yours included) can behave differently on local, staging and production; Lesson 8.4 sets production.

1.2 · YOUR LOCAL DEVELOPMENT ENVIRONMENT: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
wp db query "SELECT option_value FROM wp_options WHERE option_name = 'blogname'"
```

```
option_value
Pages &amp;amp; Co.
```

Why: `wp db query` prints the column name and then the value, tab-separated, without a table (in a terminal or not). The value is the escaped title as it sits in the database: the escaping happened when the option was SAVED.

P4

Predict the output, then run it.

```
wp eval 'var_dump( wp_mail( "", "No address", "Hello" ) );'
```

```
bool(false)
```

Why: No message at all: `wp_mail()` returns false and WordPress keeps the reason for the `wp_mail_failed` action (Lesson 1.2, Common Mistakes 3 shows how to listen to it). A silent false is the hardest kind of bug: check return values.

LAB REFERENCE SOLUTIONS

Every output below is real: the lab files run exactly as `labs/check_labs.py` runs them, on a fresh course site (WordPress 7.1.3, PHP 8.5.12, MySQL 9.7, WP-CLI 2.12.0), copied into the WordPress root and run with `wp eval-file --use-include` (PHP labs) or `sh` in the WP-CLI container (shell labs). Paths show `/var/www/html/...`, which is the learner's `wordpress/` folder.

ENVIRONMENT CHECK: LABS/MODULE-01/L1_0-ORIENTATION/CHECK_ENVIRONMENT.SH

A tool, not an exercise. On a ready machine it prints (here in a throwaway copy of the course project; on a learner's machine the Docker and Compose versions differ, and on macOS the owner line is left out):

```
PHP and WordPress Development: environment check
OK    Docker 29.6.2
OK    Docker Compose 5.3.1
OK    project folder pagesco-dev (compose.yaml of project pagesco)
OK    services running: db, mailpit, wordpress
OK    wordpress/ belongs to you (uid 1000): your editor can save in wp-content/
OK    WP-CLI 2.12.0 (the course's wpcli image)
OK    WordPress 7.1.3
OK    PHP 8.5.12 (web server), 8.5.12 (WP-CLI)
OK    site address http://localhost:8080
OK    pretty permalinks /%postname%/
OK    the home page answers 200 inside the containers
OK    WordPress can call itself (wp cron test)
OK    debugging on (WP_DEBUG, WP_DEBUG_LOG, WP_DEBUG_DISPLAY), environment local, wp-c
OK    wp_mail() sent a test message and Mailpit caught it
OK    this computer reaches http://localhost:8080/ (the site) and http://localhost:802
READY for PHP and WordPress Development
```

On PHP 8.4 the PHP line says CHECK and the verdict is `READY WITH NOTES`; a project under /mnt/c on Windows adds a CHECK line too. The FIX lines name what to do; the most common are "Docker is installed but not running", "not running: db mailpit wordpress" (`docker compose up -d`), the Linux file owner, "WordPress is not installed yet" and the missing Mailpit plugin. `--part2` adds Node 24, the composer:2 image and the starting site of Part 2 (pagesco active after Lesson 4.3): Session 9 uses it.

- DIAGNOSTIC PRACTICE: LABS/MODULE-01/L1_0-ORIENTATION/DIAGNOSTIC_STARTER.PHP → DIAGNOSTIC_SOLUTION.PHP

As shipped the starter prints PHP's answers with the three gaps still in place (the last key instead of the first, no escaping, the wrong callback):

```
D1  PHP says: 3 Emma
D2  PHP says: Hello, <b>Maya</b>
D6  PHP says: dune emma
```

The solution prints:

```
D1  PHP says: 3 Dune
D2  PHP says: Hello, &lt;b&gt;Maya&lt;/b&gt;
D6  PHP says: DUNE! EMMA!
```

Common sticking points: `htmlspecialchars` typed inside the quotes (it prints the word); `'pagesco_shout()'` with parentheses as the callback (a callback is the NAME); D2's output looks wrong in the terminal and right in a browser, which is the point of escaping.

- LAB 1.1-A (GUIDED): LABS/MODULE-01/L1_1-WORDPRESS-FOR-DEVELOPERS/CORE_TOUR_STARTER.SH → CORE_TOUR_SOLUTION.SH

As shipped:

```
1. wp-load.php loads your settings file:
50: require_once ABSPATH . 'wp-config.php';
2. wp-settings.php runs every must-use plugin:
TODO 2
3. wp-settings.php fires plugins_loaded, then init:
TODO 3
4. template-loader.php: the last chance to redirect, then the filter that can swap the
TODO 4
5. PHP files in wp-includes, the core library:
TODO 5
```

The solution prints:

```
1. wp-load.php loads your settings file:
50: require_once ABSPATH . 'wp-config.php';
2. wp-settings.php runs every must-use plugin:
506:foreach ( wp_get_mu_plugins() as $mu_plugin ) {
3. wp-settings.php fires plugins_loaded, then init:
630:do_action( 'plugins_loaded' );
779:do_action( 'init' );
4. template-loader.php: the last chance to redirect, then the filter that can swap the
23: do_action( 'template_redirect' );
114:    $template   = apply_filters( 'template_include', $template );
5. PHP files in wp-includes, the core library:
1059
```

What each step shows: (1) wp-load.php finds your settings; (2) must-use plugins run first, from a plain foreach, no activation; (3) plugins are loaded and `plugins_loaded` fires, then the theme, then `init`: code that registers a post type belongs on init; (4) `template_redirect` is the last moment to redirect before anything is drawn, and the `template_include` filter can swap the template the hierarchy chose (Lesson 3.1); (5) the size of the library you never edit. The line numbers are WordPress 7.1.3's and move between releases: the grep, not the number, is the skill.

- LAB 1.1-B: LABS/MODULE-01/L1_1-WORDPRESS-FOR-DEVELOPERS/WP_PATHS_STARTER.PHP → WP_PATHS_SOLUTION.PHP

As shipped:

```
WordPress root  /var/www/html/
wp-content      TODO
plugins         TODO
must-use        TODO
active theme    TODO
its URL         TODO
installed plugins: TODO
must-use plugins: TODO
```

The solution prints:

```
WordPress root  /var/www/html/
wp-content      /var/www/html/wp-content
plugins         /var/www/html/wp-content/plugins
must-use        /var/www/html/wp-content/mu-plugins
active theme    twentytwentyfive
its URL         http://localhost:8080/wp-content/themes/twentytwentyfive
installed plugins: akismet/akismet.php, hello.php
must-use plugins: mailpit-smtp.php
```

Accept `get_template()` for TODO 4 on this site (without a child theme the template and the stylesheet are the same theme; Lesson 3.4 shows where they differ) and `get_template_directory_uri()` for TODO 5 the same way. The point of TODO 6: some functions live in admin files that WP-CLI and the front end do not load; the Code Reference names the file.

- HANDOUT: LABS/MODULE-01/L1_1-WORDPRESS-FOR-DEVELOPERS/DEBUG_OLD_TUTORIAL.PHP (DELIBERATELY OLD)

As shipped, WordPress prints:

```
Deprecated: Function get_page_by_title is <strong>deprecated</strong> since version 6.2
Sample Page ID: 2
Deprecated: Function get_userdatabylogin is <strong>deprecated</strong> since version 3
Admin email: wordpress@example.com
Escaped: Deprecated: Function wp_specialchars is <strong>deprecated</strong> since vers
Pride &amp;amp; Prejudice &amp;lt;new edition&amp;gt;
[new lines in wp-content/debug.log]
[26-Oct-2026 12:01:29 UTC] PHP Deprecated:  Function get_page_by_title is <strong>depre
[26-Oct-2026 12:01:29 UTC] PHP Deprecated:  Function get_userdatabylogin is <strong>dep
[26-Oct-2026 12:01:29 UTC] PHP Deprecated:  Function wp_specialchars is <strong>deprec
```

1. `get_page_by_title()`, **deprecated since 6.2.0:** use a query. `get_posts( array( 'post_type' => 'page', 'title' => 'Sample Page', 'numberposts' => 1 ) )` returns an array; take `[0]`.
2. `get_userdatabylogin()`, **since 3.3.0:** `get_user_by( 'login', 'admin' )`.
3. `wp_specialchars()`, **since 2.8.0:** `esc_html()`.

Fixed (the three replacements), it prints the same three answers without a message:

```
Sample Page ID: 2
Admin email: wordpress@example.com
Escaped: Pride &amp;amp; Prejudice &amp;lt;new edition&amp;gt;
```

The point: deprecated code keeps working, which is why it survives in tutorials; the messages appear only with debugging on, and each names its replacement. The oldest call here was replaced in 2009.

- LAB 1.2-A (GUIDED): LABS/MODULE-01/L1_2-LOCAL-ENVIRONMENT/SITE_REPORT_STARTER.SH → SITE_REPORT_SOLUTION.SH

As shipped:

```
== Pages & Co. site report ==
WordPress: 7.1.3
Title: TODO
Admin email: TODO
Active theme: TODO
Inactive plugins: TODO
Pages: TODO
```

The solution prints:

```
== Pages & Co. site report ==
WordPress: 7.1.3
Title: Pages &amp;amp; Co.
Admin email: wordpress@example.com
Active theme: twentytwentyfive
Inactive plugins:
akismet
hello
Pages:
+----+----------------+-------------+
| ID | post_title     | post_status |
+----+----------------+-------------+
| 2  | Sample Page    | publish     |
| 3  | Privacy Policy | draft       |
+----+----------------+-------------+
```

`--field=name` prints one value per line without a table; `--fields=` chooses the columns of a table. WP-CLI draws the table because the script runs in a terminal (`docker compose run` gives it one); piped into another command it prints tab-separated rows instead. On a learner's site that is past Session 1 the lists grow: the commands, not the rows, are the answer.

- LAB 1.2-B: LABS/MODULE-01/L1_2-LOCAL-ENVIRONMENT/STORE_HOURS_STARTER/ → STORE_HOURS_SOLUTION/ WITH TEST_STORE_HOURS.SH

The test with the starter's `pagesco-hours.php` in mu-plugins/:

```
Home page, the plugin's paragraph:
<p class="pagesco-hours">Open Saturday: TODO</p>
debug.log, the plugin's lines (without the time stamps):
(none yet)
```

With the solution's:

```
Home page, the plugin's paragraph:
<p class="pagesco-hours">Open Saturday: 10:00-16:00</p>
debug.log, the plugin's lines (without the time stamps):
      1 pagesco-hours: footer printed
```

Reading it: the must-use plugin runs on every request without activation (Lesson 1.1); `wp_footer` is the hook the lesson's debug demo used; `esc_html()` is the habit from the fixed demo; `error_log()` writes to debug.log because WP_DEBUG_LOG is on, and its line has no PHP level, so it is a note, not a warning. One page view, one log line: the test's own request. Remove the file afterwards (`rm wordpress/wp-content/mu-plugins/pagesco-hours.php`): the course site must stay as the lessons expect.

- HANDOUT: LABS/MODULE-01/L1_2-LOCAL-ENVIRONMENT/DEBUG_HEALTH_CHECK.PHP (DELIBERATELY BROKEN)

As shipped, PHP prints:

```
WordPress 7.1          OK
Site address           OK
Warning: Undefined array key "fail" in /var/www/html/debug_health_check.php on line 28
Pretty permalinks
Environment is local   OK
Mail plugin present    OK
Warning: Undefined variable $pagesco_failed in /var/www/html/debug_health_check.php on
 check(s) failed.
```

1. **Silent, line 18:** `get_option( 'permalink' )` reads an option that does not exist (false), so "Pretty permalinks" fails although permalinks are on. The option is `permalink_structure` (Lesson 1.2's `wp option get permalink_structure`).
2. **Warning, line 28:** the words array has the keys `ok` and `failed` , the code asks for `fail` . It only shows because bug 1 made a check fail: fix bug 1 first and this bug hides until a check really fails. Fix the key anyway.
3. **Warning, line 33:** the counter is `$pagesco_failes` (line 26) and the last line reads `$pagesco_failed` . Rename the counter; `0 === null` is false, which is why the script claimed " check(s) failed." with no number.

Fixed, all five checks say OK and the last line is `All checks passed.` The lesson in it: a check that cannot fail, or fails for the wrong reason, proves nothing (the health check's Step 4).

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 1.1 · WORDPRESS FOR DEVELOPERS

1. **B** In the pagesco-books plugin in wp-content/plugins/: It is a feature of the store's data, so it belongs in the store's own plugin, in wp-content/plugins/, where it survives core updates and theme changes.
2. **C** Ask which plan they are on, because only some plans allow your own plugins: Whether you can install your own plugins on WordPress.com depends on the plan. Find out first; the answer may be an upgrade or a move to another host.
3. **A** index.php, wp-blog-header.php, wp-config.php, wp-settings.php, template-loader.php: index.php hands over to wp-blog-header.php, whose wp-load.php finds wp-config.php, which ends by loading wp-settings.php; only then do wp() and the template loader run.
4. **C** Replace the call with wp_get_current_user() and check how old the tutorial is: The message names the replacement. Switch to wp_get_current_user(), and treat the tutorial with care: it was written for a much older WordPress.
5. **B** WordPress core runs the main program and calls your code through hooks: Core runs the program and calls your functions at the moments you registered them for with hooks. This inversion of control is what Lesson 2.3 builds on.

LESSON 1.2 · YOUR LOCAL DEVELOPMENT ENVIRONMENT

1. **C** In a new wpcli container that is removed when the command ends: The alias runs docker compose run --rm wpcli wp: a one-off wpcli container that shares the WordPress files and the network, runs the command in /var/www/html and is removed afterwards.
2. **A** Run the containers as your user (.env plus compose.override.yaml) after giving the existing files to your user: Running the wordpress and wpcli containers as your own user means you, Apache and WP-CLI all create files as the same user. Repair the existing files with a root chown in a container first.
3. **B** Both are still there: the database is in a volume and the files are in wordpress/: The posts are in the db_data volume and the theme files in the wordpress folder on your disk; down only removes containers. Only down -v deletes the volume.
4. **B** mailpit-smtp.php is not in wp-content/mu-plugins/, so the sender wordpress@localhost is rejected: Without mailpit-smtp.php in mu-plugins, WordPress sends from wordpress@localhost, PHPMailer rejects that address, and wp_mail returns false. Copy the plugin back.
5. **C** npx @wordpress/env start, with phpVersion 8.5 in .wp-env.json: The official package is @wordpress/env, run with npx and a .wp-env.json that sets phpVersion 8.5. The unscoped wp-env package is someone else's code.

OPENING: READING THE ROOM

There is no retrieval warm-up in Session 1. The opening's two checks have their answers above: the environment check output (what READY looks like) and the diagnostic D1 to D6. Learners who end the clinic still NOT READY leave with a numbered plan in their workbook's Start Here box.

EXIT TICKET

1. **Your code:** in `wp-content/`: your own theme, your own plugin, or a must-use plugin. Core (`wp-admin/`, `wp-includes/`, the root `wp-*.php`) is replaced by every update, so a change there disappears, and `wp core verify-checksums` (and every security scanner) reports it.
2. **Four services:** wordpress (Apache and PHP 8.5 with WordPress, the site on port 8080), db (MySQL 9.7, the data in the db_data volume), wpcli (WP-CLI, started for each command and removed), mailpit (catches every email, inbox on 8025).
3. **A critical error:** the end of `wordpress/wp-content/debug.log`: `tail -n 20 wordpress/wp-content/debug.log` from the project folder (or `tail -f` while reloading the page; Lesson 1.2's Exercise 3 makes it `wplog`).

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 B ("the theme folder").** Mixes up WordPress updates and theme updates. Fine to discuss: Twenty Twenty-Five IS replaced, by its own updates; pagesco is not, by anything.
- **Q2 with an en dash.** Expects every title to have the separator. The home page lists posts and is titled with the site name alone; ask them to curl `/hello-world/` and compare.
- **Q3 B ("wp() first").** Thinks WordPress works out the request before loading plugins. Redraw the boxes: the plugins must be loaded first, or nobody could register a post type before the query runs.
- **Q5 D ("a file per URL").** The PHP Foundations model. Show `ls` of the root again: there is no about.php.
- **Q9 A ("/var/www/html on my disk").** Has not connected the bind mount to the folder they edit. Open wordpress/ in VS Code next to an error message.
- **Q11 B or C.** Guessing. Run `wp cron test` and explain what it requested.
- **Q12 with `development`.** Remembers the four environment types but not this site's. The value comes from compose.yaml: open it.

INSTRUCTOR KIT · ANSWER KEY

Session 2: PHP the WordPress Way and the Content Model

Answers and reasoning for every Session 2 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs, the lesson pages' own quizzes and the exit ticket.

Session 2 | 12 self-check items | Confidential

WARM-UP · SESSION 1 REVISITED

1. **W1** Which folder of the WordPress root is yours, and why is nothing else? **Answer:** `wp-content/` (themes, plugins, mu-plugins, uploads), plus your settings in `wp-config.php`. Everything else is core and every update replaces it.
2. **W2** Put them in order: the main query, `init`, the template. **Answer:** `init` (the end of loading: core, mu-plugins, plugins, the theme), then `wp()` runs the main query, then the template loader draws the page: load, query, template.
3. **W3** `docker compose down` and `docker compose down -v`: what survives each? **Answer:** `down` keeps the files and the database (the volume); `-v` deletes the database volume too.
4. **W4** A page says "There has been a critical error on this website." Where do you look first? **Answer:** The end of `wordpress/wp-content/debug.log` (`tail -n 20 wordpress/wp-content/debug.log`, or `tail -f` while reloading).
5. **W5** What does `wp option get blogname` print on the course site, and why? **Answer:** `Pages & Co.`: WordPress stored the title HTML-escaped when it was saved. Today's first lesson explains when to escape yourself.

The site check prints, on a site exactly as Session 1 left it:

```
7.1.3
+--------------+----------+
| name         | status   |
+--------------+----------+
| mailpit-smtp | must-use |
+--------------+----------+
```

1.3 · PHP THE WORDPRESS WAY: SELF-CHECK

Q1

A template prints a book cover: `<img src="..." alt="...">`. The alt text is the book's title as staff typed it. Which function goes around the title?

A) `esc_html()`

B) `esc_attr()`

C) `esc_url()`

D) `wp_kses_post()`

> **Why:** The value goes inside an attribute, so `esc_attr()` (and the attribute is quoted). Lesson 1.3's rule: escape late, for the exact place the value is printed into.
> **A** is wrong: `esc_html()` is for text between tags. Here it would happen to encode the same characters, but a reviewer or PHPCS reads the context from the function: use the one that names it.
> **C** is wrong: `esc_url()` is for the `src` and every `href`: it checks the protocol, which an alt text does not have.
> **D** is wrong: `wp_kses_post()` keeps HTML tags a post may contain; tags inside an attribute value break the page.

Q2

From the project folder, on the course site: what does this print?

```
wp eval 'echo esc_html( esc_html( "Tom & Jerry" ) ), "\n";'
```

`Tom & Jerry`

> **Why:** The first `esc_html()` turns the ampersand into `&`, and the second leaves an entity that is already there alone: WordPress's escapers do not double-encode (Lesson 1.3, the escape.php demo), so escaping at output is safe even when something upstream already escaped.

Q3

What reaches the page from `$h = '<h2>' . _e( 'Staff picks', 'pagesco' ) . '</h2>'; echo $h;` ?

A) `<h2>Staff picks</h2>`

B) Nothing: PHP stops with a fatal error

C) "Staff picks" first, then an empty `<h2></h2>`

D) `<h2>_e(Staff picks)</h2>`

> **Why:** `_e()` ECHOES the translation at once and returns null, so the text is printed while the string is still being built, and the heading ends up empty. Inside an expression you want `__()`, or better `esc_html__()` (Lesson 1.3, Common Mistakes 3). No message warns you.
> **A** is wrong: That is what __() would give.
> **B** is wrong: Concatenating null is allowed; nothing complains.
> **D** is wrong: _e is a function call, not text.

Q4

Which of these needs NO `pagesco_` prefix?

A) A class inside the namespace `PagesCo\Books`, such as `PagesCo\Books\Plugin`

B) A global function that formats prices

C) A variable defined at the top level of a plugin file

D) An option your plugin saves in wp_options

> **Why:** The namespace is the prefix (Module 5's plugin uses one). Everything global shares one PHP process with core and every plugin, and a database table with them: functions, global variables, constants, hook names, options and meta keys all get the prefix.
> **B** is wrong: Two plugins with format_price() stop the site with a fatal error.
> **C** is wrong: A plugin file runs in the global scope, so its variables are globals.
> **D** is wrong: Options share wp_options with core and every plugin: pagesco_books_settings, not settings.

Q5

What does this print?

```
wp eval 'var_dump( absint( "-7 copies" ), sanitize_key( "Staff Picks!" ) );'
```

```
int(7)
string(10) "staffpicks"
```

Why: `absint()` makes a non-negative integer: the text is cut at the first non-digit and the minus sign is dropped (validate when a negative must be REJECTED). `sanitize_key()` keeps lowercase letters, digits, dashes and underscores: option names and array keys.

Q6

`$id = wp_insert_post( array( 'post_title' => '' ) );` What is `$id`, and how do you learn why it failed?

A) A WP_Error: call `$id->get_error_message()`

B) false: read debug.log

C) An exception: wrap the call in try/catch

D) `0`: pass `true` as the second argument to get a WP_Error with the message

Why: By default `wp_insert_post()` returns 0 on failure; with `true` as its last argument it returns the WP_Error ("Content, title, and excerpt are empty."). Read the Return line in the Code Reference before you rely on a result (Lesson 1.3, errors.php).

A is wrong: Only when asked for with true; wp_insert_user() always returns one.

B is wrong: Nothing is logged: the failure is silent.

C is wrong: Core returns errors as values; it does not throw.

1.3 · PHP THE WORDPRESS WAY: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
wp eval 'echo esc_url( "example.com/books?a=1&b=2" ), "\n";'
```

> `http://example.com/books?a=1&b=2`

Why: `esc_url()` adds `http://` to an address without a protocol (it looks like a domain) and encodes the ampersand for HTML as `&`. In an href the browser turns it back into `&`. A URL you store or redirect to uses `esc_url_raw()`, which leaves the `&` alone.

P2

Predict the output, then run it.

```
wp eval 'echo _n( "%s book", "%s books", 0 ), "\n";'
```

> `%s books`

Why: English uses the plural for zero, so `_n()` picks the second string. It does NOT fill in the number: `_n()` only chooses the string, and `sprintf()` puts the count into `%s` (Lesson 1.3's book card does both: `sprintf( _n( ... ), number_format_i18n( $n ) )`).

2.1 · THE CONTENT MODEL: SELF-CHECK

Q7

Twenty Twenty-Five's header menu lists the site's pages. Which post type stores that menu?

A) `nav_menu_item`

B) `page`

C) `wp_template_part`

D) `wp_navigation`

> **Why:** Block themes keep their menus as `wp_navigation` posts (post 4 on the course site, holding a Page List block). Everything is a post with a type: one filing cabinet, many labels (Lesson 2.1).
> **A** is wrong: nav_menu_item rows are the menus of classic themes (Module 4).
> **B** is wrong: The pages are what the menu LISTS, not the menu.
> **C** is wrong: The header template part contains the Navigation block; the menu itself is a separate post.

Q8

On a fresh course site (before the lesson project): what does this print, and why are the two numbers counting different things?

```
wp eval 'echo count( get_posts( array( "post_type" => "page" ) ) ), " ", wp_count_p
```

`1 1`: one published page for `get_posts()` (it defaults to publish), and one draft page counted by `wp_count_posts()` (the Privacy Policy)

> **Why:** `get_posts()` and WP_Query default to `'post_status' => 'publish'`, so PHP sees only the Sample Page; `wp_count_posts()` counts every status, and the draft Privacy Policy is one of them. WP-CLI's `wp post list` also lists every status unless you pass `--post_status`.

Q9

Devon (Author) opens Maria's published post "Book club: Dune" (ID 17). What does `user_can( $devon, 'edit_post', 17 )` answer?

A) false: `edit_post` is a meta capability that maps to `edit_others_posts` here, which authors lack

B) true: authors have `edit_posts`

C) true: the post is published

D) An error: `edit_post` needs a role, not a user

> **Why:** WordPress maps the meta capability `edit_post` to real capabilities from the post's author and status: someone else's published post needs `edit_others_posts` (and `edit_published_posts`). Lesson 2.1's Exercise 1 prints exactly this "no".
>
> **B** is wrong: edit_posts covers his own posts only.
>
> **C** is wrong: Published makes it harder, not easier: edit_published_posts joins the list.
>
> **D** is wrong: user_can() takes a user (or ID); roles are never checked directly.

Q10

What does this print?

```
wp eval 'echo parse_blocks( "<!-- wp:separator /-->" )[0]["blockName"], "\n";'
```

`core/separator`

> **Why:** Block markup drops the `core/` namespace of core blocks (`wp:separator`), and `parse_blocks()` puts it back. A self-closing comment is a block with no saved HTML; dynamic blocks such as Latest Posts look the same and are rendered by PHP on every view.

Q11

A script creates a post with WP-CLI and then updates it twice. How many revisions does the post have, and what is in the newest one?

A) Three: the newest holds the version before the last update

B) Two: the newest is identical to the current post

C) One: WordPress keeps only the latest

D) None: only the block editor makes revisions

> **Why:** "Every update is a revision, and the most recent revision always matches the current post" (core's own comment, shown in Lesson 2.1). Creating a post is not an update, so the first version has no revision.
> **A** is wrong: That is the intuitive model, and it is wrong in both the count and the content.
> **C** is wrong: WP_POST_REVISIONS is true on the course site: every revision is kept (-1).
> **D** is wrong: WP-CLI and the editor both call wp_insert_post() / wp_update_post(); the revision comes from core.

Q12

A deployment script must set the "Site Address (URL)" field of Settings > General. Which option is that?

A) `siteurl`

B) `blogname`

C) `home`

D) `permalink_structure`

> **Why:** Site Address is `home` (where visitors find the site); WordPress Address is `siteurl` (where WordPress's files live). They are usually equal, which is why they are easy to swap. Every Settings field is a row in wp_options (Lesson 2.1's table).
> **A** is wrong: siteurl is the WordPress Address field.
> **B** is wrong: blogname is the Site Title.
> **D** is wrong: permalink_structure is Settings > Permalinks.

2.1 · THE CONTENT MODEL: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
wp post list --post_type=attachment --format=count
```

```
0
```

Why: Zero. Twenty Twenty-Five's pictures are files in its theme folder, not uploads: only what goes through the Media Library (or `wp media import`) becomes an attachment post. The lesson's demo imports one of those theme files and deletes it again.

P4

Predict the output, then run it.

```
wp eval 'var_dump( get_option( "page_on_front" ), get_option( "pagesco_no_such_opti
```

```
string(1) "0"
bool(false)
```

Why: `page_on_front` exists and is the STRING "0" (no static front page; options are strings). An option that does not exist returns `false`, which is how `get_option()` tells you it is missing (pass a second argument for another default).

LAB REFERENCE SOLUTIONS

Every output below is real: the lab files run exactly as `labs/check_labs.py` runs them, on a fresh course site (WordPress 7.1.3, PHP 8.5, MySQL 9.7, WP-CLI 2.12), copied into the WordPress root and run with `wp eval-file --use-include` (PHP labs) or `sh` in the WP-CLI container (shell labs). Paths show `/var/www/html/...`, which is the learner's `wordpress/` folder. All six files only read: they leave the site exactly as they found it.

- LAB 1.3-A (GUIDED): LABS/MODULE-02/L1_3-PHP-THE-WORDPRESS-WAY/SIGNUP_FORM_STARTER.PHP → SIGNUP_FORM_SOLUTION.PHP

As shipped:

```
name:  string(4) "TODO"
email: TODO
seats: TODO
shelf: TODO
field: <input name="name" value="TODO">
link:  <a href="TODO">TODO</a>
```

The solution prints:

```
name:  string(18) "Siobhan O'Neil VIP"
email: Siobhan.ONeil@Example.COM (valid: yes)
seats: 3
shelf: fiction
field: <input name="name" value="Siobhan O&amp;#039;Neil VIP">
link:  <a href="">Siobhan O&amp;#039;Neil VIP</a>
```

What each line shows: the name lost its tags but not the text between them, its line break and extra spaces, and kept its apostrophe because `wp_unslash()` came first; the email was trimmed (sanitize_email() keeps the case) and `is_email()` validated it; `absint()` turned -3 into 3, which a real form should REJECT instead (validate, Lesson 1.3); the shelf was not on the list, so validation fell back to fiction; `esc_attr()` encoded the apostrophe for the attribute; `esc_url()` emptied the `javascript:` link. Accept `sanitize_text_field()` without `wp_unslash()` only after the learner has seen `O\'Neil` in the output and explained it.

- `LAB 1.3-B: LABS/MODULE-02/L1_3-PHP-THE-WORDPRESS-WAY/EVENT_LINE_STARTER.PHP → EVENT_LINE_SOLUTION.PHP`

As shipped:

```
TODO: Author reading: Ada Quill
TODO: Book club: Dune
TODO: Story time & cocoa
TODO: Poetry night
TODO: Book club: Dune
TODO: Poetry night
```

The solution prints:

```
skipped: The event has no date. (pagesco_no_date)
Book club: Dune on Thursday, October 1 · 1 seat left
Story time &amp;amp; cocoa on Saturday, October 17 · 0 seats left
Author reading: Ada Quill on Thursday, November 12 · 24 seats left
Book club: Dune on Thursday, October 1
New: Author reading: Ada Quill on Nov 12 · 24 seats left
```

Reading it: the sort callback runs first, so the undated Poetry night (an empty string sorts first) is skipped with its WP_Error's message and code; `_n()` chose "1 seat left" and "0 seats left" (English uses the plural for zero);

`esc_html()` encoded the ampersand of "Story time & cocoa"; the last two calls pass options as an array and as a query string, which `wp_parse_args()` accepts too. Common sticking points: the arguments of `wp_parse_args()` swapped (the defaults then override the caller: the debug handout's silent bug), concatenating the date into the translated string instead of using `%2$s`, and printing the WP_Error object ("Object of class WP_Error could not be converted to string").

- HANDOUT: LABS/MODULE-02/L1_3-PHP-THE-WORDPRESS-WAY/DEBUG_BOOK_BADGE.PHP (DELIBERATELY BROKEN)

As shipped, PHP prints:

```
Warning: Undefined array key "autor" in /var/www/html/debug_book_badge.php on line 33
Warning: Undefined array key "price" in /var/www/html/debug_book_badge.php on line 35
Deprecated: number_format(): Passing null to parameter #1 ($num) of type float is depre
<span class="pagesco-badge">Staff pick</span> <b>The Left Hand of Darkness</b> by  $0.(
Warning: Undefined array key "autor" in /var/www/html/debug_book_badge.php on line 33
<span class="pagesco-badge">Staff pick</span> <b>Emma</b> by  $7.99
```

1. **Warning, line 33:** `$book['autor']`: the key is `author`. Both books lose their author.
2. **Warning and Deprecated, line 35:** the first book has no price, so `$book['price']` is undefined (the Warning names the lab file) and the null reaches `number_format()` inside core (the Deprecated names wp-includes/functions.php). Read the Warning just above a core Deprecated: that is your line. Fix: print the price only when it is set (`isset( $book['price'] )`).
3. **Silent, line 24:** the arguments of `wp_parse_args()` are swapped: the defaults are merged OVER the caller's options, so the second badge says "Staff pick" although the call asks for "New in paperback". `wp_parse_args( $args, $defaults )`: what the caller passed wins.

Fixed (the key, the isset() and the order), it prints both badges as the calls ask for, without a message:

```
<span class="pagesco-badge">Staff pick</span> <b>The Left Hand of Darkness</b> by Ursu
<span class="pagesco-badge">New in paperback</span> <b>Emma</b> by Jane Austen $7.99
```

The lesson in it: messages find the typos; only reading the output against the calls finds the swapped arguments. A helper with an array of arguments needs one test call that passes every option.

- LAB 2.1-A (GUIDED): LABS/MODULE-02/L2_1-CONTENT-MODEL/CONTENT_CENSUS_STARTER.SH → CONTENT_CENSUS_SOLUTION.SH

As shipped:

```
== Pages & Co. content census ==
Post types: 16
Published posts: TODO
Pages, any status: TODO
Draft pages: TODO
Categories: TODO
Home page shows: TODO
Editor capabilities: TODO
Blocks in post 1: TODO
```

The solution prints (the site before the lesson project):

```
== Pages & Co. content census ==
Post types: 16
Published posts: 1
Pages, any status: 2
Draft pages: 1
Categories: Uncategorized
Home page shows: posts
Editor capabilities: 34
Blocks in post 1:
      1 <!-- wp:paragraph
```

`--format=count` makes WP-CLI count for you; leaving `--post_status` out lists every status (two pages, one of them the draft Privacy Policy). After the project the same script says 4 published posts, 5 pages, two categories (News and Uncategorized): the commands, not the numbers, are the answer. `uniq -c` prints the count first, padded with spaces. 34 capabilities include the old `level_*` entries kept for old plugins.

- LAB 2.1-B: LABS/MODULE-02/L2_1-CONTENT-MODEL/POST_REPORT_STARTER.PHP → POST_REPORT_SOLUTION.PHP

As shipped:

```
#1 TODO (publish) "Hello world!"
   slug: hello-world · hierarchical type: TODO
   author: admin, may edit others' posts: TODO
   terms: TODO
   revisions: TODO
   top-level blocks: TODO
   address: TODO
#2 TODO (publish) "Sample Page"
   slug: sample-page · hierarchical type: TODO
   author: admin, may edit others' posts: TODO
   terms: TODO
   revisions: TODO
   top-level blocks: TODO
   address: TODO
#3 TODO (draft) "Privacy Policy"
   slug: privacy-policy · hierarchical type: TODO
   author: admin, may edit others' posts: TODO
   terms: TODO
   revisions: TODO
   top-level blocks: TODO
   address: TODO
#4 TODO (publish) "Navigation"
   slug: navigation · hierarchical type: TODO
   author: none (post_author is 0)
   terms: TODO
   revisions: TODO
   top-level blocks: TODO
   address: TODO
#999 not found
```

The solution prints:

```
#1 Post (publish) "Hello world!"
   slug: hello-world · hierarchical type: no
   author: admin, may edit others' posts: yes
   terms: category:Uncategorized
   revisions: 0
   top-level blocks: 1 (core/paragraph)
   address: http://localhost:8080/hello-world/
#2 Page (publish) "Sample Page"
   slug: sample-page · hierarchical type: yes
   author: admin, may edit others' posts: yes
   terms: none
   revisions: 0
   top-level blocks: 5 (core/paragraph, core/quote)
   address: http://localhost:8080/sample-page/
#3 Page (draft) "Privacy Policy"
   slug: privacy-policy · hierarchical type: yes
   author: admin, may edit others' posts: yes
   terms: none
   revisions: 0
   top-level blocks: 24 (core/heading, core/paragraph)
   address: http://localhost:8080/?page_id=3
#4 Navigation Menu (publish) "Navigation"
   slug: navigation · hierarchical type: no
   author: none (post_author is 0)
   terms: none
   revisions: 0
   top-level blocks: 1 (core/page-list)
   address: none (not a public type)
#999 not found
```

Points to draw out: post 4 (Twenty Twenty-Five's navigation menu) has no author, because WordPress made it, and no public address (`is_post_type_viewable()` is false: `get_permalink()` would still build one); the draft Privacy Policy has only an ugly `?page_id=3` address (no slug until it is published) and 24 top-level blocks; pages have no taxonomies at all. Accept `get_post_type_object( ... )->public` for TODO 6 on this site. After the project the IDs 12 to 17 work too: try `pagesco_lab_describe( 16 )` for the Staff picks post.

HANDOUT: LABS/MODULE-02/L2_1-CONTENT-MODEL/DEBUG_PAGE_REPORT.PHP (DELIBERATELY BROKEN)

As shipped, PHP prints:

```
Post: Hello world!
Filed under:
Page 2 categories:Warning: foreach() argument must be of type array|object, false given

Page 3 categories:
Pages are looked after by: Warning: Attempt to read property "display_name" on false in
[new lines in wp-content/debug.log]
[26-Oct-2026 13:16:27 UTC] PHP Warning:  foreach() argument must be of type array|object
[26-Oct-2026 13:16:27 UTC] PHP Warning:  Attempt to read property "display_name" on false
```

1. **Silent, line 19:** `$pagesco_post->category` prints nothing and no message: WP_Post has no such field, and for an unknown property it quietly looks up post meta with that key (empty). The categories come from `get_the_terms( $post, 'category' )`.
2. **Warning, line 24:** pages have no categories, so `get_the_terms()` returns false and `foreach` cannot loop over it. Check the result (`is_array()`) or ask first (`is_object_in_taxonomy( 'page', 'category' )`). The message shows once although the loop runs twice: PHP does not repeat the same message from the same line here.
3. **Warning, line 32:** there is no user maria yet (she arrives in the lesson project), so `get_user_by()` returns false. Check it before you use it. After the project this bug hides: the code is still wrong.

Fixed, it prints:

```
Post: Hello world!
Filed under: Uncategorized
Page 2 categories: none (pages have no categories)
Page 3 categories: none (pages have no categories)
Pages are looked after by: nobody yet (no user maria)
```

The lesson in it: core functions say "nothing" with false or an empty value, not with an exception, and what posts and pages have is decided by their registration (Lesson 2.1's `get_object_taxonomies()`).

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 1.3 · PHP THE WORDPRESS WAY

1. **C** echo 'Website';: href and src take esc_url, which encodes the URL for HTML and drops dangerous protocols such as javascript:.
2. **A** small red: wp_parse_args keeps the default size, uses the passed color, and simply carries the misspelled colour key along without using it.
3. **B** if (0 === $count) { pagesco_reset_count(); }: Spaces inside the parentheses, a Yoda condition with ===, and a prefixed snake_case function name.

4. **A** if (is_wp_error($id)) { /* show $id->get_error_message() */ }: wp_insert_user returns the new user ID or a WP_Error object; is_wp_error tells the two apart, and the error object has the code and message.

5. **C** $nick = sanitize_text_field(wp_unslash($_POST['nickname']));: Unslash first (WordPress adds slashes to request data), then sanitize on input. Escape later, when you print it.

LESSON 2.1 · THE CONTENT MODEL

1. **B** A page: Timeless content that is linked from a menu rather than listed by date is what pages are for.

2. **C** core/heading 4: Core blocks are core/name in PHP even though the comment omits the namespace, and attributes come from the JSON in the opening comment.

3. **A** current_user_can('edit_others_posts'): Check the capability the feature needs. Administrators, editors and any custom role that has it pass; nobody else does.

4. **B** At least three; the newest is identical to the current page: Every update stores a revision of the new version, so the most recent revision always matches the current page.

5. **C** In the posts_per_page option: wp option update posts_per_page 12: Every Settings field is an option; this one is posts_per_page, so wp option update posts_per_page 12 sets it.

EXIT TICKET

1. **Escapers:** `esc_html()` for text between tags, `esc_attr()` for an attribute value (quoted), `esc_url()` for every href and src; `wp_kses_post()` when the value is HTML a post may contain.

2. **Prefixes:** core, the theme and every active plugin run in ONE PHP process with one global namespace (and share one options table): a second function with the same name is a fatal error that takes the site down, and you cannot know which plugins a client installs later. A namespace counts as a prefix for classes.

3. **The content model:** all are rows of `wp_posts`; the `post_type` column tells them apart (post, page, attachment, revision, wp_navigation). That is why they share revisions, permissions and the REST API.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 A (`esc_html()`).** Thinks of escaping as one operation. It works here by accident; ask where an href goes, then show the javascript: link from Mistake 1.
- **Q2** `Tom & Jerry`. Expects double encoding. Run it: WordPress's escapers leave existing entities alone.
- **Q3 A.** Has not met "`_e` echoes". Have them run the demo's `mistake-e.php` and read the third line of its output.

- **Q4 C.** Thinks a variable in a plugin file is private. Plugin files run in the global scope; show `$GLOBALS` holding it.
- **Q6 A.** Generalizes from `wp_insert_user()`. Read the Return line of `wp_insert_post()` in the Code Reference together.
- **Q7 A.** Classic-theme knowledge. Run `wp post get 4 --field=post_type` and `--field=post_content`.
- **Q8** `2 1` **or** `1 0`**.** Expects one default for all tools. WP-CLI vs `get_posts()` is the lesson's statuses section.
- **Q11 A.** The intuitive model of revisions. Run the lesson's revision demo again and compare the newest revision with the post.
- **Q12 A.** Swaps the two addresses. WordPress Address = siteurl (files), Site Address = home (visitors).

INSTRUCTOR KIT · ANSWER KEY

Session 3: Core Architecture, the Database and Hooks

Answers and reasoning for every Session 3 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs, the lesson pages' own quizzes and the exit ticket.

Session 3 | 12 self-check items | Confidential

WARM-UP · SESSION 2 REVISITED

1. **W1** Which escaping function goes around a URL in an href attribute? **Answer:** `esc_url()` (text between tags: `esc_html()`, other attribute values: `esc_attr()`).
2. **W2** Name the four shapes a WordPress callback can take. Which one is hard for anyone else to unhook? **Answer:** A function name, a closure, `array( $object, 'method' )`, a static method (`'Class::method'` or `array( 'Class', 'method' )`). The closure: nobody else has a handle on it (Lesson 2.3 comes back to this).
3. **W3** A page and a post: which table holds each, and which column tells them apart? **Answer:** Both are rows in `wp_posts`; `post_type` is `page` or `post` (Lesson 2.1's filing cabinet).
4. **W4** Code must decide whether the current user may publish pages. What does it check, and why not the role? **Answer:** `current_user_can( 'publish_pages' )`: a capability. Roles are bundles that sites change; capabilities are what WordPress checks.
5. **W5** What does `wp_parse_args( $args, $defaults )` return? **Answer:** One array: the defaults, overridden by whatever the caller passed in `$args`.

The site check prints, on a site where Lesson 2.1's project was finished:

```
+----+-----------+----------------+-------------+
| ID | post_type | post_name      | post_author |
+----+-----------+----------------+-------------+
| 1  | post      | hello-world    | 1           |
| 2  | page      | sample-page    | 1           |
| 3  | page      | privacy-policy | 1           |
| 12 | page      | about          | 2           |
| 13 | page      | contact        | 2           |
| 14 | page      | events         | 2           |
| 15 | post      | welcome        | 2           |
| 16 | post      | staff-picks    | 3           |
| 17 | post      | book-club-dune | 2           |
+----+-----------+----------------+-------------+
+----+------------+---------------+
| ID | user_login | roles         |
+----+------------+---------------+
| 1  | admin      | administrator |
| 2  | maria      | editor        |
| 3  | devon      | author        |
+----+------------+---------------+
```

2.2 · CORE ARCHITECTURE & THE DATABASE: SELF-CHECK

Q1

A tutorial says to add `define( 'DISALLOW_FILE_EDIT', true );` to wp-config.php. You paste it as the very last line of the file, after `require_once ABSPATH . 'wp-settings.php';`. What happens?

A) It works: wp-config.php is read on every request, so the position does not matter

B) PHP stops with a parse error

C) It is defined too late: wp-settings.php has already loaded WordPress, so the file editors stay

D) WordPress moves the line above the "stop editing" comment the next time it saves the file

Why: The require_once on line 139 hands over to the loader, and WordPress reads its settings while it loads. Anything after it is defined when the decisions have been made. That is what "That's all, stop editing!" means: custom lines go between the two comments above it.

A is wrong: The position is the whole point: wp-config.php runs top to bottom, and the last line loads WordPress.

B is wrong: It is valid PHP; it simply comes too late.

D is wrong: WordPress never rewrites wp-config.php by itself.

Q2

What does this print on the course site, and what is that value?

```
wp config get DB_HOST
```

`db` : the name of the database service in compose.yaml (on a typical host it is localhost)

> **Why:** wp-config.php reads it with getenv_docker('WORDPRESS_DB_HOST', ...) from the container's environment, which compose.yaml sets. Containers on one Compose network reach each other by service name. `wp config get` reads wp-config.php for you, so a script never needs the password.

Q3

In which order does wp-settings.php load these on every request?

A) The theme's functions.php, active plugins, must-use plugins, pluggable.php

B) Must-use plugins, active plugins, pluggable.php, the theme's functions.php

C) pluggable.php, must-use plugins, the theme's functions.php, active plugins

D) Active plugins, must-use plugins, the theme's functions.php, pluggable.php

> **Why:** Must-use plugins at line 506, active plugins at 582, pluggable.php at 612, the theme at 741 (WordPress 7.1.3). Pluggable functions come after the plugins so a plugin can replace one by defining it first; that is also why calling one at the top of a plugin file fails.
> **A** is wrong: That is the order backwards: the theme comes last of the four.
> **C** is wrong: pluggable.php waits for the plugins, on purpose.
> **D** is wrong: Must-use plugins always come before normal plugins.

Q4

Devon's first name is stored once in wp_usermeta. What does this print?

```
wp eval 'var_dump( get_user_meta( 3, "first_name" ) );'
```

```
array(1) {
  [0]=>
  string(5) "Devon"
}
```

Why: Without the third argument (`true`) the metadata API returns EVERY value of the key, as an array, because a key may be stored several times. With `true` it returns the single value, the string "Devon". Lesson 2.2, Common Mistakes 4.

Q5

Someone runs `UPDATE wp_usermeta SET meta_value = REPLACE(meta_value, 'author', 'editor')` on Devon's `a:1:{s:6:"author";b:1;}`. What happens?

A) The value breaks: unserialize() fails and Devon loses every role

B) It works, but only because both words have 6 characters: the s:6 length still matches. Change roles through WordPress anyway

C) MySQL refuses: the column holds serialized PHP

D) WordPress recomputes the lengths when it reads the value

Why: Serialized values store each string's length. Same length, still valid; "Springfield" to "Portland" was not (Lesson 2.2, Common Mistakes 1). It is still the wrong tool: it skips WordPress's caches and hooks, and the next REPLACE may not be so lucky. `wp user set-role devon editor` or `$user->set_role()` is the API.

A is wrong: That happens when the length changes; here it does not.

C is wrong: To MySQL it is just LONGTEXT.

D is wrong: WordPress trusts the stored lengths; a mismatch makes unserialize() fail.

Q6

An option's autoload column says `auto-off`. What does that mean?

A) The code that saved it passed false

B) It is loaded on every request

C) The option is broken and will be deleted

D) Nobody chose, and WordPress decided not to autoload it, usually because the value is large (over 150,000 bytes)

> **Why:** on/off = the code said true/false; auto = nobody said, WordPress's default (loaded today); auto-on/auto-off = WordPress decided by itself. Lesson 2.2's autoload demo: a 200,000-byte option became auto-off. Rule for your plugins: pass `true` for small settings you read on every page, `false` for anything big or rare.
> **A** is wrong: Passing false gives off.
> **B** is wrong: auto-off is NOT in the list wp_autoload_values_to_autoload() returns.
> **C** is wrong: Nothing about it is broken; it is read on demand, with one query.

2.2 · CORE ARCHITECTURE & THE DATABASE: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
wp eval 'var_dump( get_option( "pagesco_no_such_option" ), get_option( "pagesco_no_
```

```
bool(false)
string(4) "none"
```

> **Why:** An option that does not exist returns false, or the default you pass as the second argument. Both reads cost a query (WordPress cannot know an option is missing without asking; the lesson's options-cost.php measured it).

P2

Predict the output, then run it.

```
wp db query "SELECT option_value FROM wp_options WHERE option_name = 'active_plugin
```

```
option_value
a:0:{}
```

Why: `a:0:{}` is a serialized empty array: no active plugins. `wp db query` prints the column name, then the raw value; `get_option( 'active_plugins' )` would return `array()`.

2.3 · THE HOOKS SYSTEM: SELF-CHECK

Q7

What does this print when run with wp eval-file?

```php
<?php
add_filter( 'pagesco_q7', fn( $text ) => $text . 'b', 20 );
add_filter( 'pagesco_q7', fn( $text ) => strtoupper( $text ) );
add_filter( 'pagesco_q7', fn( $text ) => $text . 'a', 5 );

echo apply_filters( 'pagesco_q7', 'x' ), "\n";
```

`XAb`

Why: Priority 5 runs first (`xa`), then the callback at the default 10 (`XA`), then 20 (`XAb`). The order in which the lines were written does not matter across priorities; only ties at the same priority keep their added order.

Q8

A members page must send visitors who are not logged in to the login screen. On which action do you call wp_safe_redirect() and exit?

A) `plugins_loaded`

B) `wp_footer`

C) `template_redirect`

D) `shutdown`

> **Why:** template_redirect fires when WordPress knows which page was asked for (conditional tags work) and before any HTML is printed, so headers can still be sent. Lesson 2.3's table of actions.
> **A** is wrong: Too early: the request has not been parsed, so you cannot know which page it is.
> **B** is wrong: Too late: the page has been printed, headers are gone.
> **D** is wrong: Far too late: the response is finished.

Q9

A theme adds `add_action( 'wp_head', 'pagesco_theme_banner' )` inside its after_setup_theme callback. Your plugin calls `remove_action( 'wp_head', 'pagesco_theme_banner' )` at the top of its main file, and the banner stays. Why?

A) The plugin ran before the theme added the hook, so there was nothing to remove: remove it on init

B) remove_action() only works on core's hooks

C) The priority is wrong: it must be 1

D) Plugins cannot change what a theme does

> **Why:** Removal rule 2 of Lesson 2.3: remove after it was added. Plugins load at line 582 of wp-settings.php, the theme at 741 and its after_setup_theme callbacks after that; init (line 779) comes later. remove_action() returned false, silently.
> **B** is wrong: It works on any hook, if the hook, callback and priority match and the hook exists already.
> **C** is wrong: The theme added it at the default 10, so 10 (the default) is right.
> **D** is wrong: They can, with hooks; that is the point of them.

Q10

What does this print, and what does the value mean?

```
wp eval 'var_dump( has_filter( "the_content", "do_blocks" ) );'
```

`int(9)`: do_blocks is attached to the_content at priority 9, before wpautop at 10

> **Why:** has_filter() with a callback returns its priority, or false when it is not attached. do_blocks turns block markup into HTML before the paragraph filter runs (Lesson 2.1 named it; Exercise 3 of 2.3 lists them all). Test the result with `false !== has_filter( ... )`: 0 is a valid priority.

Q11

After a new the_title filter, posts have no titles and the only message is `Deprecated: str_replace(): Passing null to parameter #3 ... in .../wp-includes/formatting.php on line 5777`. What is the likely bug?

A) Your callback does not return the title, so the next callback (core's, at priority 11) received null

B) A bug in WordPress 7.1's formatting.php: update core

C) PHP 8.5 deprecated str_replace()

D) The filter needs 2 accepted arguments

> **Why:** A filter must return the value. Null went down the chain to capital_P_dangit(), which handed it to str_replace(), so the message points into core. When a message blames a core file, look at what YOUR code handed to it (Lesson 2.3, Common Mistakes 1).
> **B** is wrong: Core is fine; it received a null it did not expect.
> **C** is wrong: str_replace() is fine; passing null to it is deprecated.
> **D** is wrong: That mistake is a fatal ArgumentCountError, not a Deprecated notice.

Q12

Your plugin prints a gift-card form. Other developers should be able to print a line of their own just BEFORE the form. What do you add right before you print it?

A) `apply_filters( 'pagesco_before_gift_form', '' );`

B) `add_action( 'pagesco_before_gift_form', 'pagesco_print_form' );`

C) `do_action( 'before_gift_form' );`

D) `do_action( 'pagesco_before_gift_form' );` with a DocBlock above it

> **Why:** A moment others may act at is an action; you fire it with do_action(), prefixed and documented like core's (the project's pagesco_before_store_notice). A value others may change would be a filter.
> **A** is wrong: A filter is for a value; nobody is asked to change one here.
> **B** is wrong: add_action() attaches a callback; it does not create the moment.
> **C** is wrong: Unprefixed: another plugin's before_gift_form would collide with it.

2.3 · THE HOOKS SYSTEM: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
wp eval 'add_filter( "pagesco_p", "strrev" ); echo apply_filters( "pagesco_p", "Dun
```

```
enuD Dune
```

> **Why:** `strrev` is a callback by name: PHP's own function, which takes one value and returns one, the shape a filter needs. A hook nobody filters returns the value unchanged, which is why calling apply_filters() costs nothing when no plugin uses it.

P4

Predict the output, then run it.

```
wp eval 'add_action( "pagesco_p", function () { echo "ran "; } ); var_dump( do_acti
```

```
ran NULL
```

Why: The action ran its callback (`ran`), and do_action() returns nothing: `var_dump()` of nothing is NULL. Actions are moments, not values; if you need an answer back, offer a filter.

LAB REFERENCE SOLUTIONS

Every output below is real: the lab files run exactly as `labs/check_labs.py` runs them, on the course site at the end of Lesson 2.1 (WordPress 7.1.3, PHP 8.5, MySQL 9.7, WP-CLI 2.12), copied into the WordPress root and run with `wp eval-file --use-include` (PHP labs) or `sh` in the WP-CLI container (shell labs). Paths show `/var/www/html/...`, which is the learner's `wordpress/` folder.

- LAB 2.2-A (GUIDED): LABS/MODULE-03/L2_2-CORE-DATABASE/META_PIVOT_STARTER.SH → META_PIVOT_SOLUTION.SH

As shipped:

```
== Step 1: the users ==
ID  user_login  display_name
1   admin   admin
2   maria   Maria Lopez
3   devon   Devon Park

== Step 2: first names, one join ==
Step 2: TODO
== Step 3: one row per user, two fields ==
Step 3: TODO
== Step 4: the same through WordPress ==
Step 4: TODO
== Step 5: meta rows per user ==
Step 5: TODO
```

The solution prints:

```
== Step 1: the users ==
ID  user_login  display_name
1   admin   admin
2   maria   Maria Lopez
3   devon   Devon Park

== Step 2: first names, one join ==
ID  user_login  first_name
1   admin
2   maria   Maria
3   devon   Devon

== Step 3: one row per user, two fields ==
ID  user_login  first_name  last_name
1   admin
2   maria   Maria   Lopez
3   devon   Devon   Park

== Step 4: the same through WordPress ==
+----+------------+------------+-----------+
| ID | user_login | first_name | last_name |
+----+------------+------------+-----------+
| 1  | admin      |            |           |
| 2  | maria      | Maria      | Lopez     |
| 3  | devon      | Devon      | Park      |
+----+------------+------------+-----------+
== Step 5: meta rows per user ==
user_id meta_rows
1   16
2   15
3   16
```

The key query is Step 3: one join without a key condition gives every meta row of every user; GROUP BY folds them into one row per user, and each `MAX(CASE WHEN um.meta_key = 'x' THEN um.meta_value END)` keeps the one non-NULL value of its key (MAX of one value is that value). Common wrong turns: the key condition in WHERE (then only one key survives the join and the other column is NULL); a second join per field (also correct: one join per field, the shape WP_Query writes for two meta conditions). admin's names are empty strings, not NULL: the rows exist. Step 5's counts grow when a learner changes screen options or opens the editor (WordPress stores per-user preferences as meta).

- LAB 2.2-B: LABS/MODULE-03/L2_2-CORE-DATABASE/WPDB_REPORT_STARTER.PHP → WPDB_REPORT_SOLUTION.PHP

As shipped:

```
Prefix: wp_
Comments (SQL): TODO
Comments (API): TODO
Maria's pages: TODO
permalink_structure: TODO
Autoload values WordPress loads: TODO
```

The solution prints:

```
Prefix: wp_
Comments (SQL): 1
Comments (API): 1
Maria's pages: about, contact, events
permalink_structure: /%postname%/ (autoload on)
Autoload values WordPress loads: yes, on, auto-on, auto
```

Accept `get_var()` with a string literal for TODO 1 (no value goes in, so prepare() has nothing to do) but not a typed `wp_comments`. TODO 3: the 2 goes in through `%d` even though it is a constant here; the habit is the point (Lesson 5.5). TODO 4: `get_row()` returns an object with one property per selected column. TODO 5 is the list from the lesson's version note: what `wp_load_alloptions()` loads; `auto-off` is not in it.

HANDOUT: LABS/MODULE-03/L2_2-CORE-DATABASE/DEBUG_META_REPORT.PHP (DELIBERATELY BROKEN)

As shipped, PHP prints:

```
Post 15 waiting for pings: no
Warning: foreach() argument must be of type array|object, string given in /var/www/htm
Published pages: Warning: Undefined property: stdClass::$published in /var/www/html/de
[new lines in wp-content/debug.log]
[26-Oct-2026 14:09:36 UTC] PHP Warning:  foreach() argument must be of type array|obje
[26-Oct-2026 14:09:36 UTC] PHP Warning:  Undefined property: stdClass::$published in /
```

1. **Silent, line 21:** `get_post_meta( 15, '_pingme' )` without `true` returns an array (`array( '1' )`), never identical to the string '1', so the answer is "no" although the row exists. Pass `true` (Lesson 2.2, Common Mistakes 4).
2. **Warning, line 26 (the bug is on line 25):** the meta key is `wp_capabilities`, with the table prefix. `get_user_meta()` of a key that does not exist returns '' with `true`, and foreach() on a string warns. In code the key is `$wpdb->prefix . 'capabilities'` (the prefix can be anything); better still, `get_userdata( 2 )->roles`.
3. **Warning, line 32:** wp_count_posts() names its properties after the statuses: `publish`, not `published`.

Fixed, it prints:

```
Post 15 waiting for pings: yes
Maria's role: editor
Published pages: 4
```

Two of the three bugs are the lesson's own lessons in disguise (the third argument, the prefix); the third is the most ordinary bug in PHP: a guessed property name. `wp db query "SELECT meta_key FROM wp_usermeta WHERE user_id = 2"` answers the second one in one line.

LAB 2.3-A (GUIDED): LABS/MODULE-03/L2_3-HOOKS/FILTER_CHAIN_STARTER.PHP → FILTER_CHAIN_SOLUTION.PHP

As shipped:

```
Removed the sale badge: TODO
pagesco_lab_capitalize: TODO
Line: fahrenheit 451 [SALE]
```

The solution prints:

```
Removed the sale badge: true
pagesco_lab_capitalize: 5
pagesco_lab_add_author: 10
pagesco_lab_sale_badge: false
Line: Fahrenheit 451 by Ray Bradbury
```

What each TODO shows: (1) priority 5 runs before the default 10, so the title is capitalized before the author is added; (2) `10, 2` hands the callback the hook's second value, the author (without it: the fatal ArgumentCountError of Lesson 2.3, Mistake 2); (3) the badge was added at 30, so only `remove_filter( ..., 30 )` removes it, and the true says so; (4) has_filter() returns the priority, and false for the removed badge. Accept `ucwords` written inside the author callback only if the learner can say why the separate callback is better (another plugin can remove or reorder it).

LAB 2.3-B: LABS/MODULE-03/L2_3-HOOKS/HOOK_FINDER_STARTER.SH → HOOK_FINDER_SOLUTION.SH

As shipped:

```
== 1. Where core fires wp_head ==
wp-includes/general-template.php:3441:  do_action( 'wp_head' );
== 2. Who adds the generator tag ==
2. TODO
== 3. Where the excerpt length is filtered ==
3. TODO
== 4. Asked of the running site ==
4. TODO
== 5. Callbacks on wp_head ==
5. TODO
```

The solution prints:

```
== 1. Where core fires wp_head ==
wp-includes/general-template.php:3441:  do_action( 'wp_head' );
== 2. Who adds the generator tag ==
362:add_action( 'wp_head', 'wp_generator' );
== 3. Where the excerpt length is filtered ==
wp-includes/formatting.php:4081:        $excerpt_length = (int) apply_filters( 'excerpt
wp-includes/blocks/latest-posts.php:167:                $excerpt_length = (int) apply_f
== 4. Asked of the running site ==
wpautop on the_content: 10
== 5. Callbacks on wp_head ==
30
```

Patterns must be exact: `grep -rn wp_head wp-includes/` returns every line that mentions it (callbacks attached to it, comments, the function wp_head() itself). With the quotes and spaces core's coding standard uses, `do_action( 'wp_head' )` is found once, at line 3441 of general-template.php, and the generator at line 362 of default-filters.php. Step 3 finds two callers: wp_trim_excerpt() in formatting.php (the automatic excerpt) and the Latest Posts block, which is why one filter changes both. Step 5's 30 counts the callbacks on wp_head of a WP-CLI run on this site; a front-end request can differ, because themes and plugins add their own. Line numbers move between releases; the pattern is the skill.

- LAB 2.3-C: LABS/MODULE-03/L2_3-HOOKS/TITLE_PARTS_STARTER/ → TITLE_PARTS_SOLUTION/ WITH TEST_TITLE_PARTS.SH

The test with the starter's `pagesco-title.php` in mu-plugins/:

```
/                <title>Pages &amp; Co. &#8211; TODO</title>
/book-club-dune/  <title>Book club: Dune &#8211; Pages &amp; Co.</title>
/hello-world/     <title>Hello world! &#8211; Pages &amp; Co.</title>
/about/           <title>About Us &#8211; Pages &amp; Co.</title>
```

With the solution's:

```
/                <title>Pages &amp; Co. &#8211; Independent bookstore since 1998</tit
/book-club-dune/  <title>News: Book club: Dune &#8211; Pages &amp; Co.</title>
/hello-world/     <title>Hello world! &#8211; Pages &amp; Co.</title>
/about/           <title>About Us &#8211; Pages &amp; Co.</title>
```

Reading it: document_title_parts passes an array (title, page, tagline, site) and the callback returns the array, changed or not; WordPress then joins the parts with the separator and escapes them, so `&` arrives as `&`. The front page is the blog home here (Settings > Reading: your latest posts), so its parts are the site name and the tagline. `in_category( 'news' )` takes the slug and only Book club: Dune of the four tested pages is a News post; Hello world! (Uncategorized) and About Us stay as they were. The TODO 3 trap: a `return` only inside the if,

which empties every other title. Remove the file afterwards (`rm wordpress/wp-content/mu-plugins/pagesco-title.php`).

HANDOUT: LABS/MODULE-03/L2_3-HOOKS/DEBUG_PRICE_TAG.PHP (DELIBERATELY BROKEN)

As shipped, PHP prints:

```
Deprecated: number_format(): Passing null to parameter #1 ($num) of type float is depre
Warning: Undefined variable $pagesco_shop_name in /var/www/html/debug_price_tag.php on
 price: $0.00 (holiday price)
```

1. **Deprecated, line 26 (the bug is on line 21):** pagesco_lab_round_price() assigns the rounded price and returns nothing, so the next callback receives null and number_format() complains. The message points at an innocent line, the pattern of Lesson 2.3's Mistake 1. Fix: `return round( $price, 2 );`.
2. **Warning, line 36:** a function cannot see variables from outside it, so `$pagesco_shop_name` is undefined inside the callback and the label starts with nothing. Fix: the name inside the function (or a constant). `global $pagesco_shop_name;` does NOT help here: a file run by `wp eval-file` is included inside a WP-CLI method, so its top-level variables are local to that method, not globals. Measured with a three-line probe, `global` gives:
3. **Silent, line 41:** the markup was added at priority 15, so `remove_filter()` without the 15 looks at 10, finds nothing and returns false; "(holiday price)" stays. Fix: pass 15, and `var_dump()` the result while you debug.

```
NULL
```

Fixed (all three, with the removal's result printed), it prints:

```
bool(true)
Pages & Co. price: $14.99
```

Fixing bug 1 first changes the Deprecated message into a correct price, which makes the silent bug easy to spot: the markup is visible on a real number. That order is worth asking learners to explain.

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 2.2 · CORE ARCHITECTURE & THE DATABASE

1. **A** A row in wp_postmeta with the post's ID, a meta_key and a meta_value: Extra fields of a post are rows in wp_postmeta: post_id, meta_key, meta_value. That is what update_post_meta writes.
2. **B** 10, 100, 9: meta_value is LONGTEXT, so the values are compared as text, character by character: 1 sorts before 9. Cast them, or use meta_value_num in WP_Query, for numeric order.

3. **C** auto: With no decision from the code and a small value, WordPress stores auto, which is autoloaded today. Large values become auto-off.

4. **B** It stays in wp_postmeta as orphaned rows: WordPress tables have no foreign keys, so MySQL neither refuses nor cascades. The meta and term rows stay as orphans; wp_delete_post would have removed them.

5. **C** pluggable.php, which defines it, loads only after all plugins: wp_get_current_user lives in pluggable.php, which wp-settings.php loads after all plugins so they can replace its functions. Call it from a hook such as init instead.

LESSON 2.3 · THE HOOKS SYSTEM

1. **B** add_filter('the_title', ...) with a callback that returns the title and an asterisk: the_title is a filter: WordPress passes the title in and displays whatever your callback returns, so the callback returns the title with an asterisk added.

2. **C** CBDA: C has priority 5 and runs first. B and D both have the default 10 and run in the order they were added, then A at 20.

3. **A** remove_filter('excerpt_length', 'pagesco_excerpt_length', 15);: Removal needs the same hook, the same callback and the same priority it was added with, here 15.

4. **C** ArgumentCountError: Too few arguments to function {closure}(), 1 passed ... and exactly 2 expected: add_filter defaults to one accepted argument, so the closure is called with only the title and PHP throws ArgumentCountError for the missing $id. Registering with 10, 2 fixes it.

5. **A** $count = apply_filters('pagesco_featured_count', 3);: A prefixed, documented filter on the value is exactly the extension point: the other developer hooks pagesco_featured_count and returns a different number.

EXIT TICKET

1. **Through WordPress:** WordPress's tables have no foreign keys, so PHP does what ON DELETE CASCADE would do (wp_delete_post() removes meta, term links, revisions and comments); serialized values carry their lengths, which a text REPLACE breaks; and the API keeps the object cache and every hook in step. A DELETE at the prompt leaves orphans; an UPDATE skips the caches.

2. **Banner and excerpts:** a banner at the top of every page is a moment: an action on `wp_body_open` (the project's store notice). The excerpt length is a value: a filter on `excerpt_length` whose callback returns a number.

3. **remove_action() false:** the priority differs from the one used in add_action(); it ran before the hook was added (a theme's after_setup_theme callback, removed from the top of a plugin file); or the callback is not the same one (a closure, or a method of a different object).

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 A ("position does not matter").** Sees wp-config.php as a list of settings, not a program that ends by loading WordPress. Show line 139 again.
- **Q3 A or D.** Remembers that the theme is "the outside" and plugins "the inside" but not the order. Redraw the shop: lights, always-there things, today's departments, window display.
- **Q4 with `string(5) "Devon"`.** Expects one value because there is one row. The metadata API always returns a list unless you ask for one value: `true`.
- **Q5 A.** Correct instinct (never edit serialized text), wrong mechanism: count the letters. The answer's second half (use the API anyway) is the part to keep.
- **Q7 with `XbA` or `xaB`.** Ran the lines in the order written. Ask: "which priority runs first?"
- **Q9 C.** Blames the priority, the most common removal bug, but this one is timing. Ask: "when did the theme add it, and when did your line run?"
- **Q11 D.** Mixes up the two the_title mistakes: a missing return is a Deprecated notice and an empty title; two arguments asked and one registered is a fatal error.

INSTRUCTOR KIT · ANSWER KEY

Session 4: Theme Anatomy and Building a Block Theme

Answers and reasoning for every Session 4 item: the warm-up, Checkpoint 1's model answer and rubric, both self-checks, the predict-then-run items, the lab reference outputs, the lesson pages' own quizzes and the exit ticket.

Session 4 | 12 self-check items | Checkpoint 1 | Confidential

WARM-UP

Real output on the site of Lesson 2.3.

W1

Session 3: priorities: What does this print on the site of Lesson 2.3?

```
wp eval 'echo has_filter( "pagesco_store_notice", "pagesco_notice_late_opening" ),
```

```
20
```

Why: The priority: `has_filter()` returns the priority of that callback on that hook, or false. pagesco-site.php adds the late-opening text at 20, so it runs after a callback at the default 10.

W2

Session 3: must-use plugins: Which two names does this list on your site, and why does neither need activating?

```
wp plugin list --status=must-use --fields=name,status
```

```
+--------------+----------+
| name         | status   |
+--------------+----------+
| mailpit-smtp | must-use |
| pagesco-site | must-use |
+--------------+----------+
```

> **Why:** mailpit-smtp (Lesson 1.2) and pagesco-site (Lesson 2.3). Every PHP file in wp-content/mu-plugins/ runs on every request, before the normal plugins (the load order of Lesson 2.2).

W3

Session 3: removing a hook: Why does this print 0?

```
curl -s http://localhost:8080/about/ | grep -c 'name="generator"'
```

```
0
```

> **Why:** pagesco-site.php calls `remove_action( 'wp_head', 'wp_generator' )`, so the generator meta tag is never printed (and grep -c with no match exits 1).

CHECKPOINT 1 · MODEL ANSWER

The task is on the workbook's Checkpoint 1 page; the starter is `labs/module-04/checkpoint-1/checkpoint_starter/pagesco-checkpoint.php`. The model answer, `labs/module-04/checkpoint-1/checkpoint_solution/pagesco-checkpoint.php`:

```php
<?php
/**
 * Plugin Name: Pages & Co. Checkpoint 1
 * Description: Checkpoint 1 (Modules 1 and 2), model answer: three small additions to
 */

/**
 * (a) Adds the newest News post to the store notice.
 *
 * @param string $text The notice text so far (plain text).
 * @return string
 */
function pagesco_checkpoint_latest_news( $text ) {
    $posts = get_posts(
        array(
            'category_name' => 'news',
            'numberposts'   => 1,
        )
    );

    if ( ! $posts ) {
        return $text;
    }

    return $text . ' Latest news: ' . get_the_title( $posts[0] ) . '.';
}
add_filter( 'pagesco_store_notice', 'pagesco_checkpoint_latest_news', 30 );

/**
 * (b) Prints a link to Maria's author archive inside the notice bar.
 */
function pagesco_checkpoint_ask_link() {
    $user = get_user_by( 'login', 'maria' );

    if ( ! $user ) {
        return;
    }

    echo ' · <a href="' . esc_url( get_author_posts_url( $user->ID ) ) . '">' . esc_htm
}
add_action( 'pagesco_store_notice_end', 'pagesco_checkpoint_ask_link', 20 );

/**
 * (c) Prints the number of published pages in the footer.
 */
function pagesco_checkpoint_page_count() {
    global $wpdb;
```

```
    $count = (int) $wpdb->get_var(
        $wpdb->prepare(
            "SELECT COUNT(*) FROM {$wpdb->posts} WHERE post_type = %s AND post_status =
            'page',
            'publish'
        )
    );

    echo '<p class="pagesco-pages">Published pages: ' . esc_html( $count ) . '</p>';
}
add_action( 'wp_footer', 'pagesco_checkpoint_page_count' );
```

`test_checkpoint.sh` with the model answer in mu-plugins/:

```
notice bar:
<div class="pagesco-notice">Free shipping on orders over $35. Open until 9 p.m. on Thu
footer line:
<p class="pagesco-pages">Published pages: 4</p>
```

With the starter as handed out (only the two `add_action` lines are there, so (a) does not run yet):

```
notice bar:
<div class="pagesco-notice">Free shipping on orders over $35. Open until 9 p.m. on Thu
footer line:
<p class="pagesco-pages">Published pages: TODO</p>
```

RUBRIC (2 / 1 / 0 PER ROW, 10 POINTS)

CRITERION	2 · MEETS	1 · PARTLY	0 · NOT YET
Hooks, names and priorities	`add_filter( 'pagesco_store_notice', ..., 30 )` and `add_action( 'pagesco_store_notice_end', ..., 20 )` with the callbacks named as strings; the filter RETURNS the text; the order in the bar is late opening, latest news, events link, Ask Maria.	Works, but a priority is off (Ask Maria before See events) or the filter is attached with add_action.	The filter echoes instead of returning (the notice text disappears), or a hook name is misspelled.

CRITERION	2 · MEETS	1 · PARTLY	0 · NOT YET
WordPress functions with arguments	`get_posts()` with an arguments array (`category_name` news, `numberposts` 1); `get_user_by( 'login', 'maria' )` ; `get_author_posts_url()` ; each result checked before use.	Right answer by a detour (all posts fetched and filtered in PHP, the URL typed by hand, the title or name hard-coded).	The newest post or Maria's link is wrong or missing.
Escaping at output	`esc_url()` on the link and `esc_html()` on its text; the filter returns PLAIN text, because the notice escapes it (Lesson 2.3).	One part unescaped, or the filter's text escaped a second time.	User data printed with no escaping at all.
A prepared query	ONE `$wpdb->get_var( $wpdb->prepare( ... ) )` on `{$wpdb->posts}` with `%s` for page and publish; the count printed as a number.	Correct count, but the values are written into the SQL string, or the table name is typed as wp_posts.	No query (a number typed in, or the count of the wrong post type).
Clean, standard code	The test prints the three expected lines and NO message in the page or in debug.log; every function has the pagesco_ prefix and a DocBlock; WordPress spacing and tabs.	One message, or the prefix or DocBlocks missing on some functions.	The site shows a warning or a critical error, or the work is unfinished.

- **Equally good:** `get_posts()` with `'category'` and the term ID 2, or `'posts_per_page' => 1` ; `WP_Query` with `'no_found_rows' => true` ; `wp_count_posts( 'page' )->publish` for (c) is correct WordPress but misses the point of the task (rubric row 4: 1 point, "no prepared query"). Any order of escaping inside the link is fine as long as both parts are escaped once.
- **Common wrong answers:** the filter echoes (the notice loses its text; row 1: 0); priority 10 on (b) ("Ask Maria" before "See events", because pagesco-checkpoint.php loads before pagesco-site.php: mu-plugins load alphabetically); `esc_html()` on the returned notice text (`&` would show twice if a title had one; row 3: 1); `"... WHERE post_type = 'page'"` without `prepare()` (row 4: 1); a hard-coded "Book club: Dune" (row 2: 0, and it breaks with the next News post).

3.1 · THEME ANATOMY & THE TEMPLATE HIERARCHY: SELF-CHECK

Q1

Twenty Twenty-Five is active. Which candidate list does WordPress build for the About page, `/about/` (page ID 12)?

A) `about, page, index`

B) `page-about, page-12, page, singular, index`

C) `single-page-about, single-page, single, singular, index`

D) `page-12, page-about, page, index`

> **Why:** Pages go slug first, then ID, then `page`, `singular` and `index`; Lesson 3.1's template-map log shows exactly `page-about.php, page-12.php, page.php` and then the fall-through to singular and index.
> **A** is wrong: Hierarchy names are type-slug, never the slug alone.
> **C** is wrong: single-{type}-{slug} is the chain of posts and custom post types; pages have their own.
> **D** is wrong: The slug comes before the ID, and singular is missing.

Q2

What does this print on the course site?

```
wp eval 'echo implode( ", ", get_template_hierarchy( "author-devon" ) ), "\n";'
```

`author-devon, author, archive, index`

> **Why:** Devon is user 3, but `get_template_hierarchy()` works from the slug alone, so the ID-based name `author-3` that a real request adds is missing (Lesson 3.1, Exercise 1). Every archive passes through `archive`; every list ends in `index`.

Q3

A client's classic theme has `wp_head()` in header.php and `wp_footer()` in footer.php, but no `wp_body_open()`. What happens to the Pages & Co. store notice?

A) A PHP warning names the missing function

B) WordPress calls wp_body_open itself, as it does for block themes

C) The notice moves to the footer

D) It disappears, with no message anywhere

> **Why:** pagesco-site.php adds its callback to `wp_body_open`, and in a classic theme only the theme fires that action. Nobody fires it, so nothing is printed and nothing is wrong as far as PHP knows (Lesson 3.1, Common Mistakes 3).
>
> **A** is wrong: A hook that never fires is not an error.
>
> **B** is wrong: Only template-canvas.php, for block themes, fires it on the theme's behalf.
>
> **C** is wrong: A callback runs only on the hook it was added to.

Q4

Every function in Twenty Twenty-Five's functions.php is wrapped in `if ( ! function_exists( ... ) )`. Why?

A) So a child theme can define a function of the same name first and replace it

B) So WordPress can load functions.php twice

C) PHP 8.5 requires it for theme functions

D) So the functions run only in the block editor

> **Why:** A child theme's functions.php is loaded BEFORE its parent's. If it defines twentytwentyfive_editor_style(), the parent skips its own: that is how a theme someone else maintains is customized without editing it (Lesson 3.4).
>
> **B** is wrong: Loading it twice would still be a mistake; the guard is about replacing, not repeating.
>
> **C** is wrong: PHP has no such rule; redeclaring a function is simply a fatal error.
>
> **D** is wrong: function_exists() says nothing about where code runs.

Q5

Settings > Reading shows the latest posts on the front page. The theme has front-page.html, home.html and index.html. Which template draws http://localhost:8080/?

A) home.html

B) index.html

C) front-page.html

D) The theme cannot have both front-page.html and home.html

Why: The front page's list is `front-page, home, index`: front-page comes first whatever the front page shows. home is only the list of latest posts, used here when the theme has no front-page.
A is wrong: home would win only without a front-page template.
B is wrong: index is the last resort.
D is wrong: Both are normal: front-page for the front page, home for the blog index wherever it lives.

Q6

Name the filter that changes the list of candidate templates for category archives, before WordPress looks for files.

`category_template_hierarchy`

Why: Each type has its own `{$type}_template_hierarchy` filter (template.php line 62); Lesson 3.1's template-map tool hooks all eighteen. `template_include` is the other hook: it filters the final file path, after the choice.

3.1 · THEME ANATOMY & THE TEMPLATE HIERARCHY: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
wp eval 'echo implode( ", ", get_template_hierarchy( "front-page" ) ), "\n";'
```

```
front-page, home, index
```

Why: front-page first whatever the front page shows, then home (the latest posts), then index. With a static front page the chain after front-page is the page chain instead (Lesson 3.1, Mistake 2).

P2

Predict the output, then run it.

```
wp eval 'echo count( get_block_templates( array(), "wp_template_part" ) ), "\n";'
```

```
7
```

Why: Twenty Twenty-Five has seven parts: header, vertical-header, header-large-title, footer, footer-columns, footer-newsletter and sidebar. `get_block_templates( array(), 'wp_template_part' )` lists parts instead of templates.

3.2 · BUILDING A BLOCK THEME: SELF-CHECK

Q7

On the course site after Lesson 3.2 (pagesco active), what does this print?

```
wp eval 'echo wp_get_global_settings( array( "layout", "wideSize" ) ), "\n";'
```

```
1180px
```

Why: `wp_get_global_settings()` reads the merged theme.json data (core, theme, Site Editor) by path: `settings.layout.wideSize` is pagesco's 1180px. Code that needs a design value asks for it instead of repeating the number.

Q8

In theme.json, `"fontSize": "var:preset|font-size|large"` under styles. What does WordPress print in the page's CSS?

A) `font-size: large`

B) `font-size: 1.75rem`

C) `font-size: var(--wp--preset--font-size--large)`

D) `font-size: var(--large)`

> **Why:** The shorthand points at the preset, and the preset is a CSS custom property, so a style variation or the Styles panel can change the size in one place. Lesson 3.2's body rule shows the same for colors: `background-color: var(--wp--preset--color--paper)`.
> **A** is wrong: That would be the CSS keyword large, not your preset.
> **B** is wrong: The value is never copied in: that is the point of presets.
> **D** is wrong: Custom properties from theme.json always carry the --wp--preset--TYPE-- prefix (here --wp--preset--font-size--).

Q9

Which of these blocks is written SELF-CLOSING in a template file, with no HTML of its own?

A) Group

B) Heading

C) Paragraph

D) Post Title

> **Why:** Post Title is dynamic: PHP renders it on every request from the post being viewed, so the file holds only `<!-- wp:post-title {"level":1} /-->`. Group, Heading and Paragraph are static: their exact saved HTML sits between the comments.
> **A** is wrong: Group is static: its `<div>` (or `<main>`) is in the file.
> **B** is wrong: Heading is static, with its `<h1>` in the file.
> **C** is wrong: Paragraph is static, with its `<p>` in the file.

Q10

A live site (WP_DEBUG off) gets a theme.json with a trailing comma. What do visitors see?

A) The site in core's default design: the theme's colors, fonts and sizes are gone, with no message

B) A critical error page

C) The notice wp_json_file_decode(): Syntax error at the top of every page

D) Nothing changes: WordPress skips the bad line

> **Why:** WordPress cannot parse the file, so it uses no theme.json at all (zero oxblood variables in Lesson 3.2's Mistake 1). The Notice exists, but only with WP_DEBUG on, in the page and in debug.log. The notice names the file, not the line: `$schema` in VS Code finds it.
> **B** is wrong: It is a Notice, not a fatal error: the page still renders.
> **C** is wrong: Only with WP_DEBUG and WP_DEBUG_DISPLAY on, as on the course site.
> **D** is wrong: JSON is parsed as a whole; one syntax error loses the whole file.

Q11

A template contains `<!-- wp:template-part {"slug":"footer","tagName":"footer"} /-->`. Which file of the theme does it include, and what does the tagName do?

`parts/footer.html`; it wraps the part in a `<footer>` element (the contentinfo landmark)

> **Why:** The slug is the file name in parts/ without .html (a typo such as `heder` renders nothing for visitors). tagName gives screen-reader users the header, main and footer landmarks; Lesson 3.2 checks them with curl.

Q12

What does `"defaultPalette": false` in settings.color do?

A) Deletes the theme's own palette

B) Hides WordPress's own palette from the editors, so they choose from the theme's colors

C) Turns off all color controls

D) Removes --wp--preset--color--black and --white from the page

> **Why:** The editors then offer only Paper, Ink, Oxblood, Sage and Slate. Core still prints black and white as custom properties (Lesson 3.2's preset grep shows both). Turning off the free color picker is a different setting, custom: false (the lesson's quiz Q4).
> **A** is wrong: Your palette is what remains.
> **C** is wrong: Controls stay; only the default swatches go.
> **D** is wrong: Both are still printed: the lesson's output starts with them.

3.2 · BUILDING A BLOCK THEME: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8080/nothing-here/
```

```
404
```

Why: pagesco's 404.html draws the page, and WordPress still answers with status 404 so search engines do not index it. Lesson 3.2's verification loop checks exactly this.

P4

Predict the output, then run it.

```
curl -s http://localhost:8080/ | grep -o '\.has-oxblood-color{[^}]*}'
```

```
.has-oxblood-color{color: var(--wp--preset--color--oxblood) !important;}
```

Why: Every palette color produces a class rule using its custom property, with `!important` so a color chosen for one block beats the theme's other rules.

LAB REFERENCE SOLUTIONS

Every output below is real: the lab files run exactly as `labs/check_labs.py` runs them, on the course site at the state each file names (WordPress 7.1.3, PHP 8.5, MySQL 9.7, WP-CLI 2.12): the 3.1 labs on the site of Lesson 2.3, the 3.2 labs on the site of Lesson 3.1. Paths show `/var/www/html/...`, which is the learner's `wordpress/` folder.

- LAB 3.1-A (GUIDED): LABS/MODULE-04/L3_1-THEME-ANATOMY/WHICH_TEMPLATE_STARTER.PHP → WHICH_TEMPLATE_SOLUTION.PHP

As shipped:

```
/tag/staff-picks/   tag-staff-picks > tag > archive > index
                    => TODO
/author/devon/      author-devon > author > archive > index
                    => TODO
/hello-world/       single-post-hello-world > single-post > single > singular > index
                    => TODO
/?s=dune            search > index
                    => TODO
```

The solution prints:

```
/tag/staff-picks/   tag-staff-picks > tag > archive > index
                    => archive
/author/devon/      author-devon > author > archive > index
                    => archive
/hello-world/       single-post-hello-world > single-post > single > singular > index
                    => single
/?s=dune            search > index
                    => search
/                   front-page > home > index
                    => home
/contact/           page-contact > page > singular > index
                    => page
```

Accept any loop that stops at the first match. Without `break` the last match wins and every row says `index`: the most instructive bug of the lab. On a site where pagesco is active, `/` answers `index` (pagesco has no home template), which is the template map's "index doubles as home".

- LAB 3.1-B: LABS/MODULE-04/L3_1-THEME-ANATOMY/THEME_INSPECTOR_STARTER.SH → THEME_INSPECTOR_SOLUTION.SH

The solution prints:

```
== twentytwentyfive 1.5
  Requires at least: 6.7
  kind: block
  templates: 8  parts: 7  patterns: 98
  functions.php: yes
== twentytwentyfour 1.6
  Requires at least: 6.4
  kind: block
  templates: 11  parts: 4  patterns: 57
  functions.php: yes
== twentytwentythree 1.7
  Requires at least: 6.1
  kind: block
  templates: 9  parts: 4  patterns: 7
  functions.php: no
```

All three default themes are block themes; Twenty Twenty-Three has no functions.php at all, which shows that PHP is optional in a block theme. The pattern counts change whenever a default theme updates. `grep -m 1` matters: some style.css files repeat a field in their description.

- HANDOUT: LABS/MODULE-04/L3_1-THEME-ANATOMY/DEBUG_THEME_REPORT.PHP (DELIBERATELY BROKEN)

As shipped, PHP prints:

```
Theme: Twenty Twenty-Five
Needs WordPress:
Block theme: yes
Warning: Undefined array key "colors" in /var/www/html/debug_theme_report.php on line 2
Warning: Trying to access array offset on null in /var/www/html/debug_theme_report.php
Warning: foreach() argument must be of type array|object, null given in /var/www/html/d
Colors in the palette: Warning: Undefined variable $pagesco_count in /var/www/html/debu
[new lines in wp-content/debug.log]
[26-Oct-2026 12:33:39 UTC] PHP Warning:  Undefined array key "colors" in /var/www/html/
[26-Oct-2026 12:33:39 UTC] PHP Warning:  Trying to access array offset on null in /var/
[26-Oct-2026 12:33:39 UTC] PHP Warning:  foreach() argument must be of type array|objec
[26-Oct-2026 12:33:39 UTC] PHP Warning:  Undefined variable $pagesco_count in /var/www/
```

1. **Silent, line 17:** `get( 'Requires at least' )` asks for a header by its style.css label; `WP_Theme::get()` wants the key, `RequiresWP` (others: `RequiresPHP`, `TextDomain`, `Version`). It returns false, so the line prints nothing after the colon.
2. **Warnings, line 21:** the key is `color`, not `colors` (Lesson 3.1's outline printed `color, spacing, typography`). One wrong key gives three warnings: the missing key, an offset on null, and a foreach over null.

3 **Warning, line 25:** `$pagesco_count++` on a variable that was never set. Hidden while bug 2 stops the loop from running; start the counter at 0 before the loop.

Fixed, it prints:

```
Theme: Twenty Twenty-Five
Needs WordPress: 6.7
Block theme: yes
  base      #FFFFFF
  contrast  #111111
  accent-1  #FFEE58
  accent-2  #F6CFF4
  accent-3  #503AA8
  accent-4  #686868
  accent-5  #FBFAF3
  accent-6  color-mix(in srgb, currentColor 20%, transparent)
Colors in the palette: 8
```

• LAB 3.2-A (GUIDED): LABS/MODULE-04/L3_2-BLOCK-THEME/BOOK_FAIR_STARTER/ → BOOK_FAIR_SOLUTION/ WITH TEST_BOOK_FAIR.SH

The test with the starter installed as `pagesco-fair`:

```
Success: Switched to 'Pages &amp;amp; Co. Book Fair' theme.
active theme: pagesco-fair
palette:
--wp--preset--color--night: #1b1f3b
--wp--preset--color--cream: #fff8e7
lantern: (missing)
page colors: (none: theme.json has no styles.color)
posts listed: 0
header parts: 1
footer parts: 0
footer text: (none)
```

With the solution:

```
Success: Switched to 'Pages &amp;amp; Co. Book Fair' theme.
active theme: pagesco-fair
palette:
--wp--preset--color--night: #1b1f3b
--wp--preset--color--cream: #fff8e7
--wp--preset--color--lantern: #f2b134
page colors: body{background-color: var(--wp--preset--color--cream);color: var(--wp--pr
posts listed: 4
header parts: 1
footer parts: 1
footer text: Book fair weekend · 12 Market Street
```

What each line shows: the palette lines come from settings (TODO 1); the page colors from styles, pointing at presets (TODO 2: the test only matches `var(--wp--preset--...)`, never a hex value); `posts listed` from the Query Loop with `inherit` (TODO 3); the footer from the part (TODO 4). Before TODO 1 the header's `has-lantern-background-color` class had no rule, so the band was transparent: a class is only as good as its preset. Links are night because lantern on cream is far below 4.5:1.

- LAB 3.2-B: LABS/MODULE-04/L3_2-BLOCK-THEME/THEME_JSON_READER_STARTER.PHP → THEME_JSON_READER_SOLUTION.PHP

The solution prints:

```
twentytwentyfive: theme.json version 3
content width: 645px  wide width: 1340px
palette: base, contrast, accent-1, accent-2, accent-3, accent-4, accent-5, accent-6
font sizes: small, medium, large, x-large, xx-large
page background: var:preset|color|base  text: var:preset|color|contrast
text as CSS: var(--wp--preset--color--contrast)
```

The file holds the shorthand (`var:preset|color|contrast`); the page holds the custom property; `wp_get_global_settings()` / `wp_get_global_styles()` return the MERGED data (core, theme, the owner's Site Editor changes), which is what code should read in practice. With `$pagesco_slug = 'pagesco'` the lines are pagesco's (680px and 1180px, paper to slate, `var(--wp--preset--color--ink)`).

- HANDOUT: LABS/MODULE-04/L3_2-BLOCK-THEME/DEBUG_CAFE_THEME/ WITH TEST_DEBUG_CAFE_THEME.SH (DELIBERATELY BROKEN)

As shipped, installed as `pagesco-cafe`, the test prints:

```
Success: Switched to 'Pages &amp;amp; Co. Cafe' theme.
messages in the page (how many of each):
      1 Deprecated: str_replace(): Passing null to parameter #3 ($subject) of type arra
      2 Notice: wp_json_file_decode(): Error when decoding a JSON file at path /var/www
palette:
header parts: 0
titles:
[new lines in wp-content/debug.log]
[26-Oct-2026 12:34:21 UTC] PHP Notice:  wp_json_file_decode(): Error when decoding a JS
[26-Oct-2026 12:34:21 UTC] PHP Deprecated:  str_replace(): Passing null to parameter #3
[26-Oct-2026 12:34:21 UTC] PHP Notice:  wp_json_file_decode(): Error when decoding a JS
```

1. **Notice (theme.json):** a comma after the last palette entry (`"#fdf6ec" },`). WordPress ignores the whole file: no espresso or foam variables, no page colors.
2. **Deprecated (functions.php):** `pagesco_cafe_title_cup()` assigns `$title` and returns nothing, so the filter hands null to the next callback and core's `capital_P_dangit()` complains in formatting.php: the message blames core (Lesson 2.3, Mistake 1). Every title is empty. Fix: `return '☕ ' . $title;`.
3. **Silent (templates/index.html):** `"slug":"heder"`: no part with that slug, so no header for visitors and no message (`header parts: 0`). Logged in, the page says "Template part has been deleted or is unavailable: heder".

Fixed, the same test prints:

```
messages in the page (how many of each):
palette: --wp--preset--color--espresso: #3b2a20 --wp--preset--color--foam: #fdf6ec
header parts: 1
titles:
☕ Hello world!
☕ Book club: Dune
☕ Staff picks for the season
☕ Welcome to the new Pages &#038; Co. website
```

Clean up: switch back to your theme and `wp theme delete pagesco-cafe`. The lesson in it: a block theme fails quietly; the messages that do exist appear only with WP_DEBUG on.

THE LESSON PAGES' OWN QUICK QUIZZES

◆ LESSON 3.1 · THEME ANATOMY & THE TEMPLATE HIERARCHY

1. **A** singular.html: The page chain is page-about, page-12, page, singular, index. The theme has no page template, so the first match is singular.html.
2. **C** A hybrid: PHP templates plus a theme.json: No templates/index.html, so it is not a block theme, but it has a theme.json: a classic theme with block features, a hybrid.

3. **B** page.html: There is no front-page.html, and with a static front page WordPress follows the page chain next, so page.html draws it.

4. **C** Her save stored a copy as a wp_template post, and that copy wins over the file: Saving in the Site Editor stores a wp_template post with the source custom, and that copy wins over the theme file until it is reset.

5. **B** add_filter('single_template_hierarchy', ...) returning a changed array: single_template_hierarchy is the filter WordPress applies to the candidate list for single posts; a callback receives the array and returns the list to search.

LESSON 3.2 · BUILDING A BLOCK THEME

1. **B** --wp--preset--color--oxblood: Every palette color becomes --wp--preset--color--SLUG in the global styles, which is what var:preset|color|oxblood stands for.

2. **A** style.css and templates/index.html: style.css carries the theme header and templates/index.html is the template every request can fall back to. theme.json and functions.php are optional.

3. **C** The page without a footer, and no message: The Template Part block finds no part with that slug and renders nothing. Only logged-in users see a hint, so check slugs while logged in.

4. **A** In settings.color: the palette, plus custom set to false: What the editors offer is a setting: settings.color.palette lists the colors and settings.color.custom set to false removes the free picker.

5. **B** Decide which version is right, then reset or delete the saved copy (exporting it first if it should be kept): The post is a Site Editor copy that wins over the file. Reset it in the Site Editor or delete it with wp post delete ID --force, after exporting it if the owner's change should be kept.

EXIT TICKET

1. **archive.html.** The candidates are author-maria, author-2, author, archive, index; the theme has none of the first three, so archive answers. (index would answer only without archive.)

2. **Palette:** `settings.color.palette` (what editors may choose from). **Background:** `styles.color.background`, written `var:preset|color|paper` so it follows the preset.

3. **Two of:** a template-part slug that matches no file in parts/ (silent for visitors; logged in you see "Template part has been deleted or is unavailable"); a Site Editor copy (`wp_template` or `wp_template_part` post) that wins over the file. A JSON syntax error removes the styles, not the header.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 C (single-page-...).** Applies the post chain to pages. Pages have their own chain; show the demo's log for /about/.

- **Q3 A (a warning).** Expects PHP to complain about a hook nobody fires. A hook that never fires is silent by design.
- **Q5 A (home.html).** The most common hierarchy mistake of all: home is the latest-posts list. front-page comes first.
- **Q8 B (1.75rem).** Thinks presets are copied in. They are referenced: that is what makes style variations possible (Lesson 3.3).
- **Q10 B or C.** Confuses the course site (WP_DEBUG on) with a live site. On a live site the design just goes plain.
- **Q12 D.** Has not looked at the preset grep: black and white are always printed.

INSTRUCTOR KIT · ANSWER KEY

Session 5: Patterns, Style Variations and Child Themes

Answers and reasoning for every Session 5 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs, the lesson pages' own quizzes and the exit ticket.

Session 5 | 12 self-check items | Confidential

START HERE · WARM-UP

Session 4 retrieval, not graded. W5's output on a site ready for today:

```
wp eval 'var_dump( wp_is_block_theme(), wp_theme_has_theme_json() );'
wp post list --post_type=wp_template,wp_template_part,wp_global_styles --format=count
```

```
bool(true)
bool(true)
0
```

1. **W1** In theme.json, what is the difference between settings and styles? **Answer:** settings say what editors may choose (the palette, font sizes, layout widths); styles say what is applied (the page's colors, fonts, element and block styles). Presets in settings become custom properties and classes (Lesson 3.2).
2. **W2** The front page shows the latest posts. In a block theme, which templates does WordPress look for, in order? **Answer:** front-page, then home, then index (Lesson 3.1). pagesco has only index.html so far; Lesson 3.3 adds front-page.html today.
3. **W3** The manager changed the Single Posts template in the Site Editor. Where is that change, and which version does a visitor see? **Answer:** In the database, a wp_template post whose source is custom; it wins over templates/single.html until someone resets it (Lesson 3.2, Common Mistakes 3).
4. **W4** A template part is included as {"slug":"heder"}. What do visitors see? And a logged-in administrator? **Answer:** Visitors: nothing, the header is just missing. Logged in: "Template part has been deleted or is unavailable: heder" (Lesson 3.2).
5. **W5** Run the two commands on the slide. What should a site ready for today print? **Answer:** bool(true) twice (pagesco is a block theme with a theme.json), then 0: no Site Editor copies. A learner with copies resets them first (wp post delete ID --force), or today's outputs will not match.

3.3 · PATTERNS, STYLE VARIATIONS & THE SITE EDITOR: SELF-CHECK

Q1

The live site has no `WP_DEVELOPMENT_MODE`. You deploy pagesco with a new `patterns/gift-cards.php`, and the inserter does not show it for half an hour. What makes WordPress read the patterns folder again at once, as part of every release?

A) Raise the theme's Version in style.css

B) Call `register_block_pattern()` for the file in functions.php

C) Run `wp cache flush`

D) Rename `patterns/` and back

> **Why:** The list of pattern files is cached in a transient (`wp_theme_files_patterns-…`) for 30 minutes, and rebuilt early when the theme's Version changes or the theme is switched (Lesson 3.3). Bumping Version on every deploy is the habit Lesson 8.4 builds in; the transient holds the version it was built for.
> **B** is wrong: Files in patterns/ register themselves; registering the same pattern by hand is a second, different mechanism.
> **C** is wrong: Without a persistent object cache, transients live in the options table, which a cache flush does not touch.
> **D** is wrong: The cache is keyed on the theme, not on the folder's name; renaming does nothing reliable.

Q2

On the site at the end of Lesson 3.3, what does this print, and what are the two numbers?

```
wp eval 'echo count( WP_Theme_JSON_Resolver::get_style_variations() ), " ", count(
```

`1 2`: one site variation (Evening) and two section styles (Sage band, Ink band)

> **Why:** The resolver sorts every JSON file under styles/ by one key: a file with `blockTypes` is a section style (the second call), a file without it a style variation for the whole site (the first). The folder styles/sections/ is only for order.

Q3

Which of these lines should never be written into a template file in a theme?

A) `<!-- wp:pattern {"slug":"pagesco/hero"} /-->`

B) `<!-- wp:template-part {"slug":"footer"} /-->`

C) `<!-- wp:block {"ref":20} /-->`

D) `<!-- wp:query {"query":{"inherit":true}} -->`

> **Why:** A synced pattern is referenced by its post ID, and IDs differ on every site: on another copy of the site, post 20 is something else or nothing. Themes refer to patterns by slug and to parts by slug; synced patterns are the site owner's content (Lesson 3.3, "Synced patterns are content, not theme").
> **A** is wrong: A theme pattern by slug is exactly how templates use patterns.
> **B** is wrong: Template parts are included by slug from parts/.
> **D** is wrong: The Query Loop that inherits the main query is how lists are drawn in block templates.

Q4

After the shop manager saved the front page in the Site Editor, the `wp_template` copy contains `"patternName":"pagesco/hero"` and `src="http://localhost:8080/..."`. What does that tell you?

A) The copy still refers to hero.php, so changes to the file will show up

B) The Site Editor could not read the pattern file and left a placeholder

C) Nothing: patternName is only used for the inserter's preview

D) The pattern was expanded before saving: its PHP already ran, so the URL is hard-coded and the strings are frozen in English

> **Why:** When someone edits a template that uses wp:pattern, the editor replaces each reference with the pattern's blocks and records where they came from in metadata.patternName. Copying that template over the file would lose translation, theme URLs and escaping; move the actual change into the pattern file and reset the copy.
> **A** is wrong: No wp:pattern reference is left in the copy; it is ordinary blocks now.
> **B** is wrong: The content is complete; the metadata is a note, not a placeholder.
> **C** is wrong: It is a record of where the blocks came from; the important part is that the reference is gone.

Q5

You move `section-ink.json` from `styles/sections/` up into `styles/`. What changes?

A) It becomes a style variation for the whole site

B) Nothing: it is still a section style, because it has `blockTypes`

C) WordPress ignores it: section styles must be in styles/sections/

D) It replaces `evening.json`, because both are in styles/ now

> **Why:** WordPress reads every JSON file under styles/, at any depth, and tells the two kinds apart by the `blockTypes` key alone. The subfolder exists only for people.
> **A** is wrong: A file becomes a site variation only when it has no blockTypes.
> **C** is wrong: There is no required folder name.
> **D** is wrong: Files replace each other by name only in a child theme (Lesson 3.4), and the names differ.

Q6

The page source announces Vollkorn in a @font-face rule, but every browser shows the fallback serif, and debug.log is empty. Where do you look first, and for what?

The browser's developer tools, Network tab filtered by Font: the font request answers 404, so the src path after file:./ (or the folder name) is wrong

> **Why:** WordPress builds the @font-face URL from theme.json without checking that the file exists (Lesson 3.3, Common Mistakes 2). From the WP-CLI container, `curl -s -o /dev/null -w '%{http_code}' <the font URL>` shows the same 404.

3.3 · PATTERNS, STYLE VARIATIONS & THE SITE EDITOR: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
wp eval 'var_dump( WP_Block_Patterns_Registry::get_instance()->get_registered( "pag
```

```
array(1) {
  [0]=>
  string(11) "wp_template"
}
```

Why: The header line `Post Types: wp_template` becomes an array with one entry: WordPress offers the hero only when editing templates in the Site Editor, never in a page (where its `<h1>` would compete with the page title).

P2

Predict the output, then run it.

```
curl -s http://localhost:8080/about/ | grep -c 'wp:block'
```

```
0
```

Why: Nothing of the reference reaches the browser: the page stores `<!-- wp:block {"ref":20} /-->`, and WordPress renders the synced pattern's blocks in its place. `grep -c` counts 0 and exits with status 1. The same holds for `wp:pattern` in templates.

3.4 · CHILD THEMES & CUSTOMIZING EXISTING THEMES: SELF-CHECK

Q7

A client uploads only your child theme (pagesco-tt5-child.zip) to a site where Twenty Twenty-Five has been deleted, and clicks Activate. What happens?

A) WordPress uses its default theme as the parent instead

B) It refuses: the parent theme is missing, and the theme is listed under Broken Themes

C) The child activates, unstyled, until the parent is installed

D) WordPress downloads Twenty Twenty-Five automatically

> **Why:** A child is only meaningful on top of its parent. WP-CLI says `Error: The parent theme is missing. Please install the "twentytwentyfive" parent theme.`; the Themes screen lists the child under Broken Themes (Lesson 3.4, Common Mistakes 1, where a typo in Template: caused the same). Install the parent first.
> **A** is wrong: Template: names one folder; WordPress never guesses another parent.
> **C** is wrong: Without a parent there are no templates to draw a page with; activation is refused.
> **D** is wrong: Nothing is downloaded on activation; wp theme install or the Themes screen does that.

Q8

On the site at the end of Lesson 3.4: what do the two middle commands print?

```
wp theme activate pagesco-tt5-child
wp option get stylesheet
wp option get template
wp theme activate pagesco
```

`pagesco-tt5-child`, then `twentytwentyfive`

> **Why:** With a child theme active, the stylesheet option holds the child (the theme whose style.css header names the site's theme) and the template option holds the parent (where the templates and code come from). Without a child theme both hold the same folder name. The first and last lines switch the child on and back to pagesco.

Q9

A colleague's child theme.json lists only the two colors they wanted to change, base and contrast. What happens to Twenty Twenty-Five's accent colors?

A) They disappear: the child's palette replaces the parent's whole palette, and styles that use accent variables silently lose their colors

B) They stay: theme.json is merged key by key

C) WordPress prints a notice that the palette is incomplete

D) The child's two colors are added to the end of the parent's list

Why: Single values merge key by key, but preset lists (palette, font sizes, spacing sizes) are replaced as a whole. The home page still uses `var(--wp--preset--color--accent-3)` and accent-4, now undefined, and CSS ignores an undefined variable without a message (Lesson 3.4, Common Mistakes 2). Repeat every slug, change only values.

B is wrong: That is true for single values, not for preset lists.

C is wrong: There is no message anywhere; that is what makes it dangerous.

D is wrong: Lists are not appended; the child's list is the list.

Q10

Why can a child of Twenty Twenty-Five replace `twentytwentyfive_post_format_setup()` by defining a function with that name in its own functions.php?

A) Child functions always override parent functions of the same name

B) WordPress renames the parent's function when a child theme is active

C) The parent's function is registered on a hook, and hooks prefer the child

D) The child's functions.php loads first, and the parent defines that function only `if ( ! function_exists( ... ) )`

Why: Load order (Lesson 3.4: child, then parent) plus a pluggable function. For a parent function WITHOUT the function_exists() wrapper, defining the same name in the child is a fatal `Cannot redeclare` error; remove or replace its hook instead, on `after_setup_theme`.

A is wrong: PHP does not allow two functions with one name; only the wrapper makes it possible.

B is wrong: WordPress renames nothing.

C is wrong: Hooks have priorities, not a preference for the child; and the replacement here is a function definition.

Q11

Twenty Twenty-Four enqueues no stylesheet of its own. In a child of it you write wp_enqueue_style('my-child-style', get_stylesheet_uri(), $deps, $version). What should $deps be?

A) `array( 'twentytwentyfour-style' )`

B) `array( 'wp-block-library' )`

C) `array()`

D) `get_template_directory_uri() . '/style.css'`

> **Why:** There is no parent handle to wait for, so the child depends on nothing (the session's TT4 child lab does exactly this). Read the parent's functions.php before copying a tutorial's handle: on WordPress 7.1 a dependency that is not registered stops the stylesheet from printing at all, with a notice that it was `enqueued with dependencies that are not registered` (since 6.9.1).
> **A** is wrong: That handle does not exist: WordPress drops the child's stylesheet and notes it in debug.log.
> **B** is wrong: The child's CSS has nothing to do with the block library's; the dependency only delays it for no reason.
> **D** is wrong: That is a URL, not a list of handles, and it points at the parent's style.css header file.

Q12

The shop manager wants to try a darker link color for the holidays and switch it back in January, without calling you. Site Editor, child theme, or plugin? Why?

The Site Editor's Styles: a visual choice the owner wants to change again; it is saved in the database (the user global styles) and needs no developer

> **Why:** The decision table of Lesson 3.4: who should change it later, and must it survive a theme switch? Lasting design work on a theme you did not write goes in a child theme, kept in Git; features go in plugins; owner-changeable looks belong to the Site Editor.

3.4 · CHILD THEMES & CUSTOMIZING EXISTING THEMES: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
wp theme activate pagesco-tt5-child
wp eval 'echo get_theme_file_uri( "assets/images/book-image.webp" ), "\n";'
wp theme activate pagesco
```

```
Success: Switched to 'Pages &amp;amp; Co. TT5 Child' theme.
http://localhost:8080/wp-content/themes/twentytwentyfive/assets/images/book-imag
Success: Switched to 'Pages &amp;amp; Co.' theme.
```

Why: The child has no assets/ folder, so `get_theme_file_uri()` falls back to the parent's copy, and Twenty Twenty-Five happens to ship a book-image.webp of its own. That lookup is why the hero uses `get_theme_file_uri()`: a child of pagesco could replace the image by adding its own file.

P4

Predict the output, then run it.

```
wp theme activate pagesco-tt5-child
wp eval 'var_dump( function_exists( "twentytwentyfive_post_format_setup" ), get_the
wp theme activate pagesco
```

```
Success: Switched to 'Pages &amp;amp; Co. TT5 Child' theme.
bool(true)
array(2) {
  [0]=>
  string(5) "aside"
  [1]=>
  string(5) "quote"
}
Success: Switched to 'Pages &amp;amp; Co.' theme.
```

Why: `function_exists()` is true, and the post formats are the child's two: the child's functions.php loaded first and defined `twentytwentyfive_post_format_setup()`, so the parent skipped its own definition (a pluggable function).

LAB REFERENCE SOLUTIONS

Every output below is real: the lab files run exactly as `labs/check_labs.py` runs them, on a course site at the state each file declares (WordPress 7.1.3, PHP 8.5, MySQL 9.7, WP-CLI 2.12). Folder labs are installed as a theme and activated first (the first line of their output); paths show `/var/www/html/...`, the learner's `wordpress/` folder.

- LAB 3.3-A (GUIDED): LABS/MODULE-05/L3_3-PATTERNS-STYLE-VARIATIONS/HOURS_BAND_STARTER/ → HOURS_BAND_SOLUTION/ WITH TEST_HOURS_BAND.PHP

The test with the starter's pattern:

```
Success: Switched to 'Pages &amp;amp; Co. Lab: Hours Band' theme.
Title:       Visit the shop: hours band
Categories:  (none)
Keywords:    (none)
Description: (none)
Heading:     Visit the shop
Text:        12 Market Street, open late on Thursdays.
Button:      Plan your visit -> http://localhost:8080/contact/
```

With the solution's:

```
Success: Switched to 'Pages &amp;amp; Co. Lab: Hours Band' theme.
Title:       Visit the shop: hours band
Categories:  contact
Keywords:    hours, address, visit
Description: A sage band with the shop's address and a button to the Contact page.
Heading:     VISIT THE SHOP
Text:        12 MARKET STREET, OPEN LATE ON THURSDAYS.
Button:      PLAN YOUR VISIT -> https://pagesandco.example/contact/
```

Reading it: the header fields arrive in the registry as arrays (Categories, Keywords) or strings; the content is the pattern file's output, produced the first time someone asks for it, which is why the test's two filters are in place in time. Lower case after the "translation" means a string without `esc_html_e()` in the child's text domain (`pagesco` would not change case: another theme's strings); localhost means a hard-coded URL. Common sticking points: `esc_html_e()` inside `echo` (it already prints), a missing `esc_url()` (the test cannot see it; ask for it anyway), `Categories: Contact` with a capital (category names are slugs).

- LAB 3.3-B: LABS/MODULE-05/L3_3-PATTERNS-STYLE-VARIATIONS/VARIATION_CONTRAST_STARTER.PHP → VARIATION_CONTRAST_SOLUTION.PHP

As shipped:

```
Default   ink on paper      TODO
Default   oxblood on paper  TODO
Default   slate on paper    TODO
```

The solution prints:

```
Default   ink on paper      13.9:1  AA
Default   oxblood on paper   8.8:1  AA
Default   slate on paper     5.8:1  AA
Default   ink on sage       12.0:1  AA
Default   sage on paper      1.2:1  FAIL (needs 4.5)
Evening   ink on paper      13.5:1  AA
Evening   oxblood on paper   8.3:1  AA
Evening   slate on paper     8.4:1  AA
Evening   ink on sage        9.7:1  AA
Evening   sage on paper      1.4:1  FAIL (needs 4.5)
```

The numbers confirm the lesson (Evening: ink on paper 13.5:1, oxblood links 8.3:1) and add one it does not state: sage on paper is 1.2:1, which is why sage is a background color only (Lesson 3.2's handoff calls it that). The two traps: the variation's palette is in `['settings']['color']['palette']['theme']` (keyed by origin, the resolver's internal shape); and a variation is a LAYER, so its colors are merged over the theme's (`array_merge()`), exactly as WordPress merges them.

- HANDOUT: LABS/MODULE-05/L3_3-PATTERNS-STYLE-VARIATIONS/DEBUG_PATTERN_FILES.SH (DELIBERATELY BROKEN)

As shipped:

```
== what WordPress registered ==
Notice: Function get_block_patterns was called <strong>incorrectly</strong>. Could not
pattern: pagesco/call-to-action
pattern: pagesco/featured-books
pattern: pagesco/hero
Notice: wp_json_file_decode(): Error when decoding a JSON file at path /var/www/html/wp
site variation: Evening
site variation: sepia
section style: Ink band
section style: Sage band
```

1. **doing_it_wrong notice, the pattern header (script line 22):** no `Slug:` line, so WordPress skips the file ("Slug" field missing). Add `* Slug: pagesco/gift-cards`.
2. **Notice, script line 45 (sepia.json):** a trailing comma after the last palette entry: `wp_json_file_decode()` reports a syntax error, and the file is still listed, under its FILE NAME (`sepia`) with no data. Remove the comma.
3. **Silent, script line 40 (sepia.json):** `"blockTypes"` copied from a section style. With the comma fixed, Sepia is registered as a section style for Groups, not as a look for the whole site. Delete the line. (Bug 2 hides bug 3: say so when a learner "fixes" the comma and asks why Sepia is still not a site variation.)

Fixed, the script prints:

```
== what WordPress registered ==
pattern: pagesco/call-to-action
pattern: pagesco/featured-books
pattern: pagesco/gift-cards
pattern: pagesco/hero
site variation: Evening
site variation: Sepia
section style: Ink band
section style: Sage band
```

- LAB 3.4-A (GUIDED): LABS/MODULE-05/L3_4-CHILD-THEMES/TT4_CHILD_STARTER/ → TT4_CHILD_SOLUTION/ WITH TEST_TT4_CHILD.SH

The test with the starter:

```
Success: Switched to 'Pages &amp;amp; Co. Lab: TT4 Child' theme.
template:    twentytwentyfour
stylesheet:  pagesco-lab-tt4-child
accent-3:    #d8613c
palette:     10 theme colors
child CSS:   0
footer:      (not the shop's: still the parent's footer pattern)
styles:      core/details arrow-icon-details, core/post-terms pill, core/list checkmarl
```

With the solution:

```
Success: Switched to 'Pages &amp;amp; Co. Lab: TT4 Child' theme.
template:    twentytwentyfour
stylesheet:  pagesco-lab-tt4-child
accent-3:    #7a2e2e
palette:     10 theme colors
child CSS:   1
footer:      The reading corner at Pages &amp;amp; Co. · 12 Market Street
styles:      core/list checkmark-list
```

Reading it, line by line: template/stylesheet (the starter is already a working child); accent-3 is the child's only when the child's palette lists it, and `10 theme colors` stays 10 only when the child repeats every slug (a palette with one entry makes it 1); the child's CSS is loaded once enqueued, with `array()` as dependencies because Twenty Twenty-Four registers no stylesheet handle of its own (a copied `twentytwentyfour-style` makes WordPress drop the child's stylesheet and log that the dependency is not registered); the footer is the child's once the pattern has the parent's slug; and the pluggable `twentytwentyfour_block_styles()` replaced the parent's five styles with one.

- LAB 3.4-B: LABS/MODULE-05/L3_4-CHILD-THEMES/MERGE_LAYERS_STARTER.PHP → MERGE_LAYERS_SOLUTION.PHP

As shipped:

```
parent:  8 colors, accent-3 = #503AA8
child:   TODO
merged:  TODO
link:    TODO
body:    TODO
h1 size: TODO
partial: TODO
```

The solution prints:

```
parent:  8 colors, accent-3 = #503AA8
child:   8 colors, accent-3 = #7a2e2e
merged:  8 colors, accent-3 = #7a2e2e
link:    var(--wp--preset--color--accent-3)  (from the child)
body:    var(--wp--preset--font-family--manrope)  (from the parent)
h1 size: var(--wp--preset--font-size--xx-large)  (from the parent)
partial: 2 colors (base, contrast)
```

The parent's accent-3 (#503AA8) is replaced by the child's oxblood; the link color comes from the child, the body font and the h1 size from the parent: the merge rule of Lesson 3.4 in four lines. Note that WP_Theme_JSON turned the files' `var:preset|color|accent-3` shorthand into the CSS custom property it prints. The partial palette keeps two colors: lists are replaced, never merged.

HANDOUT: LABS/MODULE-05/L3_4-CHILD-THEMES/DEBUG_CHILD_THEME.SH (DELIBERATELY BROKEN)

As shipped:

```
Success: Switched to 'Pages &amp;amp; Co. Lab: Broken Child' theme.
link:      currentColor
child CSS: 1
checkmark: Notice: Function wp_enqueue_style was called <strong>incorrectly</strong>. S
yes
== what the page request added to debug.log (time stamps and HTML removed) ==
      2 PHP Notice:  Function wp_enqueue_style was called incorrectly. Scripts and styl
      1 PHP Notice:  wp_json_file_decode(): Error when decoding a JSON file at path /va
Notice: Function wp_enqueue_style was called <strong>incorrectly</strong>. Scripts and
Success: Switched to 'Pages &amp;amp; Co.' theme.
[new lines in wp-content/debug.log]
[26-Oct-2026 13:27:20 UTC] PHP Notice:  Function wp_enqueue_style was called <strong>ir
[26-Oct-2026 13:27:20 UTC] PHP Notice:  wp_json_file_decode(): Error when decoding a JS
[26-Oct-2026 13:27:20 UTC] PHP Notice:  Function wp_enqueue_style was called <strong>ir
[26-Oct-2026 13:27:21 UTC] PHP Notice:  Function wp_enqueue_style was called <strong>ir
```

1. **Notice, script line 39 (theme.json):** a trailing comma, so `wp_json_file_decode()` fails and the child's whole theme.json is ignored: the link color is the parent's `currentColor`. Remove the comma.

2. **doing_it_wrong notice, script line 54 (functions.php):** `wp_enqueue_style()` at the top level, before `wp_enqueue_scripts` ("Scripts and styles should not be registered or enqueued until..."). The stylesheet still loads, which is why people ignore it; wrap it in a function on `wp_enqueue_scripts`. (In a theme's functions.php you get this one notice; in a must-use plugin, Lesson 4.3 shows two more.)

3. **Silent, script line 56 (functions.php):** `remove_action()` at the top level returns false (the parent has not added its hook yet): the checkmark style is still registered. Move it into a callback on `after_setup_theme`.

Fixed, the script prints:

```
Success: Switched to 'Pages &amp;amp; Co. Lab: Broken Child' theme.
link:      #7a2e2e
child CSS: 1
checkmark: no
== what the page request added to debug.log (time stamps and HTML removed) ==
Success: Switched to 'Pages &amp;amp; Co.' theme.
```

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 3.3 · PATTERNS, STYLE VARIATIONS & THE SITE EDITOR

1. **B** The cached list of the theme's pattern files is still the old one: WordPress caches the list of pattern files per theme version. Set WP_DEVELOPMENT_MODE to theme on a development site, or raise the theme Version when you deploy.

2. **C** A synced pattern: A synced pattern is one wp_block post; every page references it, so editing it once updates all six.

3. **A** One with "blockTypes": ["core/group"] and a styles section: The blockTypes key is what makes a file in styles/ a block style variation (section style) instead of a style variation for the whole site.

4. **B** Find the actual change, put it into the right theme file (often a pattern), then reset the copy: The export has the patterns expanded, with hard-coded URLs and frozen text. Move the actual change into the file it belongs to, usually a pattern, then reset the database copy.

5. **C** Check whether its license allows web use and redistribution in a theme: Desktop or print licenses usually forbid web embedding and redistribution. Check the license (or buy a web license); until then use an OFL font or a system stack.

LESSON 3.4 · CHILD THEMES & CUSTOMIZING EXISTING THEMES

1. **B** twentytwentyfive: template holds the parent's folder (twentytwentyfive); stylesheet holds the child's. Without a child theme both are the same.

2. **A** It stays: the child's file is merged over the parent's: Single values in theme.json are merged key by key, so everything the child does not set keeps the parent's value. Only preset lists such as the palette are replaced as

a whole.

3. **C** false, and the parent's stylesheet is still loaded: The child's functions.php runs before the parent's, so the action has not been added yet and there is nothing to remove. Do it from a callback on after_setup_theme.

4. **B** A child pattern with the slug twentytwentyfive/footer: The parent's footer part only references the pattern twentytwentyfive/footer, and a child pattern with the same slug wins, so the part itself stays the parent's.

5. **C** In a plugin: A form is a feature with data and security needs; it must survive a theme switch, so it belongs in a plugin (Module 5).

EXIT TICKET

1. **Opening hours on four pages:** a synced pattern, edited once, shown everywhere it is referenced. **A hero for new landing pages:** a theme pattern, a starting point that is copied and then edited per page.

2. **Site Editor storage:** `wp_template` (and `wp_template_part`) posts for templates and parts, the `wp_global_styles` post for styles; `wp post list --post_type=wp_template,wp_template_part,wp_global_styles` lists them. (Synced patterns are `wp_block` posts, created on purpose.)

3. **Load order:** the child's functions.php first, then the parent's. A parent hook does not exist yet when the child's file runs, so remove it in a callback on `after_setup_theme`; a pluggable parent function can be replaced by defining it first.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 B (register_block_pattern).** Thinks pattern files need registering. Show the registry before and after development mode: no code, only the cache.
- **Q3 A or B.** Thinks every block comment is risky in a template. The question is what the comment refers to: a slug travels, an ID does not.
- **Q4 A.** The most important miss of the session: the learner believes the copy still points at hero.php. Rerun the demo's `grep` on the copy.
- **Q5 A.** Believes the folder decides. Point at the resolver's one key: blockTypes.
- **Q9 B.** Knows "merged" from the lesson and applies it to lists. Run the partial-palette count.
- **Q10 A.** "Child wins" as a general rule. Ask what PHP does with two functions of the same name.
- **Q11 A.** Copied the lesson's handle. The TT4 lab is the cure: read the parent's functions.php.

INSTRUCTOR KIT · ANSWER KEY

Session 6: The Loop and Custom Queries

Answers and reasoning for every Session 6 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs, the lesson pages' own quizzes and the exit ticket.

Session 6 | 12 self-check items | Confidential

START HERE · WARM-UP

Session 5 retrieval, not graded. W5's output on a site ready for today:

```
wp theme list --fields=name,status
wp post list --post_type=wp_template,wp_template_part,wp_global_styles --format=count
```

```
+--------------------+----------+
| name               | status   |
+--------------------+----------+
| pagesco-tt5-child  | inactive |
| pagesco            | active   |
| twentytwentyfive   | inactive |
| twentytwentyfour   | inactive |
| twentytwentythree  | inactive |
+--------------------+----------+
0
```

1. **W1** Theme pattern or synced pattern: which one is a post in the database, and why does a theme never ship one? **Answer:** The synced pattern: a wp_block post referenced by ID, and IDs differ on every site (Lesson 3.3).
2. **W2** Two JSON files in styles/: what makes one a section style and the other a style variation? **Answer:** The blockTypes key: with it, a section style for the listed blocks; without it, a look for the whole site.
3. **W3** A child theme is active. What do the template and stylesheet options hold? **Answer:** template = the parent's folder, stylesheet = the child's (Lesson 3.4).
4. **W4** Why did the child's style.css not load until functions.php enqueued it, and why must a parent hook be removed on after_setup_theme? **Answer:** A block theme's style.css is a header file; CSS loads only when enqueued. The child's functions.php runs first, so the parent's hooks do not exist yet when it runs.
5. **W5** Run the two commands on the slide. What should a site ready for today print? **Answer:** pagesco active, pagesco-tt5-child inactive (Twenty Twenty-One deleted), and 0 Site Editor copies. Today's demo starts by

building pagesco-classic and switching to it.

4.1 · THE LOOP & TEMPLATE TAGS: SELF-CHECK

Q1

A classic theme has only index.php, singular.php and 404.php. Which file draws /author/maria/?

A) `singular.php`

B) `index.php`, with a Loop over Maria's posts

C) `author.php`, which WordPress generates

D) `404.php`, because there is no author template

> **Why:** The hierarchy for an author archive is author-{nicename}, author-{id}, author, archive, then index (Lesson 3.1). With none of the specific files, index.php runs, and its Loop walks whatever the main query found: Maria's two posts. That is why one index.php can draw every list.
> **A** is wrong: singular.php is for one post or page.
> **C** is wrong: WordPress never generates templates.
> **D** is wrong: 404 is only for requests whose main query finds nothing at all.

Q2

On the course site, what does this print, and why?

```
wp eval 'var_dump( is_home(), get_the_title() );'
```

`bool(false)` and `string(0) ""`: WP-CLI loads WordPress but runs no main query, so is_home() has nothing to answer from and there is no current post for get_the_title()

> **Why:** Conditional tags and template tags read the main query and the global `$post`, which only a front-end request (or `wp()`, as the session's debug handouts do) fills. In a template both would work.

Q3

Inside the Loop you need the post's title in a link's `title` attribute and as the link text inside an h2. Which pair is right?

A) `title="<?php the_title(); ?>"` and `echo '<h2>' . the_title() . '</h2>';`

B) `title="<?php echo get_the_title(); ?>"` and `the_title( '<h2>', '</h2>' );`

C) `title="<?php echo esc_html( the_title() ); ?>"` and `the_title();`

D) `title="<?php the_title_attribute(); ?>"` and `the_title( '<h2>', '</h2>' );`

> **Why:** the_title() prints text prepared for HTML, not for an attribute; `the_title_attribute()` (or `esc_attr( get_the_title() )`) is the attribute version. the_title() with $before and $after wraps itself; concatenating it into a string prints the title first and an empty h2 after it (Lesson 4.1, Common Mistakes 1).
> **A** is wrong: Both halves are wrong: an unescaped attribute, and the concatenation mistake.
> **B** is wrong: The h2 is right; the attribute is not escaped for its context.
> **C** is wrong: the_title() prints and returns nothing, so esc_html() escapes an empty value after the title is already out.

Q4

`get_template_part( 'template-parts/content.php' );` prints nothing, and there is no message. Why?

A) It looked for template-parts/content.php.php, found nothing and returned false

B) Template parts only work inside the Loop

C) Template parts must be in the theme's root folder

D) It found the file but needs a third argument with the post

> **Why:** get_template_part() adds .php itself and goes through locate_template() (child first, then parent). A missing file is not an error: the function returns false, a found file null. var_dump() its return value when a part seems to print nothing.
> **B** is wrong: Parts can run anywhere; content.php just uses template tags, which need a current post.
> **C** is wrong: Any path relative to the theme works.
> **D** is wrong: $args is optional (WordPress 5.5+).

Q5

A plugin must do something only on single posts and pages. Which is the EARLIEST of these hooks on which `is_singular()` gives the right answer?

A) `plugins_loaded`

B) `init`

C) `template_redirect`

D) `wp_footer`

> **Why:** Conditional tags read the main query, which runs after init (wp() in the request order, Lesson 2.2). template_redirect fires after the query and before the template, which is why the lesson's conditions tool logs there. wp_footer works too, but much later.
> **A** is wrong: Before the query: the tag returns false, with a "called incorrectly" notice.
> **B** is wrong: Still before the query.
> **D** is wrong: It works, but it is not the earliest; by then the page is printed.

Q6

singular.php shows only one page. Why does it still use while (have_posts()) { the_post(); ... }?

The main query holds exactly one post, and the_post() is what sets the global $post that every template tag reads (and fires loop_start / loop_end for plugins)

> **Why:** On a single post or page the Loop has one item. Skipping it leaves template tags without a current post, and plugins that hook into the Loop never see it.

4.1 · THE LOOP & TEMPLATE TAGS: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
curl -s http://localhost:8080/author/devon/ | grep -o '<h1>.*</h1>' | sed 's/<[^>]*
```

```
Author: Devon Park
```

> **Why:** No author branch in index.php: `the_archive_title()` knows every archive type and prints "Author: Devon Park" (the tags removed by sed). One call covers category, tag, author and date archives.

P2

Predict the output, then run it.

```
wp eval 'var_dump( did_action( "wp" ), have_posts() );'
```

```
int(0)
bool(false)
```

Why: WP-CLI loads WordPress but never runs `wp()`, so the `wp` action has fired 0 times and the main query has no posts: `have_posts()` is false. That is why the session's debug handouts call `wp( 's=dune' )` themselves.

4.2 · CUSTOM QUERIES: SELF-CHECK

Q7

On the site at the end of Lesson 4.1: what does this print, and why the second number?

```
wp eval '$q = new WP_Query( array( "post_type" => "page", "posts_per_page" => 2, "n
```

`2 0`: two pages loaded; found_posts is 0 because no_found_rows skips the counting query

Why: no_found_rows is right for a list you will not paginate (Lesson 4.2's query-args demo). Without it found_posts would be the four published pages.

Q8

A pre_get_posts callback reads `if ( is_home() ) { $query->set( 'posts_per_page', 3 ); }` and nothing else. What goes wrong?

A) Nothing: is_home() makes sure only the blog home changes

B) Every other WP_Query that runs on the blog home (a box, a widget, a block) is also cut to three posts

C) It is a filter, so it must return $query

D) The admin's Posts screen breaks, but the front end is fine

Why: The callback runs for every WP_Query. The global is_home() describes the request, not the query being built, so secondary queries on the blog home pass the test too. Guard with is_admin(), `$query->is_main_query()` and the query's own `$query->is_home()` (Lesson 4.2).

A is wrong: It checks the request, not which query is about to run.

C is wrong: pre_get_posts is an action: the object is changed in place.

D is wrong: is_home() is false in the admin, so the admin is not the problem here.

Q9

A query with `'posts_per_page' => 3` matches 7 posts. On page 3, what are post_count, found_posts and max_num_pages?

A) 1, 7, 3

B) 3, 7, 3

C) 1, 3, 7

D) 3, 3, 3

Why: post_count is what this page holds (7 = 3 + 3 + 1), found_posts the total match, max_num_pages ceil(7 / 3). The session's query_shelf lab prints all three for every query.

B is wrong: Page 3 holds only the remainder.

C is wrong: found_posts and max_num_pages are swapped.

D is wrong: found_posts is the total, not the page size.

Q10

In which case does your code need NO `wp_reset_postdata()`?

A) After a WP_Query Loop at the end of a template part

B) After a secondary Loop inside the main Loop of singular.php

C) After a secondary Loop in front-page.php, before the main Loop

D) After `get_posts()`, when each WP_Post is passed to the functions you call (`get_the_title( $post )`)

> **Why:** Only $query->the_post() (and setup_postdata()) replaces the global $post. get_posts() returns an array and leaves the global alone, as long as you pass each post along instead of calling setup_postdata() (Lesson 4.2, "get_posts() needs no reset").
> **A** is wrong: Somebody will add a template tag after your part one day; reset.
> **B** is wrong: That is exactly the broken post navigation of Common Mistakes 2.
> **C** is wrong: The main Loop's first the_post() sets the post again, but tags between the two Loops would talk about the wrong post: reset anyway.

Q11

An old template starts with `query_posts( 'cat=2&posts_per_page=5' );`. How do you translate it?

A) Keep it, and call wp_reset_query() at the end

B) Replace it with get_posts() and the same string

C) If it changes the page's main list: pre_get_posts on the main query (category_name, posts_per_page); if it adds a list: new WP_Query plus wp_reset_postdata()

D) Move it into functions.php so it runs earlier

> **Why:** query_posts() throws the main query away and runs a second one, so pagination and conditional tags disagree with what is shown; its own documentation says not to use it (Lesson 4.2 FAQ).
> **A** is wrong: The reset only cleans up after the damage.
> **B** is wrong: get_posts() returns data; it does not change what the template's Loop shows.
> **D** is wrong: query_posts() in functions.php runs before the main query exists and breaks more.

Q12

Module 5 brings a Book post type. The shop wants (a) a "three newest books" box under every post and (b) 12 books per page on /books/. Which tool for each?

(a) a secondary WP_Query (or get_posts()) with posts_per_page 3, plus wp_reset_postdata() after a Loop; (b) pre_get_posts on the main query (is_post_type_archive('book'), posts_per_page 12)

> **Why:** Extra lists are secondary queries; the page's own list is the main query, changed before it runs so that pagination and the 404 decision agree (the lesson's decision table and journal prompt).

4.2 · CUSTOM QUERIES: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
wp eval '$q = new WP_Query( array( "category_name" => "news", "posts_per_page" => 2
```

```
2 3 2
```

> **Why:** post_count 2 (this page), found_posts 3 (all News posts), max_num_pages 2 (ceil(3 / 2)).

P4

Predict the output, then run it.

```
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8080/page/2/
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8080/category/news/page/2
```

```
200
404
```

> **Why:** The home page's main query has three per page (pre_get_posts), so its page 2 exists; the News archive still has ten per page and three posts, so its page 2 does not, and WordPress answers 404 before any template runs.

LAB REFERENCE SOLUTIONS

Every output below is real: the lab files run exactly as `labs/check_labs.py` runs them, on a course site at the state each file declares (WordPress 7.1.3, PHP 8.5, MySQL 9.7, WP-CLI 2.12). Paths show `/var/www/html/...`, the learner's `wordpress/` folder.

- LAB 4.1-A (GUIDED): LABS/MODULE-06/L4_1-LOOP-TEMPLATE-TAGS/SHELF_THEME_STARTER/ → SHELF_THEME_SOLUTION/ WITH TEST_SHELF_THEME.SH

The test with the starter:

```
Success: Switched to 'Pages &amp;amp; Co. Lab: Shelf' theme.
/category/news/   TODO
    TODO: the Loop
/author/devon/    TODO
    TODO: the Loop
/?s=dune          TODO
    TODO: the Loop
/?s=zzz           TODO
    TODO: the Loop
/book-club-dune/  TODO
    TODO: the Loop
the first link:
```

With the solution:

```
Success: Switched to 'Pages &amp;amp; Co. Lab: Shelf' theme.
/category/news/   Shelf: News
    Book club: Dune · 2026-10-01 · Maria Lopez
    Staff picks for the season · 2026-09-15 · Devon Park
    Welcome to the new Pages &amp;#038; Co. website · 2026-09-01 · Maria Lopez
/author/devon/    Archive
    Staff picks for the season · 2026-09-15 · Devon Park
/?s=dune          Found for: dune
    Book club: Dune · 2026-10-01 · Maria Lopez
    Staff picks for the season · 2026-09-15 · Devon Park
/?s=zzz           Found for: zzz
    Nothing on this shelf.
/book-club-dune/  Book club: Dune
    Book club: Dune · 2026-10-01 · Maria Lopez
the first link: <li class="shelf-item"><a href="http://localhost:8080/book-club-dune/"
```

Reading it: one template, five addresses, and the Loop never asked for posts. /author/devon/ says Archive because the heading has no is_author() branch (fine: the task did not ask for one; a good extension). /book-club-dune/ is a Loop of one. The `&` in Welcome's title is the_title()'s display filter writing the ampersand as an entity, which the browser shows as &. The title attribute comes from the_title_attribute(). Accept `the_title( '<a href="' . esc_url( get_permalink() ) . '">', '</a>' )` with the attribute added any correct way, and an if/elseif chain inside the h1 instead of a variable.

- LAB 4.1-B: LABS/MODULE-06/L4_1-LOOP-TEMPLATE-TAGS/TEMPLATE_TAGS_STARTER.PHP → TEMPLATE_TAGS_SOLUTION.PHP

As shipped:

```
post_title:        Welcome to the new Pages & Co. website
title:             Welcome to the new Pages &#038; Co. website
title attribute:   TODO
permalink:         TODO
date (Y-m-d):      TODO
author:            TODO
categories:        TODO
raw content:       TODO
rendered content:  TODO
excerpt:           TODO
classes:           TODO
```

The solution prints:

```
post_title:        Welcome to the new Pages & Co. website
title:             Welcome to the new Pages &#038; Co. website
title attribute:   Welcome to the new Pages &#038; Co. website
permalink:         http://localhost:8080/welcome/
date (Y-m-d):      2026-09-01
author:            Maria Lopez
categories:        <a href="http://localhost:8080/category/news/" rel="category tag">Ne
raw content:       <!-- wp:paragraph -->
rendered content:  <p class="wp-block-paragraph">Our new website is open! Browse our pa
excerpt:           Our new website is open! Browse our pages, read the shop news, and s
classes:           post-15 post type-post status-publish format-standard hentry categor
```

The answers to the workbook's questions: post_title is stored with a plain &, the display filters (wptexturize) turn it into `&`; the attribute version is the same here because the title has no tags or quotes; the raw content starts with a block comment, the rendered one with the paragraph's HTML (only the content filters render blocks); the excerpt was built from the content (no hand-written excerpt).

- HANDOUT: LABS/MODULE-06/L4_1-LOOP-TEMPLATE-TAGS/DEBUG_LOOP_REPORT.PHP (DELIBERATELY BROKEN)

As shipped:

```
Book club: Dune-  (2026-10-01)
Staff picks for the season-  (2026-09-15)
Warning: Undefined variable $pagesco_foud in /var/www/html/debug_loop_report.php on lir
0 result(s) for "dune"
```

1. **Silent, line 17:** `is_search()` before `wp( 's=dune' )`: no query has run, so it is false and the heading never prints. Move the check after the `wp()` call (in a plugin: after template_redirect).
2. **Silent, line 26:** `the_title()` inside the string prints the title first and returns null, hence `Book club: Dune-` with the dash after it. Use `get_the_title()`.

3 **Warning, line 30:** `$pagesco_foud` is undefined (a typo for `$pagesco_found`), so the count prints 0.

Fixed, it prints:

```
Search results
- Book club: Dune (2026-10-01)
- Staff picks for the season (2026-09-15)
2 result(s) for "dune"
```

- LAB 4.2-A (GUIDED): LABS/MODULE-06/L4_2-CUSTOM-QUERIES/QUERY_SHELF_STARTER.PHP → QUERY_SHELF_SOLUTION.PHP

As shipped:

```
Maria's posts:     TODO
news, page 2:      TODO
before October:    TODO
pages, A to Z:     TODO
how many news?     TODO
```

The solution prints:

```
Maria's posts:     Book club: Dune, Welcome to the new Pages &#038; Co. website  (post_
news, page 2:      Welcome to the new Pages &#038; Co. website  (post_count 1, found_p
before October:    Welcome to the new Pages &#038; Co. website, Staff picks for the sea
pages, A to Z:     About Us, Contact, Events  (post_count 3, found_posts 3, max_num_pag
how many news?     Book club: Dune  (post_count 1, found_posts 3, max_num_pages 3)
```

Accept other correct arrays: `'author' => 2` for Maria works on this site but is not portable; a date_query with `'year' => 2026, 'month' => 9` gives the same two posts today. `before October` excludes Hello world! because its date is the day the site was installed. The last line is the point: found_posts counts all matches although one ID was loaded; max_num_pages is pages of ONE.

- LAB 4.2-B: LABS/MODULE-06/L4_2-CUSTOM-QUERIES/ARCHIVE_RULES_STARTER/ → ARCHIVE_RULES_SOLUTION/ WITH TEST_ARCHIVE_RULES.SH

The test with the starter (the callback changes nothing):

```
/author/maria/      200  Book club: Dune | Welcome to the new Pages &#038; Co. website
/2026/09/           200  Staff picks for the season | Welcome to the new Pages &#038; C
/2026/09/page/2/    404  (no list)
WP-CLI post list:   4 posts
a September box:    2 posts (a secondary query: must stay 2)
```

With the solution:

```
/author/maria/      200  Welcome to the new Pages &#038; Co. website | Book club: Dune
/2026/09/           200  Staff picks for the season
/2026/09/page/2/    200  Welcome to the new Pages &#038; Co. website
WP-CLI post list:   4 posts
a September box:    2 posts (a secondary query: must stay 2)
```

Reading it: Maria's archive is now oldest first; September has one post per page, so /2026/09/page/2/ answers 200 (the main query decides that page exists, Lesson 4.2); and the two queries that are not the main query are unchanged. Without the is_main_query() guard the last line would say 1: the September box is a date query too. Remove the file afterwards.

HANDOUT: LABS/MODULE-06/L4_2-CUSTOM-QUERIES/DEBUG_RELATED_BOX.PHP (DELIBERATELY BROKEN)

As shipped:

```
More from the shop:
- Staff picks for the season
- Welcome to the new Pages &#038; Co. website
Warning: Undefined array key 2 in /var/www/html/wp-includes/class-wp-query.php on line
-
Titles:
This page is about:
```

1. **Warning, line 31:** a `for` loop that runs three times over a query that holds two posts: the third `the_post()` reads past the end of `$posts` (Undefined array key 2 in class-wp-query.php, the core line that reads it) and leaves no post. Loop with `while ( $pagesco_related->have_posts() )`.
2. **Silent, line 36:** `wp_list_pluck()` gets the query object instead of its `->posts` array, so the Titles line is empty. Pass `$pagesco_related->posts`.
3. **Silent, line 15 (it appears once bug 1 is fixed):** no `wp_reset_postdata()` after the box, so the last line names Welcome, the box's last post, instead of Book club: Dune. Reset right after the loop.

Fixed, it prints:

```
More from the shop:
- Staff picks for the season
- Welcome to the new Pages &#038; Co. website
Titles: Staff picks for the season, Welcome to the new Pages & Co. website
This page is about: Book club: Dune
```

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 4.1 · THE LOOP & TEMPLATE TAGS

1. **C** Dune : PHP evaluates the expression first: the_title prints Dune and returns null, then echo prints the empty paragraph. Inside a string use get_the_title with esc_html.

2. **A** the_title_attribute(), or esc_attr(get_the_title()): the_title_attribute prints the title escaped for an attribute; esc_attr(get_the_title()) is the returning equivalent. the_title prepares HTML text, not attribute values.

3. **B** index.php: The search chain is search, then index. A search is not an archive, so archive.php is never a candidate and index.php draws it.

4. **C** It returns false, and WordPress notices that it was called incorrectly: The main query has not run when must-use plugins load, so conditional tags return false, and with WP_DEBUG on WordPress reports that they were called incorrectly. Ask on template_redirect instead.

5. **A** template-parts/content-none.php: The name is joined to the slug with a hyphen: template-parts/content-none.php, looked up in the child theme and then the parent. Only if it is missing does WordPress fall back to template-parts/content.php.

LESSON 4.2 · CUSTOM QUERIES

1. **A** A pre_get_posts callback that sets posts_per_page to 5 when the main query is_category('news'): Changing the main query before it runs keeps the 404 decision, the template, the conditional tags and the_posts_pagination in agreement.

2. **B** The title of the last post of the secondary Loop: Each the_post() of the secondary query replaced the global post, so the last post of that Loop is still current until wp_reset_postdata() restores the main query's post.

3. **C** 4, 10, 3: post_count is the 4 posts on this page, found_posts the 10 that match in total, and max_num_pages is 10 divided by 4, rounded up: 3.

4. **C** is_admin(): Admin list screens run a main query too, so is_main_query alone does not exclude them; is_admin() returns early for every admin request.

5. **B** All four published posts, without any warning: cat takes category IDs; the string news is not one, so the filter is dropped silently and every published post comes back. category_name takes the slug.

EXIT TICKET

1. **The Loop** walks the posts the main query already found; **the_post()** moves to the next one and sets the global $post (and fires loop_start the first time).

2. **A pair:** the_title() prints HTML-ready text for the page; get_the_title() returns the value, for an attribute (with esc_attr()), a string, or a comparison. Same for the_permalink() / get_permalink() (esc_url()) and the_date() / get_the_date() (esc_html()).

3. **Three per page:** a pre_get_posts callback in functions.php (or a plugin), returning early on is_admin() or ! $query->is_main_query(), then `if ( $query->is_home() ) $query->set( 'posts_per_page', 3 );`.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 C.** Believes WordPress makes missing templates. Show the hierarchy falling through to index.php.
- **Q3 B.** Knows the_title($before, $after) but not that attributes need their own escaping.
- **Q5 A or B.** Mixes up "WordPress is loaded" and "the main query has run". Redraw the request order.
- **Q8 A.** Thinks the global is_home() describes the query being built. Ask how many queries run on the blog home.
- **Q9 B.** Confuses post_count with posts_per_page.
- **Q10 B or C.** Thinks the reset is optional inside templates. Rerun the broken navigation.

INSTRUCTOR KIT · ANSWER KEY

Session 7: functions.php: Supports, Assets, Menus & Widgets

Answers and reasoning for every Session 7 item: the warm-up, the self-check (Lesson 4.3 and the Part 1 project skills), the predict-then-run items, the Part 1 kickoff outputs, the lab reference outputs, the lesson page's own quiz and the exit ticket.

Session 7 | 12 self-check items | Confidential

START HERE · WARM-UP

Session 6 retrieval, not graded. W5's output on a site ready for today:

```
wp theme list --status=active --field=name
curl -s http://localhost:8080/ | grep -c 'class="entry-title"'
```

```
pagesco-classic
3
```

1. **W1** What does the Loop do, and what does the_post() do inside it? **Answer:** It walks the posts the main query already found; the_post() moves to the next one and sets the global $post (Lesson 4.1).
2. **W2** the_title() or get_the_title() inside echo '<h2>' . … . '</h2>'? **Answer:** get_the_title(), escaped: the_title() prints on the spot and leaves an empty heading.
3. **W3** posts_per_page 2, three matches: post_count, found_posts, max_num_pages on page 1? **Answer:** 2, 3, 2 (Lesson 4.2).
4. **W4** Name the three guards of a pre_get_posts callback that changes the blog home. **Answer:** is_admin(), $query->is_main_query(), $query->is_home().
5. **W5** Run the two commands on the slide. What should a site ready for today print? **Answer:** pagesco-classic (active since Lesson 4.1), then 3: three news items on the home page (the 4.2 callback).

4.3 · FUNCTIONS.PHP: SUPPORTS, ASSETS, MENUS & WIDGETS: SELF-CHECK

Q1

Which of these actions fires FIRST in a front-end request?

A) `wp_enqueue_scripts`

B) `widgets_init`

C) `after_setup_theme`

D) `wp_head`

> **Why:** The order of Lesson 4.3's timeline: the theme's functions.php is included, after_setup_theme fires (supports, menu locations), then init with widgets_init at priority 1, then the main query and the template, and wp_head runs wp_enqueue_scripts at priority 1.
> **A** is wrong: It runs inside wp_head, after the query.
> **B** is wrong: It runs on init, after after_setup_theme.
> **D** is wrong: wp_head is printed by the template, near the end.

Q2

At the end of Lesson 4.3, what does this print, and why does the version column matter for the theme's CSS?

```
wp theme list --name=pagesco-classic --fields=name,status,version
```

`pagesco-classic | inactive | 1.1.0`: pagesco is active again; 1.1.0 is the Version the stylesheet is enqueued with (style.css?ver=1.1.0), so raising it on a release makes every browser fetch the new CSS

> **Why:** The enqueue reads the version from the theme header (wp_get_theme(get_template())->get('Version')), never a typed number.

Q3

pagesco-classic enqueued its stylesheet with `get_stylesheet_uri()` instead of `get_parent_theme_file_uri( 'style.css' )`. A client builds a child theme of it. What happens?

A) The page loads the CHILD's style.css under the parent's handle, and the parent's CSS is gone

B) Nothing changes: both functions return the parent's file

C) WordPress loads both files automatically

D) A notice: get_stylesheet_uri() cannot be used in a parent theme

> **Why:** get_stylesheet_uri() is the ACTIVE (child) theme's style.css; get_parent_theme_file_uri() always the theme that holds the code (Lesson 3.4's template and stylesheet). The same reasoning picks wp_get_theme(get_template()) for the version.
> **B** is wrong: Without a child they are the same, which is why the mistake hides.
> **C** is wrong: Only what is enqueued loads.
> **D** is wrong: There is no such notice; the mistake is silent.

Q4

Why does header.php wrap `wp_nav_menu()` in `has_nav_menu( 'primary' )`?

A) It makes the menu load faster

B) So a location without an assigned menu prints nothing instead of wp_page_menu()'s list of every page

C) wp_nav_menu() refuses to run without it

D) It registers the location

> **Why:** With no menu in the location, wp_nav_menu() calls its fallback, wp_page_menu(), which lists all published pages, Sample Page included (Lesson 4.3, Common Mistakes 3). The guard (or 'fallback_cb' => false) keeps the header empty until the owner assigns a menu.
> **A** is wrong: It saves nothing measurable.
> **C** is wrong: It runs fine; that is the problem.
> **D** is wrong: register_nav_menus() registers locations, on after_setup_theme.

Q5

pagesco-classic ships languages/es_ES.l10n.php and says Requires at least: 6.6. Does it still need load_theme_textdomain()?

A) Yes, every theme needs it on every version

B) No, never: translations always load themselves

C) Only plugins need it

D) Not on WordPress 6.8 and later, which load the active theme's languages/ folder itself; it stays for 6.6 and 6.7

> **Why:** Since 6.8 wp-settings.php calls $wp_theme->load_textdomain() for the active theme (the lesson's version note, checked in the 6.7 and 6.8 images). A theme that supports older versions keeps the call; on 7.1 it is harmless.
> **A** is wrong: Measured otherwise on 6.8+.
> **B** is wrong: Not before 6.8.
> **C** is wrong: Plugins and themes follow the same rule since 6.8.

Q6

A must-use plugin calls wp_enqueue_style() at the top level of its file. What does WordPress say, and what is the real damage?

A "called incorrectly" notice naming the handle (scripts and styles should not be enqueued until wp_enqueue_scripts...), plus two follow-on notices; the real damage: top-level code runs on every request, so the CSS also loads on admin screens and the login page

> **Why:** Lesson 4.3's Common Mistake 1. Read the first notice; wrap the call in a function on wp_enqueue_scripts.

4.3 · FUNCTIONS.PHP: SUPPORTS, ASSETS, MENUS & WIDGETS: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
wp eval 'var_dump( current_theme_supports( "title-tag" ), current_theme_supports( "
```

```
bool(false)
bool(false)
```

Why: Neither: pagesco-classic declares no supports yet and registers no location. Registering a location (Lesson 4.3) is what switches menus on; nobody needs add_theme_support('menus').

P2

Predict the output, then run it.

```
wp eval 'var_dump( current_theme_supports( "title-tag" ), current_theme_supports( "
```

```
bool(false)
bool(true)
```

Why: pagesco declares nothing, yet html5 search-form is true: _add_default_theme_supports() adds it for block themes. title-tag is false although every page has a title: block themes print it unconditionally (the lesson's Exercise 3).

PART 1 PROJECT: SELF-CHECK (PROJECT SKILLS)

Q7

Fernhill is a block theme. What is its style.css for?

A) It names the theme: the header fields WordPress reads (Theme Name, Text Domain, Version, Requires at least ...)

B) It is the theme's stylesheet, loaded on every page

C) It is a fallback when theme.json has an error

D) It registers the patterns

> **Why:** Block themes do not load style.css on the front end (Lesson 3.2's FAQ); the design lives in theme.json, and extra CSS is enqueued (R5: `fernhill-style`). The header is R1 and the self-check's first line.
> **B** is wrong: Not in a block theme; Fernhill enqueues assets/css/fernhill.css instead.
> **C** is wrong: A broken theme.json is just ignored, with a decode notice.
> **D** is wrong: Pattern files register themselves from patterns/.

Q8

The brief wants links and buttons in the primary color. Where does that go?

A) A hex color typed into each template's markup

B) assets/css/fernhill.css

C) `theme.json`: `styles.elements.link` and `styles.elements.button` with `var:preset|color|primary`

D) A pattern that wraps every link

> **Why:** The design system lives in theme.json (Lesson 3.2): one preset, used everywhere, and a style variation (Autumn, R4) can recolor it without touching a template. Hard-coded hex in markup costs a rubric point.
> **A** is wrong: The rubric's Partly column names exactly this.
> **B** is wrong: Possible, but not where a variation can change it.
> **D** is wrong: Patterns hold sections, not global styles.

Q9

R6: the blog home lists three posts per page with working pagination. Which way does the brief want?

A) A Query Loop with `perPage` 3 that does not inherit

B) A `pre_get_posts` callback in functions.php with the three guards: not the admin, the main query only, the blog home only

C) Settings > Reading set to 3

D) A second WP_Query in the front-page pattern

> **Why:** One main query, changed before it runs, so the Query Loop that inherits it, the pagination and the 404 decision all agree (Lesson 4.2). The self-check's line 11 counts 3 on / and 1 on /page/2/.
> **A** is wrong: A non-inheriting Query Loop runs a second query; /page/2/ then disagrees.
> **C** is wrong: That is the owner's setting, not the theme's, and it changes every list.
> **D** is wrong: The second-query mistake of Lesson 4.2.

Q10

You tried a header change in the Site Editor and clicked Save. Now your edits to parts/header.html no longer show. What happened, and what do you do?

A) The theme cache needs 30 minutes

B) parts/ files are only read on activation: switch themes twice

C) theme.json overrides parts

D) A wp_template_part copy in the database wins over the file: move the change into header.html, then delete the copy

> **Why:** Lesson 3.2's Mistake 3 and 3.3's loop: list the template, template-part and global-styles posts (wp post list), find the change, put it in the file, delete the copy. A copy left behind is also missing from the zip you hand in (R8).
> **A** is wrong: That cache is for pattern files, and development mode is on.
> **B** is wrong: Parts are read on every request.
> **C** is wrong: theme.json declares part areas; it does not replace markup.

Q11

Odette wants the autumn look she can switch on in October (R4). Which file, and what may it change?

`styles/autumn.json`, titled Autumn, without blockTypes: the same five color slugs with autumn values, nothing else

> **Why:** A style variation is a layer over theme.json (Lesson 3.3); keeping the slugs means every template and pattern recolors by itself. With blockTypes it would be a section style instead.

Q12

You stop working on Fernhill for the day. Which command do you run, and why?

`wp theme activate pagesco`: the labs, Session 8's assessment and Part 2 (Lesson 5.1) expect pagesco active, the state at the end of Lesson 4.3

> **Why:** The self-check switches nothing; you activate fernhill yourself and switch back yourself (the brief's rules).

PART 1 PROJECT: WHAT THE KICKOFF SHOWS

Milestone 1, the self-check on the starter:

```
Success: Switched to 'Fernhill' theme.
Fernhill self-check (Part 1 project, BRIEF.md)
TODO  1  Theme header: Fernhill 1.0.0, text domain fernhill, requires WordPress 7.1 and
OK    2  A block theme with a valid theme.json, version 3
TODO  3  Palette: base, contrast, primary, secondary, accent (in that order); the defau
TODO  4  Layout: content 680px, wide 1100px
TODO  5  Templates: index, front-page, single, page, 404 (missing: front-page, single,
TODO  6  Template parts: header (area header) and footer (area footer)
TODO  7  Patterns: fernhill/hero and fernhill/volunteer-call in the fernhill category;
TODO  8  Style variation: Autumn
TODO  9  Front page: header, main, the hero heading, the volunteer heading, footer
TODO 10  Stylesheet: fernhill-style, assets/css/fernhill.css?ver=1.0.0
TODO 11  Main query: 3 posts on the front page, 1 on /page/2/ (now: 4 and 0)
TODO 12  404: status 404 and the heading "This path is overgrown"
Checks passed: 1 of 12
```

After the room's Milestone 2 time (the starter with the model solution's style.css and theme.json): `Checks passed: 4 of 12`. The model solution, for comparison (never handed out before the feedback):

```
Success: Switched to 'Fernhill' theme.
Fernhill self-check (Part 1 project, BRIEF.md)
OK    1  Theme header: Fernhill 1.0.0, text domain fernhill, requires WordPress 7.1 and
OK    2  A block theme with a valid theme.json, version 3
OK    3  Palette: base, contrast, primary, secondary, accent (in that order); the defau
OK    4  Layout: content 680px, wide 1100px
OK    5  Templates: index, front-page, single, page, 404
OK    6  Template parts: header (area header) and footer (area footer)
OK    7  Patterns: fernhill/hero and fernhill/volunteer-call in the fernhill category;
OK    8  Style variation: Autumn
OK    9  Front page: header, main, the hero heading, the volunteer heading, footer
OK   10  Stylesheet: fernhill-style, assets/css/fernhill.css?ver=1.0.0
OK   11  Main query: 3 posts on the front page, 1 on /page/2/ (now: 3 and 1)
OK   12  404: status 404 and the heading "This path is overgrown"
Checks passed: 12 of 12
```

The rubric, the scoring guidance and the model solution notes are in the Part 1 section of `final-assessment.html` (Session 8). The learners' brief is `labs/module-08/p1-project/BRIEF.md`.

LAB REFERENCE SOLUTIONS

Every output below is real: the lab files run exactly as `labs/check_labs.py` runs them, on a course site at the state each file declares (WordPress 7.1.3, PHP 8.5, MySQL 9.7, WP-CLI 2.12).

- LAB 4.3-A (GUIDED): LABS/MODULE-07/L4_3-FUNCTIONS-PHP/CORNER_THEME_STARTER/ → CORNER_THEME_SOLUTION/ WITH TEST_CORNER_THEME.SH

The test with the starter:

```
Success: Switched to 'Pages &amp;amp; Co. Lab: Reading Corner' theme.
<title>:      0
search form:  xhtml (id="searchform")
style:        (not enqueued)
script:       (not enqueued)
menu places:  (none)
widget areas: (none)
```

With the solution:

```
Success: Switched to 'Pages &amp;amp; Co. Lab: Reading Corner' theme.
<title>:      1
search form:  html5 (type="search")
style:        style.css?ver=2.0.0
script:       corner.js?ver=2.0.0, defer
menu places:  corner (Reading corner menu)
widget areas: corner-footer
```

Reading it: no title at all until title-tag (the header has no hand-written one: the lesson's two-titles mistake cannot happen); the xhtml form until html5 with an ARRAY; the stylesheet and the deferred script carry the theme's Version 2.0.0 read from the header; registering the location and the area is enough for WP-CLI to list them (they stay empty until the owner fills them, which is why the templates guard with has_nav_menu() and is_active_sidebar()). Accept the menu and the supports in one callback; insist on one function per hook family and the prefix.

- LAB 4.3-B: LABS/MODULE-07/L4_3-FUNCTIONS-PHP/MENUS_WIDGETS_STARTER.SH → MENUS_WIDGETS_SOLUTION.SH

As shipped:

```
Success: Switched to 'Pages &amp;amp; Co. Classic' theme.
== locations ==
+----------+--------------+
| location | description  |
+----------+--------------+
| primary  | Primary menu |
+----------+--------------+
Second menu: TODO
Second widget: TODO
footer widgets after cleanup: block-7
Success: Switched to 'Pages &amp;amp; Co.' theme.
```

The solution prints:

```
Success: Switched to 'Pages &amp;amp; Co. Classic' theme.
== locations ==
+----------+--------------+
| location | description  |
+----------+--------------+
| primary  | Primary menu |
+----------+--------------+
== a second menu ==
Success: Created menu 5.
Success: Menu item added.
Success: Menu item added.
+------------+---------------------------------+
| title      | link                            |
+------------+---------------------------------+
| Events     | http://localhost:8080/events/   |
| Gift cards | http://localhost:8080/contact/ |
+------------+---------------------------------+
== one more widget ==
Success: Added widget to sidebar.
+-------+---------+----------+
| name  | id      | position |
+-------+---------+----------+
| block | block-8 | 1        |
| block | block-7 | 2        |
+-------+---------+----------+
== cleanup ==
Success: Deleted 1 of 1 widgets.
Deleted menu 'shop-links'.
Success: Deleted 1 of 1 menus.
footer widgets after cleanup: block-7
Success: Switched to 'Pages &amp;amp; Co.' theme.
```

Menu and item IDs come from the site's counters (5, and the posts after it): a learner's numbers may differ. `wp widget add` with position 1 puts the search widget before the lesson's block-7; the cleanup deletes it by the first id of the list, then the menu (wp menu delete also removes its items).

HANDOUT: LABS/MODULE-07/L4_3-FUNCTIONS-PHP/DEBUG_THEME_SETUP.PHP (DELIBERATELY BROKEN)

As shipped:

```
Notice: Function register_sidebar was called <strong>incorrectly</strong>. No <code>id<
widget area id: sidebar-2
Notice: Function add_theme_support( 'html5' ) was called <strong>incorrectly</strong>.
html5 for: comment-form, comment-list, search-form, gallery, caption, style, script
stylesheet: ?ver=7.1.3 (the theme is at 1.0.0)
```

1. **doing_it_wrong notice, line 20:** register_sidebar() without an id: WordPress numbers it (sidebar-N, depending on what was registered before) and says so. Give it `'id' => 'footer-shop'`.
2. **doing_it_wrong notice, line 24:** add_theme_support('html5', 'search-form'): "You need to pass an array of types", and the call is ignored (pagesco, a block theme, has html5 by default anyway, which is why the list looks fine). Pass `array( 'search-form' )`.
3. **Silent, line 28:** wp_enqueue_style() without a version: `?ver=7.1.3`, WordPress's own version, which changes only with core updates. Pass the theme's Version.

Fixed, it prints:

```
widget area id: footer-shop
html5 for: comment-form, comment-list, search-form, gallery, caption, style, script, s
stylesheet: ?ver=1.0.0 (the theme is at 1.0.0)
```

THE LESSON PAGE'S OWN QUICK QUIZ

LESSON 4.3 · FUNCTIONS.PHP: SUPPORTS, ASSETS, MENUS & WIDGETS

1. **B** In a functions.php callback hooked to after_setup_theme: Theme supports are declared on after_setup_theme, which fires right after functions.php is included and long before any template runs.
2. **C** ?ver=7.1.3: Without a version WordPress adds its own, 7.1.3, which does not change when your CSS does, so browsers keep the old file. Pass the theme's Version instead.
3. **A** A list of all published pages, from the fallback wp_page_menu(): With no menu found, wp_nav_menu calls its default fallback, wp_page_menu, which lists every published page. fallback_cb false or a has_nav_menu check prevents it.
4. **B** widgets_init: widgets_init fires during init (at priority 1), the moment WordPress sets up widgets, on every request including the admin's Widgets screen.
5. **C** Add the standard .screen-reader-text CSS that clips it visually but keeps it for screen readers: Core marks text meant for screen readers with screen-reader-text and expects the theme to hide it visually only. Block themes get this CSS from core; classic themes must add it.

EXIT TICKET

1. **Hooks:** after_setup_theme (supports, menu locations); wp_enqueue_scripts (styles and scripts); widgets_init (widget areas).
2. **The version:** it ends the file's URL (?ver=), so a new release means a new URL and every browser and cache fetches the new file; the theme's Version changes exactly when you ship.

3 **Personal.** Use the answers to plan the Session 8 studio: group learners by the requirement they named.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 A or D.** Thinks assets come first because they are "the theme". Redraw the timeline.
- **Q3 B.** Has never built a child theme of a classic theme; tie it to Lesson 3.4's template/stylesheet.
- **Q5 A.** Copied from older tutorials; show the 6.8 version note.
- **Q7 B.** Still thinks a block theme loads style.css: Lesson 3.2's FAQ and Lesson 3.4's "stylesheet nobody loads".
- **Q9 A.** A non-inheriting Query Loop: the most common Fernhill bug in the studio; /page/2/ gives it away.

INSTRUCTOR KIT · ANSWER KEY

Session 8: Part 1 Project and Assessment

Answers and reasoning for Session 8: the warm-up, the Part 1 self-check, the predict items, the review lab and handout, and the project's and the assessment's self-check outputs (rubrics and scoring in final-assessment.html).

Session 8 | 12 self-check items | Confidential

WARM-UP

1. **W1** Hooks (Lesson 2.3), themes (Modules 3 and 4), plugins (Module 5), blocks (Module 6), APIs (Module 7). Part 1 covered the first two.
2. **W2** Block: templates/*.html and parts/*.html, block markup. Classic: *.php in the theme root (index.php, singular.php...), PHP with template tags.
3. **W3** Not is_admin(); $query->is_main_query(); $query->is_home() (the query's method, not the global function).
4. **W4** esc_html() text, esc_attr() attributes, esc_url() href/src, wp_kses_post() HTML with safe tags (and the _e / __ variants for translated text).
5. **W5** Each learner's own number: the status round (see the studio triage).

SELF-CHECK: PART 1

Q1

The site shows the latest posts on its front page. The active block theme has `templates/front-page.html`, `templates/home.html` and `templates/index.html`. Which template draws `/`?

A) `home.html`, because the front page lists posts

B) `index.html`, the fallback for everything

C) `front-page.html`

D) `page.html`, because the front page is a page

> **Why:** `front-page` is checked first for the front page, whatever it shows (Lesson 3.1's hierarchy). `home` draws the posts list only when it is NOT the front page. That is why Fernhill's news list lives in front-page.html.
>
> **A** is wrong: home.html would win only if front-page.html were missing (or the posts list were on another page).
>
> **B** is wrong: index.html is the last resort, used when nothing more specific exists.
>
> **D** is wrong: With show_on_front set to posts there is no front PAGE at all.

Q2

What does this print?

```
wp eval 'add_filter( "the_title", function ( $t ) { return "A" . $t; }, 20 );
add_filter( "the_title", function ( $t ) { return "B" . $t; } );
echo apply_filters( "the_title", "x" ), "\n";'
```

`ABx`

> **Why:** Callbacks run by priority, lowest first: the default 10 ("B") before 20 ("A"). Each filter receives the value the previous one returned, so A wraps the result of B (Lesson 2.3).

Q3

Which file must a block theme have, besides `style.css`, for WordPress to accept it?

A) `functions.php`

B) `templates/index.html`

C) `theme.json`

D) `index.php`

> **Why:** A block theme needs style.css (its name) and templates/index.html (the last resort of the hierarchy); without the template WordPress reports "Template is missing" (Lesson 3.1, Mistake 1). theme.json is strongly recommended, functions.php optional.
> **A** is wrong: Optional: many block themes have none.
> **C** is wrong: Without theme.json the theme works on core's defaults.
> **D** is wrong: That is the CLASSIC theme's required template.

Q4

pagesco-tt5-child is a child theme of Twenty Twenty-Five. What does the middle command print?

```
wp theme activate pagesco-tt5-child
wp eval 'echo get_template(), " ", get_stylesheet(), "\n";'
wp theme activate pagesco
```

`twentytwentyfive pagesco-tt5-child`

> **Why:** `get_template()` is the PARENT's folder (the Template header), `get_stylesheet()` the active theme's own folder: the child (Lesson 3.4). Without a child theme both print the same name.

Q5

A learner changed Fernhill's footer in the Site Editor yesterday. Today every edit to parts/footer.html is ignored. Why, and what fixes it?

A) The database copy of the part (a wp_template_part post) wins over the file: copy what you want into the file, then reset or delete the copy

B) OPcache: restart the containers

C) The footer area in theme.json is wrong: change it to "footer-1"

D) Template parts are cached for 30 minutes: wait

> **Why:** A Site Editor save creates a database copy (source custom) that WordPress prefers to the theme's file. Export or copy the markup into the file, then Reset in the Site Editor or delete the post (Lesson 3.2, Mistake 3; the project's rule R8).
> **B** is wrong: OPcache caches PHP, and the course setup revalidates on every request; .html parts are not PHP.
> **C** is wrong: The area only says where the part belongs in the editor; it does not decide which copy wins.
> **D** is wrong: The 30-minute cache is for PATTERN files, and WP_DEVELOPMENT_MODE theme switches it off on the course site.

Q6

This callback forgets the main-query guard. What does the last line print on the course site (four published posts)?

```
wp eval 'add_action( "pre_get_posts", function ( $q ) {
    if ( is_admin() ) { return; }
    if ( $q->is_home() ) { $q->set( "posts_per_page", 3 ); }
} );
$q = new WP_Query( array( "posts_per_page" => 5 ) );
echo $q->post_count, "\n";'
```

A) 4

B) 5

C) 0

D) 3

Why: A WP_Query without query variables counts as the blog home (`is_home()` is true), so the unguarded callback changes YOUR query too: 3 posts instead of the four you asked for. `$query->is_main_query()` is the guard (Lesson 4.2, Mistake 3; the project's R6).

A is wrong: That is what the query returns WITH the guard (all four posts, fewer than five).

B is wrong: There are only four published posts.

C is wrong: Nothing is excluded; the query only gets a smaller page size.

Q7

What does this print? (Three posts are in News.)

```
wp eval '$q = new WP_Query( array( "category_name" => "news", "posts_per_page" => 2
echo $q->post_count, " ", $q->found_posts, "\n";'
```

A) 2 2

B) 2 3

C) 3 3

D) 3 2

> **Why:** `post_count` is how many posts this page of results holds (`posts_per_page` 2), `found_posts` how many match in all (3). `max_num_pages` would be 2: that is what pagination needs (Lesson 4.2).
> **A** is wrong: found_posts ignores the page size.
> **C** is wrong: post_count respects the page size.
> **D** is wrong: The two are swapped.

Q8

A pattern prints a link to an address that comes from `home_url()`. Which function goes around it inside `href="..."`?

A) `esc_html()`

B) `esc_attr()`

C) `esc_url()`

D) `wp_kses_post()`

> **Why:** `esc_url()` checks the protocol and encodes the characters a URL may not carry; it is the escaping for href and src, even for an address WordPress built (Lessons 1.3 and 3.3). Escape late, at the echo.
> **A** is wrong: For text between tags, not for URLs.
> **B** is wrong: For other attribute values; it does not check the protocol (javascript: would pass).
> **D** is wrong: For HTML that may keep safe tags, such as post content.

Q9

What does this print? (Post 17 is "Book club: Dune".)

```
wp eval 'global $post; $post = get_post( 17 ); setup_postdata( $post );
echo "<h2>" . the_title() . "</h2>\n";'
```

`Book club: Dune<h2></h2>`

> **Why:** `the_title()` ECHOES and returns nothing, so the title is printed while PHP builds the string, before the concatenated `<h2></h2>`. Use `the_title( "<h2>", "</h2>" )` or `get_the_title()` with esc_html() (Lesson 4.1, Mistake 1).

Q10

Fernhill must load `assets/css/fernhill.css` on the front end. On which hook does `wp_enqueue_style()` belong?

A) `init`

B) `after_setup_theme`

C) `wp_head`

D) `wp_enqueue_scripts`

> **Why:** `wp_enqueue_scripts` is the hook for front-end styles and scripts (admin_enqueue_scripts and login_enqueue_scripts for those screens). Called anywhere earlier, WordPress says the function was called incorrectly; wp_head is too late for the queue to be printed in order (Lesson 4.3, Mistake 1).
>
> **A** is wrong: Too early: "wp_enqueue_style was called incorrectly".
>
> **B** is wrong: For theme supports and menus, not assets.
>
> **C** is wrong: wp_head is where the queue is PRINTED (at priority 8); enqueueing on it is too late or out of order.

Q11

A theme has `styles/autumn.json` with a title and a palette, and no `blockTypes`. What is it?

A) A style variation the site owner can switch on for the whole site

B) A section style for Group blocks

C) A child theme of the theme

D) A pattern

> **Why:** A JSON file in styles/ without blockTypes is a style variation: it overlays the theme's theme.json for the whole site when chosen in Styles (it saves as user global styles). With blockTypes it is a section (block style) variation (Lesson 3.3).
> **B** is wrong: Section styles name the block types they apply to.
> **C** is wrong: A child theme is its own folder with Template: in style.css (Lesson 3.4).
> **D** is wrong: Patterns are PHP (or HTML) files in patterns/ with a Slug header.

Q12

pagesco's theme.json has the palette color oxblood, #7a2e2e. What does this print?

```
wp eval 'echo wp_get_global_stylesheet( array( "variables" ) ), "\n";' \
    | grep -o -- '--wp--preset--color--oxblood: [^;]*'
```

`--wp--preset--color--oxblood: #7a2e2e`

> **Why:** Every preset becomes a CSS custom property, `--wp--preset--color--<slug>`; `var:preset|color|oxblood` in theme.json and the has-oxblood-* classes point at it (Lesson 3.2). That is why slugs name ROLES: a variation changes the value, never the name.

PREDICT, THEN RUN

P1

Predict the output, then run it.

```
wp eval 'echo wp_get_theme( "pagesco-tt5-child" )->get( "Template" ), "\n";'
```

> twentytwentyfive

Why: The child's style.css names its parent FOLDER in the Template header (Lesson 3.4); WP_Theme reads it as `Template`.

P2

Predict the output, then run it.

```
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8080/page/2/
```

> 404

Why: pagesco shows ten posts per page (the Reading setting) and the site has four posts, so there is no page 2: WordPress answers 404. On Fernhill (three per page) or Brightwater (two) it exists.

P3

Predict the output, then run it.

```
wp eval '$q = new WP_Query( array( "post_type" => "page", "orderby" => "title", "or
echo implode( ", ", wp_list_pluck( $q->posts, "post_title" ) ), "\n";'
```

> About Us, Contact, Events, Sample Page

Why: A WP_Query of pages with no post_status finds only published pages: the Privacy Policy is a draft. `orderby` title sorts alphabetically; posts_per_page defaults to the Reading setting (ten).

P4

Predict the output, then run it.

```
wp eval 'echo esc_html( "<b>Fernhill</b> & friends" ), "\n";'
```

```
&lt;b&gt;Fernhill&lt;/b&gt; &amp; friends
```

Why: esc_html() turns `<`, `>` and `&` into entities. The browser shows the text exactly as written; that is the point, even for your own strings (a translation might contain anything).

LAB REFERENCE SOLUTIONS

Every output below is real: the lab files run exactly as `labs/check_labs.py` runs them, on a course site at the end of Lesson 4.3 (WordPress 7.1.3, PHP 8.5), pagesco active.

REVIEW LAB: LABS/MODULE-08/P1-REVIEW/REQUEST_MAP_STARTER.SH → REQUEST_MAP_SOLUTION.SH

As shipped:

```
/  200  TODO
/book-club-dune/  200  TODO
```

The solution prints:

```
/  200  home blog
/page/2/  404  error404
/book-club-dune/  200  single single-post
/about/  200  page-template-default page
/category/news/  200  archive category category-news
/?s=dune  200  search search-results
/no-such-path/  404  error404
```

Reading it: body classes are WordPress's answer to "what is this request?" (the query flags is_home, is_single, is_page, is_category, is_search, is_404), printed by body_class() in every theme. The hierarchy then looks for a template for that answer. `/page/2/` is a 404 on pagesco because ten posts per page leave no page 2; `/about/` adds `page-template-default` because no page template is assigned.

HANDOUT: LABS/MODULE-08/P1-REVIEW/DEBUG_QUERY_REVIEW.PHP (DELIBERATELY BROKEN)

As shipped:

```
On the screen: About Us
Latest news:
- Hello world!
- Book club: Dune
News posts in all:
First title again: Warning: Undefined variable $pagesco_posts in /var/www/html/debug_qu
Warning: Trying to access array offset on null in /var/www/html/debug_query_review.php
Warning: Attempt to read property "post_title" on null in /var/www/html/debug_query_rev

On the screen after the list: Book club: Dune
[new lines in wp-content/debug.log]
[26-Oct-2026 12:39:33 UTC] PHP Warning:  Undefined variable $pagesco_posts in /var/www/
[26-Oct-2026 12:39:33 UTC] PHP Warning:  Trying to access array offset on null in /var/
[26-Oct-2026 12:39:33 UTC] PHP Warning:  Attempt to read property "post_title" on null
```

1. **Silent, line 23:** `'category' => 'news'` is not a WP_Query argument, so it is ignored and the list shows the two newest posts of ANY category (Hello world! is Uncategorized). The argument is `category_name` (the slug) or `cat` (the ID): Lesson 4.2, Mistake 4.
2. **Silent, line 34:** `found_post` is a typo for `found_posts`. WP_Query answers unknown properties through its magic `__get()` with nothing, so no warning: the line just prints an empty count.
3. **Loud, line 35:** `$pagesco_posts` was never defined; the results are in `$pagesco_news->posts`.
4. **Silent, no line:** no `wp_reset_postdata()` after the loop, so the current post is still the last post of the list (Book club: Dune) instead of the page on the screen (About Us). In a template it moves everything that follows: the Brightwater assessment's Task 5.

Fixed (all four), it prints:

```
On the screen: About Us
Latest news:
- Book club: Dune
- Staff picks for the season
News posts in all: 3
First title again: Book club: Dune
On the screen after the list: About Us
```

THE PART 1 PROJECT: FERNHILL

Brief, rubric, scoring guidance and answer notes: `final-assessment.html`, section Part 1 Project. The learner's brief is `labs/module-08/p1-project/BRIEF.md`. The self-check with the starter, as handed out in Session 7:

```
Success: Switched to 'Fernhill' theme.
Fernhill self-check (Part 1 project, BRIEF.md)
TODO  1  Theme header: Fernhill 1.0.0, text domain fernhill, requires WordPress 7.1 and
OK    2  A block theme with a valid theme.json, version 3
TODO  3  Palette: base, contrast, primary, secondary, accent (in that order); the defau
TODO  4  Layout: content 680px, wide 1100px
TODO  5  Templates: index, front-page, single, page, 404 (missing: front-page, single,
TODO  6  Template parts: header (area header) and footer (area footer)
TODO  7  Patterns: fernhill/hero and fernhill/volunteer-call in the fernhill category;
TODO  8  Style variation: Autumn
TODO  9  Front page: header, main, the hero heading, the volunteer heading, footer
TODO 10  Stylesheet: fernhill-style, assets/css/fernhill.css?ver=1.0.0
TODO 11  Main query: 3 posts on the front page, 1 on /page/2/ (now: 4 and 0)
TODO 12  404: status 404 and the heading "This path is overgrown"
Checks passed: 1 of 12
```

With the model solution `fernhill_solution/` (never handed out before the feedback):

```
Success: Switched to 'Fernhill' theme.
Fernhill self-check (Part 1 project, BRIEF.md)
OK    1  Theme header: Fernhill 1.0.0, text domain fernhill, requires WordPress 7.1 and
OK    2  A block theme with a valid theme.json, version 3
OK    3  Palette: base, contrast, primary, secondary, accent (in that order); the defau
OK    4  Layout: content 680px, wide 1100px
OK    5  Templates: index, front-page, single, page, 404
OK    6  Template parts: header (area header) and footer (area footer)
OK    7  Patterns: fernhill/hero and fernhill/volunteer-call in the fernhill category;
OK    8  Style variation: Autumn
OK    9  Front page: header, main, the hero heading, the volunteer heading, footer
OK   10  Stylesheet: fernhill-style, assets/css/fernhill.css?ver=1.0.0
OK   11  Main query: 3 posts on the front page, 1 on /page/2/ (now: 3 and 1)
OK   12  404: status 404 and the heading "This path is overgrown"
Checks passed: 12 of 12
```

THE PART 1 ASSESSMENT: BRIGHTWATER

Tasks, rubric, scoring guidance, the traps and the completion rule: `final-assessment.html`, section Part 1 Assessment. The learner's brief is `labs/module-08/p1-assessment/BRIEF.md`. The self-check with the starter:

```
Success: Switched to 'Brightwater' theme.
Brightwater self-check (Part 1 assessment, BRIEF.md)
OK    1  A classic theme: Brightwater, text domain brightwater
TODO  2  T1 Title: title-tag support, exactly one <title> (now: 1)
TODO  3  T1 wp_body_open(): the store notice band from mu-plugins/pagesco-site.php appe
TODO  4  T2 Stylesheet: brightwater-style, style.css?ver=1.0.0
TODO  5  T1 Menu: the primary location shows About Us, Events, Contact in the header na
TODO  6  T3 Blog home: 2 posts with linked h2 titles, a link to page 2 (now: 0 posts, (
TODO  7  T3 /page/2/: status 200 and the other 2 posts
TODO  8  T4 Search: the term is printed as text (&lt;b&gt;), never as HTML
OK    9  T5 Related news on /book-club-dune/: Staff picks, then Welcome
TODO 10  T5 Post navigation on /book-club-dune/: previous Staff picks, next Hello worlc
Stretch: footer widget area not registered
Checks passed: 2 of 10
```

With the model solution `brightwater_solution/`:

```
Success: Switched to 'Brightwater' theme.
Brightwater self-check (Part 1 assessment, BRIEF.md)
OK    1  A classic theme: Brightwater, text domain brightwater
OK    2  T1 Title: title-tag support, exactly one <title> (now: 1)
OK    3  T1 wp_body_open(): the store notice band from mu-plugins/pagesco-site.php appe
OK    4  T2 Stylesheet: brightwater-style, style.css?ver=1.0.0
OK    5  T1 Menu: the primary location shows About Us, Events, Contact in the header na
OK    6  T3 Blog home: 2 posts with linked h2 titles, a link to page 2 (now: 2 posts, 2
OK    7  T3 /page/2/: status 200 and the other 2 posts
OK    8  T4 Search: the term is printed as text (&lt;b&gt;), never as HTML
OK    9  T5 Related news on /book-club-dune/: Staff picks, then Welcome
OK   10  T5 Post navigation on /book-club-dune/: previous Staff picks, next Hello worlc
Stretch: footer widget area registered
Checks passed: 10 of 10
```

EXIT TICKET

1. **pre_get_posts:** a callback in functions.php, guarded (not admin, `$query->is_main_query()`, `$query->is_home()`), sets posts_per_page to 3. The Query Loop inherits the main query, so pagination and /page/2/ work; a second query in the template would not page, and the main query would still run.
2. **esc_html_e():** it makes the string translatable with the theme's text domain and escapes it as it prints; a translation is data the theme did not write. It is also the habit that keeps a later dynamic value safe.
3. **Own answer:** use the "review" answers to choose Session 9's Part 1 refresher topics.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 A (home.html).** Mixes up "shows posts" with "is the posts page". front-page wins for the front page whatever it shows.
- **Q2 BAx.** Assumes the order of add_filter() calls is the order of running; priority decides, then the order of adding.
- **Q5 B or D.** Blames caching. Ask: where does the Site Editor save? (The database: Lesson 3.2.)
- **Q6 A.** Trusts the code to do what it says. The guard is the whole point; that is also why the self-check cannot catch the missing guard and the rubric reads the code.
- **Q9** `<h2>Book club: Dune</h2>`**.** the_ functions echo; the get_ functions return (Lesson 4.1).
- **Q10 C (wp_head).** Remembers that styles print in the head. Enqueue on wp_enqueue_scripts; WordPress prints the queue in wp_head.

Part 2: Plugins to Launch

Sessions 9 to 18. Modules 5 to 9: the pagesco-books plugin (custom post types, security, settings, custom tables), custom blocks with the Interactivity API, the REST API and custom endpoints, external APIs and cron, debugging, testing, performance, internationalization and deployment, ending with the final project: the Pages & Co. website, launched.

INSTRUCTOR KIT · ANSWER KEY

Session 9: Your First Plugin and Custom Post Types

Answers and reasoning for every Session 9 item: the Part 1 refresher, the Part 2 check, both self-checks, the predict-then-run items, the lab reference outputs, the lesson pages' own quizzes and the exit ticket.

Session 9 | 12 self-check items | Confidential

START HERE · PART 1 REFRESHER

Not graded. Each answer says where Part 2 uses it.

R1

The store notice of Lesson 2.3 lives in which file, and which of its three hooks is a filter?

`wp-content/mu-plugins/pagesco-site.php`; `pagesco_store_notice` is the filter (the text), `pagesco_before_store_notice` and `pagesco_store_notice_end` are actions. Lesson 5.1 hooks a plugin feature into the filter and the end action.

R2

Load, query, template: in which of the three do plugins load, and which action tells them WordPress is ready to register things?

Load (wp-settings.php): must-use plugins, plugins, `plugins_loaded`, the theme, then `init`. pagesco-books starts its features on `plugins_loaded`; post types register on `init`.

R3

What makes `pagesco` a block theme, as `wp_is_block_theme()` sees it?

A `templates/index.html` (Lesson 3.1); its `theme.json` holds the design. Part 2 never edits the theme's looks: books get their template in Module 6.

R4

Why does a `pre_get_posts` callback check `is_admin()` and `$query->is_main_query()` first?

Because the action fires for every query, in the admin too: without the guards it would change the Books screen, widgets and REST lists (Lesson 4.2). The book archive at /books/ is a main query again.

R5

A feature should survive the next redesign. Theme or plugin?

A plugin (Lessons 3.4 and 4.3: the theme is the looks, the plugin is what the shop does). That rule is why Part 2 builds pagesco-books.

START HERE · THE PART 2 CHECK

`check_environment.sh --part2` prints every Session 1 line and then three more. A learner who is ready for Sessions 9 and 10 usually sees `OK` for Composer and the site, and the note `CHECK no Node 24 on this computer (v20.20.2): use the node:24 image as Lesson 6.1 shows (docker run ... node:24 npm ...)` (verdict `READY WITH NOTES`). The FIX lines and what to do:

- `FIX the composer:2 image does not run: docker pull composer:2 (Lesson 1.3)`: Docker cannot run the composer:2 image (offline, or not pulled yet): `docker pull composer:2`, on a good connection.
- `FIX no Node 24 and no node:24 image: install Node 24 LTS (nvm install 24) before Module 6`: neither Node 24 nor the image: not a problem for Sessions 9 and 10; fix it before Session 11 (Lesson 6.1).
- `FIX the site is not where Part 2 starts (active theme 'twentytwentyfive'): finish Lesson 4.3, or import the Part 2 starting site (the course download pagesco-part2-start.zip: its README.txt)` (the active theme is named): the site is not the Part 2 starting site; import `pagesco-part2-start.zip` (its README.txt; the setup guide's Part 2 section), or finish Lesson 4.3.

5.1 · YOUR FIRST PLUGIN: SELF-CHECK

Q1

A client's server runs PHP 8.1. pagesco-books says `Requires PHP: 8.2`. What happens when the administrator clicks Activate?

A) The plugin activates, and the first page that uses PHP 8.2 syntax dies with a fatal error

B) WordPress refuses to activate it and says which PHP version the plugin requires

C) The plugin activates with the features that need PHP 8.2 switched off

D) Composer's platform_check.php deletes the vendor folder

> **Why:** The requirement lines are read before a line of the plugin runs; activation fails with "Current PHP version ... does not meet minimum requirements" (Lesson 5.1, Exercise 1). Without the header, PHP 8.2 syntax on an old server would have been a fatal error instead.
> **A** is wrong: That is what happens WITHOUT the header line.
> **C** is wrong: WordPress knows nothing about your features; it refuses the whole plugin.
> **D** is wrong: platform_check.php stops with a message on an old PHP; it never deletes anything, and WordPress refuses earlier anyway.

Q2

On the course site after Lesson 5.1, what does this print?

```
wp eval 'echo PAGESCO_BOOKS_DIR, "\n";'
```

`/var/www/html/wp-content/plugins/pagesco-books/`

> **Why:** `plugin_dir_path( __FILE__ )` returns the folder of the main file WITH a trailing slash, as the container sees it (your `wordpress/wp-content/plugins/pagesco-books/`). The trailing slash is why the main file can write `PAGESCO_BOOKS_DIR . 'vendor/autoload.php'` with no slash in between.

Q3

A learner creates `src/Admin/BookColumns.php` but writes `namespace PagesCo\Books;` at the top of it, and lists `Admin\BookColumns::class` in feature_classes(). What happens on the next request?

A) Nothing: PSR-4 only looks at the file name

B) Composer must be run again before the class is found

C) A fatal error: Class `PagesCo\Books\Admin\BookColumns` not found

D) WordPress shows a "called incorrectly" notice and skips the feature

> **Why:** The autoloader finds and includes src/Admin/BookColumns.php, but that file declares PagesCo\Books\BookColumns, so the class boot() asked for still does not exist: the same fatal as the project's Step 3. The namespace line must match the folder: PagesCo\Books\Admin for src/Admin/.
> **A** is wrong: PSR-4 builds the file name from the class name, then expects the file to declare exactly that class.
> **B** is wrong: dump-autoload is only needed when composer.json's autoload section changes.
> **D** is wrong: WordPress is not involved: this is PHP's own class loading, and it is fatal.

Q4

A site owner deactivates pagesco-books for five minutes to find a conflict. Which of these must the deactivation callback NOT do?

A) Delete the Bookseller role and the plugin's options

B) Clear the plugin's scheduled events

C) Flush the rewrite rules

D) Nothing at all (an empty callback)

> **Why:** Deactivation is a pause: it undoes what is temporary (scheduled events, rewrite rules) and keeps the owner's data. Deleting data is uninstall.php's job, and only when the plugin is deleted (Lesson 5.1, the life-cycle table).
> **B** is wrong: Clearing scheduled events is exactly a deactivation job (Lesson 7.3).
> **C** is wrong: Lesson 5.2's deactivate() does this so /books/ stops answering while the plugin is off.
> **D** is wrong: An empty callback is fine: Lesson 5.1's first Activator::deactivate() was one.

Q5

On the course site after Lesson 5.1, what does the second command print, and why?

```
wp plugin uninstall pagesco-books --deactivate --skip-delete > /dev/null
wp eval 'var_dump( get_role( "pagesco_bookseller" ) );'
```

`NULL` : uninstall.php removed the role, so get_role() finds nothing

> **Why:** `--skip-delete` runs the uninstall procedure (uninstall.php) without deleting the files: the safe way to test cleanup. Afterwards `wp plugin activate pagesco-books` re-creates the role, because activation runs Activator::activate() again.

Q6

Why must register_activation_hook() be called at the top level of the main file, not inside a plugins_loaded callback?

A) Because hooks registered on plugins_loaded run twice

B) Because the activation hook needs the autoloader, which loads later

C) Because WordPress only reads the main file's header at the top level

D) Because on the activation request plugins_loaded has already fired before WordPress includes the new plugin's file

> **Why:** WordPress activates a plugin by including its main file during an admin (or WP-CLI) request that has already loaded everything else. A plugins_loaded callback added then never runs, so the activation hook is never registered, and nothing warns you (Lesson 5.1, Common Mistakes 3).
>
> **A** is wrong: Callbacks run once per firing; plugins_loaded fires once per request.
>
> **B** is wrong: The autoloader is required at the top of the main file, before the hooks are registered.
>
> **C** is wrong: The header is read without running the file at all; it has nothing to do with where hooks are registered.

5.1 · YOUR FIRST PLUGIN: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
wp plugin deactivate pagesco-books
curl -s http://localhost:8080/ | grep -o '<div class="pagesco-notice">.*</div>'
wp plugin activate pagesco-books
```

```
Plugin 'pagesco-books' deactivated.
Success: Deactivated 1 of 1 plugins.
<div class="pagesco-notice">Free shipping on orders over $35.</div>
Plugin 'pagesco-books' activated.
Success: Activated 1 of 1 plugins.
```

Why: With pagesco-books off, StoreNotice is never created, so the notice shows only the must-use plugin's own text: the split Lesson 5.1's Step 2 set up. The last command switches it back on.

P2

Predict the output, then run it.

```
wp eval 'var_dump( class_exists( "PagesCo\Books\StoreNotice", false ), class_exists
```

```
bool(true)
bool(true)
```

Why: Both true: boot() ran on plugins_loaded and created StoreNotice, which made the autoloader include src/StoreNotice.php. `false` as the second argument only says "do not autoload just for this question"; the class is already there.

5.2 · CUSTOM POST TYPES, TAXONOMIES & META: SELF-CHECK

Q7

BookPostType registers `'has_archive' => 'books'` and `'rewrite' => array( 'slug' => 'books' )`. Which address lists all books?

A) `/book/`

B) `/books/`

C) `/?post_type=books`

D) `/wp-json/wp/v2/book`

> **Why:** has_archive gives the archive its slug; the rewrite slug is the first part of each single book's address (/books/dune/). After a flush the rule books/?$ maps to index.php?post_type=book (Lesson 5.2's `wp rewrite list`).
> **A** is wrong: That would be the default if rewrite and has_archive were just true: the type's NAME, book.
> **C** is wrong: The query variable is the type's name, book, not the archive slug.
> **D** is wrong: The REST route uses rest_base, books: /wp-json/wp/v2/books.

Q8

The Great Gatsby (book 33) is out of stock; pagesco_in_stock is registered as a boolean. What does this print on the finished Lesson 5.2 site?

```
wp eval 'var_export( get_post_meta( 33, "pagesco_in_stock", true ) ); echo "\n";'
```

`''` (an empty string)

> **Why:** Meta values are stored as strings: true is saved as '1', false as ''. get_post_meta() returns the stored string; only the REST API (and block bindings) cast it back to the registered type, which is why /wp/v2/books/33 says false. Compare a value with '' or cast it with (bool), as BookColumns does.

Q9

The shop wants a page /publisher/penguin/ listing every Penguin book, and filters by publisher on the Books screen. How should the publisher be stored?

A) As a taxonomy on the book type

B) As a meta key, pagesco_publisher

C) As an option

D) In the post content of each book

> **Why:** Many books share one publisher, and the shop wants to browse by it: that is a taxonomy, with its own archive at /publisher/<slug>/ and an admin column for free. The lesson's test: would you want a page listing everything with that value? (Lesson 5.2 FAQ.)
> **B** is wrong: Meta suits a fact that belongs to one item and that you read rather than browse, such as a price.
> **C** is wrong: An option is one site-wide value, not one per book.
> **D** is wrong: Content cannot be filtered, queried or listed by value.

Q10

"To be safe", a developer calls `flush_rewrite_rules()` on init. What is wrong with that?

A) Nothing: it is the recommended place

B) The rules are flushed before the post type is registered, so books 404

C) WordPress refuses to flush on init and logs a notice

D) It rewrites the whole rules option on every page load: a needless, slow database write

> **Why:** Rebuilding the rules is expensive, which is why WordPress keeps them in an option and only rebuilds them when something flushes. Flush once, in the activator, after registering the type and taxonomy; deactivation flushes again (Lesson 5.2, Common Mistakes 1).
> **A** is wrong: The handbook and the lesson both say: on activation, never on every request.
> **B** is wrong: Registration at the default priority runs first and the URLs work; the cost is the problem.
> **C** is wrong: WordPress lets you; it just costs a write per request.

Q11

Genre 8 is Fantasy. What does this print on the finished Lesson 5.2 site?

```
curl -s 'http://localhost:8080/wp-json/wp/v2/books?genres=8&_fields=title.rendered&
```

```
[{"title":{"rendered":"The Hobbit"}},{"title":{"rendered":"Lord of the Rings"}}]
```

Why: The genre taxonomy is registered with rest_base genres, so the books route accepts `genres=8` as a filter (term IDs, not slugs); `_fields` trims the response. Two books are filed under Fantasy (the term list's count).

Q12

A WP-CLI seeding script creates a book with wp_insert_post() and must give it two genres. Which reliably works?

A) `'tax_input' => array( 'genre' => $genres )` in the post array

B) A 'genres' key in the post array

C) `wp_set_object_terms( $id, $genres, 'genre' )` after the insert

D) `update_post_meta( $id, 'genre', $genres )`

Why: tax_input is checked against the current user's assign_terms capability, and a script runs as no user, so the genres are silently dropped (Lesson 5.2, Common Mistakes 2). wp_set_object_terms() sets them without a capability check.
A is wrong: It works in the editor, as a logged-in user, and silently does nothing in a script.
B is wrong: wp_insert_post() ignores keys it does not know.
D is wrong: That stores a meta value named genre; the taxonomy never hears of it.

5.2 · CUSTOM POST TYPES, TAXONOMIES & META: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8080/genre/fantasy/
```

```
200
```

Why: 200: the seeding created the Fantasy term, and the activator's flush gave /genre/<slug>/ its rule. Before the seeding it was a 404 for lack of a term, not of a rule.

P4

Predict the output, then run it.

```
wp eval 'var_dump( get_post_meta( 30, "pagesco_pages", true ),
    get_post_meta( 30, "pagesco_pages" ) );'
```

```
string(3) "412"
array(1) {
  [0]=>
  string(3) "412"
}
```

Why: A string both times: the database stores meta as text, and get_post_meta() hands back what is stored (the single value, or an array of all values without `true`). The REST API casts it to an integer because the key is registered as one.

LAB REFERENCE SOLUTIONS

Every output below is real: the lab files run exactly as `labs/check_labs.py` runs them, on the site each declares (`after:lesson_13` = the Part 2 starting site, `after:lesson_14` = after Lesson 5.1, `after:lesson_15` = after Lesson 5.2), with WordPress 7.1.3, PHP 8.5, MySQL 9.7 and WP-CLI 2.12. Paths show `/var/www/html/...`, which is the learner's `wordpress/` folder.

- LAB 5.1-A (GUIDED): LABS/MODULE-09/L5_1-FIRST-PLUGIN/GIFT_WRAP_STARTER/ → GIFT_WRAP_SOLUTION/ WITH TEST_GIFT_WRAP.SH

The test with the starter installed and activated:

```
Plugin 'pagesco-gift-wrap' activated.
Success: Activated 1 of 1 plugins.
header: requires WordPress TODO, PHP TODO
direct request: 500
stored price: (not stored)
notice:
<div class="pagesco-notice">Free shipping on orders over $35. Open until 9 p.m. on Thu
Deactivating 'pagesco-gift-wrap'...
Plugin 'pagesco-gift-wrap' deactivated.
Ran uninstall procedure for 'pagesco-gift-wrap' plugin without deleting.
Success: Uninstalled 1 of 1 plugins.
after uninstall: (not stored)
```

With the solution:

```
Plugin 'pagesco-gift-wrap' activated.
Success: Activated 1 of 1 plugins.
header: requires WordPress 6.8, PHP 8.2
direct request: 200
stored price: 2.50
notice:
<div class="pagesco-notice">Free shipping on orders over $35. Open until 9 p.m. on Thu
Deactivating 'pagesco-gift-wrap'...
Plugin 'pagesco-gift-wrap' deactivated.
Ran uninstall procedure for 'pagesco-gift-wrap' plugin without deleting.
Success: Uninstalled 1 of 1 plugins.
after uninstall: (not stored)
```

What each TODO shows: (1) the requirement lines are data WordPress reads without running the file (get_plugin_data()); (2) the guard turns a direct request from a 500 (the file ran without WordPress and died on add_filter(); the fatal went to the web server log, not debug.log) into a quiet 200; (3) the activation hook at the top level, with `__FILE__`, is what made activation store the price; (4) `get_option()` returns false when nothing is stored, so check before you print; (5) uninstall.php with its own guard deletes the option, which deactivation never does. Accept `add_option()` or `update_option()` in the activator; `add_option()` is the better choice because it never overwrites a price the shop changed.

- LAB 5.1-B: LABS/MODULE-09/L5_1-FIRST-PLUGIN/PSR4_PATHS_STARTER.PHP → PSR4_PATHS_SOLUTION.PHP

As shipped:

```
PagesCo\Books\Plugin                TODO
PagesCo\Books\Admin\SettingsPage    TODO
PagesCo\Books\Rest\BooksController  TODO
PagesCo\Shop\Cart                   TODO
pagesco\books\plugin                TODO
```

The solution prints:

```
PagesCo\Books\Plugin                src/Plugin.php
PagesCo\Books\Admin\SettingsPage    src/Admin/SettingsPage.php
PagesCo\Books\Rest\BooksController  src/Rest/BooksController.php
PagesCo\Shop\Cart                   shop/src/Cart.php
pagesco\books\plugin                (no rule matches: not autoloaded)
```

The point of the last line: PHP compares class names without case, but an autoloader compares strings and builds a FILE name, and Linux file names are case-sensitive. That is the project's Step 3 in one line. Composer's real ClassLoader does exactly this, plus a cache.

- HANDOUT: LABS/MODULE-09/L5_1-FIRST-PLUGIN/DEBUG_PLUGIN_AUDIT.PHP (DELIBERATELY BROKEN)

As shipped, PHP prints:

```
Warning: Undefined array key "RequiresPhp" in /var/www/html/debug_plugin_audit.php on l
Akismet Anti-spam: Spam Protection inactive needs PHP 7.4  TOO OLD for this plugin
Warning: Undefined array key "inactive" in /var/www/html/debug_plugin_audit.php on line
Warning: Undefined array key "RequiresPhp" in /var/www/html/debug_plugin_audit.php on l
Hello Dolly  inactive needs PHP 7.4  TOO OLD for this plugin
2 plugin(s) installed, 2 inactive
```

1. **Warning, line 20:** `'RequiresPhp'` is not a key of get_plugins()'s data; the key is `'RequiresPHP'` (array keys are case-sensitive: the theme of the whole lesson). With the key missing every plugin falls back to 7.4.
2. **Warning, line 23:** `++$pagesco_totals[ $pagesco_status ]` on an empty array: the first increment of each key warns. Start with `array( 'active' => 0, 'inactive' => 0 )`; the totals line then also has its key.
3. **Silent, line 21:** `version_compare( PHP_VERSION, $pagesco_needs, '<' )` is true when this server is OLDER than the requirement, so every plugin is called TOO OLD on PHP 8.5. The question is `'>='`. This is the check WordPress makes on activation (Lesson 5.1, Exercise 1).

Fixed (the three changes), it prints, without a message:

```
Akismet Anti-spam: Spam Protection inactive needs PHP 7.2  runs here
Hello Dolly  inactive needs PHP 7.4  runs here
2 plugin(s) installed, 2 inactive
```

Akismet's header asks for PHP 7.2; Hello Dolly declares none, so the script's 7.4 default applies. Both run here; neither is active on the Part 2 starting site.

- LAB 5.2-A (GUIDED): LABS/MODULE-09/L5_2-CUSTOM-POST-TYPES/BOOK_QUOTES_STARTER/ → BOOK_QUOTES_SOLUTION/ WITH TEST_BOOK_QUOTES.SH

The test with the starter:

```
Plugin 'pagesco-quotes' activated.
Success: Activated 1 of 1 plugins.
archive /quotes/: 404
REST /wp/v2/quotes:
{"code":"rest_no_route","message":"No route was found matching the URL and request meth
quotes left: 0
```

With the solution:

```
Plugin 'pagesco-quotes' activated.
Success: Activated 1 of 1 plugins.
archive /quotes/: 200
REST /wp/v2/quotes:
[{"title":{"rendered":"A truth universally acknowledged"},"meta":{"pagesco_quote_source
quotes left: 0
```

Reading it: (1) `public` plus `has_archive => 'quotes'` and the rewrite slug give /quotes/; with `has_archive => true` the archive would be /pagesco_quote/; (2) `show_in_rest` plus `rest_base` give /wp/v2/quotes (without rest_base the route is the type name); (3) the meta key needs `show_in_rest` to appear under meta, and supports `custom-fields` (already there) to reach REST at all; (4) the activator calls `pagesco_quotes_register()` before `flush_rewrite_rules()`: on the activation request init has already run without this plugin. A learner whose activator only flushes still sees a 404, which is the lesson's warning box made real. The test deletes its quote, so the count at the end is 0 either way.

- LAB 5.2-B (HOMEWORK): LABS/MODULE-09/L5_2-CUSTOM-POST-TYPES/SHELF_REPORT_STARTER.PHP → SHELF_REPORT_SOLUTION.PHP

As shipped (on the finished Lesson 5.2 site):

```
Fiction:         11
Science Fiction: 3
Classic:         6
Fantasy:         2
Dystopian:       3
Cheapest in stock: TODO
Average price: TODO
Published before 1950: TODO
```

The solution prints:

```
Fiction:         11
Science Fiction: 3
Classic:         6
Fantasy:         2
Dystopian:       3
Cheapest in stock: Pride and Prejudice ($7.99)
Average price: $10.76
Published before 1950: 6
```

Sticking points: in-stock is stored as `'1'` (true) and `''` (false), so the meta_query compares with `'1'`; `meta_value_num` sorts prices as numbers (`meta_value` would sort them as text, the Lesson 2.2 trap); the average of the eleven prices is $10.76; `'type' => 'DATE'` makes MySQL compare the Y-m-d strings as dates (string comparison happens to agree for this format, but the type says what you mean). Six books were first published before 1950.

HANDOUT: LABS/MODULE-09/L5_2-CUSTOM-POST-TYPES/DEBUG_MEETING_TYPE.PHP (DELIBERATELY BROKEN)

As shipped, WordPress and PHP print:

```
Notice: Function register_post_type was called <strong>incorrectly</strong>. Post type
Notice: Function register_meta was called <strong>incorrectly</strong>. When registerir
type: Warning: Attempt to read property "label" on null in /var/www/html/debug_meeting_

taxonomies on meetings: (none)
meta keys: (none)
[new lines in wp-content/debug.log]
[26-Oct-2026 13:25:20 UTC] PHP Notice:  Function register_post_type was called <strong>
[26-Oct-2026 13:25:20 UTC] PHP Notice:  Function register_meta was called <strong>incor
[26-Oct-2026 13:25:20 UTC] PHP Warning:  Attempt to read property "label" on null in /v
```

1. **"called incorrectly", register_post_type:** a post type name has at most 20 characters (it is stored in a varchar(20) column); `pagesco_bookclub_meeting` has 24, so WordPress refuses it and returns a WP_Error. Use `pagesco_meeting`.
2. **Warning, line 43:** follows from bug 1: get_post_type_object() returns null for a type that was never registered. Fixing bug 1 fixes it.
3. **"called incorrectly", register_meta:** an `array` meta key shown in REST needs a schema for its items (`show_in_rest => array( 'schema' => array( 'type' => 'array', 'items' => array( 'type' => 'string' ) ) )`); without it the key is not registered at all, hence `meta keys: (none)`.
4. **Silent, line 27:** the taxonomy is attached to `pagesco_meetings` (a plural that does not exist), so `get_object_taxonomies()` finds nothing on the real type: no message, just `(none)`. The second argument of register_taxonomy() is the post type, exactly (self-check, lesson quiz Q4).

Fixed, it prints:

```
type: Meetings
taxonomies on meetings: pagesco_room
meta keys: pagesco_reading
```

The order of the lesson's feature classes (type before taxonomy before meta) is the same for a reason: each one names the one before it.

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 5.1 · YOUR FIRST PLUGIN

1. **C** A Plugin Name line in the header comment: WordPress reads the header comment without running the file, and Plugin Name is the one required field; everything else is optional, though the requirement lines are

strongly recommended.

2. **A** Once, during the request that activates the plugin: It runs once, during the request that activates the plugin, which is why one-time setup such as add_role belongs there and per-request code does not.

3. **B** src/Rest/BooksController.php: PSR-4 replaces the prefix PagesCo\Books\ with src/, turns the remaining backslashes into slashes and adds .php, with the exact same letters and case.

4. **C** In uninstall.php: uninstall.php runs only when the plugin is deleted, guarded by WP_UNINSTALL_PLUGIN. Deactivation is a pause and must keep the data.

5. **A** Nothing; the autoloader finds the new class by its name: Nothing: the PSR-4 autoloader finds new classes by name. composer dump-autoload is only needed when the autoload section of composer.json changes, as the project showed with StoreNotice.

LESSON 5.2 · CUSTOM POST TYPES, TAXONOMIES & META

1. **B** init: init is the action WordPress fires once it is loaded and ready for plugins to register post types, taxonomies and meta; registering earlier or later misses the editor and REST setup.

2. **A** The rewrite rules were not flushed after the type was registered: New rewrite rules are written only when flush_rewrite_rules() runs, which is why the activation hook flushes once; without it the pretty URL matches nothing.

3. **C** A leading underscore would make them protected: not writable through REST and refused by block bindings: A leading underscore marks meta as protected: hidden from the Custom Fields box, not writable through REST without an auth_callback, and refused by block bindings. The shop wants these keys editable through the API and usable in bindings, so it leaves the underscore off.

4. **B** The post type (or array of types) the taxonomy applies to: The second argument is the object type (or array of types) the taxonomy is attached to, here the book post type; that is what makes genre a taxonomy of books.

5. **C** wp_set_object_terms($id, $genres, 'genre'): wp_set_object_terms($id, $genres, 'genre') assigns terms directly, with no capability check, so it works in a script where no user is logged in.

EXIT TICKET

1. **Three files:** a class in `src/` that implements `Feature` (its hooks in `register()`); one line in `Plugin::feature_classes()`; and, if the feature stores data, a line in uninstall.php that removes it.

2. **The role:** activation creates it (Activator::activate()), deactivation keeps it (a pause), uninstall.php removes it (and the next activation creates it again).

3. **The 404:** the rewrite rules in the database were saved before the book type existed and nothing rebuilt them. `wp rewrite flush` fixes it today; the permanent fix is the activator: register the type and taxonomy, then `flush_rewrite_rules()` (and flush again in deactivate()).

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 A ("it activates, then dies").** Has not seen the header do work. Run Exercise 1 of Lesson 5.1 (Requires PHP 9.0) together.
- **Q3 B ("run Composer again").** Thinks the autoloader keeps a list of classes. Show vendor/composer/autoload_psr4.php: one line per PREFIX, not per class.
- **Q4 C or D.** Reads "what must deactivation NOT do" as "what does it do". Point at the life-cycle line on the board.
- **Q7 A (/book/).** Mixes up the type's name and its archive slug. `wp rewrite list --match=/books/`.
- **Q8 `false`.** Expects get_post_meta() to return the registered type. It returns the stored string; REST casts (P4 is the same idea).
- **Q9 B (meta).** Fine for a fact you only display; the /publisher/penguin/ page is the clue.
- **Q12 A (tax_input).** It works in the editor. The script runs as no user: Lesson 5.2's Common Mistakes 2.

INSTRUCTOR KIT · ANSWER KEY

Session 10: Security and the Settings API

Answers and reasoning for every Session 10 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs, the lesson pages' own quizzes and the exit ticket.

`Session 10` `12 self-check items` `Confidential`

START HERE · WARM-UP

The five retrieval questions on Session 9, with the answers to give.

W1

A new feature for pagesco-books: which three places does it touch?

A class in `src/` implementing `Feature` (hooks in `register()`), one line in `Plugin::feature_classes()`, and uninstall.php if it stores data.

W2

Why does `Activator::activate()` register the book type itself before it flushes?

On the activation request the plugin's own init callback has not run, so the rules would be rebuilt without the type.

W3

`get_post_meta( 33, 'pagesco_in_stock', true )` on The Great Gatsby returns what?

`''`: meta is stored as strings, false as an empty string. REST casts it to false.

W4

Where did Lesson 5.2's meta box already use a nonce?

`wp_nonce_field( 'pagesco_books_save_details', ... )` in `render()` and `wp_verify_nonce()` in `save()`: today's lesson hardens that save.

W5

Deactivating pagesco-books: what happens to the eleven books?

Nothing: they stay in wp_posts; the type is unregistered, so they leave the admin and /books/ is a 404 until it is active again.

5.3 · SECURITY IN WORDPRESS: SELF-CHECK

Q1

A book title goes into a link's tooltip: `<a href="/books/dune/" title="...">`. Which function escapes it there?

A) `esc_html()`

B) `esc_url()`

C) `esc_attr()`

D) `sanitize_text_field()`

Why: The destination decides: inside an attribute value it is esc_attr(), which also escapes the double quote that would end the attribute (Lesson 5.3, "Escape Late"). The meta box uses it on every value="…".
A is wrong: esc_html() is for text between tags; it happens to escape quotes too, but say what you mean: the attribute function.
B is wrong: esc_url() is for href and src, and would mangle a title into a URL.
D is wrong: Sanitizing cleans input on the way IN; output is escaped on the way out.

Q2

What does this print?

```
wp eval 'echo esc_html( "Pride & Prejudice <2nd ed.>" ), "\n";'
```

`Pride & Prejudice <2nd ed.>`

Why: esc_html() turns &, < and > into entities: the browser shows the text, never runs it. In a terminal you see the entities themselves, which is how you check escaping from WP-CLI (the lesson's XSS demo did the same with the Books column).

Q3

A form handler calls check_admin_referer() and it passes. What does that prove?

A) The user is allowed to make the change

B) The request came from a page this site served to this user, for this action, recently

C) The submitted values are safe to store

D) The request was not sent by a script

> **Why:** A nonce is tied to a user, an action and a few hours: it stops a forged request from another site (CSRF). Whether the user may do it is a separate question, current_user_can(); whether the values are clean is a third, sanitizing (Lesson 5.3, Common Mistakes 2).
> **A** is wrong: Any logged-in user who can open the form gets a valid nonce.
> **C** is wrong: A nonce says nothing about the values; sanitize and validate them.
> **D** is wrong: A script with the user's cookies and a fresh nonce passes too; the nonce defeats forgery from ANOTHER site.

Q4

What does this print?

```
wp eval 'global $wpdb; echo $wpdb->prepare( "SELECT ID FROM {$wpdb->posts} WHERE po
```

`SELECT ID FROM wp_posts WHERE post_title = 'Ender\'s Game'`

> **Why:** prepare() quoted the %s value and escaped the apostrophe inside it, so the title cannot end the string early: SQL injection is impossible through this placeholder. `{$wpdb->posts}` is WordPress's own value (the prefixed table name), never user input.

Q5

This handler is reached from a link on the Books screen. Which defense is missing?

```
$book_id = absint( $_GET['book'] );
check_admin_referer( 'pagesco_books_feature_' . $book_id );
update_post_meta( $book_id, 'pagesco_featured', 1 );
```

A) `current_user_can( 'edit_post', $book_id )` before the update

B) A nonce check

C) `esc_html()` on the book ID

D) `$wpdb->prepare()` around the update

> **Why:** The nonce is checked, the ID is forced to an integer, and update_post_meta() builds safe SQL itself; but nothing asks whether THIS user may edit THIS book. Database functions trust their caller (Lesson 5.3, Common Mistakes 3), so a subscriber with a valid link could feature any book.
> **B** is wrong: check_admin_referer() is the nonce check.
> **C** is wrong: Nothing is printed; esc_html() is for output.
> **D** is wrong: update_post_meta() prepares its own query.

Q6

The meta box's "First published" field receives `soon`. What keeps it out of the database?

A) `sanitize_text_field()`: it removes words that are not dates

B) `esc_attr()` when the box prints the value

C) `absint()`, which turns it into 0

D) Validating it: DateTime::createFromFormat('Y-m-d', $v) must give back exactly $v, else store ''

> **Why:** Sanitizing cleans characters; it happily stores "soon". A value that must have a shape needs validating: accept or reject. The hardened save's valid_date() helper does exactly the round trip in option D (Lesson 5.3, Exercise 1).
> **A** is wrong: sanitize_text_field() strips tags and extra whitespace, not words.
> **B** is wrong: Escaping on output is still required, but it does not stop a bad value being stored.
> **C** is wrong: A date is not an integer; absint() would store 0.

5.3 · SECURITY IN WORDPRESS: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
wp eval 'echo esc_url( "javascript:alert(1)" ) === "" ? "(empty)" : "kept", "\n";'
```

```
(empty)
```

Why: esc_url() only lets through the protocols WordPress allows (http, https, mailto and a few more); a javascript: URL becomes an empty string, so an href built from it cannot run code.

P2

Predict the output, then run it.

```
wp eval 'echo sanitize_text_field( "  Pages <b>&</b> Co.\n\tBooks  " ), "|\n";'
```

```
Pages & Co. Books|
```

Why: sanitize_text_field() strips the tags (keeping their text), turns the line break and tab into nothing it keeps, and trims the outer spaces: one clean line. The ampersand stays a plain &: sanitizing is not escaping; escape when you print it.

5.4 · ADMIN PAGES, SETTINGS API & OPTIONS: SELF-CHECK

Q7

An administrator presses Save Changes on Settings > Pages & Co. Books. Which code checks the nonce and the capability?

A) Core's own handler, wp-admin/options.php, before it calls your sanitize callback

B) Your SettingsPage::render() method

C) Your Settings::sanitize() method

D) Nothing: the Settings API has no nonce

> **Why:** The form posts to options.php, which checks the nonce settings_fields() printed (action <group>-options), current_user_can('manage_options') and the allowed options list, then runs update_option() and with it your sanitize callback (Lesson 5.4, "What Save Changes Does").
> **B** is wrong: render() only prints the form; its own capability check is defense in depth.
> **C** is wrong: The sanitize callback cleans values; by the time it runs the checks have passed.
> **D** is wrong: settings_fields() prints the nonce; options.php verifies it.

Q8

On the finished Lesson 5.4 site (the uninstall test deleted the row; nothing was saved since), what does this print?

```
wp option get pagesco_books_settings --format=json
```

```
{"notice_extra":"Open until 9 p.m. on Thursdays.","show_events_link":true,"books_per_page":6,"events_limit":5}
```

> **Why:** register_setting()'s default is handed out while the row is missing, so WP-CLI prints the defaults although wp_options has no row named pagesco_books_settings (count it: 0). Nothing is written until someone saves.

Q9

The Events link checkbox is unticked when the form is saved. What does options.php receive for it?

A) `show_events_link=0`

B) `show_events_link=false`

C) Nothing: an unticked checkbox is not sent at all

D) `show_events_link=off`

> **Why:** Browsers send only checked boxes. A sanitize callback that reads the key without a test warns (Undefined array key) on every save with the box off; `! empty( $input['show_events_link'] )` turns "missing" into false (Lesson 5.4, Common Mistakes 2).
> **A** is wrong: That would need a hidden field with the same name before the box.
> **B** is wrong: HTML forms send strings, never booleans.
> **D** is wrong: "on" is what a checkbox WITHOUT a value attribute sends when it is ticked; unticked sends nothing.

Q10

On the finished Lesson 5.4 site, what does this print, and why is the notice sentence empty?

```
wp eval 'echo wp_json_encode( sanitize_option( "pagesco_books_settings",
    array( "books_per_page" => "0", "events_limit" => "7" ) ) ), "\n";'
```

`{"notice_extra":"","show_events_link":false,"books_per_page":6,"events_limit":7}`: 0 is out of range, so books_per_page keeps the saved value (6, the default); notice_extra and show_events_link were not in the array, so they come back empty and false

> **Why:** sanitize() always returns the complete option: a missing key means empty (the form always sends every field, code does not). That is Common Mistakes 3: merge before you update, or `wp option patch update`. The rejected 0 also queued an add_settings_error() message.

Q11

Maria, an editor, opens options-general.php?page=pagesco-books and gets a 403. Which line decided that?

A) The current_user_can() check inside render()

B) The nonce in settings_fields()

C) register_setting()'s type 'object'

D) The manage_options capability passed to add_options_page()

Why: The capability argument guards both the menu item and the URL: WordPress refuses the page before your callback runs ("Sorry, you are not allowed to access this page."). The check inside render() is the second, defensive layer.
A is wrong: It would also refuse, but WordPress stops the request earlier.
B is wrong: No form is posted when a page is opened.
C is wrong: The type describes the value, for REST and sanitizing.

Q12

Which value deserves its own option instead of a key in the pagesco_books_settings array?

A) The notice sentence

B) A counter of page views, updated on every request

C) Books per list

D) Whether the notice shows the events link

Why: One array option per plugin is the usual choice: one row, one autoloaded read, one delete_option(). Split out a value that changes on every request (rewriting the whole array to bump a number is wasteful) or one that is large and rarely needed (autoload off) (Lesson 5.4, the box "One array option, or several?").
A is wrong: A setting the shop edits now and then: exactly what the array is for.
C is wrong: The same; Module 6 reads it through Settings::get().
D is wrong: The same; a boolean in the array.

5.4 · ADMIN PAGES, SETTINGS API & OPTIONS: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
wp eval 'var_dump( get_option( "pagesco_no_such_option", array() ) );'
```

```
array(0) {
}
```

Why: The option does not exist, so get_option() returns its second argument, here an empty array (without it: false). Settings::get() relies on exactly this, then fills the keys with wp_parse_args().

P4

Predict the output, then run it.

```
wp eval 'echo wp_json_encode( sanitize_option( "pagesco_books_settings", "not an ar
```

```
{"notice_extra":"","show_events_link":false,"books_per_page":6,"events_limit":5}
```

Why: The callback treats hostile input as hostile: not an array becomes an empty array, every key falls back to its cleaned value (an empty sentence, the box off) and the two numbers keep the saved values (the defaults). It returns a complete option either way.

LAB REFERENCE SOLUTIONS

Every output below is real: the lab files run exactly as `labs/check_labs.py` runs them, on the site each declares (`after:lesson_15` = after Lesson 5.2, `after:lesson_16` = after Lesson 5.3, `after:lesson_17` = after Lesson 5.4), with WordPress 7.1.3, PHP 8.5, MySQL 9.7 and WP-CLI 2.12. Paths show `/var/www/html/...`, the learner's `wordpress/` folder.

- LAB 5.3-A (GUIDED): LABS/MODULE-10/L5_3-SECURITY/STAFF_NOTE_STARTER.PHP → STAFF_NOTE_SOLUTION.PHP

As shipped:

```
no token, as Maria              saved               stored: Signed copy, ask at the c
forged token, as Maria          saved               stored: Signed copy, ask at the c
real token, as Devon            saved               stored: Signed copy, ask at the c
real token, as Maria, with tag  saved               stored: Signed copy<script>alert(1
```

The solution prints:

```
no token, as Maria              refused: no token    stored: (nothing)
forged token, as Maria          refused: bad token   stored: (nothing)
real token, as Devon            refused: not allowed stored: (nothing)
real token, as Maria, with tag  saved                stored: Signed copy, ask at the c
```

The order matters: the cheap structural check first (is there a token at all?), then the nonce, then the capability for THIS book, then the values. Devon carries a real token (he could open the form) and is still refused: a nonce is not a permission. sanitize_text_field() removed the script tag on the way in; any place that prints the note still escapes it on the way out. Accept `wp_verify_nonce()` with or without sanitize_key(); the lesson's save uses sanitize_key(wp_unslash()) on the real $_POST.

LAB 5.3-B: LABS/MODULE-10/L5_3-SECURITY/PREPARED_SEARCH_STARTER.PHP → PREPARED_SEARCH_SOLUTION.PHP

As shipped:

```
SELECT COUNT(*) FROM wp_postmeta WHERE meta_key = 'pagesco_price' AND meta_value + 0 <
=> 5 books under $9.99
TODO
TODO
```

The solution prints:

```
SELECT COUNT(*) FROM wp_postmeta WHERE meta_key = 'pagesco_price' AND meta_value + 0 <
=> 5 books under $9.99
SELECT p.post_title FROM wp_posts p INNER JOIN wp_postmeta m ON m.post_id = p.ID WHERE
=> Lord of the Rings, The Hobbit
SELECT p.post_title FROM wp_posts p INNER JOIN wp_postmeta m ON m.post_id = p.ID WHERE
=> 0 rows
```

What to point at: `%f` printed 9.990000 (a number, never a string that could carry SQL); `esc_like()` before the `%` wildcards are added, so a word containing % or _ cannot widen the search; in the hostile line the apostrophes are backslash-escaped inside the quoted string, so `OR '1'='1` is just text to compare and nothing matches. Without `remove_placeholder_escape()` the printed SQL shows a long random string where each literal % is: prepare() protects them until the query runs.

HANDOUT: LABS/MODULE-10/L5_3-SECURITY/DEBUG_BOOK_SEARCH.PHP (DELIBERATELY BROKEN)

As shipped, WordPress prints:

```
Under $9: 4 book(s): Pride and Prejudice, Animal Farm, The Great Gatsby, The Catcher in
Notice: Function wpdb::prepare was called <strong>incorrectly</strong>. The query argu
Notice: Function wpdb::prepare was called <strong>incorrectly</strong>. The query does
In stock (whole catalog): 9
Notice: Function wpdb::prepare was called <strong>incorrectly</strong>. The query does
Cheapest: $9
```

1. **Silent, line 15:** `absint( '9.99' )` is 9: an integer sanitizer for a decimal price, so the first query asks for books under $9 and finds four instead of five. Use `(float)` (and `%f`, which the query already has). The wrong sanitizer is as bad as none.
2. **"called incorrectly" twice, line 27:** the in-stock query has no placeholder at all but is given an argument. WordPress says so ("must have a placeholder", then "the correct number of placeholders (0)"). A query with no values needs no prepare(): run it directly.
3. **"called incorrectly", line 35:** two placeholders, one argument (the price is missing). prepare() refuses and returns nothing, and `$wpdb->get_var( null )` then hands back a value of the PREVIOUS query (9, the in-stock count): a loud bug with a silent consequence.

Fixed (the three changes), it prints, without a message:

```
Under $9.99: 5 book(s): Pride and Prejudice, Animal Farm, The Great Gatsby, The Catcher
In stock (whole catalog): 9
Cheapest: $7.99
```

LAB 5.4-A (GUIDED): LABS/MODULE-10/L5_4-SETTINGS-API/NEWSLETTER_SETTINGS_STARTER/ → NEWSLETTER_SETTINGS_SOLUTION/ WITH TEST_NEWSLETTER_SETTINGS.SH

The test with the starter's must-use plugin:

```
rows: 1
<label for="pagesco-lab-signup-text">Sign-up text</label>
options.php answered: 302
Settings saved.
stored: {"signup_text":"<b>Join<\/b> our list","frequency":"daily"}
as maria: 403
```

With the solution's:

```
rows: 3
<label for="pagesco-lab-signup-text">Sign-up text</label>
<label for="pagesco-lab-frequency">Frequency</label>
<label for="pagesco-lab-show-form">Sign-up form</label>
options.php answered: 302
Frequency must be weekly or monthly.
stored: {"signup_text":"Join our list","frequency":"monthly","show_form":false}
as maria: 403
```

Reading it: the starter has no sanitize callback (TODO 1) and a callback that returns `$input` (TODO 2), so the tags and the impossible frequency were stored exactly as sent, and the unticked box simply vanished from the value. The solution's callback returns a complete array every time: the tags stripped, `daily` refused with an error and the saved `monthly` kept, the missing box as false. The two extra rows are TODO 3. Maria's 403 comes from `manage_options` in add_options_page(), in both versions. The test deletes the option at the end, so the learner's site is unchanged.

- LAB 5.4-B (HOMEWORK): LABS/MODULE-10/L5_4-SETTINGS-API/SETTINGS_PROBE_STARTER.PHP → SETTINGS_PROBE_SOLUTION.PHP

As shipped (on the finished Lesson 5.4 site):

```
group pagesco_books, type object, sanitized by sanitize()
TODO
rows in wp_options: TODO
filters: TODO
```

The solution prints:

```
group pagesco_books, type object, sanitized by sanitize()
{"notice_extra":"Signed copies this week.","show_events_link":false,"books_per_page":6,
books_per_page: Books per list must be between 1 and 24.
rows in wp_options: 0
sanitize_option filter: yes, default_option filter: yes
```

What it shows: sanitize_option() is the function options.php calls, so the result is exactly what Save Changes would store: the tags stripped, the box off (not sent), 30 refused with its message and 6 kept, 3 accepted. Nothing was stored (0 rows). And register_setting() is two filters on the option's name, one for every write and one for the missing-row default.

- HANDOUT: LABS/MODULE-10/L5_4-SETTINGS-API/DEBUG_NEWSLETTER_SANITIZE.PHP (DELIBERATELY BROKEN)

As shipped, WordPress and PHP print:

```
Notice: Function register_setting was called <strong>incorrectly</strong>. When registe
box ticked: {"signup_text":"<b>Join<\/b> our list","frequency":"weekly","show_form":"1"
box unticked: Warning: Undefined array key "show_form" in /var/www/html/debug_newslette
{"signup_text":"Join our list","frequency":"daily"}
[new lines in wp-content/debug.log]
[26-Oct-2026 13:35:54 UTC] PHP Notice:  Function register_setting was called <strong>ir
[26-Oct-2026 13:35:54 UTC] PHP Warning:  Undefined array key "show_form" in /var/www/ht
```

1. **"called incorrectly", line 33:** a setting of type `array` shown in REST needs a schema for its items. This setting is an associative array: type `object` (as Lesson 5.4's Settings uses), or no REST.
2. **Warning, line 24:** `$input['show_form']` on the form with the box unticked: the key was not sent. `! empty( $input['show_form'] )`.
3. **Silent, line 26:** the callback builds `$clean` and returns `$input`. Every cleaning step is thrown away: the tags stay, `daily` stays, the box stays the string "1". Return the cleaned value (and build it from scratch, not from a copy of the input, so unknown keys cannot ride along).

Fixed, it prints:

```
box ticked: {"signup_text":"Join our list","frequency":"weekly","show_form":true}
box unticked: {"signup_text":"Join our list","frequency":"monthly","show_form":false}
```

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 5.3 · SECURITY IN WORDPRESS

1. **B** At the point of output, for its context: Escape late: at the point of output, with the function for that context (esc_html, esc_attr, esc_url), because you cannot be sure where a stored value has been.
2. **A** Any logged-in user, regardless of role, can perform the action: A valid nonce only proves the request came from this user's genuine page; any logged-in user, whatever their role, gets one. The capability check decides whether they are allowed to act.
3. **C** $wpdb->prepare($sql, $value): The prepare() method inserts each value through a typed placeholder (%s, %d, %f, %i), quoting and escaping it so it cannot break out of the query.
4. **B** An autosave carries no form fields and would wipe the saved values: save_post fires on autosaves and revisions, which carry no meta-box fields; without the guard an autosave would overwrite the saved values with empty ones.
5. **C** No; you must check the capability yourself before calling it: No. wp_insert_post() and update_post_meta() trust the caller; you must call current_user_can() yourself before writing. The REST API and block editor add that check, but your own handlers must too.

1. **A** settings_fields('pagesco_books') and do_settings_sections('pagesco-books'): settings_fields takes the option group from register_setting, which names the nonce and the allowed options list; do_settings_sections takes the page slug the sections and fields were added to.

2. **C** 7, and the page shows the error: 0 fails the range check, so the callback queues the error and puts the saved value, 7, back. The other fields are still stored; the page shows the error instead of Settings saved.

3. **B** A 403: Sorry, you are not allowed to access this page: add_options_page was given manage_options, which editors lack, so WordPress hides the menu item and answers the URL with a 403 Sorry, you are not allowed to access this page.

4. **C** So keys added in a later version still have a value on sites that saved an older array: A site that saved the settings before a new key existed has an array without that key. Merging with the defaults gives every key a value, so code can rely on it.

5. **A** Delete it, because it only makes sense with the plugin: The option belongs to the plugin and is useless without it, so uninstall deletes it. Deactivation keeps it, so a reactivated plugin finds its settings again.

EXIT TICKET

1. **Four checks:** genuine (the nonce: check_admin_referer() or wp_verify_nonce()), allowed (current_user_can() for this object), clean (sanitize, and validate what must have a shape), and safe output (escape for the context); with SQL by hand, every value through $wpdb->prepare().

2. **options.php:** check_admin_referer('<group>-options'), current_user_can('manage_options') (filterable), the allowed options list built from register_setting(), then update_option() for each option of the group, which runs sanitize_option() and so your callback; then a redirect back with settings-updated=true, and the page prints "Settings saved." or the queued errors.

3. **A refused value:** store the old (saved) value, never nonsense, and call add_settings_error($option, $code, $message); settings_errors() prints it above the form on the next page load.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 A (esc_html).** Works by accident here; ask what function the meta box uses in value="…" and why.
- **Q3 A ("the user is allowed").** The most important confusion of the lesson. Run the lesson's Common Mistakes 2 again.
- **Q5 B ("a nonce check").** Did not recognize check_admin_referer() as the nonce check; put the two functions side by side on the board.
- **Q6 A.** Expects sanitizing to judge values. Exercise 1's validator makes the difference visible.

- **Q7 B or C.** Thinks the plugin's code saves the form. Show the form's action attribute: options.php.
- **Q9 A.** Expects HTML to send a zero. Show the request with the box unticked (the test's own post has no such field).
- **Q11 A.** True as a second layer; the 403 comes before render() runs (the page never prints).

INSTRUCTOR KIT · ANSWER KEY

Session 11: Custom Tables, Shortcodes and Block Setup

Answers and reasoning for every Session 11 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs, the lesson pages' own quizzes and the exit ticket.

Session 11 | 12 self-check items | Confidential

START HERE · WARM-UP

W1

A handler that changes data: the four checks, in order?

Genuine (nonce), allowed (`current_user_can()` for this object), sanitize and validate, escape on output; SQL by hand through `$wpdb->prepare()` .

W2

`settings_fields()` takes the group or the page slug?

The group (`pagesco_books`), the first argument of `register_setting()` ; `do_settings_sections()` takes the slug.

W3

What does an unticked checkbox send?

Nothing: missing means off (`! empty( ... )`).

W4

`$wpdb->prepare()` : which placeholder for a table name?

`%i` (WordPress 6.2+), today's lesson builds a whole table with it.

W5

What does uninstall.php remove, and what does it keep?

The plugin's own data (the settings option, the role); the shop's books stay. Today the events table joins the "remove" list.

5.5 · SHORTCODES, $WPDB & CUSTOM TABLES: SELF-CHECK

Q1

What does this print?

```
wp eval 'echo wp_json_encode( shortcode_atts( array( "limit" => 5, "location" => ""
    array( "limit" => "3", "color" => "red" ), "pagesco_events" ) ), "\n";'
```

`{"limit":"3","location":""}`

Why: shortcode_atts() keeps exactly the keys of the defaults: limit takes the given value, as the STRING "3" (attributes are always strings: absint() it before use), location keeps its default, and color, never declared, is dropped. The third argument creates the shortcode_atts_pagesco_events filter.

Q2

Which of these do you get for free with a post type, but must build yourself for a custom table?

A) A REST route, admin screens, revisions and export

B) Fast queries on an indexed column

C) A schema you control column by column

D) JOINs with wp_posts

Why: That is the price of a table (Lesson 5.5, "When a Custom Table Is Right"): the events got a shortcode now, a REST route in Lesson 7.2, and their cleanup in uninstall.php, all written by hand. In return the table has typed columns and an index on starts_at.

B is wrong: That is what the table GIVES you (KEY starts_at); meta queries sort a string column.

C is wrong: Also a benefit of the table, and a duty: you write the migrations.

D is wrong: Any table can be joined; the wishlist lab joins wp_posts.

Q3

On the finished Lesson 5.5 site, what does this print?

```
wp eval 'global $wpdb; echo $wpdb->prepare( "SELECT title FROM %i WHERE location =
    PagesCo\Books\EventsTable::name(), "Café" ), "\n";'
```

`SELECT title FROM `wp_pagesco_events` WHERE location = 'Café'`

> **Why:** %i quotes an identifier with backticks (WordPress 6.2+), %s quotes and escapes a value. The table name comes from EventsTable::name(), built from $wpdb->prefix, never a hard-coded wp_.

Q4

EventsTable::add() calls $wpdb->insert(). Where does it find the new row's ID?

A) In the return value of insert()

B) `$wpdb->insert_id`

C) `$wpdb->last_query`

D) Only with a second query, SELECT LAST_INSERT_ID()

> **Why:** insert() returns the number of rows inserted (1) or false; the auto-increment ID of the new row is in $wpdb->insert_id right after it (the lesson's add() returns it, or 0 when the insert failed).
> **A** is wrong: The return value is a count (or false), not an ID.
> **C** is wrong: last_query is the SQL that ran, useful for debugging.
> **D** is wrong: Not needed: wpdb reads it for you.

Q5

Why does EventsTable check the schema version on every request (maybe_upgrade() on init), and not only on activation?

A) Because dbDelta() must run on every request

B) Because the activation hook cannot create tables

C) Because a plugin update replaces the files but does not run the activation hook again

D) Because the version option is not autoloaded

> **Why:** An update ships a higher DB_VERSION and a new schema(); the next request sees the stored version is lower and runs install() once. The check costs one autoloaded get_option() (Lesson 5.5, Project Step 1: no reactivation).
> **A** is wrong: dbDelta() runs only when the stored version is older than the code.
> **B** is wrong: Activation does create it: Activator::activate() calls install().
> **D** is wrong: update_option() autoloads it, which is why the check is cheap.

Q6

On the finished Lesson 5.5 site, what does this print?

```
wp eval 'echo do_shortcode( "[pagesco_events location=\"Reading corner\" limit=\"1\
```

```
<ul class="pagesco-events"><li class="pagesco-event"><strong>Book club: Dune</strong> <time datetime="2027-11-04T19:00">Thursday, November 4, 7:00 pm</time> <span class="pagesco-event__where">Reading corner, 20 seats</span></li></ul>
```

> **Why:** location narrows the list (an exact match, through a %s placeholder in EventsTable::find()), limit caps it; the dates come from mysql2date() in the shop's local time. Two events are in the Reading corner; the first by date is the Dune book club.

5.5 · SHORTCODES, $WPDB & CUSTOM TABLES: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
wp eval 'echo do_shortcode( "[pagesco_book id=\"14\"]" ) === "" ? "(empty)" : "a ca
```

```
(empty)
```

Why: Page 14 is a page, not a published book, so render() returns an empty string and the tag disappears: a visitor never sees an error because an editor mistyped an ID.

P2

Predict the output, then run it.

```
wp eval 'global $wpdb; echo $wpdb->get_var( "SELECT COUNT(*) FROM " . PagesCo\Books
```

```
6
```

Why: Six: the seeding script added six events after the uninstall test had recreated the table (IDs 1 to 6).

6.1 · BLOCK DEVELOPMENT SETUP: SELF-CHECK

Q7

Which file in build/blocks/book-card/ has no source file of its own in src/?

A) `block.json`

B) `index.js`

C) `style-index.css`

D) `index.asset.php`, the dependency list and version hash

> **Why:** The build writes index.asset.php itself: the WordPress script handles index.js needs (react-jsx-runtime, wp-block-editor, wp-blocks, wp-i18n) and a hash of its content for cache busting. block.json is copied; index.js and style-index.css are compiled from the sources.
>
> **A** is wrong: block.json is copied from src/blocks/book-card/block.json.
>
> **B** is wrong: index.js is compiled from index.js, edit.js and save.js.
>
> **C** is wrong: style-index.css is compiled from style.scss.

Q8

On the finished Lesson 6.1 site, what does this print?

```
wp eval 'echo implode( ", ", WP_Block_Type_Registry::get_instance()
    ->get_registered( "pagesco/book-card" )->style_handles ), "\n";'
```

`pagesco-book-card-style`

> **Why:** Registration turned block.json's "style": "file:./style-index.css" into a registered style with a handle derived from the block name. That handle is what a block theme prints (inlined) on pages that contain the block, and only there.

Q9

You worked all afternoon with npm start running. What do you run before you commit or deploy?

A) `npm run build` : the minified production files, without .map files

B) `npm install` again

C) `npx @wordpress/create-block` again

D) Nothing: npm start already wrote build/

> **Why:** npm start writes a development build (not minified, with source maps) and watches for changes; npm run build rewrites build/ with the small production files (Lesson 6.1, "While you work").
> **B** is wrong: npm install only installs the tools; it builds nothing.
> **C** is wrong: create-block scaffolds a NEW block.
> **D** is wrong: It did, but the development version, with .map files that should not ship.

Q10

Posts already contain the Book Card. A developer renames the block from pagesco/book-card to pagesco/staff-pick. What happens to those posts?

A) WordPress renames the block in every post automatically

B) Nothing visible: names are cosmetic

C) Their stored comments still say wp:pagesco/book-card, a block that no longer exists: the editor cannot handle them

D) The posts are deleted

> **Why:** The name is stored in every post that uses the block (), so it is a permanent identifier: never rename it once content exists (Lesson 6.1, Reading block.json). Change the title instead; it is only a label.
> **A** is wrong: Nothing rewrites stored content for you.
> **B** is wrong: The name is the block's identity; the title is the cosmetic part.
> **D** is wrong: The content stays; only the editor no longer knows the block.

Q11

On the finished Lesson 6.1 site, what does this print, and why not 4?

```
grep -c "'wp-" wp-content/plugins/pagesco-books/build/blocks/book-card/index.asset.
```

`3`: wp-block-editor, wp-blocks and wp-i18n; the fourth dependency, react-jsx-runtime, does not start with wp-

> **Why:** The build replaced every @wordpress/* import with the matching global script handle and listed them; WordPress loads them before index.js. That is why the block's own file is about a kilobyte.

Q12

What does `npx @wordpress/create-block@latest book-card --no-plugin --target-dir=src/blocks/book-card` NOT create?

A) A block.json

B) A plugin PHP file, a package.json and node_modules/

C) An edit.js and a save.js

D) A view.js

> **Why:** --no-plugin writes only the block's own files into the target folder: block.json, index.js, edit.js, save.js, view.js and the two SCSS files. The plugin, its package.json and the npm install are yours (Lesson 6.1, Building with @wordpress/scripts).
>
> **A** is wrong: block.json is the first file it writes.
>
> **C** is wrong: Both are scaffolded (edit for the editor, save for the stored HTML).
>
> **D** is wrong: It writes view.js too; the project deletes it.

6.1 · BLOCK DEVELOPMENT SETUP: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
wp eval 'echo implode( ", ", array_keys( require WP_PLUGIN_DIR . "/pagesco-books/bu
```

```
book-card
```

Why: The manifest is plain PHP that returns an array keyed by the block's folder in build/blocks: one block, book-card.

P4

Predict the output, then run it.

```
curl -s http://localhost:8080/ | grep -c 'pagesco-book-card-style'
```

```
0
```

Why: 0 (and grep exits 1): block themes print a block's styles only on pages that contain the block. The home page has no Book Card.

LAB REFERENCE SOLUTIONS

Every output below is real: the lab files run exactly as `labs/check_labs.py` runs them, on the site each declares (`after:lesson_17` = after Lesson 5.4, `after:lesson_18` = after Lesson 5.5).

- LAB 5.5-A (GUIDED): LABS/MODULE-11/L5_5-CUSTOM-TABLES/WISHLIST_TABLE_STARTER.PHP → WISHLIST_TABLE_SOLUTION.PHP

As shipped:

```
1. TODO
2. TODO
3. TODO
4. TODO
5. TODO
```

The solution prints:

```
Created table wp_pagesco_lab_wishlist
rows: 4, last ID 4
Dune: 2 reader(s)
Fahrenheit 451: 1 reader(s)
The Hobbit: 1 reader(s)
updated: 1 row(s)
tables left: 0
```

dbDelta() returned one message (it created the table); a second CREATE would return an empty array. The JOIN puts both table names through %i; the update has a data format and a where format; DROP TABLE goes through prepare() too, and information_schema proves the site is as it was.

- LAB 5.5-B: LABS/MODULE-11/L5_5-CUSTOM-TABLES/OPEN_HOURS_STARTER/ → OPEN_HOURS_SOLUTION/ WITH TEST_OPEN_HOURS.SH

The test with the starter (the list is ECHOED: it appears before the quotes, and the callback returned ''):

```
[pagesco_open_hours day="saturday"] => <ul class="pagesco-hours"><li>Monday: 9:00-18:0(
[pagesco_open_hours day="SATURDAY"] => <ul class="pagesco-hours"><li>Monday: 9:00-18:0(
[pagesco_open_hours day="holiday"] => <ul class="pagesco-hours"><li>Monday: 9:00-18:00<
[pagesco_open_hours] => <ul class="pagesco-hours"><li>Monday: 9:00-18:00</li><li>Tuesda
```

With the solution:

```
[pagesco_open_hours day="saturday"] => '<span class="pagesco-hours">Saturday: 10:00-16:
[pagesco_open_hours day="SATURDAY"] => '<span class="pagesco-hours">Saturday: 10:00-16:
[pagesco_open_hours day="holiday"] => ''
[pagesco_open_hours] => '<ul class="pagesco-hours"><li>Monday: 9:00-18:00</li><li>Tuesd
```

shortcode_atts() gives day its default; sanitize_key() lowercases SATURDAY; an unknown day returns '' instead of an error; the whole list is built as a string and returned.

- HANDOUT: LABS/MODULE-11/L5_5-CUSTOM-TABLES/DEBUG_EVENTS_REPORT.PHP (DELIBERATELY BROKEN)

As shipped:

```
Notice: Function wpdb::prepare was called <strong>incorrectly</strong>. The query argu
Warning: Undefined property: stdClass::$capacity in /var/www/html/debug_events_report.p
2027-11-04 19:00:00  Book club: Dune            0 seats
2027-11-06 10:30:00  Story time: The Hobbit     0 seats
2027-11-11 18:30:00  An evening with Ana Ruiz   0 seats
Café: 2 seats
Reading corner: 2 seats
Children's corner: 1 seats
Front of the shop: 1 seats
```

1. **"called incorrectly", line 19:** prepare() with no placeholder at all. The table name is interpolated and LIMIT 3 is literal: pass both through %i and %d (or drop prepare()).
2. **Warning, line 22:** the SELECT never asked for capacity, so the row object has no such property and every event shows 0 seats. Name every column you use.
3. **Silent, line 26:** COUNT(capacity) counts rows (2, 2, 1, 1), SUM(capacity) adds seats. No message, a wrong report.

Fixed, it prints:

```
2027-11-04 19:00:00  Book club: Dune            20 seats
2027-11-06 10:30:00  Story time: The Hobbit     15 seats
2027-11-11 18:30:00  An evening with Ana Ruiz   40 seats
Café: 70 seats
Reading corner: 40 seats
Children's corner: 15 seats
Front of the shop: 0 seats
```

- LAB 6.1-A (GUIDED): LABS/MODULE-11/L6_1-BLOCK-SETUP/SHELF_LABEL_STARTER/ → SHELF_LABEL_SOLUTION/ WITH TEST_SHELF_LABEL.SH

The test with the starter:

```
Plugin 'pagesco-shelf-label' activated.
Success: Activated 1 of 1 plugins.
pagesco/shelf-label: not registered
front end: 0 stylesheet(s)
<p class="wp-block-pagesco-shelf-label">Staff picks</p>
```

With the solution:

```
Plugin 'pagesco-shelf-label' activated.
Success: Activated 1 of 1 plugins.
Shelf Label (text, icon tag)
keywords: shelf, sign
handles: pagesco-shelf-label-editor-script, pagesco-shelf-label-style
front end: 1 stylesheet(s)
<p class="wp-block-pagesco-shelf-label">Staff picks</p>
```

Unregistered, the saved HTML still renders (it is in the post) but no style is printed. TODO 1 is the Blocks feature's call with __DIR__; the handles come from the block name. The keywords line needs TODO 2's rebuild: WordPress reads the manifest in build/, never src/.

- LAB 6.1-B: LABS/MODULE-11/L6_1-BLOCK-SETUP/CORE_BLOCKS_TOUR_STARTER.SH → CORE_BLOCKS_TOUR_SOLUTION.SH

As shipped:

```
1. block.json files in wp-includes/blocks:
115
2. blocks in core's manifest (blocks-json.php, the same idea as your blocks-manifest.ph
TODO
3. apiVersion of every core block:
TODO
4. core/quote's folder (where is its JavaScript?):
TODO
5. registered block types without a block.json:
TODO
```

The solution prints:

```
1. block.json files in wp-includes/blocks:
115
2. blocks in core's manifest (blocks-json.php, the same idea as your blocks-manifest.ph
115
3. apiVersion of every core block:
    115 "apiVersion": 3
4. core/quote's folder (where is its JavaScript?):
block.json         style-rtl.css      style-rtl.min.css  style.css          style.min.c
5. registered block types without a block.json:
core/post-comments
```

Core has 115 block.json files and the same number of entries in its own manifest (blocks-json.php), all apiVersion 3; core/quote's folder has styles and no JavaScript, because the editor code of every core block is one bundle (wp-includes/js/dist/block-library.js). The registry has 116 types: core/post-comments is registered from PHP alone, a legacy block.

HANDOUT: LABS/MODULE-11/L6_1-BLOCK-SETUP/DEBUG_BLOCK_REGISTRATION.PHP (DELIBERATELY BROKEN)

As shipped:

```
Notice: Function WP_Block_Type_Registry::register was called <strong>incorrectly</stror
Notice: Function WP_Block_Type_Registry::register was called <strong>incorrectly</stror
pagesco/sale-badge: NOT registered
pagesco/gift-card: NOT registered
pagesco/reading-list: registered, apiVersion 2
```

1. **"called incorrectly", line 18:** block names are lowercase: pagesco/sale-badge.
2. **"called incorrectly", line 26:** a block name needs a namespace: pagesco/gift-card.
3. **Silent, line 38:** api_version 2 registers without a word from PHP; the editor logs a deprecation for apiVersion 2 or lower and does not put the block in the iframed canvas the same way. Use 3, as every core block does.

Fixed, it prints:

```
pagesco/sale-badge: registered, apiVersion 3
pagesco/gift-card: registered, apiVersion 3
pagesco/reading-list: registered, apiVersion 3
```

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 5.5 · SHORTCODES, $WPDB & CUSTOM TABLES

1. **B** At the very top of the page, above the doctype: echo prints immediately, and a block theme renders the content before it prints the head, so the HTML ends up above the doctype; the shortcode's place in the content stays empty.

2. **C** Thousands of bookings, each with the same few fields, listed by date: Many rows with the same fields, queried by those fields and not edited as content, are where a custom table beats post meta: real column types, indexes and simple SQL.

3. **A** A higher DB_VERSION, so maybe_upgrade() runs install() again: maybe_upgrade only calls dbDelta when the stored version is lower than DB_VERSION. Without raising the constant, the new statement never runs.

4. **C** {"limit":5,"location":""}: shortcode_atts compares keys exactly and keeps only declared ones, so LIMIT and color are both ignored and the defaults stay. In a real tag, the parser lowercases names before this step, which is why [pagesco_book ID=35] worked.

5. **B** $wpdb->get_results($wpdb->prepare('SELECT title FROM %i WHERE location = %s', $table, $loc)): prepare puts the table name in through %i and the visitor's value through %s, quoted and escaped, so it can never change the query.

LESSON 6.1 · BLOCK DEVELOPMENT SETUP

1. **B** build/blocks/ and its blocks-manifest.php: The Blocks feature passes build/blocks and its manifest to the registration function; src/ is only read by the build, which is why an unbuilt change never shows up.

2. **A** To add the block to the existing pagesco-books plugin instead of creating a second plugin: Without it create-block scaffolds a separate plugin with its own PHP file and build setup; --no-plugin writes only the block files, so the block lives in pagesco-books with everything else.

3. **C** It lists the WordPress scripts index.js depends on, and a version hash for caching: The build writes the script handles the file needs (wp-blocks, wp-block-editor...) and a content hash; WordPress loads those dependencies first and uses the hash as the version.

4. **B** The build has not run since the change: Registration reads build/blocks-manifest.php and build/, which still hold the old title until npm run build (or npm start) runs again.

5. **C** Only the styles from style-index.css; the HTML is already in the post: The HTML was stored when the editor saved the post; the front end only needs the shared styles, which a block theme adds (here inlined) only on pages that contain the block.

EXIT TICKET

1. **Return, not echo:** the callback runs while the content is filtered, before the page is printed; in a block theme the template is rendered before the <head>, so echoed text lands above the doctype (in a classic theme at the top of the content).
2. **DB_VERSION:** raise it; maybe_upgrade() compares it with pagesco_books_db_version on init and runs install(), whose dbDelta() adds the column. Plugin updates replace files without running activation.
3. **The path:** src/blocks/book-card → npm run build → build/blocks/book-card + blocks-manifest.php → Blocks::register_blocks() on init → registered type and handles → the editor loads index.js; visitors get the saved HTML and style-index.css.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 with limit as the number 3.** Attributes are strings: absint() them.
- **Q2 B.** Lists a benefit of tables as a cost: read the question again.
- **Q5 B.** Thinks activation cannot create tables; it does, the problem is updates.
- **Q7 A.** block.json is copied (and reformatted), not generated.
- **Q10 B.** Thinks the name is a label; the title is.

INSTRUCTOR KIT · ANSWER KEY

Session 12: Static and Dynamic Blocks

Answers and reasoning for every Session 12 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs, the lesson pages' own quizzes and the exit ticket.

Session 12 | 12 self-check items | Confidential

WARM-UP: LESSONS 5.5 AND 6.1

1. **W1.** The HTML appears at the top of the content (wherever the filter runs), not where the shortcode is: shortcodes RETURN their output. The [pagesco_events] callback returns it.
2. **W2.** Because the location comes from shortcode attributes, REST arguments (Lesson 7.2) and anyone who edits a post: every value goes through placeholders, every time.
3. **W3.** Into build/blocks/ (`blocks-manifest.php` and `book-card` on the projector site), and build/blocks/blocks-manifest.php, read by wp_register_block_types_from_metadata_collection() in the Blocks feature.
4. **W4.** None: the manifest lists every built block and the Blocks feature registers them all. Today's Book List proves it.
5. **W5.** Did you build? The editor loads build/blocks/book-card/index.js. `npm start` rebuilds on every save while you work.

The projector site's build folder (W3), and what `[pagesco_events]` returns, which proves it RETURNS its HTML (W1):

```
ls wp-content/plugins/pagesco-books/build/blocks
```

```
blocks-manifest.php  book-card
```

```
wp eval 'echo do_shortcode( "[pagesco_events limit=\"1\"]" ), "\n";'
```

```
<ul class="pagesco-events"><li class="pagesco-event"><strong>Book club: Dune</strong> <
```

6.2 · STATIC BLOCKS: ATTRIBUTES, CONTROLS & SUPPORTS: SELF-CHECK

Q1

A block attribute is declared `"url": { "type": "string", "source": "attribute", "selector": "a", "attribute": "href" }`. Where is the URL stored once the post is saved?

A) In the block comment, as JSON

B) In post meta, under the key url

C) In the saved HTML: the `href` of the first `<a>` inside the block

D) Nowhere: the editor asks for it again

> **Why:** An attribute with a source lives in the HTML that save() wrote, and the editor finds it again through the selector. Only attributes without a source go into the comment (and only when they differ from their default).
> **A** is wrong: Only source-less attributes such as headingLevel go into the comment.
> **B** is wrong: Post meta needs a binding or code of your own; "source": "meta" is the deprecated old way.
> **D** is wrong: Then every saved card would lose its link.

Q2

On the course site after Lesson 6.2: what does this print?

```
wp eval 'echo wp_json_encode( parse_blocks( "<!-- wp:pagesco/book-card {\"showBadge
```

`{"showBadge":false}`

> **Why:** parse_blocks() returns the comment's JSON only. The title Emma is sourced from the HTML, so it is not in `attrs`; showBadge is there because false differs from its default, true. A card with the badge on would print `[]`.

Q3

Cards are already on the site. Which change to the Book Card needs a new deprecated entry?

A) A new rule in style.scss that rounds the corners more

B) A new sidebar control in edit.js only

C) Wrapping the author in a `<cite>` in save()

D) A new attribute whose default makes save() print nothing new

> **Why:** Validation compares the stored HTML with what the CURRENT save() returns for the same attributes. A new element changes that HTML, so every stored card turns invalid until a deprecation describes the old save().
> **A** is wrong: CSS is not part of the stored HTML.
> **B** is wrong: edit is never compared with anything.
> **D** is wrong: Lesson 6.2's url and headingLevel were exactly that: no deprecation needed.

Q4

A designer wants the card's heading level in the toolbar that floats above the selected block, like core's Heading block. Where does the control go?

A) Inside `<BlockControls>` in edit

B) Inside `<InspectorControls>` in save

C) Inside `<InspectorControls>` in edit

D) In block.json under supports

> **Why:** BlockControls fills the block toolbar, InspectorControls the sidebar's Settings tab; both belong in edit, which is the editor's interface. save never renders controls.
> **B** is wrong: save returns the stored HTML; controls there would be saved into the post.
> **C** is wrong: That puts it in the sidebar, where the lesson's Card settings panel is.
> **D** is wrong: supports switch on core's own features (colors, spacing); there is no support for a custom control.

Q5

An editor picks the theme's Sage background for a card. What does the post store?

A) The hex color in an inline style

B) The class `has-sage-background-color` in the HTML and `"backgroundColor":"sage"` in the comment

C) A row in wp_options

D) Nothing: the theme decides at view time

> **Why:** Preset choices are saved as a slug and a class; the theme's global styles define .has-sage-background-color. Change Sage's hex value in theme.json and every sage card follows, which is why supports beat hand-made color attributes.
> **A** is wrong: Inline styles are for values that are not presets, such as a custom padding per side.
> **C** is wrong: Block settings are content: they live in post_content.
> **D** is wrong: The choice is the editor's, so it has to be stored with the card.

Q6

Why does the course write the badge text `Staff pick` as a plain string in save() instead of `__( 'Staff pick', 'pagesco-books' )`?

save() must return the same HTML for the same attributes in every language: a translated string makes cards invalid as soon as someone edits in another locale

> **Why:** Validation re-runs save() in the editor of whoever opens the post. Text in save is content: a literal, or an attribute. Translate edit's labels and placeholders freely (Common Mistake 3).

6.2 · STATIC BLOCKS: ATTRIBUTES, CONTROLS & SUPPORTS: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
wp eval 'echo wp_json_encode( parse_blocks( "<!-- wp:heading --><h2 class=\"wp-bloc
```

> `[]`

> **Why:** A heading at its default level 2 stores nothing in the comment, and its text is rich-text sourced from the h2: `[]` (an empty array as JSON). Save the same heading as an h3 and you would get `{"level":3}`.

P2

Predict the output, then run it.

```
wp eval 'echo wp_json_encode( WP_Block_Type_Registry::get_instance()->get_registere
```

```
{"type":"rich-text","source":"rich-text","selector":"p","role":"content"}
```

Why: Core declares the paragraph's text with the source `rich-text` and the selector `p`: it lives in the HTML. `role: content` marks it as the block's content (the editor's content-only editing and pattern overrides use it).

6.3 · DYNAMIC BLOCKS & BLOCK BINDINGS: SELF-CHECK

Q7

The Book List already sits on twenty pages. You change render.php so every item gets a second line. What else must you do?

A) Add a deprecated entry to the block

B) Open and update each of the twenty pages

C) Run `npm run build` (render.php is copied into build/), then nothing: every page follows on its next view

D) Bump the block's version so caches notice

Why: A dynamic block stores only its comment, so there is no stored HTML to validate. The build copies render.php into build/blocks/book-list/, the file WordPress runs; after that, the next view of every page uses it.
A is wrong: Deprecations describe old save() output; a dynamic block saves none.
B is wrong: Nothing in the pages changed: they hold only the comment.
D is wrong: The version only changes the ?ver= of the block's CSS and JS.

Q8

On the course site after Lesson 6.3 (six Classic books): what does this print?

```
wp eval 'echo substr_count( do_blocks( "<!-- wp:pagesco/book-list {\"genre\":\"clas
```

`4`

Why: do_blocks() runs render.php with the comment's attributes, exactly as a page view does. count 4 is inside the 1 to 24 clamp, and the genre filter is a tax_query on the slug, so four of the six Classic books.

Q9

In render.php the wrapper is `<ul <?php echo get_block_wrapper_attributes(); ?>>`, with no escaping function around it. Why is that safe?

A) It is not: wrap it in esc_html()

B) Block attributes are always numbers

C) Only administrators can add blocks

D) get_block_wrapper_attributes() returns attributes that are already escaped (class and style from the supports)

> **Why:** It is the PHP twin of useBlockProps.save(): it builds the class and style attributes and escapes them itself. Core's own blocks print it the same way. Everything else render.php prints is escaped where it is printed.
> **A** is wrong: esc_html() would turn the quotes into entities and break the markup.
> **B** is wrong: Attributes can be strings typed by anyone who can edit posts (genre, for example).
> **C** is wrong: Any editor or author can insert blocks; escaping is about the data, not the person.

Q10

A list item is bound to `core/post-meta` with the key `pagesco_in_stock`. What does it show on The Great Gatsby's page (sold out)?

A) Sold out

B) The fallback text of the list item

C) false

D) Nothing: an empty list item

> **Why:** core/post-meta returns the raw stored value: false arrives as an empty string, so the list item is empty (Dune's would say 1). Binding is not formatting; that is why the shop writes its own source, pagesco/book.
> **A** is wrong: That wording comes from the shop's own source, not from core/post-meta.
> **B** is wrong: The fallback shows only when the source returns null (an unknown or protected key).
> **C** is wrong: PHP's false becomes an empty string when it is printed.

Q11

You register a binding source without `uses_context`. What does a bound paragraph in single-book.html show?

A) Its fallback text, with no error

B) A PHP warning about the missing context

C) The value for the first book in the catalog

D) The value for the post being shown, as usual

> **Why:** A paragraph never asks for postId on its own. uses_context makes WordPress hand postId and postType to every block bound to the source; without it the callback sees no post, returns null, and the fallback stays: silently.
> **B** is wrong: Bindings fail silently by design.
> **C** is wrong: The callback has no post ID at all.
> **D** is wrong: Only with uses_context.

Q12

On the course site after Lesson 6.3: what does this print (book 40 is Animal Farm, published 1945-08-17)?

```php
<?php
$parsed = parse_blocks( '<!-- wp:paragraph {"metadata":{"bindings":{"content":{"sou
$block  = new WP_Block( $parsed, array( 'postId' => 40, 'postType' => 'book' ) );

echo wp_strip_all_tags( $block->render() ), "\n";
```

`First published in 1945`

> **Why:** new WP_Block($parsed, $context) renders the paragraph with a made-up context, so pagesco/book's year field runs for book 40 and replaces the fallback. It is Lesson 6.3's Exercise 1 technique, the way a PHPUnit test would check a source (Lesson 8.1).

6.3 · DYNAMIC BLOCKS & BLOCK BINDINGS: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
wp eval 'echo substr_count( do_blocks( "<!-- wp:pagesco/book-list {\"count\":30} /-
```

```
11
```

Why: render.php clamps count to 1 to 24, and the catalog has eleven books, so all eleven print. The clamp protects the query from a huge number typed into a comment by hand.

P4

Predict the output, then run it.

```
wp eval 'echo get_post_meta( 37, "pagesco_in_stock", true ) === "" ? "empty string"
```

```
empty string
```

Why: Booleans are stored as '1' and '' (the empty string), which is why core/post-meta leaves an empty list item for Gatsby and Catcher, and why Exercise 3 compares pagesco_in_stock with '1'.

LAB REFERENCE SOLUTIONS

Every output below is real: the lab files run exactly as `labs/check_labs.py` runs them, on a fresh course site at the state each file declares (Lesson 6.2's labs at `after:lesson_19`, Lesson 6.3's at `after:lesson_20`: what each lesson starts from), copied into the WordPress root and run with `wp eval-file --use-include` (PHP labs) or `sh` in the WP-CLI container (shell labs); folder labs installed and activated first. Paths show `/var/www/html/...`, which is the learner's `wordpress/` folder.

- LAB 6.2-A (GUIDED): LABS/MODULE-12/L6_2-STATIC-BLOCKS/ATTRIBUTE_SOURCES_STARTER.PHP → ATTRIBUTE_SOURCES_SOLUTION.PHP

As shipped:

```
core/heading content: TODO
core/quote content: TODO
core/button content: TODO
parse_blocks() sees: TODO
```

The solution prints:

```
core/heading content: in the HTML (rich-text, h1,h2,h3,h4,h5,h6)
core/heading level: in the comment (default 2)
core/quote value: in the HTML (html, blockquote)
core/quote citation: in the HTML (rich-text, cite)
core/button tagName: in the comment (default "a")
core/button type: in the comment (default "button")
core/button url: in the HTML (attribute, a @href)
core/button title: in the HTML (attribute, a,button @title)
core/button text: in the HTML (rich-text, a,button)
core/button linkTarget: in the HTML (attribute, a @target)
core/button rel: in the HTML (attribute, a @rel)
parse_blocks() sees:
core/heading {"level":3}
core/image {"id":42,"sizeSlug":"large"}
```

Reading it: core uses `rich-text` as a source too (the editor's rich-text value, stored as HTML like `html`); the quote's whole inner HTML is one attribute; the button's tagName and type are settings, so comment attributes with defaults. parse_blocks() returns `{"level":3}` and the image's id and size: the src, the alt and the heading's text are in the HTML only. Accept any equivalent formatting of TODOs 2 and 3; `array_key_exists()` matters because a default can be false.

- LAB 6.2-B: LABS/MODULE-12/L6_2-STATIC-BLOCKS/CARD_PRICE_STARTER/ → CARD_PRICE_SOLUTION/ WITH TEST_CARD_PRICE.SH

The test with the starter's `pagesco-card-price.php` in mu-plugins/:

```
The cards and what follows them:
<h3 class="wp-block-pagesco-book-card__title">The Hobbit</h3>
<h3 class="wp-block-pagesco-book-card__title">A book without a link</h3>
price lines: 0
```

With the solution's:

```
The cards and what follows them:
<h3 class="wp-block-pagesco-book-card__title">The Hobbit</h3>
<p class="pagesco-card-price">Today: $10.99</p>
<h3 class="wp-block-pagesco-book-card__title">A book without a link</h3>
price lines: 1
```

Reading it: `render_block_pagesco/book-card` passes the card's stored HTML; WP_HTML_Tag_Processor finds the link by class and reads its href (never a regular expression on HTML); `url_to_postid()` turns the book's address into its ID; the price is read on every view, so it is live although the card is static. The card without a link gets nothing. It is a dynamic ADDITION to a static block, a preview of Lesson 6.3's reason for dynamic blocks; the test creates and force-deletes its own post.

HANDOUT: LABS/MODULE-12/L6_2-STATIC-BLOCKS/DEBUG_CARD_REPORT.PHP (DELIBERATELY BROKEN)

As shipped, PHP prints:

```
Warning: Undefined array key "title" in /var/www/html/debug_card_report.php on line 28
Warning: Undefined array key "innerHtml" in /var/www/html/debug_card_report.php on line
Deprecated: strlen(): Passing null to parameter #1 ($string) of type string is deprecat
: h2, 0 characters of HTML
Warning: Undefined array key "title" in /var/www/html/debug_card_report.php on line 28
Warning: Undefined array key "innerHtml" in /var/www/html/debug_card_report.php on line
Deprecated: strlen(): Passing null to parameter #1 ($string) of type string is deprecat
: h2, 0 characters of HTML
```

1. **Warning, line 28:** `$block['attrs']['title']`: the title is sourced from the HTML, so it is never in the comment's attrs (Lesson 6.2, Common Mistake 1). Read it with WP_HTML_Processor (the fix below), as the lesson's Exercise 1 does.
2. **Warning + Deprecated, line 30:** `innerHtml` is spelled `innerHTML` in a parsed block; the missing key gives null, and strlen(null) is deprecated since PHP 8.1. One bug, two messages.
3. **Silent, line 29:** `?? 2`: the card's default heading level is 3 (block.json), and a card at its default stores no headingLevel. The second card is an h3 but the report says h2. Use `?? 3`, or better, the registered default.

Fixed (all three), it prints:

```
Dune: h2, 166 characters of HTML
Emma: h3, 164 characters of HTML
```

The fixed script is in the generator (blocks_S12.py, CARD_REPORT_FIXED), run on the same site.

LAB 6.3-A (GUIDED): LABS/MODULE-12/L6_3-DYNAMIC-BLOCKS/NEXT_EVENTS_STARTER/ → NEXT_EVENTS_SOLUTION/ WITH TEST_NEXT_EVENTS.PHP

With the starter's plugin installed and activated:

```
Plugin 'pagesco-next-events' activated.
Success: Activated 1 of 1 plugins.
== no attributes
<p class="wp-block-pagesco-next-events">TODO: the next events</p>
== 2 at the Café
<p class="wp-block-pagesco-next-events">TODO: the next events</p>
== rooftop
<p class="wp-block-pagesco-next-events">TODO: the next events</p>
count 99: 0 events
```

With the solution's:

```
Plugin 'pagesco-next-events' activated.
Success: Activated 1 of 1 plugins.
== no attributes
<ul class="wp-block-pagesco-next-events">
            <li><time datetime="2027-11-04T19:00">Nov 4</time> Book club: Dune</li>
            <li><time datetime="2027-11-06T10:30">Nov 6</time> Story time: The Hobbit</
            <li><time datetime="2027-11-11T18:30">Nov 11</time> An evening with Ana Ru
            <li><time datetime="2027-11-18T19:30">Nov 18</time> Poetry open mic</li>
            <li><time datetime="2027-12-02T19:00">Dec 2</time> Book club: Fahrenheit 45
    </ul>
== 2 at the Café
<ul class="wp-block-pagesco-next-events">
            <li><time datetime="2027-11-11T18:30">Nov 11</time> An evening with Ana Ru
            <li><time datetime="2027-11-18T19:30">Nov 18</time> Poetry open mic</li>
    </ul>
== rooftop
<p class="wp-block-pagesco-next-events">No events at this location yet.</p>
count 99: 6 events
```

Reading it: no count means the Events to show setting (5, the default); the location is sanitized and passed to EventsTable::find(), which prepares the SQL (Lesson 5.5); the empty case prints a sentence inside the same wrapper; count 99 is clamped to 10, and the table has six events. Common answers to accept: `?? (int) Settings::get(...)` instead of isset(); `wp_date()` is wrong here (starts_at is already local time; mysql2date() formats it as stored). Remove the plugin afterwards.

- LAB 6.3-B: LABS/MODULE-12/L6_3-DYNAMIC-BLOCKS/CORE_SOURCES_STARTER.PHP → CORE_SOURCES_SOLUTION.PHP

As shipped:

```
Dune      pagesco_author     [TODO]
Dune      pagesco_price      [TODO]
Gatsby    pagesco_in_stock   [TODO]
Dune      pagesco_in_stock   [TODO]
Dune      _edit_lock         [TODO]
Dune      pagesco_writer     [TODO]
Dune      link               [TODO]
```

The solution prints:

```
Dune      pagesco_author     [Frank Herbert]
Dune      pagesco_price      [14.99]
Gatsby    pagesco_in_stock   []
Dune      pagesco_in_stock   [1]
Dune      _edit_lock         [No data]
Dune      pagesco_writer     [No data]
Dune      link               [http://localhost:8080/books/dune/]
```

Reading it: core/post-meta returns raw values (14.99 without a dollar sign; the boolean as '' or 1); a protected key (`_edit_lock`) and an unknown key both return null, so the fallback "No data" stays, silently; core/post-data's link field gives the permalink. The same rendering with a made-up context is how the lesson's Exercise 1 tests pagesco/book.

HANDOUT: LABS/MODULE-12/L6_3-DYNAMIC-BLOCKS/DEBUG_GENRE_COUNTS.PHP (DELIBERATELY BROKEN)

As shipped, PHP prints:

```
Warning: Undefined array key "limit" in /var/www/html/debug_genre_counts.php on line 18
Warning: Undefined variable $pagesco_items in /var/www/html/debug_genre_counts.php on 1
<ul class="wp-block-pagesco-lab-genres"><li>Fiction: 11</li></ul>
<ul class="wp-block-pagesco-lab-genres"><li>Fiction: 11</li></ul>
```

1. **Warning, line 18:** `$attributes['limit']` on a block without a limit: the attribute has no default, so the key is missing (Lesson 6.3, Common Mistake 1). `isset( $attributes['limit'] ) ? ... : 3`.
2. **Warning, line 32:** `$pagesco_items .=` on a variable that was never set. Add `$pagesco_items = '';` before the loop. (PHP shows the warning once per line with this site's settings, so it appears once although both blocks hit it.)
3. **Silent, line 19:** `min( 1, max( 10, ... ) )` is the clamp written backwards: it always gives 1, so both blocks show one genre. `min( 10, max( 1, $limit ) )`.

Fixed, the two blocks print three and four genres, the biggest first:

```
<ul class="wp-block-pagesco-lab-genres"><li>Fiction: 11</li><li>Classic: 6</li><li>Sci
<ul class="wp-block-pagesco-lab-genres"><li>Fiction: 11</li><li>Classic: 6</li><li>Sci
```

Dystopian and Science Fiction both have three books; their order among themselves is the database's, so accept either. The fixed script is GENRES_FIXED in blocks_S12.py.

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 6.2 · STATIC BLOCKS: ATTRIBUTES, CONTROLS & SUPPORTS

1. **A** In the saved HTML, inside the element the selector points to: A sourced attribute lives only in the saved HTML; the editor reads it back with the selector, and the comment JSON stays free of it.
2. **B** A ToggleControl inside InspectorControls, in edit: Whatever edit renders inside InspectorControls appears in the sidebar while the block is selected; PanelBody groups it into a collapsible panel.
3. **C** Spreading useBlockProps.save() onto the wrapper in save: Supports add their classes and styles through useBlockProps.save(); a wrapper without it saves none of them.

4. **C** {"headingLevel":4}: parse_blocks returns exactly the comment JSON: the sourced title stays in the HTML, and defaults such as showBadge are not filled in.

5. **B** A deprecated entry with the previous attributes, supports and save: A deprecated entry with the old attributes, supports and save lets the editor recognize existing cards and open them as valid blocks of the new version.

LESSON 6.3 · DYNAMIC BLOCKS & BLOCK BINDINGS

1. **B** Only the block comment with its attributes: A dynamic block saves only its comment with the non-default attributes; render.php produces the HTML on every view.

2. **A** 8: count has no default, so it is missing from the comment and render.php uses Settings::get('books_per_page'), now 8.

3. **B** The paragraph's fallback text: Keys starting with an underscore are protected; core/post-meta returns null for them, so the paragraph keeps its fallback text, silently.

4. **C** Sold out: The context names book 37, the source reads pagesco_in_stock, finds it false and returns the formatted Sold out, which replaces the paragraph's content.

5. **C** When something asks for the block's variations, such as the editor: block.json names the file, and WordPress requires it lazily when the variations are requested, for example by the editor through the block-types REST route.

EXIT TICKET

1. **Title and level:** the title in the saved HTML, inside the element its selector names; the heading level in the block comment, only when it is not the default 3.

2. **render.php and attributes:** check optional attributes (isset() or a default in block.json), cast and clamp numbers, sanitize strings (sanitize_key() for a slug), and escape everything printed; start the wrapper with get_block_wrapper_attributes().

3. **A fallback on a book page:** a misspelled meta key, a key not registered with show_in_rest or starting with an underscore (core/post-meta), a custom source without uses_context, or a source that returned null for that field.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 A ("in the comment").** Thinks every attribute is JSON. Show the stored Card test post: the URL is in the href, the comment is empty.
- **Q2 with the title in it.** The same picture: parse_blocks() never reads the HTML. Run the command.

- **Q3 A or B.** Thinks validation is about files. It compares stored HTML with save(): CSS and edit are never compared.
- **Q5 A ("the hex color").** Remembers inline styles; presets are classes and slugs, so the theme can change the color later.
- **Q7 A ("a deprecation").** Carries Lesson 6.2's rule over to dynamic blocks; there is no stored HTML to validate.
- **Q9 A ("wrap it in esc_html()").** Good instinct, wrong place: the function escapes its own output, and double escaping breaks the attribute.
- **Q10 A ("Sold out").** Confuses core/post-meta with the shop's source. Show Gatsby's first template version: an empty item.

INSTRUCTOR KIT · ANSWER KEY

Session 13: The Interactivity API and Using the REST API

Answers and reasoning for every Session 13 item: the warm-up, both self-checks, the predict-then-run items, Checkpoint 2's rubric and model answer, the lab reference outputs, the lesson pages' own quizzes and the exit ticket.

Session 13 | 12 self-check items | Checkpoint 2 | Confidential

WARM-UP: LESSONS 6.2 AND 6.3

1. **W1.** Only the comment's: `{"backgroundColor":"sage"}`. Title, author, recommendation and link are in the HTML; headingLevel and showBadge are at their defaults.
2. **W2.** When content with the old HTML exists that must stay editable (a client's posts): always. While the block exists only on a test site: delete the test posts instead.
3. **W3.** render.php, on every view: no count means the Books per list setting (6), clamped to 1 to 24.
4. **W4.** `'uses_context' => array( 'postId', 'postType' )` in register_block_bindings_source(); without it the fallback stays, silently.
5. **W5.** `$10.99`: the large paragraph is bound to pagesco/book's price field, which formats pagesco_price on every view.

On the projector site (after Lesson 6.3):

```
wp eval '$b = wp_list_filter( parse_blocks( get_page_by_path( "staff-picks", OBJECT, "p
```

```
{"backgroundColor":"sage"}
```

```
curl -s http://localhost:8080/books/the-hobbit/ | grep -o 'has-large-font-size wp-block
```

```
has-large-font-size wp-block-paragraph">$10.99
```

6.4 · THE INTERACTIVITY API: SELF-CHECK

Q1

A visitor opens /find-a-book/?genre=fantasy. Where does the browser's store get state.genre's starting value from?

A) From the address bar, read by view.js

B) From the state JSON that render.php's wp_interactivity_state() put into the page

C) From localStorage

D) From a REST request the store sends on load

Why: render.php validates ?genre= and passes it to wp_interactivity_state(); WordPress prints the plain values as JSON (the script tag with the id wp-script-module-data-@wordpress/interactivity), and the runtime merges it into the store's state before anything runs.
A is wrong: view.js only WRITES the address (history.replaceState()).
C is wrong: Nothing in the block uses browser storage.
D is wrong: The Book Filter makes no requests: all books are in the HTML (Lesson 7.2 designs the REST version).

Q2

Which of these directives does the SERVER not evaluate when it renders the block?

A) `data-wp-bind--hidden="!state.isVisible"`

B) `data-wp-text="state.visibleCount"`

C) `data-wp-class--is-selected="state.isSelected"`

D) `data-wp-on--click="actions.selectGenre"`

Why: Events only exist in the browser, so data-wp-on (and watch, init, run) are browser-only. bind, class, style, text, context, each and interactive are processed on the server too, which is why the page is correct before any JavaScript runs.
A is wrong: The server writes hidden on the nine non-fantasy books.
B is wrong: The server fills the span with 2.
C is wrong: The server adds class="is-selected" to the Fantasy link.

Q3

On the course site after Lesson 6.4 (three Dystopian books of eleven): what does this print?

```
curl -s 'http://localhost:8080/find-a-book/?genre=dystopian' | grep -c '<li hidden'
```

`8`

> **Why:** All eleven books are in the HTML; the server ran data-wp-bind--hidden for each and wrote hidden on the eight that are not dystopian. curl runs no JavaScript: this is what every visitor gets first.

Q4

Each genre link must know its own slug. Where does that slug belong?

A) In the element's context, `wp_interactivity_data_wp_context( array( 'slug' => $slug ) )`

B) In the state, as one key per genre

C) In a global JavaScript variable

D) In the link's class name

> **Why:** State is shared by the whole store (one genre is selected for everyone); context is data attached to one element and visible to the directives inside it. getContext() reads it in getters and actions, wp_interactivity_get_context() in the PHP closures.
> **B** is wrong: Works, but every link's directives would need to name their own key: context is what per-element data is for.
> **C** is wrong: The runtime would not track it, so nothing would update.
> **D** is wrong: Classes are for styling; the store cannot read them as data.

Q5

An action calls event.preventDefault() to stop a link. What does WordPress 6.8 and later require for it?

A) Nothing: actions always run synchronously

B) data-wp-on-async--click instead of data-wp-on--click

C) Wrapping the action in withSyncEvent()

D) Returning false from the action

> **Why:** Since 6.8 the runtime may run actions after the browser has handled the event, too late to prevent anything. withSyncEvent() says the action needs the event right away; without it 7.1 still works but logs a deprecation warning (Common Mistake 2).
> **A** is wrong: That was true before 6.8.
> **B** is wrong: on-async is for actions that need NOTHING from the event.
> **D** is wrong: Return values of actions are ignored.

Q6

A visitor's browser blocks the Book Filter's script module, and she clicks Fantasy. What happens, and why?

The link is followed (a page load) and the server renders the filtered page: the links are real ?genre= links and render.php reads the address

> **Why:** Progressive enhancement: the HTML works on its own; JavaScript only makes it faster. It came almost for free because the state starts in PHP and the derived values have PHP closures.

6.4 · THE INTERACTIVITY API: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
curl -s 'http://localhost:8080/find-a-book/?genre=romance' | grep -o 'visibleCount"
```

> ```
> visibleCount">11
> ```

> **Why:** render.php accepts only slugs of genres that have books, so romance falls back to All books: 11. Nothing from the address reaches the page unchecked (the lesson's Exercise 1).

P2

Predict the output, then run it.

```
wp eval 'echo wp_interactivity_process_directives( "<div data-wp-interactive=\"page
```

```
<div data-wp-interactive="pagesco/x"><b data-wp-text="state.missing"></b></div>
```

Why: The server evaluates `data-wp-text` against the store's state; `state.missing` has no value, so the element's text is REPLACED by nothing: the old text is gone. No message: check the namespace and the state key when a text is empty.

7.1 · USING THE REST API: SELF-CHECK

Q7

Anonymously, `GET /wp-json/wp/v2/books/30?context=edit`. What comes back?

A) 200 with the raw title and the content as block markup

B) 404 rest_no_route

C) 401 rest_forbidden_context

D) 403 rest_cannot_edit

Why: context=edit returns raw values only to users who could edit the item. Nobody is logged in, so it is "who are you?": 401. Maria with an application password gets the raw fields; Devon, an author, 403.
A is wrong: Only for users who may edit the book.
B is wrong: The route exists; only the context is refused.
D is wrong: 403 means "I know you, and no": the request is anonymous.

Q8

On the course site (six Classic books, genre ID 7): what does this print?

```
curl -s 'http://localhost:8080/wp-json/wp/v2/books?genres=7&per_page=4&_fields=id'
```

```
X-WP-TotalPages: 2
```

Why: X-WP-TotalPages is the number of pages at this per_page: six books in pages of four are two pages. The body (sent to /dev/null here) would hold the first four; a client loops up to the header's number.

Q9

A Dashboard page sends Maria's cookie and an X-WP-Nonce header with an old, made-up value. What does the REST API answer?

A) 403 rest_cookie_invalid_nonce: the page must fetch a fresh nonce

B) 200: the cookie is enough

C) 401 rest_not_logged_in, as if no cookie had been sent

D) 500: the nonce check crashed

> **Why:** With no nonce, the cookie is ignored and the request is anonymous. With a WRONG nonce, WordPress refuses outright, so a stale page knows it must refresh (apiFetch does that by itself through the rest-nonce endpoint).
> **B** is wrong: The nonce is what protects the cookie from CSRF.
> **C** is wrong: That is the answer WITHOUT a nonce.
> **D** is wrong: A failed check is a normal error answer.

Q10

`/wp/v2/books/30?_embed=wp:term&_fields=id,title` answers without any genres. Why?

A) The book has no genres

B) _embed works only for authenticated requests

C) wp:term is not an embeddable relation

D) _fields removed _links, so there was nothing to follow, and _embedded is cut off too

> **Why:** Since 5.4 WordPress embeds only links that survive _fields. Add _links and _embedded to the list: _fields=id,title,_links,_embedded (Common Mistake 1).
> **A** is wrong: Dune is Fiction and Science Fiction.
> **B** is wrong: Public data embeds anonymously.
> **C** is wrong: The genre link says embeddable: true.

Q11

Why can you create application passwords on the course site although it runs on plain http://localhost:8080?

A) Application passwords never need HTTPS

B) WP_ENVIRONMENT_TYPE is local; elsewhere WordPress offers them only on HTTPS

C) Docker encrypts the traffic

D) Because the administrator created them

> **Why:** Basic authentication only encodes the password; on plain HTTP anyone on the way can read it. WordPress therefore offers application passwords on HTTPS sites or where the environment type is local, as compose.yaml sets it (Lesson 1.2).
> **A** is wrong: They do, outside local environments.
> **C** is wrong: The containers talk plain HTTP.
> **D** is wrong: Maria created hers with WP-CLI; the rule is about the connection.

Q12

Name two things wp.apiFetch() adds to a request that a plain fetch() on the Dashboard does not.

The REST nonce (X-WP-Nonce) for the logged-in user, and the API's root URL (also _locale=user, a JSON body and the server's error object on failure)

> **Why:** The page prints apiFetch's setup when a script depends on wp-api-fetch: createRootURLMiddleware, the nonce middleware, the nonce endpoint for refreshing. Without the nonce the request is anonymous (Common Mistake 2).

7.1 · USING THE REST API: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
curl -s 'http://localhost:8080/wp-json/wp/v2/books?search=the&per_page=1&_fields=id
```

```
X-WP-TotalPages: 4
```

> **Why:** search matches the title, content and excerpt; four books contain "the" in their title (or text), so at one per page X-WP-TotalPages is 4. The header answers "how many" without downloading the books.

P4

Predict the output, then run it.

```
curl -s 'http://localhost:8080/wp-json/wp/v2/genres/8?_fields=name,count'; echo
```

```
{"count":2,"name":"Fantasy"}
```

Why: One term by its ID, two fields: `_fields` works on every route, single items included. Fantasy is genre 8 with two books.

CHECKPOINT 2: RUBRIC AND MODEL ANSWER

Five rows, 0 to 2 points each (10 in all). Score from the learner's file (run it once on your projector site at the end of Lesson 6.3; it changes nothing) and the Part D page. **8 to 10:** on track. **5 to 7:** review the weakest row before Session 14. **Under 5:** a one-to-one before Session 15.

CRITERION	STRONG (2)	PARTIAL (1)	MISSING (0)
Part A · the query (5.2, 5.3)	One get_posts()/WP_Query: meta_query with price <= max as DECIMAL (or NUMERIC) and stock '1'; meta_key + orderby meta_value_num; output matches line A	Right books but sorted by title, or the price compared as a string, or two queries merged in PHP	No meta_query (filters in PHP after loading all books), or wrong books
Part B · safe SQL (5.5)	$wpdb->prepare() with %i for the table (or the table name from EventsTable::name()) and %s for the location; get_var(); (int); the hostile location counts 0 events and breaks nothing	Prepared, but the table name hard-coded as wp_pagesco_events, or get_results() + count()	The location concatenated into SQL, or Part B not working
Part C · blocks (6.2, 6.3)	C1: the comment built with wp_json_encode() and rendered with do_blocks(), titles read from the output; C2: parse_blocks() of the post, filtered by blockName, count and the first card's attrs	One of C1 and C2 right, or C1 calls get_posts() itself instead of rendering the block	Neither, or the card's title read from attrs (Lesson 6.2's Mistake 1)

CRITERION	STRONG (2)	PARTIAL (1)	MISSING (0)
Output and escaping	The four lines match the model answer; prices formatted with two decimals; nothing printed unescaped that came from a user (the location is only counted)	One line differs (a missing $, an extra space), or one value printed without esc_html() in a function that builds HTML	Several lines differ or the script prints a PHP message
Part D · written (Modules 5 and 6)	D1 to D3 match the model answers, each with its reason	Two right, or right answers without reasons	One or none

MODEL ANSWER: LABS/MODULE-13/CHECKPOINT-2/CHECKPOINT_SOLUTION.PHP

```php
<?php
/**
 * checkpoint_solution.php: Checkpoint 2 (Modules 5 and 6), the Weekly Picks report: MO
 * state: after:lesson_21
 * expect: A  Under $10, in stock: Pride and Prejudice ($7.99), Animal Farm ($8.49), Br
 * expect: B  Café: 2 events · Rooftop: 0 events · Reading corner'; DROP TABLE x; --: 0
 * expect: C1 Fantasy, cheapest first: The Hobbit, Lord of the Rings
 * expect: C2 Staff picks post: 1 Book Card, comment attributes {"backgroundColor":"sag
 *
 * Instructor copy. The shop wants a weekly report for its newsletter, built from the p
 * are the scored code (three functions); Part D (three written answers) is on the work
 * The main section at the end is the same in the starter, and the script changes nothi
 * Run: wp eval-file --use-include checkpoint_solution.php
 */

/**
 * Part A (Lessons 5.2, 5.3): books in stock that cost at most $max, cheapest first, as
 *
 * @param float $max The highest price.
 * @return string[]
 */
function pagesco_cp_cheap_books( float $max ): array {
    $books = get_posts(
        array(
            'post_type'      => 'book',
            'post_status'    => 'publish',
            'posts_per_page' => -1,
            'meta_key'       => 'pagesco_price',
            'orderby'        => 'meta_value_num',
            'order'          => 'ASC',
            'meta_query'     => array(
                array(
                    'key'     => 'pagesco_price',
                    'value'   => $max,
                    'compare' => '<=',
                    'type'    => 'DECIMAL(10,2)',
                ),
                array(
                    'key'   => 'pagesco_in_stock',
                    'value' => '1',
                ),
            ),
        )
    );

    $lines = array();
    foreach ( $books as $book ) {
```

```
        $lines[] = sprintf( '%s ($%s)', $book->post_title, number_format_i18n( (float)
    }

    return $lines;
}

/**
 * Part B (Lesson 5.5): how many events take place at a location: one prepared COUNT qu
 *
 * @param string $location The exact location, as typed by anyone.
 */
function pagesco_cp_events_at( string $location ): int {
    global $wpdb;

    return (int) $wpdb->get_var(
        $wpdb->prepare( 'SELECT COUNT(*) FROM %i WHERE location = %s', PagesCo\Books\Ev
    );
}

/**
 * Part C (Lessons 6.2, 6.3): C1 renders a Book List block for a genre, cheapest first,
 * C2 describes the Book Cards stored in a post.
 *
 * @param string $genre A genre slug.
 * @return string[]
 */
function pagesco_cp_list_titles( string $genre ): array {
    $attributes = array(
        'genre'   => $genre,
        'orderBy' => 'price',
    );
    $html       = do_blocks( '<!-- wp:pagesco/book-list ' . wp_json_encode( $attributes

    preg_match_all( '#<a href="[^"]*">([^<]*)</a>#', $html, $found );

    return $found[1];
}

/**
 * @param string $slug A post slug.
 * @return array{count: int, attrs: array}
 */
function pagesco_cp_cards_in( string $slug ): array {
    $post  = get_page_by_path( $slug, OBJECT, 'post' );
    $cards = array_values( wp_list_filter( parse_blocks( $post ? $post->post_content :

    return array(
        'count' => count( $cards ),
```

```
        'attrs' => $cards ? $cards[0]['attrs'] : array(),
    );
}

// ------------------------------------------------------------- main: the same in t
echo 'A  Under $10, in stock: ', esc_html( implode( ', ', pagesco_cp_cheap_books( 10.0

$pagesco_places = array( 'Café', 'Rooftop', "Reading corner'; DROP TABLE x; --" );
echo 'B  ', implode( ' · ', array_map( fn( $place ) => $place . ': ' . pagesco_cp_event

echo 'C1 Fantasy, cheapest first: ', esc_html( implode( ', ', pagesco_cp_list_titles( '

$pagesco_cards = pagesco_cp_cards_in( 'staff-picks' );
printf( "C2 Staff picks post: %d Book Card, comment attributes %s\n", $pagesco_cards['c
```

It prints (the site after Lesson 6.3):

```
A  Under $10, in stock: Pride and Prejudice ($7.99), Animal Farm ($8.49), Brave New Wor
B  Café: 2 events · Rooftop: 0 events · Reading corner'; DROP TABLE x; --: 0 events
C1 Fantasy, cheapest first: The Hobbit, Lord of the Rings
C2 Staff picks post: 1 Book Card, comment attributes {"backgroundColor":"sage"}
```

As shipped, the starter prints the four lines with TODO and -1 values and no message. Accept: WP_Query instead of get_posts(); `%i` or the table name from EventsTable::name() concatenated (it is not user input); any way of reading the titles from C1's HTML that does not query the books again. **Do not accept `'type' => 'NUMERIC'` for the price:** NUMERIC makes MySQL cast to a whole number (`CAST('10.49' AS SIGNED)` is 10), so with a maximum of 10 Fahrenheit 451 ($10.49) and The Hobbit ($10.99) slip into line A, measured on the course site; that is Partial in row 1. Without any type the comparison is between strings and finds nothing.

PART D: MODEL ANSWERS

1. **D1.** The review: static, it is content an editor writes and it should stay as written (and survive the plugin). The three cheapest: dynamic, the data (prices, stock, new books) changes elsewhere, so render.php must ask on every view; a static block would go stale like Lesson 3.3's pattern.

2. **D2.** A nonce (wp_nonce_field() in the form, check_admin_referer() or wp_verify_nonce() before saving; the Settings API does both). It stops cross-site request forgery: another site making a logged-in administrator's browser submit the form.

3. **D3.** The genre choices are real links to ?genre= addresses, and render.php reads and validates ?genre= to set the starting state, with PHP closures for the derived state, so the server renders each filtered page itself (hidden on the other books, aria-current on the link).

LAB REFERENCE SOLUTIONS

Every output below is real: the lab files run exactly as `labs/check_labs.py` runs them, on a fresh course site at the state each file declares (Lesson 6.4's labs at `after:lesson_21`, Lesson 7.1's at `after:lesson_22`), copied into the WordPress root and run with `wp eval-file --use-include` (PHP labs) or `sh` in the WP-CLI container (shell labs).

- LAB 6.4-A (GUIDED): LABS/MODULE-13/L6_4-INTERACTIVITY-API/OPEN_TODAY_STARTER.PHP → OPEN_TODAY_SOLUTION.PHP

As shipped:

```
<div data-wp-interactive="pagesco/hours"><p>TODO: today's hours</p>
<button aria-expanded="false" data-wp-on--click="actions.toggleWeek" data-wp-bind--aria
<li data-wp-context='{"day":"Friday"}'>Friday: 9:00-18:00</li>
<li data-wp-context='{"day":"Saturday"}'>Saturday: 10:00-16:00</li>
<li data-wp-context='{"day":"Sunday"}'>Sunday: closed</li>
</ul></div>
```

The solution prints:

```
<div data-wp-interactive="pagesco/hours"><p data-wp-text="state.todayText">Today (Satur
<button aria-expanded="false" data-wp-on--click="actions.toggleWeek" data-wp-bind--aria
<li aria-current="false" hidden data-wp-context='{"day":"Friday"}' data-wp-bind--hidden
<li aria-current="true" class="is-today" data-wp-context='{"day":"Saturday"}' data-wp-b
<li aria-current="false" hidden data-wp-context='{"day":"Sunday"}' data-wp-bind--hidden
</ul></div>
```

Reading it: the server filled the paragraph (data-wp-text), wrote hidden on the two days that are not today (isHidden: the week is folded), aria-current true/false and the class on Saturday. data-wp-on--click is left for the browser: the store's `toggleWeek` action (`state.showWeek = ! state.showWeek`) and two getters mirroring the closures would make the button work, the lesson's view.js pattern. Accept closures that call `wp_interactivity_state( 'pagesco/hours' )` with the namespace.

- LAB 6.4-B: LABS/MODULE-13/L6_4-INTERACTIVITY-API/EVENT_LIST_STARTER.PHP → EVENT_LIST_SOLUTION.PHP

As shipped:

```
Showing  of 5 events
(no <li> yet: TODO 1)
```

The solution prints:

```
Showing 2 of 5 events
<li data-wp-each-child="pagesco/events::state.events" hidden data-wp-bind--hidden="stat
<li data-wp-each-child="pagesco/events::state.events" hidden data-wp-bind--hidden="stat
<li data-wp-each-child="pagesco/events::state.events" data-wp-bind--hidden="state.isEls
<li data-wp-each-child="pagesco/events::state.events" data-wp-bind--hidden="state.isEls
<li data-wp-each-child="pagesco/events::state.events" hidden data-wp-bind--hidden="stat
```

Reading it: data-wp-each repeated the template once per event and marked each copy `data-wp-each-child`, so the browser's store can take over the list (and add to it: Lesson 7.2's Load more design); inside each copy the context is `{ event: ... }`; the server evaluated the derived state per item. shown counts the Café's events from the same state. The `<template>` stays in the page for the browser.

- HANDOUT: LABS/MODULE-13/L6_4-INTERACTIVITY-API/DEBUG_STOCK_BADGES.PHP (DELIBERATELY BROKEN)

As shipped, WordPress prints:

```
Notice: Function WP_Interactivity_API::evaluate was called <strong>incorrectly</strong>:
Warning: Undefined array key "instock" in /var/www/html/debug_stock_badges.php on line
<h2 data-wp-text="state.shop"></h2>
<div data-wp-interactive="pagesco/badges"><p>In stock: <span data-wp-text="state.inStoc
<li data-wp-context='{"title":"Dune","inStock":true}'>Dune <span class="badge" data-wp-
<li data-wp-context='{"title":"The Great Gatsby","inStock":false}'>The Great Gatsby <sp
<li data-wp-context='{"title":"The Catcher in the Rye","inStock":false}'>The Catcher in
```

1. **doing_it_wrong, line 38:** the `<h2>` with `data-wp-text` sits OUTSIDE the element with `data-wp-interactive`, so the directive has no namespace ("Namespace or reference path cannot be empty"). Move the heading inside the wrapper.
2. **Warning, line 30:** the context key is `inStock`; `instock` is undefined, so isSold is `! null` = true and every badge shows (Dune's too).
3. **Silent, line 33:** `wp_interactivity_state( 'pagesco/badge' )` names a store that does not exist, so the count is 0 instead of 1. Use the namespace `pagesco/badges` (or no argument inside a closure).

Fixed (all three), it prints:

```
<div data-wp-interactive="pagesco/badges"><h2 data-wp-text="state.shop">Pages &amp;amp; Co
<li data-wp-context='{"title":"Dune","inStock":true}'>Dune <span hidden class="badge" d
<li data-wp-context='{"title":"The Great Gatsby","inStock":false}'>The Great Gatsby <sp
<li data-wp-context='{"title":"The Catcher in the Rye","inStock":false}'>The Catcher in
```

The fixed script is BADGES_FIXED in blocks_S13.py, run on the same site.

- LAB 7.1-A (GUIDED): LABS/MODULE-13/L7_1-REST-API/CATALOG_QUERIES_STARTER.SH → CATALOG_QUERIES_SOLUTION.SH

As shipped:

```
Books in the catalog: 11
The Dystopian genre: TODO
Its books, by title: TODO
Search for farm: TODO
The site's pages, oldest first: TODO
Book 99: TODO
The two biggest genres: TODO
```

The solution prints:

```
Books in the catalog: 11
The Dystopian genre:
[{"id":9,"name":"Dystopian"}]
Its books, by title:
[{"slug":"1984"},{"slug":"brave-new-world"},{"slug":"fahrenheit-451"}]
Search for farm:
[{"id":40,"title":{"rendered":"Animal Farm"}}]
The site's pages, oldest first:
[{"slug":"sample-page"},{"slug":"about"},{"slug":"contact"},{"slug":"events"},{"slug":"
Book 99: 404
The two biggest genres:
[{"count":11,"name":"Fiction"},{"count":6,"name":"Classic"}]
```

Reading it: the total is a header (HEAD request, -I); the genres filter needs the ID (9), found by the slug; search matches title, content and excerpt; pages include Find a Book (Lesson 6.4) and the install's Sample Page; a missing book is a 404 with a JSON body; genres sort by count. Learners whose site lacks Find a Book are one lesson behind.

LAB 7.1-B: LABS/MODULE-13/L7_1-REST-API/WHO_MAY_STARTER.SH → WHO_MAY_SOLUTION.SH

As shipped:

```
request                anon devon  maria  admin
users/me               401  TODO   TODO   TODO
books/30?context=edit  401  TODO   TODO   TODO
books?status=draft     400  TODO   TODO   TODO
users?context=edit     401  TODO   TODO   TODO
settings               401  TODO   TODO   TODO
Lab passwords revoked.
```

The solution prints:

```
request                 anon devon  maria  admin
users/me                401  200    200    200
books/30?context=edit   401  403    200    200
books?status=draft      400  200    200    200
users?context=edit      401  403    403    200
settings                401  403    403    200
Lab passwords revoked.
```

Reading it: users/me needs any login (401 anonymously); context=edit on Maria's book: Devon is known but may not (403); anonymous `status=draft` is a 400, "Status is forbidden", because the argument itself is refused before any permission check; listing users with context=edit and reading settings need administrator capabilities (list_users, manage_options), so Maria gets 403 too. The script revokes its three passwords: a lab leaves no credentials behind.

HANDOUT: LABS/MODULE-13/L7_1-REST-API/DEBUG_STOCK_LIST.PHP (DELIBERATELY BROKEN)

As shipped, PHP prints:

```
Warning: Undefined array key "meta" in /var/www/html/debug_stock_list.php on line 21
Warning: Trying to access array offset on null in /var/www/html/debug_stock_list.php on
Warning: Array to string conversion in /var/www/html/debug_stock_list.php on line 22
Array: sold out
Warning: Undefined array key "meta" in /var/www/html/debug_stock_list.php on line 21
Warning: Trying to access array offset on null in /var/www/html/debug_stock_list.php on
Warning: Array to string conversion in /var/www/html/debug_stock_list.php on line 22
Array: sold out
Warning: Undefined array key "meta" in /var/www/html/debug_stock_list.php on line 21
Warning: Trying to access array offset on null in /var/www/html/debug_stock_list.php on
Warning: Array to string conversion in /var/www/html/debug_stock_list.php on line 22
Array: sold out
Warning: Undefined array key "meta" in /var/www/html/debug_stock_list.php on line 21
Warning: Trying to access array offset on null in /var/www/html/debug_stock_list.php on
Warning: Array to string conversion in /var/www/html/debug_stock_list.php on line 22
Array: sold out
Total in the catalog: 11
```

1. **Warnings, line 21:** `_fields=id,title` (line 17) left `meta` out of the answer, so `$book['meta']` is undefined (two warnings: the missing key, then an offset on null). Add `meta` to `_fields`.
2. **Warning, line 22:** `title` is an object `{ rendered }`: printing the array gives "Array". Use `$book['title']['rendered']`.
3. **Silent:** per_page=4 and no loop: four books listed, "Total in the catalog: 11" printed underneath. Loop over `page` up to X-WP-TotalPages (the lesson's Exercise 1).

Fixed (all three), it prints:

```
1984: in stock
Animal Farm: in stock
Brave New World: in stock
Dune: in stock
Fahrenheit 451: in stock
Lord of the Rings: in stock
Pride and Prejudice: in stock
The Catcher in the Rye: sold out
The Great Gatsby: sold out
The Hobbit: in stock
To Kill a Mockingbird: in stock
Listed 11 of 11
```

The fixed script is STOCK_LIST_FIXED in blocks_S13.py, run on the same site.

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 6.4 · THE INTERACTIVITY API

1. **A** It also builds viewScriptModule files as script modules: It adds a module build, so viewScriptModule files such as view.js are built with a view.asset.php of type module; without it they are skipped silently.
2. **C** All eleven books, nine of them with the hidden attribute: render.php prints every book, and the server evaluates data-wp-bind--hidden through the isVisible closure, so the nine others arrive with the hidden attribute.
3. **B** data-wp-text="state.visibleCount" on the span: data-wp-text sets the element's text content from a value, on the server and in the browser; here the span holds state.visibleCount.
4. **A** So the server can evaluate the directives and send correct HTML before any JavaScript runs: The server evaluates directives while rendering; with the closure it can compute isVisible per book, so the HTML is already filtered and nothing changes when the page hydrates.
5. **C** The link is followed and the server renders the fantasy books, with a page load: No store is wired to the directives, so the click follows the link and the server renders the filtered page: correct, but a full page load each time, with no message.

LESSON 7.1 · USING THE REST API

1. **B** GET /wp-json/wp/v2/books?per_page=5&_fields=id,title.rendered: per_page=5 gives five books, newest first by default, and _fields keeps only id and the rendered title.
2. **C** 401: the request is treated as anonymous, and the edit is refused: Without an X-WP-Nonce header WordPress ignores the cookie, treats the request as anonymous and refuses the edit with 401. wp.apiFetch adds the nonce.

3. **A** With an application password of a user whose role can edit books, over HTTPS: A program gets its own application password, for a user with no more rights than it needs, sent with Basic authentication over HTTPS, and revocable on its own.
4. **C** 403: Devon is authenticated, so it is not 401; his role cannot edit another author's book, so the answer is 403 Forbidden.
5. **B** 3: Eleven books in pages of four make three pages (4, 4 and 3); the header is the same on every page.

EXIT TICKET

1. **State and derived state:** the state is the selected genre (with each book's genres); derived are isVisible, isSelected and visibleCount.
2. **401** means "who are you?" (not authenticated); 403 is "I know you, and no" (authenticated, not allowed).
3. **The partner's script:** an application password of a WordPress user whose role can edit books (an editor), sent with Basic authentication over HTTPS; never the account password.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 A ("from the address bar").** Has not seen the state JSON: show the `<script id="wp-script-module-data-...">` from the demo.
- **Q2 B or C.** Thinks the server only prints HTML: run the curl for ?genre=fantasy again.
- **Q4 B ("in the state").** Works for small cases; context is per element. Redraw the three columns.
- **Q7 D (403).** Confuses 401 and 403: the request has no user at all.
- **Q9 C (401 as anonymous).** Remembers the no-nonce case; a WRONG nonce is refused outright.
- **Q10 A or B.** Blames the data or the login: it is _fields cutting the links.
- **Q11 A.** Missed the HTTPS rule: application passwords are offered only on HTTPS or a local environment.

INSTRUCTOR KIT · ANSWER KEY

Session 14: Custom Endpoints, External APIs and Cron

Answers and reasoning for every Session 14 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs, the lesson pages' own quizzes and the exit ticket.

Session 14 | 12 self-check items | Confidential

WARM-UP: LESSONS 6.4 AND 7.1

1. **W1.** `9` of eleven: the server evaluated data-wp-bind--hidden for each book (two fantasy books stay visible).
2. **W2.** It calls event.preventDefault(); since 6.8 actions may run after the event, too late to stop the link.
3. **W3.** The role: 403 means "I know you, and no" (Devon on Maria's book). 401 would be the credentials.
4. **W4.** A HEAD request (`curl -I`) and the header: `X-WP-TotalPages: 3` on the projector site.
5. **W5.** apiFetch sends Maria's cookie AND the page's wp_rest nonce; a script has neither, so it needs an application password.

On the projector site (after Lesson 7.1):

```
curl -s 'http://localhost:8080/find-a-book/?genre=fantasy' | grep -c '<li hidden'
```

```
9
```

```
curl -s -I 'http://localhost:8080/wp-json/wp/v2/books?per_page=5' | grep -i '^X-WP-Tota
```

```
X-WP-TotalPages: 3
```

7.2 · CUSTOM REST ENDPOINTS: SELF-CHECK

Q1

A route registers `'methods' => WP_REST_Server::CREATABLE`. Which HTTP method does that endpoint answer?

A) GET

B) POST

C) POST, PUT and PATCH

D) DELETE

> **Why:** READABLE is GET, CREATABLE POST, EDITABLE 'POST, PUT, PATCH', DELETABLE DELETE. The constants say what you mean; the events route uses READABLE and CREATABLE on one route.
> **A** is wrong: That is READABLE.
> **C** is wrong: That is EDITABLE.
> **D** is wrong: That is DELETABLE.

Q2

On the course site after Lesson 7.2 (Books per list = 6, eleven books): what does this print?

```
wp eval '$r = rest_do_request( new WP_REST_Request( "GET", "/pagesco/v1/books" ) );
```

`200 6 2`

> **Why:** rest_do_request() runs the route without HTTP: the same validation, permission check and callback. per_page defaults to the Books per list setting (6), so six books and two pages.

Q3

An anonymous client POSTs to /pagesco/v1/events WITHOUT a title. Which answer does it get first?

A) 400 rest_missing_callback_param: arguments are validated before the permission check

B) 401 rest_forbidden: the permission check comes first

C) 500: the callback fails on a missing title

D) 201: the event is created without a title

> **Why:** The order is validate, sanitize, permission, callback. A missing required argument stops the request at the first desk, before anyone asks who the client is (the predict item P2 shows it with rest_do_request()).
> **B** is wrong: Only once the arguments are valid.
> **C** is wrong: The callback never runs.
> **D** is wrong: required stops it.

Q4

Maria sends a bad starts_at AND a negative capacity. The answer names only starts_at. Why?

A) capacity is not validated

B) The schema stops at the first error it finds in alphabetical order

C) A negative capacity is turned into 0

D) validate_callbacks run first; the schema checks of the other arguments run in the sanitizing step, which a failed validation never reaches

> **Why:** Lesson 7.2, Step 3: starts_at has a validate_callback; capacity's minimum is checked by the schema during sanitizing. Clients should expect errors one round at a time.
> **A** is wrong: Fix the date and the capacity error comes next.
> **B** is wrong: There is no alphabetical rule.
> **C** is wrong: The schema refuses it; nothing is coerced.

Q5

Which permission_callback fits a route that DELETES events?

A) '__return_true': the callback checks the ID

B) 'is_user_logged_in'

C) A check of `current_user_can( 'edit_others_posts' )` that returns a WP_Error with rest_authorization_required_code()

D) None: DELETE routes are protected by WordPress

> **Why:** Every write needs a capability check as specific as the job; rest_authorization_required_code() gives 401 to nobody and 403 to a logged-in user who may not. The lesson's Exercise 3 reuses can_manage_events().
> **A** is wrong: Anyone on the internet could delete events.
> **B** is wrong: Any subscriber is logged in.
> **D** is wrong: WordPress protects nothing you do not check (and warns when the key is missing).

Q6

`GET /pagesco/v1/books/14` asks for the Events page (a page, not a book). Why does the answer say only "No book was found with that ID." and not "that is a page"?

An endpoint must not tell visitors what exists behind an ID they may not see: drafts, private books and other post types all get the same 404

> **Why:** Leaking existence is information disclosure. get_book() requires a published book and answers one 404 for everything else.

7.2 · CUSTOM REST ENDPOINTS: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
curl -s 'http://localhost:8080/wp-json/pagesco/v1/books?per_page=0'; echo
```

> ```
> {"code":"rest_invalid_param","message":"Invalid parameter(s): per_page","data":
> ```

> **Why:** per_page has a minimum of 1: the schema refuses 0 with 400 before the callback runs, with the same message core's routes use.

P2

Predict the output, then run it.

```
wp eval '$r = rest_do_request( new WP_REST_Request( "POST", "/pagesco/v1/events" )
```

```
400 rest_missing_callback_param
```

Why: Validation comes before the permission check: the missing required title stops the request at the first desk (400 rest_missing_callback_param), although nobody is logged in.

7.3 · EXTERNAL APIS, TRANSIENTS & WP-CRON: SELF-CHECK

Q7

The supplier's host does not resolve, and code calls `wp_remote_retrieve_body( $response )` on the WP_Error. What does it get?

A) A fatal error

B) null and a Warning

C) The error message

D) An empty string

Why: The wp_remote_retrieve_*() helpers accept a WP_Error and return '' (or 0) instead of failing: safe to call, but '' decodes to null, so check is_wp_error() first anyway.

A is wrong: That is why the helpers exist.

B is wrong: No message at all.

C is wrong: That is $response->get_error_message().

Q8

What does this print, for a transient nobody ever set?

```
wp eval 'var_dump( get_transient( "pagesco_books_nothing_here" ) );'
```

`bool(false)`

Why: get_transient() returns false for "missing or expired". That is why a cache must never STORE false (it would look missing), and why the project stores an array even for a failure.

Q9

A plugin fetches a book cover from a URL an editor types into a field. Which function?

A) `wp_safe_remote_get()` : it refuses private networks and odd ports

B) `wp_remote_get()` with a short timeout

C) `file_get_contents()`

D) `wp_remote_head()`

> **Why:** A URL from a user could point at http://192.168.1.1/ or a cloud metadata service: server-side request forgery. The safe variants refuse those before connecting; the site's own host stays allowed.
> **B** is wrong: A timeout does not stop the request reaching an internal address.
> **C** is wrong: It bypasses the HTTP API's protections and filters.
> **D** is wrong: It is just a HEAD request.

Q10

You want to run the daily sync now, while developing, without moving tomorrow's 03:00 run. What do you type?

A) `wp cron event run pagesco_books_daily_sync`

B) `wp eval 'do_action( "pagesco_books_daily_sync" );'`

C) `wp cron test`

D) `curl -s http://localhost:8080/wp-cron.php`

> **Why:** WP-CLI's event run reschedules a recurring event from now (a 03:00 job run at 10:15 moves to 10:15); firing the hook yourself runs the work and leaves the schedule alone (Lesson 7.3's warning box).
> **A** is wrong: It moves the schedule.
> **C** is wrong: It only checks the loopback.
> **D** is wrong: It runs what is DUE; tomorrow's 03:00 is not.

Q11

Where does the supplier's API key belong?

A) In the plugin's Client class, as a constant

B) In a transient, refreshed daily

C) In wp-config.php (or an environment variable it reads), on each site

D) In the REST API response, for the JavaScript

> **Why:** A key in code ends up in Git, backups and every copy of the plugin, and the live site needs a different one. `wp config set` writes the constant; the live site gets its own.
> **A** is wrong: Code is shared; secrets are not.
> **B** is wrong: Transients can vanish at any time.
> **D** is wrong: Never send secrets to browsers.

Q12

On the course site after Lesson 7.3: what does this print?

```
wp cron event list --hook=pagesco_books_daily_sync --fields=hook,recurrence
```

```
| pagesco_books_daily_sync | 1 day |
```

> **Why:** Exactly one daily event: Activator schedules it once, guarded by wp_next_scheduled(), and deactivation clears it. Two rows here would be Common Mistake 2 (scheduling on every request).

7.3 · EXTERNAL APIS, TRANSIENTS & WP-CRON: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
wp eval 'var_dump( is_wp_error( wp_remote_get( "http://localhost:8080/supplier-sand
```

```
bool(false)
```

> **Why:** The sandbox answers 401 with JSON, which is a perfectly good HTTP response: not a WP_Error. Only the status code tells you it failed.

P4

Predict the output, then run it.

```
wp eval 'echo wp_safe_remote_get( "http://10.0.0.1/" )->get_error_message(), "\n";'
```

> A valid URL was not provided.

> **Why:** wp_safe_remote_get() refuses private networks before connecting: "A valid URL was not provided." (the site's own host stays allowed).

LAB REFERENCE SOLUTIONS

Every output below is real: the lab files run exactly as `labs/check_labs.py` runs them, on a fresh course site at the state each file declares (Lesson 7.2's labs at `after:lesson_23`, Lesson 7.3's at `after:lesson_24`), copied into the WordPress root and run with `wp eval-file --use-include` (PHP labs) or `sh` in the WP-CLI container (shell labs); folder labs installed and activated first.

- LAB 7.2-A (GUIDED): LABS/MODULE-14/L7_2-CUSTOM-ENDPOINTS/OPENING_HOURS_STARTER/ → OPENING_HOURS_SOLUTION/ WITH TEST_OPENING_HOURS.SH

With the starter's plugin:

```
Plugin 'pagesco-opening-hours' activated.
Success: Activated 1 of 1 plugins.
== Saturday
{"todo":"TODO: the hours"}
== Sunday
{"todo":"TODO: the hours"}
== A day that does not exist
{"todo":"TODO: the hours"}
== The whole week: how many days, and Thursday
0
(no Thursday)
== What OPTIONS says about the argument
(no day argument)
```

With the solution's:

```
Plugin 'pagesco-opening-hours' activated.
Success: Activated 1 of 1 plugins.
== Saturday
{"day":"saturday","closed":false,"opens":"10:00","closes":"16:00"}
== Sunday
{"day":"sunday","closed":true}
== A day that does not exist
{"code":"rest_invalid_param","message":"Invalid parameter(s): day","data":{"status":400
== The whole week: how many days, and Thursday
7
{"day":"thursday","closed":false,"opens":"9:00","closes":"21:00"}
== What OPTIONS says about the argument
"day":{"description":"One day of the week, in lowercase English. Without it: the whole
```

Reading it: the 400 for funday comes from the enum, not from the learner's code; without a default the argument is null when not sent, which means the whole week; Sunday is `closed: true` with no times (a client tests the flag); OPTIONS documents the argument. `__return_true` is right: opening hours are public. Remove the plugin afterwards.

- LAB 7.2-B: LABS/MODULE-14/L7_2-CUSTOM-ENDPOINTS/SCHEMA_RULES_STARTER.PHP → SCHEMA_RULES_SOLUTION.PHP

As shipped:

```
price = "14.99"                   TODO
price = "free"                    TODO
in_stock = "true"                 TODO
in_stock = "maybe"                TODO
per_page = "500"                  TODO
orderby = "pages"                 TODO
genres = "fantasy,classic"        TODO
title = ""                        TODO
starts_at = "2027-12-16T19:00:00" TODO
starts_at = "16/12/2027 7pm"      TODO
```

The solution prints:

```
price = "14.99"                   yes: 14.99
price = "free"                    no: price is not of type number.
in_stock = "true"                 yes: true
in_stock = "maybe"                no: in_stock is not of type boolean.
per_page = "500"                  no: per_page must be between 1 (inclusive) and 100 (
orderby = "pages"                 no: orderby is not one of title, newest, and price.
genres = "fantasy,classic"        yes: ["fantasy","classic"]
title = ""                        no: title must be at least 1 character long.
starts_at = "2027-12-16T19:00:00" yes: "2027-12-16T19:00:00"
starts_at = "16/12/2027 7pm"      no: Invalid date.
```

Reading it: query strings carry strings, and the schema turns "14.99" into a float and "true" into a boolean; "fantasy,classic" becomes a list; the messages are the ones clients get in data.params. A date-time format must be RFC 3339 (`2027-12-16T19:00:00`): the shop's own "2027-12-16 19:00" needs Lesson 7.2's validate_starts_at().

- HANDOUT: LABS/MODULE-14/L7_2-CUSTOM-ENDPOINTS/DEBUG_STAFF_PICKS_ROUTE.PHP (DELIBERATELY BROKEN)

As shipped, WordPress prints:

```
Notice: Function register_rest_route was called <strong>incorrectly</strong>. The REST
?limit=2   200 ["1984","Animal Farm"]
Warning: Undefined array key "limit" in /var/www/html/debug_staff_picks_route.php on l
(none)     200 ["1984","Animal Farm","Brave New World","Dune","Fahrenheit 451"]
?limit=50  200 ["1984","Animal Farm","Brave New World","Dune","Fahrenheit 451","Lord o
```

1. **doing_it_wrong (route at line 20):** no permission_callback. Outside a REST request the Notice prints; inside one it would only be an X-WP-DoingItWrong header. Add `'permission_callback' => '__return_true'` (a public read).
2. **Warning, line 43:** `get_query_params()` holds only what the client sent, so `limit` is missing without ?limit=; the default lives in the request's parameters: `$request['limit']`.
3. **Silent, line 30:** `'sanitize_callback' => 'absint'` switches the schema off for limit, so 50 passes and all eleven books come back (Lesson 7.2, Mistake 2). Remove it (the type sanitizes), or add `'validate_callback' => 'rest_validate_request_arg'`.

Fixed (all three), it prints:

```
?limit=2   200 ["1984","Animal Farm"]
(none)     200 ["1984","Animal Farm","Brave New World"]
?limit=50  400 "rest_invalid_param"
```

The fixed script is PICKS_FIXED in blocks_S14.py (it prints an error's code instead of the whole error).

- LAB 7.3-A (GUIDED): LABS/MODULE-14/L7_3-EXTERNAL-APIS-CRON/REMOTE_ANSWERS_STARTER.PHP → REMOTE_ANSWERS_SOLUTION.PHP

As shipped:

```
genre=fantasy     pagesco_todo: TODO: ask and check the answer
genre=romance     pagesco_todo: TODO: ask and check the answer
books/99          pagesco_todo: TODO: ask and check the answer
a web page        pagesco_todo: TODO: ask and check the answer
shop.invalid      pagesco_todo: TODO: ask and check the answer
```

The solution prints:

```
genre=fantasy     2 books: The Hobbit, Lord of the Rings
genre=romance     pagesco_remote_http_error (400): Invalid parameter(s): genre
books/99          pagesco_remote_http_error (404): No book was found with that ID.
a web page        pagesco_remote_bad_response: The answer is not JSON.
shop.invalid      http_request_failed: cURL error 6: Could not resolve host: shop.inval
```

Reading it: the 400 and the 404 are normal responses with JSON bodies, turned into WP_Errors that keep the status; the About page is a 200 that is not JSON (the third check); only shop.invalid gives no answer at all, a WP_Error from the transport. Accept a status check of `200 !== $code` for this API.

- LAB 7.3-B: LABS/MODULE-14/L7_3-EXTERNAL-APIS-CRON/EVENTS_CACHE_STARTER.PHP → EVENTS_CACHE_SOLUTION.PHP

As shipped:

```
View 1: 5 events, error "TODO: no cache yet", requests so far: 1
View 2: 5 events, error "TODO: no cache yet", requests so far: 2
View 3: 5 events, error "TODO: no cache yet", requests so far: 3
After the failure, view 1: 0 events, error "TODO: no cache yet", requests so far: 4
After the failure, view 2: 0 events, error "TODO: no cache yet", requests so far: 5
rows left: 0
```

The solution prints:

```
View 1: 5 events, error "", requests so far: 1
View 2: 5 events, error "", requests so far: 1
View 3: 5 events, error "", requests so far: 1
After the failure, view 1: 0 events, error "cURL error 7: Failed to connect", requests
After the failure, view 2: 0 events, error "cURL error 7: Failed to connect", requests
rows left: 0
```

Reading it: one request for three views; after the simulated failure (pre_http_request answers instead of the network) the failure itself is cached for a minute, so the second view does not ask again; the value is always an array, never false; the cleanup leaves no rows. `global $pagesco_requests` is deliberate: WP-CLI runs the file inside a function.

- LAB 7.3-C: LABS/MODULE-14/L7_3-EXTERNAL-APIS-CRON/CRON_PLAN_STARTER.SH → CRON_PLAN_SOLUTION.SH

As shipped:

```
Events of pagesco_books_lab_job after three schedule calls: 0
+------+------------+
| hook | recurrence |
+------+------------+
+------+------------+
First run (site time): TODO
Removed: 0 + 0
Left: 0
```

The solution prints:

```
Events of pagesco_books_lab_job after three schedule calls: 1
+-----------------------+------------+
| hook                  | recurrence |
+-----------------------+------------+
| pagesco_books_lab_job | 12 hours   |
+-----------------------+------------+
First run (site time): 04:30
+---------------------------+---------------+------+
| hook                      | recurrence    | args |
+---------------------------+---------------+------+
| pagesco_books_lab_reminder | Non-repeating | [7]  |
+---------------------------+---------------+------+
Removed: 1 + 1
Left: 0
```

Reading it: three schedule calls, one event, because each checks wp_next_scheduled() first (and the start time is fixed, so even an exact duplicate would be refused); the first run is 04:30 in the site's time zone; the reminder carries its argument, and clearing it needs the same arguments.

- HANDOUT: LABS/MODULE-14/L7_3-EXTERNAL-APIS-CRON/DEBUG_EVENT_COUNTS.PHP (DELIBERATELY BROKEN)

As shipped, PHP prints:

```
Warning: Undefined array key "items" in /var/www/html/debug_event_counts.php on line 35
Warning: foreach() argument must be of type array|object, null given in /var/www/html/d
Warning: Undefined array key "items" in /var/www/html/debug_event_counts.php on line 35
Warning: foreach() argument must be of type array|object, null given in /var/www/html/d
Warning: Undefined array key "items" in /var/www/html/debug_event_counts.php on line 35
Warning: foreach() argument must be of type array|object, null given in /var/www/html/d
Warning: Undefined variable $pagesco_totals in /var/www/html/debug_event_counts.php on
Café: 0 events (asked the API 3 times in 3 views)
Warning: Undefined array key "items" in /var/www/html/debug_event_counts.php on line 35
Warning: foreach() argument must be of type array|object, null given in /var/www/html/d
Warning: Undefined array key "items" in /var/www/html/debug_event_counts.php on line 35
Warning: foreach() argument must be of type array|object, null given in /var/www/html/d
Warning: Undefined array key "items" in /var/www/html/debug_event_counts.php on line 35
Warning: foreach() argument must be of type array|object, null given in /var/www/html/d
Warning: Undefined variable $pagesco_totals in /var/www/html/debug_event_counts.php on
Rooftop: 0 events (asked the API 3 times in 3 views)
```

1. **Warnings, line 35:** the events route answers a LIST, not `{ items: ... }`: `$data['items']` is undefined and foreach gets null. Count the list: `is_array( $data ) ? count( $data ) : 0`.
2. **Warning, line 48:** `$pagesco_totals` is a typo for `$pagesco_total`.
3. **Silent, line 26:** `if ( $cached )` treats a cached 0 as missing, so the Rooftop (0 events) asks the API on every view. Use `false !== $cached` (the reason Lesson 7.3 says never to store false, and to compare with false).

Fixed (all three), it prints:

```
Café: 2 events (asked the API 1 times in 3 views)
Rooftop: 0 events (asked the API 1 times in 3 views)
```

The fixed script is COUNTS_FIXED in blocks_S14.py.

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 7.2 · CUSTOM REST ENDPOINTS

1. **A** rest_api_init: rest_api_init fires when WordPress builds the REST server, only on requests that need it; registering anywhere else triggers a called incorrectly notice.
2. **C** 8 books, and X-WP-Total: 11, X-WP-TotalPages: 2: per_page defaults to Settings::get('books_per_page'), now 8, so page 1 has 8 books and the headers say 11 in total on 2 pages.
3. **B** The callback receives 500: the maximum is not checked: With a sanitize_callback of its own, the argument is no longer run through rest_parse_request_arg, so the maximum is never checked and the callback receives 500.

4. **A** A function that returns current_user_can('edit_others_posts'), or a WP_Error with rest_authorization_required_code(): A write needs a capability check; edit_others_posts lets editors and administrators add shop-wide events, and the function answers 401 or 403 otherwise.

5. **C** 500: Without a status of 404 in the error data, WordPress sends 500 Internal Server Error with data null.

LESSON 7.3 · EXTERNAL APIS, TRANSIENTS & WP-CRON

1. **C** A normal response array; wp_remote_retrieve_response_code() returns 503: An error status is still a response: an array whose code wp_remote_retrieve_response_code() reports as 503, with the HTML as the body. Only a missing answer is a WP_Error.

2. **A** In the object cache only, where it may be evicted before the hour is over: With an external object cache, transients are stored in the cache only, and may be evicted before they expire; without one they are two rows in wp_options.

3. **B** In the activation callback, after checking wp_next_scheduled(), with wp_clear_scheduled_hook() on deactivation: Schedule once on activation, guarded by wp_next_scheduled(), and clear it with wp_clear_scheduled_hook() on deactivation; the callback itself is added on every request.

4. **B** With the first request after 3:00, around 8 a.m.: WP-Cron checks for due events on requests; the first request after 3:00, here the first visitor around 8 a.m., starts it. A system cron calling wp cron event run --due-now removes that dependency.

5. **C** So that a broken supplier is not asked, and waited for, on every screen view: Without a cached failure, every Books screen view would call the broken supplier again and wait for its timeout; five minutes protects both sites and still retries soon.

EXIT TICKET

1. **The four desks:** validate (schema and validate_callbacks), sanitize (schema or sanitize_callbacks, defaults), permission (permission_callback), callback.

2. **Three failures:** no answer (a WP_Error: DNS, connection, timeout); an error status (401, 404, 503: a normal response); a body that is not the expected data (HTML, no items).

3. **The daily job:** in the activation callback, `if ( ! wp_next_scheduled( $hook ) )`; wp_clear_scheduled_hook() on deactivation; the callback added on every request.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 C.** Mixes CREATABLE and EDITABLE: show WP_REST_Server's constants.
- **Q3 B (401 first).** Thinks permission comes first: redraw the four desks and run P2.

- **Q4 B.** Guessing at the schema's behavior: the validate_callbacks run first, then sanitizing.
- **Q5 B (is_user_logged_in).** Any subscriber is logged in: a capability names the job.
- **Q7 B or C.** Has not met the helpers' tolerance of WP_Error; still check is_wp_error() first.
- **Q10 A.** Uses WP-CLI's event run: it moves the schedule.
- **Q11 A.** Puts secrets in code: they end up in Git and in every copy.

INSTRUCTOR KIT · ANSWER KEY

Session 15: Debugging, Testing and Performance

Answers and reasoning for every Session 15 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs, the lessons' recorded cards, the lesson pages' own quizzes and the exit ticket.

Session 15 | 12 self-check items | Confidential

WARM-UP

1. **W1 (Lesson 7.3).** No answer at all (a WP_Error: DNS, refused, timeout); an answer that is not a success (an array with a 401 or 503 status); a success whose body is not the JSON you expect. Only the first is a WP_Error.
2. **W2 (Lesson 7.3).** Availability::get() reads through: it calls the supplier, stores the answer again (12 hours, 5 minutes after an error) and returns it. A transient may vanish at any time, so code never relies on it being there.
3. **W3 (Lesson 7.3).** wp cron event run on a recurring event reschedules it to now plus the interval: the 03:00 job would move to whenever you ran it.
4. **W4 (Lesson 7.2).** A 500. The route's callback should give its WP_Error array('status' => 404) (or the right code); permission callbacks use rest_authorization_required_code().
5. **W5 (Lesson 7.3).** In wp-config.php (PAGESCO_BOOKS_SUPPLIER_URL and _KEY, set with wp config set): configuration per environment, never in the plugin's code. Lesson 8.4 moves them to the server's environment.

8.1 · DEBUGGING & TESTING: SELF-CHECK

Q1

The live Pages & Co. shop should keep a record of PHP warnings for the developer, and show visitors none. Where should `WP_DEBUG_LOG` point?

A) `true`, so the messages go to `wp-content/debug.log`

B) A file outside the web root, such as `/home/pagesco/logs/wordpress.log`

C) `false`: a live site must never record anything

D) The theme's folder, where only the theme developer looks

> **Why:** With display off and the log in a folder the web server does not serve, the messages are kept and nobody can download them. Lesson 8.1's Mistake 1: on the course site `curl` gets `wp-content/debug.log` with status 200.
> **A** is wrong: That is the default, inside the web root: bots try that address on every site.
> **C** is wrong: Then a live problem leaves no evidence at all; logging is fine, the location is the point.
> **D** is wrong: Also inside the web root, and a theme update would delete it.

Q2

On the course site after Lesson 7.3, with WordPress loaded (`wp eval-file`): what does this print?

```php
<?php
add_filter(
    'pre_http_request',
    fn() => array(
        'headers'  => array(),
        'body'     => '<h1>Service Unavailable</h1>',
        'response' => array( 'code' => 503, 'message' => 'Service Unavailable' ),
        'cookies'  => array(),
        'filename' => null,
    )
);
$result = ( new PagesCo\Books\Supplier\Client() )->availability( array( '9780441172
echo $result->get_error_code(), ' ', $result->get_error_data()['status'], "\n";
```

`pagesco_supplier_http_error 503`

> **Why:** `pre_http_request` answers in place of the network, so the client never calls the sandbox. A 503 is an answer, not a transport failure: Client::decode() turns any status outside 200 to 299 into `pagesco_supplier_http_error` with the status in the error's data. That exact code and that exact number are what SupplierClientTest asserts, and why a test that only asked "is it an error?" would miss the bug of Lesson 8.1's red run.

Q3

A plugin crashes on `init` and every WP-CLI command now dies with the same fatal error. Which command still deactivates it?

A) `wp plugin deactivate pagesco-crash --force`

B) `wp eval 'deactivate_plugins( "pagesco-crash/pagesco-crash.php" );'`

C) `wp plugin delete pagesco-crash`

D) `wp --skip-plugins=pagesco-crash plugin deactivate pagesco-crash`

> **Why:** WP-CLI loads WordPress, and with it every active plugin, before it runs any command. --skip-plugins loads WordPress without the named plugin, so the deactivation can run. Recovery mode is the browser's way out; for a must-use plugin only renaming the file helps.
> **A** is wrong: There is no --force that skips loading the plugin: the fatal error happens first.
> **B** is wrong: wp eval loads WordPress too, so it dies before your code runs.
> **C** is wrong: Also loads WordPress first; and deleting is not the first move when deactivating will do.

Q4

Why does the lesson name its temporary backtrace logger `0-pagesco-debug-backtrace.php`?

A) Must-use plugins load in alphabetical order: it must be there before the plugin that asks too early

B) WordPress ignores must-use plugins whose names start with a letter

C) The 0- tells WordPress to load it only when WP_DEBUG is on

D) So that PHPCS skips the file

> **Why:** A hook added after the notice fired would miss it. `wp_get_mu_plugins()` sorts the files by name, so `0-` loads before `pagesco-too-early.php`, adds its `doing_it_wrong_run` callback, and catches the notice with the call stack.
> **B** is wrong: Every PHP file directly in mu-plugins/ loads.
> **C** is wrong: WordPress gives file names no meaning beyond their order.
> **D** is wrong: PHPCS only checks what its ruleset names; mu-plugins are not in the plugin's ruleset at all.

Q5

PHPCS reports 30 times `PrefixAllGlobals.NonPrefixedVariableFound` in the blocks' `render.php` files. What did Lesson 8.1 do, and why?

A) Renamed every variable to start with pagesco_books_

B) Turned the rule off for the whole plugin

C) Excluded that rule for those two file patterns in phpcs.xml.dist, with a comment: WordPress includes them inside a function, so the variables are local

D) Added // phpcs:ignore to each of the 30 lines

> **Why:** A ruleset exception that is narrow (one rule, the files it does not fit) and explained (the comment) is a decision a reviewer can read. Thirty ignore comments would hide the same decision thirty times; a plugin-wide switch would hide real globals elsewhere.
> **A** is wrong: Possible, but noisy, and it fixes nothing: the variables are not globals.
> **B** is wrong: Too wide: a real global anywhere else would go unreported.
> **D** is wrong: phpcs:ignore is for a single line that is right despite the rule; thirty of them is a ruleset decision.

Q6

In the plugin's folder, with the wp-env test environment started: which command runs the plugin's PHPUnit tests, and why not just `phpunit`?

`npx @wordpress/env run cli --env-cwd=wp-content/plugins/pagesco-books vendor/bin/phpunit`: the plugin's own PHPUnit 9.6; the container's global phpunit is 10.5, which the WordPress 7.1 test library does not support

> **Why:** Two versions of PHPUnit in one process fail with "Call to undefined method PHPUnit\Framework\TestSuite::empty()" (Lesson 8.1, Common Mistake 2). --env-cwd is the folder to run in, relative to the WordPress root inside the container. composer test runs the same vendor/bin/phpunit.

8.1 · DEBUGGING & TESTING: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
wp eval 'var_dump( WP_DEBUG_DISPLAY, ini_get( "display_errors" ) );'
```

```
bool(true)
string(6) "stderr"
```

Why: WP_DEBUG_DISPLAY is true, and WP-CLI sets PHP's display_errors to stderr: messages go to the terminal's error stream, which is why your terminal shows warnings that a pipe into a file would not catch. On a web request the same setting prints them into the page.

P2

Predict the output, then run it.

```
wp eval 'add_filter( "pre_http_request", fn() => new WP_Error( "http_request_failed
```

```
WP_Error: down
```

Why: pre_http_request answered with a WP_Error before any connection was tried (the fictional address would not resolve anyway). wp_remote_get() returns it unchanged: a transport failure is a WP_Error, an HTTP error status is not (Lesson 7.3).

8.2 · PERFORMANCE & CACHING: SELF-CHECK

Q7

On the course site: what does this print?

```
wp eval 'global $wpdb; $b = $wpdb->num_queries; get_option( "pagesco_lab_nope" ); g
```

```
1
```

Why: The first `get_option()` of an option with no row asks the database and finds nothing; WordPress then remembers the name in the `notoptions` cache for the rest of the request, so the second costs nothing. One query per request is what `pagesco_books_settings` cost every page until Lesson 8.2 gave it an autoloaded row.

Q8

On a site without a persistent object cache, these two commands run one after the other. What does the second print?

```
wp eval 'wp_cache_set( "stock", 24, "pagesco_demo" );'
wp eval 'var_dump( wp_cache_get( "stock", "pagesco_demo" ) );'
```

A) `int(24)`

B) `bool(false)`

C) `NULL`

D) A Warning: the group pagesco_demo does not exist

> **Why:** Each WP-CLI command is its own PHP process, a request of its own, and WordPress's own object cache lives in that process's memory: the second starts empty and wp_cache_get() returns false for a miss. With an object-cache.php drop-in (Redis, Memcached, the lesson's SQLite Object Cache) it would print int(24).
> **A** is wrong: That needs a persistent object cache; the course site has none (wp_using_ext_object_cache() is NULL).
> **C** is wrong: A cache miss is false, which is why the lesson's code checks is_array() or is_string() for a hit.
> **D** is wrong: Groups need no registration; an unknown key is simply a miss.

Q9

A shop with 500 books runs `get_posts( array( 'post_type' => 'book', 'fields' => 'ids', 'posts_per_page' => -1 ) )`, then `get_post_meta()` for each ID, with an empty cache. About how many queries is that, and what is the fix?

A) 2 queries: WordPress loads the meta with the IDs

B) 500 queries; the fix is a persistent object cache

C) 501 queries; the fix is update_postmeta_cache($ids) before the loop: 2 queries

D) 1 query; meta is autoloaded like options

> **Why:** With 'fields' => 'ids' get_posts() skips priming the meta cache, so each get_post_meta() asks for its own post: N+1. update_postmeta_cache() loads the meta of all IDs in one query (Lesson 8.2 measured 12 to 2 for eleven books; the lab `query_count` shows 11 and 1).
> **A** is wrong: That is what a normal WP_Query does; fields => ids is the shortcut that skips it.
> **B** is wrong: Off by one (the ID query), and a warm persistent cache only hides the problem until it is flushed.
> **D** is wrong: Post meta is never autoloaded; only options have autoload.

Q10

EventsTable::find() treats a cached value as a hit with `if ( is_array( $events ) )`. A colleague suggests the shorter `if ( $events )`. What would break?

A) Nothing: both conditions treat every cached value the same way

B) A cached empty result, array(), would count as a miss: a location without events would query the database on every call

C) A cached WP_Error would reach the shortcode

D) Two different limits would share one cache entry

> **Why:** An empty array is falsy in PHP, so a truthiness check turns a valid, cached answer (no events here) into a miss, every time. `is_array()` states what a hit is; `false !== $events` would work too, since a miss is false (run: `wp eval 'var_dump( false !== array(), (bool) array() );'`). Two limits sharing an entry is a different bug, prevented by the key: the debug handout `debug_events_cache.php` has exactly that one.
> **A** is wrong: array() is falsy: if (array()) does not run its block.
> **C** is wrong: find() never stores a WP_Error.
> **D** is wrong: That is prevented by the key, md5($limit . '|' . $location), not by the check.

Q11

In Lesson 8.2's lab measurement the front page had a CLS of 0.101, just over the 0.1 limit. What caused it, and what fixed it?

A) The web fonts; font-display: swap fixed it

B) The speculation rules prefetching the next page

C) The Navigation block's external stylesheet; inlining it fixed it

D) The hero image had no reserved space; the Image block's aspectRatio attribute fixed it

> **Why:** The browser laid the page out without knowing the image's height, and moved everything below it when it arrived. Block templates get no automatic width and height (wp_filter_content_tags skips the template context), so the theme adds the aspect ratio itself: CLS fell to 0.001.
> **A** is wrong: The font swap caused only the 0.001 that was left.
> **B** is wrong: Prefetching happens on a click and moves nothing on the current page.
> **C** is wrong: A render-blocking stylesheet delays the first paint; it does not shift content afterwards.

Q12

Which WordPress filter adds a `<link rel="preload">` for the front page's hero image to the head, and which attribute tells the browser to fetch it first?

`wp_preload_resources`, with `'fetchpriority' => 'high'` (and `'as' => 'image'`)

> **Why:** The browser finds the preload in the head before it reads the body. In the lesson's lab measurement LCP went from about 1.48 s to about 1.40 s: a modest gain, honestly reported, because the image was in the HTML anyway.

8.2 · PERFORMANCE & CACHING: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
wp eval 'global $wpdb; $b = $wpdb->num_queries; get_post( 30 ); get_post( 30 ); ech
```

```
1
```

> **Why:** The first get_post() loads the row and caches it in the posts group; the second comes from the object cache. Within one request WordPress never asks twice for the same post.

P4

Predict the output, then run it.

```
wp eval 'var_dump( wp_using_ext_object_cache() );'
```

```
NULL
```

> **Why:** NULL: no object-cache.php drop-in, so WordPress uses its own in-memory cache (wp_using_ext_object_cache() is only set to true by a drop-in). In Lesson 8.2 it printed bool(true) while the SQLite Object Cache was installed.

LAB REFERENCE SOLUTIONS

Every output below is real: the lab files run exactly as `labs/check_labs.py` runs them, on a fresh course site at the state each file declares (WordPress 7.1.3, PHP 8.5, MySQL 9.7, WP-CLI 2.12), copied into the WordPress root and

run with `wp eval-file --use-include` (PHP labs) or `sh` in the WP-CLI container (shell labs). Paths show `/var/www/html/...`, which is the learner's `wordpress/` folder.

- LAB 8.1-A (GUIDED): LABS/MODULE-15/L8_1-DEBUGGING-TESTING/FAKE_SUPPLIER_STARTER.PHP → FAKE_SUPPLIER_SOLUTION.PHP

As shipped:

```
PASS a good answer is keyed by ISBN
TODO check 2: a 401 becomes pagesco_supplier_http_error with the status in its data
TODO check 3: a 200 that is not JSON becomes pagesco_supplier_bad_response
TODO check 4: a network failure comes back unchanged
1 of 4 checks passed, 1 requests answered by the fake, none by the network
```

The solution prints:

```
PASS a good answer is keyed by ISBN
PASS a 401 becomes pagesco_supplier_http_error with the status in its data
PASS a 200 that is not JSON becomes pagesco_supplier_bad_response
PASS a network failure comes back unchanged
4 of 4 checks passed, 4 requests answered by the fake, none by the network
```

What each check pins down: (2) a status outside 200 to 299 is `pagesco_supplier_http_error`, with the status in the data, even with a JSON body; (3) a 200 alone is not trusted: a body that is not JSON is `pagesco_supplier_bad_response`; (4) a transport failure comes back unchanged, so the Supplier column can tell "not reachable" from "answered 503". Common sticking points: `'401'` compared with `===` (the data holds the integer 401); a fake left in place from the previous check (`pagesco_lab_fake()` removes the old one first: ask why that matters); `global` in the script, which fails because `--use-include` runs the file inside a function.

- LAB 8.1-B: LABS/MODULE-15/L8_1-DEBUGGING-TESTING/NEXT_EVENT_STARTER/ → NEXT_EVENT_SOLUTION/ WITH TEST_NEXT_EVENT.SH

The test with the starter's `pagesco-books-next-event.php` in mu-plugins/ (state after Lesson 8.1's Step 2):

```
PHPCS with the plugin's ruleset:
FILE: mu-plugins/pagesco-books-next-event.php
----------------------------------------------------------------------------------
FOUND 6 ERRORS AFFECTING 3 LINES
----------------------------------------------------------------------------------
  1 | ERROR | [ ] Missing file doc comment (Squiz.Commenting.FileComment.Missing)
 10 | ERROR | [x] There must be no blank lines between the function comment and the dec
    |       |     (Squiz.Commenting.FunctionComment.SpacingAfter)
 17 | ERROR | [ ] All output should be run through an escaping function (see the Secur
    |       |     WordPress Developer Handbooks), found '__'.
    |       |     (WordPress.Security.EscapeOutput.OutputNotEscaped)
 17 | ERROR | [ ] A function call to __() with texts containing placeholders was found,
    |       |     accompanied by a "translators:" comment on the line above to clarify
    |       |     placeholders. (WordPress.WP.I18n.MissingTranslatorsComment)
 17 | ERROR | [ ] Mismatched text domain. Expected 'pagesco-books' but got 'pagesco'.
    |       |     (WordPress.WP.I18n.TextDomainMismatch)
 17 | ERROR | [ ] All output should be run through an escaping function (see the Secur
    |       |     WordPress Developer Handbooks), found '$event'.
    |       |     (WordPress.Security.EscapeOutput.OutputNotEscaped)
----------------------------------------------------------------------------------
PHPCBF CAN FIX THE 1 MARKED SNIFF VIOLATIONS AUTOMATICALLY
----------------------------------------------------------------------------------
The footer line, as a visitor gets it:
<p class="pagesco-next-event">Next event: Book club: Dune</p>
```

With the solution's:

```
PHPCS with the plugin's ruleset:
PHPCS: no findings
The footer line, as a visitor gets it:
<p class="pagesco-next-event">Next event: Book club: Dune</p>
```

Reading the report: the plugin header is a DocBlock, but PHPCS takes it for the function's comment (line 10) and finds no file comment (line 1): the fix is a file DocBlock with `@package` and a separate one for the function. Line 17 holds four findings: the output is not escaped (twice: `__()` and `$event`), the translators comment is missing, and the text domain is the theme's. The solution escapes the whole sentence once with `esc_html()` around the `sprintf()`, puts `/* translators: ... */` right before the string, and uses `pagesco-books`. The visitor sees exactly the same line: standards are about the code, not the page. Remove the file afterwards.

- HANDOUT: LABS/MODULE-15/L8_1-DEBUGGING-TESTING/DEBUG_SUPPLIER_LOG.PHP (DELIBERATELY BROKEN)

As shipped, PHP prints:

```
Warning: Attempt to read property "ID" on null in /var/www/html/debug_supplier_log.php
Warning: Undefined array key "X-API-Key" in /var/www/html/debug_supplier_log.php on li
dune         24 copies at the supplier
1984         31 copies at the supplier
hobbit       0 copies at the supplier
```

1. **Warning, line 34:** the slug `hobbit` does not exist (it is `the-hobbit`), so `get_page_by_path()` returns null and `->ID` reads a property of null. Its quiet consequence is the last line: an empty ISBN, so "0 copies" for a book the supplier has. Evidence beats the output: the Warning says which line, the output only looks odd.
2. **Warning, line 24:** the header is called `X-Api-Key` (Client::headers()), not `X-API-Key`: PHP array keys are case-sensitive, even when HTTP headers are not.
3. **Silent:** with the key's name fixed, the logger writes the whole API key into debug.log (below). No message, and the most serious bug of the three: logs are read by more people, and kept longer, than the code. Log the first eight characters, as Lesson 8.1's logger does.

The two loud bugs fixed, the new line of debug.log (the silent bug, visible):

```
[26-Oct-2026 13:23:25 UTC] supplier GET http://localhost:8080/supplier-sandbox/v1/avai
```

All three fixed: the log line and the output:

```
[26-Oct-2026 13:23:25 UTC] supplier GET http://localhost:8080/supplier-sandbox/v1/avai
```

```
dune         24 copies at the supplier
1984         31 copies at the supplier
the-hobbit   9 copies at the supplier
```

- LAB 8.2-A (GUIDED): LABS/MODULE-15/L8_2-PERFORMANCE-CACHING/QUERY_COUNT_STARTER.PHP → QUERY_COUNT_SOLUTION.PHP

As shipped:

```
call                                                                  first     aga
get_option( 'blogname' )                                                 0
TODO row 2: an option that has no row
TODO row 3: two meta values of one book
TODO row 4: the same WP_Query twice
TODO row 5: one meta read per book
TODO row 6: one meta query for all books
```

The solution prints:

```
call                                                                  first     aga
get_option( 'blogname' )                                                  0
get_option( 'pagesco_lab_missing' )                                       1
get_post_meta( 33, 'pagesco_price' ) and 'pagesco_author'                 1
WP_Query: the three cheapest books                                        6
get_post_meta() over 11 books, one by one                                11
get_post_meta() over 11 books, after update_postmeta_cache()              1
```

Reading the rows: (1) autoloaded options were loaded in one query before the script ran; (2) a missing option costs one query, then WordPress remembers it in `notoptions`; (3) the first meta read loads all of the book's meta, the second is free; (4) WP_Query's six queries (IDs, posts, meta, terms) and its own result cache (6.1); (5) N+1, one query per book; (6) `update_postmeta_cache()` turns them into one. Every "again" column is 0: within one request WordPress never asks twice. Accept a different row 4 count on a site with other book data; the 0 is the point.

- LAB 8.2-B: LABS/MODULE-15/L8_2-PERFORMANCE-CACHING/CHEAPEST_CACHE_STARTER.PHP → CHEAPEST_CACHE_SOLUTION.PHP

As shipped (no cache, no invalidation):

```
first call: 1 query, Pride and Prejudice at $7.99
second call: 1 query, Pride and Prejudice at $7.99
after Dune's price drop: 1 query, Dune at $1.99
after the price is back: 1 query, Pride and Prejudice at $7.99
Dune's price restored to 14.99
TODO 1 and 2: the second call should cost 0 queries, the call after each price change 1
```

The solution prints:

```
first call: 1 query, Pride and Prejudice at $7.99
second call: 0 queries, Pride and Prejudice at $7.99
after Dune's price drop: 1 query, Dune at $1.99
after the price is back: 1 query, Pride and Prejudice at $7.99
Dune's price restored to 14.99
```

With TODO 1 alone (the cache, but no `wp_cache_set_last_changed()` in the writer), the call after the price drop prints `0 queries, Pride and Prejudice at $7.99`: a stale answer, with no message, exactly Lesson 8.2's Mistake 1. TODO 2 makes every write renew the key. Why `is_string()` for the hit: a miss is false. The script restores Dune's price; a learner who stopped it halfway restores it with `wp post meta update 30 pagesco_price 14.99`.

- LAB 8.2-C: LABS/MODULE-15/L8_2-PERFORMANCE-CACHING/PAGE_WEIGHT_STARTER.SH → PAGE_WEIGHT_SOLUTION.SH

As shipped:

```
page            KB html  styles css files  modules  images
/                    82    TODO      TODO     TODO    TODO
/books/dune/         78    TODO      TODO     TODO    TODO
/find-a-book/        81    TODO      TODO     TODO    TODO
/events/             75    TODO      TODO     TODO    TODO
hero image reserves its space: TODO
hero image preloaded: TODO
lazy images on the front page: TODO
```

The solution prints, on the site as Lesson 8.1 leaves it:

```
page            KB html  styles css files  modules  images
/                    82      20         1        2       1
/books/dune/         78      16         0        2       0
/find-a-book/        81      15         0        3       0
/events/             75      14         0        2       0
hero image reserves its space: no (no width, height or aspect-ratio)
hero image preloaded: no
lazy images on the front page: 0
```

And on the site as Lesson 8.2's project leaves it (the same script; the front page has one more element in the head, the preload link):

```
page            KB html  styles css files  modules  images
/                    82      20         1        2       1
/books/dune/         78      16         0        2       0
/find-a-book/        81      15         0        3       0
/events/             75      14         0        2       0
hero image reserves its space: yes
hero image preloaded: yes
lazy images on the front page: 0
```

`grep -c` counts lines, and WordPress prints several tags on one line: `grep -o ... | wc -l` counts matches. The one external stylesheet on the front page is the Navigation block's (too big for the 40,000-byte inline budget with SCRIPT_DEBUG on); the other pages inline everything.

- HANDOUT: LABS/MODULE-15/L8_2-PERFORMANCE-CACHING/DEBUG_EVENTS_CACHE.PHP (DELIBERATELY BROKEN)

As shipped, PHP prints:

```
Next event: Warning: Undefined property: stdClass::$titel in /var/www/html/debug_events

Next three: Book club: Dune
Next five:  Book club: Dune
Warning: Undefined variable $pagesco_seats in /var/www/html/debug_events_cache.php on 1
Seats in those five: 20
[new lines in wp-content/debug.log]
[26-Oct-2026 13:26:42 UTC] PHP Warning:  Undefined property: stdClass::$titel in /var/w
[26-Oct-2026 13:26:42 UTC] PHP Warning:  Undefined variable $pagesco_seats in /var/www/
```

1. **Warning, line 36:** `->titel` is not a property of the row (`title`). The line prints "Next event: " with nothing after it.
2. **Warning, line 41:** `$pagesco_seats` is added to before it exists; PHP treats it as 0 and says so. Start it at 0 before the loop.
3. **Silent, line 24:** the key has no `$limit`, so the first call's answer (one event) is returned for three and for five: "Next three" and "Next five" show one title, and the seats are one event's. The key must name everything the result depends on (Lesson 8.2, the events cache's `md5( $limit . '|' . $location )`).

Fixed (the three changes), it prints:

```
Next event: Book club: Dune
Next three: Book club: Dune, Story time: The Hobbit, An evening with Ana Ruiz
Next five:  Book club: Dune, Story time: The Hobbit, An evening with Ana Ruiz, Poetry c
Seats in those five: 125
```

The order of discovery matters: the two Warnings are found first, and only then does the output look plausible enough to hide the third. Counting the titles is the evidence.

RECORDED IN THE LESSONS (QUOTE, DO NOT RUN IN CLASS)

These cards are the lessons' own recordings (wp-env and PHPUnit need Node.js on the computer; the SQLite Object Cache needs the internet). The Facilitator Guide shows them; learners make them at home.

◆ LESSON 8.1: THE FIRST PHPCS REPORT, BY RULE

```
Composer could not detect the root package (pagesco/pagesco-books) version, defaulting

PHP CODE SNIFFER VIOLATION SOURCE SUMMARY
--------------------------------------------------------------------------------
    SOURCE                                                                  COUNT
--------------------------------------------------------------------------------
[ ] WordPress.NamingConventions.PrefixAllGlobals.NonPrefixedVariableFound  30
[x] WordPress.Arrays.ArrayDeclarationSpacing.AssociativeArrayFound         5
[ ] WordPress.Security.EscapeOutput.OutputNotEscaped                       5
[ ] WordPress.DB.DirectDatabaseQuery.DirectQuery                           4
[ ] WordPress.DB.DirectDatabaseQuery.NoCaching                             3
[ ] WordPress.DB.SlowDBQuery.slow_db_query_meta_key                        2
[ ] WordPress.DB.SlowDBQuery.slow_db_query_tax_query                       2
[ ] WordPress.Security.NonceVerification.Recommended                       2
[x] Generic.Formatting.MultipleStatementAlignment.NotSameWarning           1
[ ] Universal.NamingConventions.NoReservedKeywordParameterNames.classFoun  1
[ ] WordPress.DB.DirectDatabaseQuery.SchemaChange                          1
[ ] WordPress.DB.PreparedSQL.NotPrepared                                   1
[ ] WordPress.DB.SlowDBQuery.slow_db_query_meta_query                      1
--------------------------------------------------------------------------------
A TOTAL OF 58 SNIFF VIOLATIONS WERE FOUND IN 13 SOURCES
--------------------------------------------------------------------------------
PHPCBF CAN FIX THE 2 MARKED SOURCES AUTOMATICALLY (6 VIOLATIONS IN TOTAL)
--------------------------------------------------------------------------------

...

Script phpcs handling the lint event returned with error code 2
```

LESSON 8.1: THE TEST ENVIRONMENT AND THE SUITE AS TEST NAMES

```
WordPress development site started at http://localhost:8889
MySQL is listening on port 45899

 ✔ Done! (in 40s 62ms)
```

```
PHPUnit 9.6.38 by Sebastian Bergmann and contributors.

Book Shortcode (PagesCo\Books\Tests\BookShortcode)
 ✔ Renders a published book
 ✔ Prints nothing for drafts and pages
 ✔ Escapes the author

Books Endpoint (PagesCo\Books\Tests\BooksEndpoint)
 ✔ Filters by genre and sorts by price
 ✔ Rejects an unknown genre
 ✔ Every book matches the schema

Daily Sync (PagesCo\Books\Tests\DailySync)
 ✔ Schedules one daily event at three
 ✔ Run removes only past events

Supplier Client (PagesCo\Books\Tests\SupplierClient)
 ✔ Reads availability by isbn
 ✔ Turns an error status into a wp error
 ✔ Rejects an answer without items
 ✔ Passes on a network failure

Time: 00:00.101, Memory: 46.50 MB

OK (12 tests, 30 assertions)
```

LESSON 8.2: THE SQLITE OBJECT CACHE INSTALLED (ITS VERSION MAY DIFFER)

```
Installing SQLite Object Cache (1.6.6)
Downloading installation package from https://downloads.wordpress.org/plugin/sqlite-obj
Unpacking the package...
Installing the plugin...
Plugin installed successfully.
Activating 'sqlite-object-cache'...
Plugin 'sqlite-object-cache' activated.
Success: Installed 1 of 1 plugins.
bool(true)
+------------------+--------------------------------+
| name             | title                          |
+------------------+--------------------------------+
| object-cache.php | SQLite Object Cache (Drop-in) |
+------------------+--------------------------------+
```

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 8.1 · DEBUGGING & TESTING

1. **B** WP_DEBUG true, WP_DEBUG_DISPLAY false, WP_DEBUG_LOG set to a path outside the web root: Errors are recorded but never shown, and the log lives where the web server does not serve it, unlike the default wp-content/debug.log.

2. **A** An email with the plugin's name, the error and a recovery mode link: The fatal error handler finds the plugin from the file in the error and emails a recovery mode link that logs in with that plugin paused.

3. **C** So that the test controls the answer, including failures, and does not depend on the network: The filter answers in place of the network, so each test decides exactly what the supplier says, including 503s and broken JSON, and the result depends only on your code.

4. **C** A phpcs:ignore comment for that rule on that line, with the reason: The output is escaped by core, which PHPCS cannot know; an ignore comment for exactly that rule on exactly that line, with the reason, documents the decision.

5. **A** The test library reinstalls that site's database on every run: The test library drops and reinstalls the tables of the database it is configured with on every run, so it must never be a database you care about.

LESSON 8.2 · PERFORMANCE & CACHING

1. **B** One query per request; an autoloaded row for the option removes it: Each request asks the database once and learns that the row does not exist; an autoloaded row is read with all other autoloaded options, for no extra query.

2. **C** Until the end of the current request: The default object cache is an array in PHP's memory, emptied at the end of every request; only a drop-in such as Redis keeps it between requests.

3. **A** So that every change to the table makes all older cached results unreachable: Every write renews the group's last-changed value, so all keys built before the write are never read again: no stale result without having to find and delete each entry.

4. **B** Because WordPress skips loading optimizations for block templates, and the image is not in the Media Library: wp_get_loading_optimization_attributes() returns early for the template context, and width and height are only added for Media Library images; the theme must take care of its own images.

5. **C** That a change you make can be compared against it on the same machine; field data from real visitors decides the rest: Lab measurements compare changes on one machine with the same settings; real visitors' phones and networks, measured as field data after launch, decide the Core Web Vitals.

EXIT TICKET

1. **A critical error:** the server's error log (on development: wordpress/wp-content/debug.log) and the recovery mode email WordPress sent the admin address (it names the plugin, the file and the line); from the command line `wp --skip-plugins=<plugin> plugin deactivate <plugin>` works although every other command dies. A must-use plugin: rename the file.

2. **The last-changed value in the key:** flush_cache() renews it after every insert or delete, so after a change every key is new and every old answer is simply never read again; nothing has to find and delete old entries. Without it, a persistent cache would show the old list until it expired.
3. **Lab against field:** a lab measurement compares before and after a change on the same machine and setup (CLS 0.101 to 0.001 is real); it cannot say what visitors on mid-range phones over weak connections experience, how a full catalog or real traffic behaves, or what a page cache changes: field data after the launch (Lesson 8.4).

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 A ("true, wp-content/debug.log").** Remembers that logging is good, not where the log goes. Run Mistake 1's curl again: 200.
- **Q2 with `pagesco_supplier_bad_response`.** Thinks an HTML body makes it a bad response first. The status is checked before the body (Client::decode()): read the method's three numbered comments.
- **Q3 B ("wp eval deactivate_plugins").** Thinks wp eval runs before WordPress loads. Everything in WP-CLI except a few commands loads WordPress, plugins included.
- **Q8 A ("int(24)").** Does not separate one request from the next. Two WP-CLI commands are two PHP processes; show P4 (`wp_using_ext_object_cache()` is NULL).
- **Q9 B ("500, persistent cache").** A warm persistent cache hides N+1; flush it and the 501 queries are back. The fix is in the code.
- **Q10 A ("nothing breaks").** Forgets that an empty array is falsy. Run `wp eval 'var_dump( false !== array(), (bool) array() );'`: the second value is the truthiness of array(). A good moment to say: verify claims about code by running them.
- **Q11 A ("the fonts").** Remembers the 0.001 that was left. The 0.1 jump was the image without reserved space.

INSTRUCTOR KIT · ANSWER KEY

Session 16: Distribution and Deployment

Answers and reasoning for every Session 16 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs, the lessons' recorded cards, the lesson pages' own quizzes and the exit ticket.

Session 16 | 12 self-check items | Confidential

WARM-UP

1. **W1 (Lesson 8.1).** To a log outside the web root (or the server's log), display always off: wp-content/debug.log is a public download (200 for anyone).
2. **W2 (Lesson 8.1).** npx @wordpress/env run cli --env-cwd=wp-content/plugins/pagesco-books vendor/bin/phpunit, in the plugin's own wp-env test site on port 8889, never in the site you develop on.
3. **W3 (Lesson 8.2).** N+1 queries; update_postmeta_cache($ids) before the loop loads all the meta in one query.
4. **W4 (Lesson 8.2).** add() calls flush_cache(), which renews wp_cache_get_last_changed('pagesco_events'), and that value is part of every key.
5. **W5 (Lesson 8.2).** The Image block's aspectRatio (CLS 0.101 to 0.001: the space is reserved) and a preload with fetchpriority high in wp_preload_resources (LCP a little earlier).

8.3 · INTERNATIONALIZATION, ACCESSIBILITY & DISTRIBUTION: SELF-CHECK

Q1

make-pot prints `Warning: Multiple placeholders should be ordered.` for `__( 'Showing %s of %s books', 'pagesco-books' )`. What is the fix?

A) Nothing: it is only a warning, and the POT file was written

B) Split it into two strings, one per number

C) Numbered placeholders, `'Showing %1$s of %2$s books'`, with a translators comment naming both

D) Use `_n()` instead of `__()`

> **Why:** With %1$s and %2$s a language can put the numbers in another order (the lab's Spanish translation does: "De %2$s libros, se muestran %1$s"); the comment tells the translator what each number is. make-pot writes the file anyway and exits 0, which is why its warnings scroll by unnoticed.
> **A** is wrong: The string ships untranslatable in languages with another word order.
> **B** is wrong: Fragments are worse: a translator cannot know how they join.
> **D** is wrong: _n() chooses a plural form; it does not number placeholders.

Q2

The Book Card's editor script is `build/blocks/book-card/index.js`. In the WP-CLI container this prints its hash. Which file holds its Spanish strings?

```
printf '%s' 'build/blocks/book-card/index.js' | md5sum | cut -d' ' -f1
```

`languages/pagesco-books-es_ES-f9065c2ff2c4d291b5b56d1be2a45f3f.json`

> **Why:** WordPress looks for the JSON whose name contains the MD5 of the script's path relative to the plugin; make-json computes it from the PO's #: references, which is why the POT must point at the built file (--exclude=src/blocks). Lesson 8.3's languages/ folder has exactly this file.

Q3

The contrast script printed `slate on paper 5.8:1 AA`. What does that allow?

A) Normal-size text in slate on paper meets WCAG AA (4.5:1), but not AAA (7:1)

B) Only large text: normal text needs 7:1

C) Nothing: anything under 7:1 fails WCAG

D) Slate is safe on every background of the palette

> **Why:** AA needs 4.5:1 for normal text and 3:1 for large text and control outlines; 7:1 is AAA. The pair passes AA, which is what most contracts mean. Exercise 2 shows slate failing on ink and on oxblood: a ratio belongs to a pair, not to a color.
> **B** is wrong: 4.5:1 is the AA limit for normal text; 3:1 for large.
> **C** is wrong: AAA is the stricter level; AA is the usual target.
> **D** is wrong: Slate on ink is 2.4:1 and slate on oxblood 1.5:1 (Exercise 2).

Q4

Why does the new uninstall.php read the opt-in with `get_option( 'pagesco_books_settings', array() )` and not with `Settings::get( 'delete_data' )`?

A) get_option() is faster

B) Settings::get() would translate the value

C) uninstall.php runs after the option is deleted

D) During uninstall the plugin is not loaded: its classes, post type and taxonomy are not there

> **Why:** WordPress runs uninstall.php without booting the plugin. That is also why the file registers the genre taxonomy before get_terms() (Common Mistake 2: invalid_taxonomy, and five orphaned genres). An empty row means delete_data is missing, which means off.
> **A** is wrong: Both read the same autoloaded row; speed is not the point.
> **B** is wrong: Settings::get() translates nothing about delete_data.
> **C** is wrong: uninstall.php reads the option first and deletes it near the end.

Q5

Which of these must NOT be in the release ZIP of Pages & Co. Books?

A) `build/blocks/blocks-manifest.php`

B) `phpcs.xml.dist`

C) `languages/pagesco-books-es_ES.l10n.php`

D) `vendor/autoload.php`

> **Why:** Development files (rulesets, phpunit.xml.dist, .wp-env.json, tests/) do not ship: Plugin Check reports them (application_detected, hidden_files), and the lab release_check.sh fails a ZIP that contains them. The build, the translations and the autoloader are what the plugin runs.
> **A** is wrong: Without the manifest no block is registered.
> **C** is wrong: The Spanish translation is part of the release.
> **D** is wrong: The plugin requires it on every load; only the development packages must stay out.

Q6

What does `--no-purge` change in `wp i18n make-json pagesco-books-es_ES.po --no-purge`?

It keeps the JavaScript strings in the PO file; without it make-json moves them into the JSON files and removes them from the PO

> **Why:** Strings used in both PHP and JavaScript ("Mark sold out") would otherwise go missing from the next .mo, and the source PO loses entries.

8.3 · INTERNATIONALIZATION, ACCESSIBILITY & DISTRIBUTION: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
wp eval 'echo get_locale(), " ", determine_locale(), "\n";'
```

> `en_US en_US`

> **Why:** Both en_US: the site language is English, and in WP-CLI determine_locale() has no user or request to change it. After Lesson 8.3 installs es_ES the site stays English until `wp site switch-language es_ES`; a user's locale applies to wp-admin only.

P2

Predict the output, then run it.

```
printf '%s' 'build/blocks/book-card/index.js' | md5sum | cut -d' ' -f1
```

> `f9065c2ff2c4d291b5b56d1be2a45f3f`

> **Why:** The MD5 of the path relative to the plugin: the long part of the Book Card's Spanish JSON file name (self-check Q2).

8.4 · DEPLOYMENT & MAINTENANCE: SELF-CHECK

Q7

Maria changed the header in the live site's Site Editor. You deploy theme 1.2.0 with a new parts/header.html. What do visitors see?

A) The new header from the theme's file

B) Maria's header: Site Editor changes are saved in the database, and they win over the theme's files

C) Both headers, one above the other

D) An error until someone resets the template part

> **Why:** Code goes up, content comes down: an edit in the Site Editor on the live site is content (a wp_template_part post). Changes that should become code are exported into the theme, committed and deployed (Lesson 3.3); then the database copy is reset.
> **A** is wrong: Only if the template part had never been edited on the live site.
> **C** is wrong: WordPress uses one version of a template part.
> **D** is wrong: Nothing is broken; the database version simply wins.

Q8

On the course site after Lesson 8.3: what does this print, and which table makes the difference?

```
echo "$(wp db tables | wc -l) $(wp db tables --all-tables-with-prefix | wc -l)"
```

`12 13` : `wp_pagesco_events`, the plugin's own table, which WordPress does not register

> **Why:** wp db tables and wp search-replace work on the registered (core) tables by default; --all-tables-with-prefix adds the plugin's table. A launch export without it has 12 tables, and the live Events page says "No events are scheduled" (Common Mistake 2; the debug handout debug_launch_check.sh).

Q9

Why does every migration command in Lesson 8.4 say `--skip-columns=guid`?

A) The GUID is an identifier feed readers use to recognize posts, not a link: it must not change

B) GUID columns are serialized and would break

C) The guid column is too long for search-replace

D) WordPress rewrites GUIDs itself on the first request

> **Why:** http://localhost:8080/?p=30 looks like a URL but identifies the post; changing it makes feed readers show every post as new. The launch database keeps 34 GUIDs with the old address, and that is right.
> **B** is wrong: The guid column holds plain strings.
> **C** is wrong: Length is not the issue.
> **D** is wrong: WordPress never rewrites GUIDs.

Q10

The project's .gitignore says `!/themes/`, then `/themes/*`, then `!/themes/pagesco/`. Why not just `!/themes/pagesco/` after `/*`?

A) Git reads .gitignore from the bottom up

B) The order does not matter; it is only for readability

C) Without the first line Git would also track Twenty Twenty-Five

D) Git cannot let a file back in when its folder is ignored: the folder is let in, its contents ignored again, then the one subfolder let in

> **Why:** /* ignores the themes folder itself, so a rule for something inside it is never even read. The three-line pattern is the documented way. `git check-ignore -v` names the line that ignores a file when you wonder why it is missing.
> **A** is wrong: Git reads it top to bottom; the last matching rule wins.
> **B** is wrong: With only the last line, themes/pagesco/ stays ignored.
> **C** is wrong: Twenty Twenty-Five is ignored by /themes/* either way.

Q11

In a clean copy of the tagged plugin (git archive), which Composer command builds the production autoloader, and what does it leave out?

`composer install --no-dev --optimize-autoloader`: the development packages (PHPUnit, PHPCS), so vendor/composer has no autoload_files.php that requires them

> **Why:** The working copy's autoloader requires files from development packages on every load (`deep_copy.php`, PHPUnit's Functions.php, the polyfills); a ZIP without those packages dies with "Failed opening required" (Lesson 8.3's Mistake 3). --optimize-autoloader writes a class map.

Q12

On the rehearsal server the account pagesco_app has no LOCK TABLES right. What happens to `wp db export "$BACKUP"`?

A) It works: WP-CLI locks nothing

B) It asks for the root password

C) It fails with error 1044, and leaves a small header-only file under the backup's name

D) It exports only the tables the account may lock

> **Why:** mariadb-dump locks each table while it reads it. The recorded run shows the error and an 831-byte file: a file in the backup folder is not a backup; its size and a restore test are what count. The lesson grants exactly LOCK TABLES, as root, and then proves the backup by restoring a deleted book.
> **A** is wrong: wp db export runs mariadb-dump, which uses LOCK TABLES.
> **B** is wrong: WP-CLI uses wp-config.php's account and asks nothing.
> **D** is wrong: It stops at the first table it cannot lock.

8.4 · DEPLOYMENT & MAINTENANCE: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
wp db tables | wc -l
```

> 12

> **Why:** The twelve tables WordPress registers; the plugin's wp_pagesco_events is not among them (--all-tables-with-prefix adds it).

P4

Predict the output, then run it.

```
wp search-replace 'http://localhost:8080' 'https://www.pagesco.example' --all-table
```

> 9

Why: Nine replacements, as for 8081: the address counts, not its length. --format=count prints only the number.

LAB REFERENCE SOLUTIONS

Every output below is real: the lab files run exactly as `labs/check_labs.py` runs them, on a fresh course site at the state each declares (WordPress 7.1.3, PHP 8.5, MySQL 9.7, WP-CLI 2.12).

- LAB 8.3-A (GUIDED): LABS/MODULE-16/L8_3-I18N-A11Y-DISTRIBUTION/PLURAL_STRINGS_STARTER.PHP → PLURAL_STRINGS_SOLUTION.PHP

As shipped:

```
en  1 book in stock | 9 books in stock
en  Showing 3 of 11 books
en  Order | Order
en  Books in Fantasy
es  1 book in stock | 9 books in stock
es  Showing 3 of 11 books
es  Order | Order
es  Books in Fantasy
TODO 1 to 4: every es line should be Spanish
```

The solution prints:

```
en  1 book in stock | 9 books in stock
en  Showing 3 of 11 books
en  Order | Order
en  Books in Fantasy
es  1 libro disponible | 9 libros disponibles
es  De 11 libros, se muestran 3
es  Orden | Pedido
es  Libros de Fantasy
```

Why each starter string fails: (1) two fragments and no plural form; (2) unnumbered placeholders, so the translator's reordered string never matches the msgid; (3) one msgid for two meanings, so the file's two contexts

never match; (4) a fragment with the genre glued on. The translation file stays unchanged: the code changes. Context keys in the file are "context\x04msgid", exactly what make-php writes.

- LAB 8.3-B: LABS/MODULE-16/L8_3-I18N-A11Y-DISTRIBUTION/A11Y_SCAN_STARTER.SH → A11Y_SCAN_SOLUTION.SH

As shipped:

```
/             lang=en-US  main=1  h1=TODO  img-no-alt=TODO  skip-link=TODO  notice-lar
/books/dune/  lang=en-US  main=1  h1=TODO  img-no-alt=TODO  skip-link=TODO  notice-lar
/find-a-book/ lang=en-US  main=1  h1=TODO  img-no-alt=TODO  skip-link=TODO  notice-lar
/events/      lang=en-US  main=1  h1=TODO  img-no-alt=TODO  skip-link=TODO  notice-lar
Not checked here, and never by a machine alone: alt texts that describe, focus you can
```

The solution, on the site as Lesson 8.2 leaves it:

```
/             lang=en-US  main=1  h1=1  img-no-alt=0  skip-link=yes  notice-landmark=r
/books/dune/  lang=en-US  main=1  h1=1  img-no-alt=0  skip-link=yes  notice-landmark=r
/find-a-book/ lang=en-US  main=1  h1=1  img-no-alt=0  skip-link=yes  notice-landmark=r
/events/      lang=en-US  main=1  h1=1  img-no-alt=0  skip-link=yes  notice-landmark=r
Not checked here, and never by a machine alone: alt texts that describe, focus you can
```

And on the site as Lesson 8.3's project leaves it (the notice a region, the navigation as links):

```
/             lang=en-US  main=1  h1=1  img-no-alt=0  skip-link=yes  notice-landmark=y
/books/dune/  lang=en-US  main=1  h1=1  img-no-alt=0  skip-link=yes  notice-landmark=y
/find-a-book/ lang=en-US  main=1  h1=1  img-no-alt=0  skip-link=yes  notice-landmark=y
/events/      lang=en-US  main=1  h1=1  img-no-alt=0  skip-link=yes  notice-landmark=y
Not checked here, and never by a machine alone: alt texts that describe, focus you can
```

`tr '\n' ' '` first: WordPress breaks some tags over lines. The two columns that change are axe's two findings; what no grep can check is in the last line.

- LAB 8.3-C: LABS/MODULE-16/L8_3-I18N-A11Y-DISTRIBUTION/RELEASE_CHECK_STARTER.SH → RELEASE_CHECK_SOLUTION.SH

As shipped:

```
versions agree: 1.1.0 1.1.0 1.1.0 1.1.0
requires WordPress: TODO
translations: TODO
the ZIP: TODO
release check: 0 problems
```

The solution prints (after Lesson 8.3):

```
versions agree: 1.1.0 1.1.0 1.1.0 1.1.0
requires WordPress 6.9 in both
translations: ./languages/pagesco-books.pot
the ZIP: 72 entries
release check: 0 problems
```

With Lesson 8.4's version bump the four values must all say 1.1.1 (Step 7 changes them with sed); a learner who forgets package.json sees VERSIONS DIFFER.

- HANDOUT: LABS/MODULE-16/L8_3-I18N-A11Y-DISTRIBUTION/DEBUG_SEATS_SPANISH.PHP (DELIBERATELY BROKEN)

As shipped:

```
Warning: Undefined property: stdClass::$seats in /var/www/html/debug_seats_spanish.php
Book club: Dune                              entrada libre
Warning: Undefined variable $pagesco_total in /var/www/html/debug_seats_spanish.php on
Warning: Undefined property: stdClass::$seats in /var/www/html/debug_seats_spanish.php
Story time: The Hobbit                       entrada libre
An evening with Ana Ruiz                     entrada libre
75 seats in the next 3 events
```

1. **Warning, line 32:** `->seats` does not exist (`capacity`): null is falsy, so every event prints "entrada libre" although it has seats.
2. **Warning, line 37:** `$pagesco_total` is added to before it exists; start it at 0.
3. **Silent, line 41:** the total uses the text domain `pagesco-books` , whose translations know no such string: it stays English with no message. The translator's file is for `pagesco-lab` .

Fixed:

```
Book club: Dune                              20 plazas
Story time: The Hobbit                       15 plazas
An evening with Ana Ruiz                     40 plazas
75 plazas en los próximos 3 eventos
```

- LAB 8.4-A (GUIDED): LABS/MODULE-16/L8_4-DEPLOYMENT-MAINTENANCE/MIGRATION_PREVIEW_STARTER.SH → MIGRATION_PREVIEW_SOLUTION.SH

As shipped:

```
tables WordPress registers: 12, all with the prefix: 13
the tables only the second list has: wp_pagesco_events
dry run, every table with the prefix, GUIDs left alone: TODO
export without --all-tables-with-prefix: TODO
export with --all-tables-with-prefix: TODO
new address in the export: TODO
options with the address inside a serialized value: TODO
the database still says: http://localhost:8080
```

The solution prints:

```
tables WordPress registers: 12, all with the prefix: 13
the tables only the second list has: wp_pagesco_events
dry run, every table with the prefix, GUIDs left alone: 9 replacements
export without --all-tables-with-prefix: 12 tables
export with --all-tables-with-prefix: 13 tables
new address in the export: 15, old address left (the GUIDs): 34
options with the address inside a serialized value: 0
the database still says: http://localhost:8080
```

Nothing changed: dry runs and exports only. 9 replacements for any new address; 15 new addresses in the export because --export processes every column in PHP; 34 GUIDs left on purpose.

- LAB 8.4-B: LABS/MODULE-16/L8_4-DEPLOYMENT-MAINTENANCE/RESTORE_DRILL_STARTER.SH → RESTORE_DRILL_SOLUTION.SH

As shipped:

```
marker post created
backup: 13 tables, events table included: 1
after the accident: TODO
after the restore: TODO
drill over: 0 "Restore drill" posts, the dump deleted
```

The solution prints:

```
marker post created
backup: 13 tables, events table included: 1
after the accident: 0 "Restore drill" posts
after the restore: 1 "Restore drill" post
drill over: 0 "Restore drill" posts, the dump deleted
```

`wp cache flush` after the import matters with a persistent object cache, which would still remember the deleted post. The marker's ID stays used.

- LAB 8.4-C: LABS/MODULE-16/L8_4-DEPLOYMENT-MAINTENANCE/CRON_AUDIT_STARTER.PHP → CRON_AUDIT_SOLUTION.PHP

As shipped:

```
scheduled events: 13
  daily        6
  hourly       1
  twicedaily   4
  weekly       2
the shop's job: TODO
WP-Cron spawning on page views: TODO
on the live server, its crontab needs:
TODO
```

The solution prints:

```
scheduled events: 13
  daily        6
  hourly       1
  twicedaily   4
  weekly       2
the shop's job: pagesco_books_daily_sync, daily, next run scheduled: yes
WP-Cron spawning on page views: on (the development default)
on the live server, its crontab needs:
*/5 * * * * cd /var/www/html && wp cron event run --due-now --quiet
```

The crontab line uses the site's own folder (here the container's /var/www/html; on a server something like /var/www/pagesco). The shop's job's next time is not printed: it depends on the day the site was saved.

- HANDOUT: LABS/MODULE-16/L8_4-DEPLOYMENT-MAINTENANCE/DEBUG_LAUNCH_CHECK.SH (DELIBERATELY BROKEN)

As shipped:

```
== Pre-launch report ==
Error: Could not get 'homeurl' option. Does it exist?
Site address:
Error: The constant or variable 'PAGESCO_BOOKS_SUPPLIER_ULR' is not defined in the 'wp
Did you mean 'PAGESCO_BOOKS_SUPPLIER_URL'?
Supplier address:
Tables in the launch export: 12
== End of report ==
```

1. **Error, line 16:** the option is `home` (or `siteurl`), not `homeurl`.
2. **Error, line 17:** a typo, `ULR`; WP-CLI even suggests the right name.
3. **Silent, line 18:** no `--all-tables-with-prefix`, so the launch export has 12 tables: the events table is missing, the Common Mistake 2 of the lesson. "Success" and a plausible number.

Fixed:

```
== Pre-launch report ==
Site address: http://localhost:8080
Supplier address: http://localhost:8080/supplier-sandbox/v1/
Tables in the launch export: 13
== End of report ==
```

The supplier's address it prints is the sandbox's: the report's job is to show that it must change for the live site.

RECORDED IN THE LESSONS (QUOTE, DO NOT RUN IN CLASS)

LESSON 8.3: THE SPANISH FRONT END

```
Success: Language activated.
<html lang="es"
<div class="pagesco-notice">Free shipping on orders over $35. Abierto hasta las 21:00 T
<span class="pagesco-event__where">Reading corner, 20 plazas
<span class="pagesco-event__where">Children&#039;s corner, 15 plazas
Success: Language activated.
```

LESSON 8.4: LAUNCH-DAY CLEAN-UP ON THE REHEARSAL SERVER (YOUR PASSWORDS DIFFER)

```
Reset password for admin.
Password: 4Zpd(Zbyl3*P@LS3d@lk^w0^
Reset password for maria.
Password: jh@XhNgavQVNted*dcZ22ine
Reset password for devon.
Password: I#3CzKYQuLy!$PKH16WqAmpf
Success: Passwords reset for 3 users.
Success: Deleted all application passwords.
Success: 4 transients deleted from the database.
Success: Deleted post 1.
Success: Deleted post 2.
Deleted 'akismet' plugin.
Deleted 'hello' plugin.
Success: Deleted 2 of 2 plugins.
Deleted 'twentytwentythree' theme.
Deleted 'twentytwentyfour' theme.
Success: Deleted 2 of 2 themes.
```

LESSON 8.4: THE FIRST BACKUP, REFUSED

```
mariadb-dump: Got error: 1044: "Access denied for user 'pagesco_app'@'%' to database 'p
total 4
-rw-r--r--    1 1000     1000           831 Oct 26 06:54 pagesco-20261026-0654.sql
```

LESSON 8.4: RELEASE 2 DEPLOYED

```
Unpacking the package...
Installing the plugin...
Removing the old version of the plugin...
Plugin updated successfully.
Success: Installed 1 of 1 plugins.
Unpacking the package...
Installing the theme...
Removing the old version of the theme...
Theme updated successfully.
Success: Installed 1 of 1 themes.
+---------------+--------+---------+
| name          | status | version |
+---------------+--------+---------+
| pagesco-books | active | 1.1.1   |
+---------------+--------+---------+
+---------+--------+---------+
| name    | status | version |
+---------+--------+---------+
| pagesco | active | 1.1.1   |
+---------+--------+---------+
```

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 8.3 · INTERNATIONALIZATION, ACCESSIBILITY & DISTRIBUTION

1. **C** 'Showing %1$s of %2$s books' with a translators comment naming both numbers: A whole sentence with numbered placeholders and a translators comment: the translator sees what each part is and may change the order.

2. **A** The MD5 hash of the script's path in the plugin, assets/js/stock-widget.js: WordPress hashes the path of the script it is about to print, relative to the plugin, and looks for a JSON file with that hash; make-json takes the paths from the PO references.

3. **B** The page passed the rules a machine can check; keyboard and screen reader tests are still needed: Automated rules catch a part of the problems, such as markup and contrast; focus visibility, meaningful alternative text, reading order and screen reader experience need people to test.

4. **C** Because the plugin is not loaded, so the genre taxonomy is not registered: During uninstall the plugin is not loaded, so its taxonomy is not registered, and get_terms() rejects unregistered taxonomies with invalid_taxonomy; register it in uninstall.php first.

5. **B** Because the development autoloader requires files from packages the ZIP does not contain: With development packages installed, the autoloader requires some of their files on every load; the ZIP does not contain them, so the plugin would die with a fatal error on the client's site.

1. **B** Only the plugin ZIP built from the release tag: Code goes up, content comes down: after the launch the live database is the original, so an update deploys only the release files built from the new tag.
2. **C** The plugin's own events table: By default WP-CLI works on the tables WordPress registers; the plugin's wp_pagesco_events table is not one of them, so the export silently leaves it out.
3. **A** The new pattern stays unknown until the cached pattern list expires: Without development mode WordPress caches the theme's pattern list in a site transient for the current Version, for up to 30 minutes; a new Version makes the cache invalid at once.
4. **C** Read and change the shop's database, and nothing beyond it: Least privilege limits the damage: the account can read and change the shop's own database, which is serious, but it reaches no other database and cannot manage users or the server.
5. **B** It never runs: the event stays scheduled and nothing starts it: With page-view spawning off, nothing runs due events until something calls wp-cron.php or wp cron event run: the job waits, scheduled, forever.

EXIT TICKET

1. **Code, content, configuration:** the plugin or theme (Git, a tagged release, a ZIP deployed: up); the books, events and pages (the database: once at launch, then created live; copies come down); wp-config.php's secrets and the supplier's key (written per environment, never copied).
2. **String rules:** any two of: whole sentences with placeholders; numbered placeholders when there are several; a translators comment; `_n()` for counts; `_x()` for ambiguous words; literal strings only; escape the translation.
3. **Hard requirements:** accept any honest pair with the right layer; the brief's own table ("Where each requirement is handled") is the reference for the conversation in Session 17.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 A ("only a warning").** Treats warnings as noise. Show the lab's Spanish line that reorders the numbers.
- **Q3 C ("under 7:1 fails").** Mixes AA and AAA.
- **Q5 A or D.** Thinks build/ or the autoloader are development files; they are what runs.
- **Q7 A ("the new header").** The code/content split has not landed: show Lesson 3.2's wp_template_part posts.
- **Q9 B ("serialized").** Confuses GUIDs with options; the guid column is a plain string.
- **Q12 A ("it works").** Has never restored a backup; `restore_drill` is the cure.

INSTRUCTOR KIT · ANSWER KEY

Session 17: Final Project Kickoff and Studio I

Answers and reasoning for every Session 17 item: the brief by layer, the self-check, the predict-then-run items, the studio reference outputs for Milestones 1 to 4, the lab reference solutions and the exit ticket.

Session 17 | 12 self-check items | Confidential

START HERE · THE BRIEF BY LAYER

#	REQUIREMENT	LAYER AND CLASS (THE LESSON'S MAP)
1	Reserve, three days, expired	`Holds::place()` computes `pickup_by` from the setting; `Holds::status()` computes "expired"
2	Limits, duplicates, sold out, settings	`Holds::place()`; settings `hold_days` and `max_holds` on Settings > Pages & Co. Books
3	My holds, cancel, only your own	the My Holds block; `HoldsTable::finish()` with the user ID in its `WHERE`; a "not found" for anyone else's hold
4	Staff list and "collected"	Books > Holds (`Admin\HoldsPage`), the collect handler, capability `edit_others_posts`
5	The email	the plugin's own action `pagesco_books_hold_placed`, and `Holds\HoldMail` listening to it
6	No JavaScript, screen readers, no reload	forms that post to `admin-post.php`; an Interactivity API store that sends the same form to the REST API; `role="status"` messages
7	Live prices on the home page	the `pagesco/book` binding source learns a book's slug; the theme's staff picks pattern binds its prices
8	Personal data, tested, safe deploys	the daily job deletes holds 30 days after their pickup date; uninstall drops the table; lint, tests, smoke test; a code-only deploy with a backup first

9.1 · SELF-CHECK

Q1

Why does the reference solution keep holds in a custom table rather than a post type?

A) A post type cannot store a date

B) A hold is one customer, one book, a status and a date: typed columns with indexes answer "this customer's holds" and "what is waiting" directly, and old rows can be deleted cleanly

C) Custom tables are always faster than wp_posts

D) Post types cannot be private

> **Why:** Lesson 9.1, "The data: why a custom table": a hold has no title, content, URL or revisions; as posts every question would be a meta query, and holds would leak into places built for content (exports, search, the posts routes).
> **A** is wrong: Post meta can store any date; the problem is querying it.
> **C** is wrong: Not always: the point is the shape of the data and the two questions the shop asks.
> **D** is wrong: They can (public => false); that is not the reason.

Q2

Where does the reference solution keep the status `expired`?

A) In the status column, set by the daily cron job

B) In a post meta field on the book

C) In a transient that expires after hold_days

D) Nowhere: `Holds::status()` computes it from `pickup_by` and today's date every time a hold is shown

> **Why:** A stored "expired" needs a job to update it and is wrong between the moment it should change and the moment the job runs. The table stores waiting, collected or cancelled; expired is derived.
> **A** is wrong: The daily job deletes old holds 30 days after pickup; it never marks them.
> **B** is wrong: Holds are not book data, and a book can have many holds.
> **C** is wrong: A transient could vanish at any time (Lesson 7.3): never the only home of data.

Q3

On the course site at the end of Lesson 8.4, what does this print? And after Milestone 4?

```
grep -c '::class,' wp-content/plugins/pagesco-books/src/Plugin.php
```

`20` (twenty features); `25` after Milestone 4 (five new features, one line each)

> **Why:** Lesson 5.1's rule: one class in src/, one line in Plugin::feature_classes(). Add each line together with its file: a line for a class that does not exist yet stops every request with "Class not found".

Q4

The plugin is already active on your development site. You add `Holds\HoldsTable::install()` to `Activator::activate()`. When is the holds table created on YOUR site?

A) On the next request that loads WordPress: `maybe_upgrade()` runs on init, finds no version option and installs it

B) Never, until you deactivate and reactivate the plugin

C) When you run `wp db query`

D) At 3 a.m., with the daily job

> **Why:** An active plugin's activation hook does not run again, and neither does it on an update (Lesson 8.4). Milestone 2's check proves it: `wp db query` (which does not start WordPress) sees no table, one `curl` later the table and version 1 exist. The same happens on the live site in Milestone 9.
>
> **B** is wrong: That would work too, but the design makes it unnecessary, and it is impossible on a live update.
>
> **C** is wrong: wp db query talks to MySQL without loading WordPress: no PHP of yours runs.
>
> **D** is wrong: The daily job deletes old holds; it does not create tables.

Q5

A reviewer finds these lines in Holds::cancel() (the reference has one call on one line). What can a logged-in customer now do?

```
$done = HoldsTable::finish(
    $hold_id,
    'cancelled'
);
if ( ! $user_id || ! $done ) {
```

A) Nothing new: the $user_id check still stops it

B) Cancel only their own holds, as before

C) Cancel ANY customer's waiting hold by sending its ID

D) Collect holds as staff

Why: finish() puts the user ID into the SQL WHERE only when it is passed. Without it, the WHERE is id and status: any logged-in customer cancels anyone's hold. This is the insecure direct object reference of the Security and Privacy Checklist; HoldsTest's test_only_the_customer catches it (Milestone 7).
A is wrong: $user_id only proves someone is logged in, not that the hold is theirs.
B is wrong: That needs the owner in the query, which this line no longer passes.
D is wrong: Collecting is another method and another capability.

Q6

Why does `Holds::place()` answer the limit, a duplicate and a sold-out book with status 409, not 400 or 403?

A) 409 is the only status WordPress allows in a WP_Error

B) The request is well formed and the customer is allowed to ask; the shop's current state says no (Conflict)

C) 403 would log the customer out

D) 409 makes the browser retry automatically

> **Why:** 400 means a malformed request (a missing or wrong argument, which the REST route's args catch), 401/403 who is asking, 404 no such thing. Every door passes the status in the error's data on, so the REST route and the form handler answer the same.
> **A** is wrong: Any status can go in the data array.
> **C** is wrong: No status logs anyone out.
> **D** is wrong: Browsers do not retry on 409.

Q7

Common Mistake 2, run as Devon (an author) about Maria's post 15. What does it print, and which check belongs in a handler?

```
wp eval 'var_dump( current_user_can( "delete_posts" ), current_user_can( "delete_po
```

`bool(true)` then `bool(false)`: the meta capability with the ID, `delete_post`, 15

> **Why:** delete_posts asks "may this user delete posts at all?" (his own, yes). A handler must ask about THIS post. Holds are not posts and have no meta capability, so there ownership goes into the SQL WHERE with the user ID from the login.

Q8

Why does the confirmation email live in `HoldMail`, listening to `pagesco_books_hold_placed`, instead of a `wp_mail()` call inside `Holds::place()`?

A) wp_mail() cannot be called from a class

B) Actions run faster than function calls

C) Emails must be sent from a must-use plugin

D) The rules class knows nothing about email: others (an SMS, a staff notice in Exercise 2) attach to the same moment without changing the rules

> **Why:** An action is an extension point for a moment others may want to act at (Lesson 2.3). The rules class stays one job; Exercise 2 adds the shop's email at priority 20 from outside the plugin.
> **A** is wrong: Any PHP code can call wp_mail().
> **B** is wrong: do_action() adds a little work, not speed.
> **C** is wrong: Any plugin can send email.

Q9

Which line computes the last day of pickup, three days from today, the way the reference does?

A) `wp_date( 'Y-m-d', time() + $days * DAY_IN_SECONDS )`

B) `date( 'Y-m-d', time() + $days * DAY_IN_SECONDS )`

C) `current_time( 'mysql', true )`

D) `gmdate( 'Y-m-d' ) . ' + 3'`

> **Why:** pickup_by is a date in the shop's time zone (the counter's calendar): wp_date() formats a timestamp in the site's time zone. date() and gmdate() use PHP's own zone (UTC in WordPress), which moves the day near midnight.
> **B** is wrong: date() ignores the site's time zone setting.
> **C** is wrong: That is now, in UTC, as a datetime: what created_at stores.
> **D** is wrong: String concatenation, not date arithmetic.

Q10

`HoldsTable::waiting()` uses this trick. What does it print, and what does it mean for the query?

```
wp eval 'global $wpdb; echo $wpdb->prepare( "WHERE ( %d = 0 OR user_id = %d )", 0,
```

`WHERE ( 0 = 0 OR user_id = 0 )`: always true, so the user filter is switched off (all customers); with 4 it becomes `( 4 = 0 OR user_id = 4 )`

> **Why:** One prepared statement serves every combination of filters: a 0 turns its filter off, any other value keeps it, and every value still goes through prepare().

Q11

Milestone 4 sends a forged "Mark collected" from Maria's own logged-in session, but without the form's nonce. What does admin-post.php answer?

A) 303, back to Books > Holds with ?pagesco_hold=collected

B) 401: Maria is not logged in

C) 403 with `The link you followed has expired.`

D) 400: the action is unknown

> **Why:** check_admin_referer() fails without a valid nonce for pagesco_books_collect_hold_5, and WordPress dies with Lesson 5.3's message and 403. The nonce is what tells a real button press from a request another page sent with Maria's cookies.
> **A** is wrong: That is what the real button gets.
> **B** is wrong: Her cookies are valid: that is what makes the forgery dangerous.
> **D** is wrong: The handler exists; 400 is what admin-post.php answers when no handler does.

Q12

The lesson's mental model is the shop's counter. Name its three layers and the one job of each.

the data (a table class that only stores), the rules (one class, Holds, that only decides), the doors (REST routes, forms, blocks that only check who is asking and pass the question on)

> **Why:** When a rule changes, it changes in one place and every door follows. The guide's requirement map shows which layer handles each line of the brief.

9.1 · PREDICT, THEN RUN

P1

Predict the output, then run it.

```
wp eval 'var_dump( current_user_can( "edit_post", 15 ), current_user_can( "edit_pos
```

```
bool(false)
bool(true)
```

Why: Same lesson as Q7, other verb: `edit_post` with the ID asks about Maria's post (no), `edit_posts` asks about posts in general (yes: his own). Every "may I change THIS?" check takes the ID.

P2

Predict the output, then run it.

```
grep -c '::class,' wp-content/plugins/pagesco-books/src/Plugin.php
```

```
20
```

Why: Twenty features before the project; Milestone 2 makes it 21, Milestone 3 22, Milestone 4 25 (Q3).

P3

Predict the output, then run it.

```
wp eval 'echo get_option( "date_format" ), " | ", wp_date( get_option( "date_format
```

```
F j, Y | November 5, 2026
```

Why: `Holds::label()` formats `pickup_by` with the site's `date_format` option through `wp_date()`; the date is read at noon so no time zone can move it to the day before.

P4

Predict the output, then run it.

```
wp eval 'global $wpdb; echo $wpdb->prepare( "WHERE ( %d = 0 OR user_id = %d )", 4,
```

```
WHERE ( 4 = 0 OR user_id = 4 )
```

Why: With a real ID the condition keeps the filter: only user 4's rows. With 0 it is always true (Q10).

STUDIO REFERENCE OUTPUTS (MILESTONES 1 TO 4)

The lesson's own files and checks, run in page order on a site at the end of Lesson 8.4. Every output here equals the lesson's output card (the generator compares them); the lines the lesson prints as `...` (the phpcs time, the pickup date) change with every run. Use them at the check-ins.

MILESTONE 1: THE BASELINE

```
Success: Added the constant 'SAVEQUERIES' to the 'wp-config.php' file with the raw valu
[26-Oct-2026 12:57:28 UTC] /: 22 queries; missing options: can_compress_scripts, site_
[26-Oct-2026 12:57:28 UTC] /books/dune/: 25 queries; missing options: can_compress_scr
[26-Oct-2026 12:57:28 UTC] /find-a-book/: 25 queries; missing options: can_compress_scr
Success: Deleted the constant 'SAVEQUERIES' from the 'wp-config.php' file.
```

MILESTONE 2: THE LINE COUNT, THE TABLE ON THE FIRST REQUEST, THE SETTINGS

```
21
```

```
holds_tables
0

COLUMN_NAME COLUMN_TYPE
id  bigint unsigned
book_id bigint unsigned
user_id bigint unsigned
status  varchar(20)
pickup_by   date
created_at  datetime

1
```

```
int(3)
int(3)
```

```
<h2 id="wp-settings-section-pagesco_books_notice-1">Store notice</h2>
<h2 id="wp-settings-section-pagesco_books_lists-2">Catalog and events</h2>
<h2 id="wp-settings-section-pagesco_books_holds-3">Reserve & Pick Up</h2>
<h2 id="wp-settings-section-pagesco_books_uninstall-4">Uninstall</h2>
```

MILESTONE 3: HOLDS-CHECK.PHP, THE NEWEST EMAIL, THE TABLE

```
Jordan reserves Dune                 hold 1: waiting
Jordan reserves Dune again           pagesco_hold_duplicate
Jordan reserves The Great Gatsby     pagesco_hold_sold_out
Jordan reserves 1984                 hold 2: waiting
Jordan reserves The Hobbit           hold 3: waiting
Jordan reserves Animal Farm          pagesco_hold_limit
Priya cancels hold 1                 pagesco_hold_not_found
Jordan cancels hold 1                hold 1: cancelled
Jordan reserves Animal Farm          hold 4: waiting
A visitor reserves Dune              pagesco_hold_login_required
Animal Farm waits until: 3 days from today
```

```
From: WordPress <wordpress@example.com>
To: jordan@example.com
Subject: Your hold at Pages & Co.: Animal Farm

Hello Jordan,

We have put Animal Farm aside for you. Pick it up at the counter by:
...

Your holds: http://localhost:8080/my-holds/

See you soon at Pages & Co.
```

```
+----+---------+---------+-----------+
| id | book_id | user_id | status    |
+----+---------+---------+-----------+
|  1 |      30 |       4 | cancelled |
|  2 |      31 |       4 | waiting   |
|  3 |      35 |       4 | waiting   |
|  4 |      40 |       4 | waiting   |
+----+---------+---------+-----------+
```

MILESTONE 4: THE ROUTES AND THE DOORS

The routes, then a visitor:

```
/pagesco/v1/holds
/pagesco/v1/holds/(?P<id>\d+)
```

```
{
  "code": "rest_forbidden",
  "message": "Log in to reserve books.",
  "data": {
    "status": 401
  }
}
```

Jordan (three waiting) and Priya:

```
{
  "code": "pagesco_hold_limit",
  "message": "You already have 3 books waiting. Collect or cancel one first.",
  "data": {
    "status": 409
  }
}
```

```
HTTP/1.1 201 Created

{
  "id": 5,
  "book": 30,
  "title": "Dune",
  "status": "waiting"
}
```

Jordan's DELETE of Priya's hold 5, then Jordan's list:

```
{
  "code": "pagesco_hold_not_found",
  "message": "You have no waiting hold with that ID.",
  "data": {
    "status": 404
  }
}
```

```
[
  {
    "id": 4,
    "title": "Animal Farm",
    "status": "waiting"
  },
  {
    "id": 3,
    "title": "The Hobbit",
    "status": "waiting"
  },
  {
    "id": 2,
    "title": "1984",
    "status": "waiting"
  },
  {
    "id": 1,
    "title": "Dune",
    "status": "cancelled"
  }
]
```


Books > Holds as Maria (the headings), as Devon, and the forged POST:

```
Book
Customer
Pick up by
Action
```

```
Sorry, you are not allowed to access this page.
```

```
HTTP/1.1 403 Forbidden

<div class="wp-die-message">The link you followed has expired.</div>
```

The commit:

```
Reserve & Pick Up: holds table, settings, rules, email, REST routes, form handlers, sta
```

LAB REFERENCE SOLUTIONS

Every output below is real: the lab files run exactly as `labs/check_labs.py` runs them, on a site at the end of Lesson 8.4 (`after:lesson_29`), copied into the WordPress root and run with `wp eval-file --use-`

`include`.

- LAB 9.1-A (GUIDED): LABS/MODULE-17/L9_1-FINAL-PROJECT/OWNERSHIP_SQL_STARTER.PHP → OWNERSHIP_SQL_SOLUTION.PHP

As shipped:

```
Devon closes wish 1          TODO
Devon closes wish 3          TODO
Maria closes wish 1          TODO
Maria closes wish 1 again    TODO
A visitor closes wish 2      TODO
Maria's wishes, newest first:
  TODO (pagesco_lab_wishes_of() returns nothing yet)
```

The solution prints:

```
Devon closes wish 1          pagesco_wish_not_found (404)
Devon closes wish 3          closed
Maria closes wish 1          closed
Maria closes wish 1 again    pagesco_wish_not_found (404)
A visitor closes wish 2      pagesco_wish_not_found (404)
Maria's wishes, newest first:
  wish 2: The Hobbit, open
  wish 1: Dune, closed
```

What to look for: the owner in the update's WHERE (three keys, three formats), `1 ===` on `$wpdb->update()` (it returns the number of rows changed, 0 when nothing matched, false on an error), the user from `get_current_user_id()`, one error for all refusals. Maria's second attempt is also a 404: the wish is no longer open, the same "not found" as Lesson 9.1's cancel of a hold that is not waiting.

- LAB 9.1-B: LABS/MODULE-17/L9_1-FINAL-PROJECT/RSVP_RULES_STARTER.PHP → RSVP_RULES_SOLUTION.PHP

As shipped (the duplicate and the limit are not checked yet, the status says TODO):

```
A visitor books event 1          pagesco_rsvp_login_required (401)
Jordan books event 99            pagesco_rsvp_no_event (404)
Jordan books event 2             Story time: The Hobbit
Jordan books Story time again    Story time: The Hobbit
Jordan books event 5             Book club: Fahrenheit 451
Jordan books event 7             Book club: Pride and Prejudice
Priya books event 7              Book club: Pride and Prejudice
Book club: Dune on 2027-11-04    TODO
Book club: Dune on 2027-11-05    TODO
```

The solution prints:

```
A visitor books event 1          pagesco_rsvp_login_required (401)
Jordan books event 99            pagesco_rsvp_no_event (404)
Jordan books event 2             Story time: The Hobbit
Jordan books Story time again    pagesco_rsvp_duplicate (409)
Jordan books event 5             Book club: Fahrenheit 451
Jordan books event 7             pagesco_rsvp_limit (409)
Priya books event 7              Book club: Pride and Prejudice
Book club: Dune on 2027-11-04    upcoming
Book club: Dune on 2027-11-05    past
```

The order of the rules matters: who is asking first (no query for visitors), then whether the thing exists, then the shop's state. Count the existing seats BEFORE adding (`>= 2`); a learner who counts after adding refuses the second seat. The computed status compares two strings of the same format, as `Holds::status()` does with dates.

LAB 9.1-C: LABS/MODULE-17/L9_1-FINAL-PROJECT/OWN_ACTION_STARTER/ → OWN_ACTION_SOLUTION/ WITH TEST_OWN_ACTION.PHP

With the starter's `pagesco-lab-requests.php` in mu-plugins/:

```
Listeners on the action: 1
Listeners heard: nothing yet (TODO 1)
```

With the solution's:

```
Listeners on the action: 2
10 customer note: Dune for jordan
20 shop note: order "Dune" (request 1)
10 customer note: Emma for priya
20 shop note: order "Emma" (request 2)
```

The customer's note at 10 runs before the shop's at 20 for each request; the shop's listener needs `20, 2` (priority, accepted arguments): without the 2, PHP stops with an ArgumentCountError the moment the action fires (worth letting a learner see once). Remove the file afterwards: `rm wordpress/wp-content/mu-plugins/pagesco-lab-requests.php`.

HANDOUT: LABS/MODULE-17/L9_1-FINAL-PROJECT/DEBUG_TRASH_DOOR.PHP (DELIBERATELY BROKEN)

As shipped, PHP and WordPress print:

```
Notice: Function wpdb::prepare was called <strong>incorrectly</strong>. The query does
devon may trash "3": yes
Reason: Warning: Undefined array key "why" in /var/www/html/debug_trash_door.php on lir
[new lines in wp-content/debug.log]
[26-Oct-2026 12:59:26 UTC] PHP Notice:  Function wpdb::prepare was called <strong>incor
[26-Oct-2026 12:59:26 UTC] PHP Warning:  Undefined array key "why" in /var/www/html/deb
```

1. **doing_it_wrong (a Notice), line 32:** three placeholders (`%i`, `%d`, `%s`) and two values. `prepare()` refuses and returns an empty string, and `$wpdb->get_var( '' )` returns the PREVIOUS query's answer: the `"3"` is Devon's user ID from `get_user_by()`. Add the third value, `'post'`.
2. **Warning, line 36:** the request has `reason`, the code reads `why`. Read keys from the place that defines them.
3. **Silent, line 28:** `current_user_can( 'delete_posts' )` asks whether Devon may delete posts at all (his own: yes). The door must ask about this post: `current_user_can( 'delete_post', $pagesco_post_id )`. Lesson 9.1's Common Mistake 2; the dangerous one, because nothing announces it.

Fixed, it prints:

```
devon may trash "Welcome to the new Pages & Co. website": no
Reason: duplicate
```

OPENING: READINESS

There is no graded warm-up. The readiness check's correct state: `git tag -n` lists release-1 ("First launch") and release-2 ("Plugin 1.1.1, theme 1.1.1"), the working copy is clean, the plugin is 1.1.1 and pagesco-live's three services run. Learners who were not there leave with a note on the board and fix it before Session 18.

EXIT TICKET

1. **Three layers:** data (HoldsTable: the table and its queries, no rules), rules (Holds: every rule, the status, the messages), doors (HoldActions for the forms, HoldsEndpoint for REST, the Reserve and My Holds blocks; HoldsPage is the staff's screen).
2. **Priya's DELETE:** `404 pagesco_hold_not_found` ("You have no waiting hold with that ID."). `Holds::cancel()` calls `HoldsTable::finish( $hold_id, 'cancelled', $user_id )` with her ID from the login; the update's WHERE matches no row.
3. **Next milestone:** any honest answer with the lesson's done-when; Milestone 4's is the forged request answered 403 and the commit, Milestone 5's the smoke test's eight ok lines.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 C ("tables are faster").** Speed is not the argument; the shape of the data and its two questions are. Ask what a hold's title would be.
- **Q2 A ("the cron job marks them").** Thinks of a status as something stored. Ask: what does the page show between midnight and the job's run?

- **Q4 B ("reactivate").** Has not yet met the update path; it matters on the live site, where nobody reactivates (Milestone 9).
- **Q5 A.** Confuses "logged in" with "owns it". The guided lab separates the two.
- **Q6 C or D.** Guessing at statuses: hand out the five used (200/201, 401, 403, 404, 409) as a card.
- **Q9 B ("date()").** Works on most days and moves the pickup day near midnight. Show `wp_timezone_string()` on their site.
- **Q11 A.** Thinks the logged-in cookie is enough: exactly why forgeries work without a nonce.

INSTRUCTOR KIT · ANSWER KEY

Session 18: Final Project Studio II, Showcase and Next Steps

Answers and reasoning for every Session 18 item: both self-checks and predict items, the studio reference outputs for Milestones 5 to 9, the lab reference solutions, Lesson 9.2's reference outputs, the lesson pages' own quizzes and the exit ticket.

`Session 18` `12 self-check items` `Confidential`

9.1 · SELF-CHECK

Q1

Milestone 7: `composer lint` finds 11 warnings, all in HoldsTable.php (DirectQuery and NoCaching). What does the reference do?

A) Adds a phpcs:ignore comment to every query

B) Turns the two sniffs off in phpcs.xml.dist for the whole plugin

C) Documents ONE exception in phpcs.xml.dist, for that file, with its reason: a custom table has no WordPress API, and holds are per user and change with every click, so a cache would mostly be invalidated before it is read

D) Moves the holds into post meta so the warnings go away

Why: Lesson 8.1's rule: fix a finding when the fix is cheap, document an exception when it is not, where the events table's exception already is. The events, read by every visitor, ARE cached (Lesson 8.2): the decision is per table, with a reason.

A is wrong: Eleven ignore comments hide the decision instead of writing it down once.

B is wrong: That would also silence the sniffs for code that should be fixed.

D is wrong: That trades a documented decision for the slow meta queries the lesson argued against.

Q2

On the reference after Milestone 8, what does this print? Why did the THEME's version change too?

```
wp eval 'echo wp_get_theme( "pagesco" )->get( "Version" ), " ", PAGESCO_BOOKS_VERSI
```

`1.2.0 1.2.0` : the theme's template and pattern changed, and without development mode the live site caches pattern files until the theme's Version changes (Lesson 8.4)

> **Why:** A new feature is a minor version (1.2.0) for the plugin; the theme gets one because templates/single-book.html and patterns/featured-books.php changed. Version_audit in the kit's labs reads all five places a version lives.

Q3

Milestone 9, on the rehearsal server: right after `wp plugin install /deploy/pagesco-books-1.2.0.zip --force`, `SHOW TABLES LIKE` finds no holds table. Why not yet?

A) `wp plugin install` ran with the OLD plugin in memory, and `wp db query` does not start WordPress: no new code has run init yet

B) The ZIP was built without src/Holds

C) The database user may not create tables

D) Tables are only created by the activation hook, so it never will be

> **Why:** The next command that loads WordPress (wp theme install, in the lesson) runs the new code; on init, HoldsTable::maybe_upgrade() finds no version option and creates the table. On a real server the first page view does the same.
>
> **B** is wrong: That ZIP would stop every request with "Class not found" (Common Mistake 1), not leave a quiet gap.
>
> **C** is wrong: pagesco_app has CREATE (Lesson 8.4's grants); the table appears one command later.
>
> **D** is wrong: That is exactly what maybe_upgrade() is for.

Q4

After the deploy, the rehearsal server answers 404 for /my-holds/. What is the right fix?

A) Import the development database on the live site

B) Deploy the plugin again with --force

C) Flush the permalinks on the development site

D) Create the page ON the live site (it is content: it lives in the database), with `wp post create`, as the readme's upgrade notice says

> **Why:** Code goes up, content stays (Lesson 8.4). The page made in Milestone 5 is a row in the development database. The recorded run: 404 before, 200 after creating it.
>
> **A** is wrong: That would overwrite the live shop's real data with test customers and test holds.
>
> **B** is wrong: The code is already there; the page is not code.
>
> **C** is wrong: The live site's permalinks are fine: the page simply does not exist there.

Q5

Milestone 7 measures the front page at 31 queries for a visitor; the baseline was 22. Where do the nine come from, and why does the reference accept them?

three per staff pick (get_page_by_path() for the slug, then the book and its meta) for the live prices; indexed primary-key lookups under a millisecond each, the page has no per-user content so a page cache serves it, and an object cache keeps them

> **Why:** Decide with numbers (Lesson 8.2) and write the decision down. A book's page stays at 25: the Reserve block leaves before its first query when nobody is logged in.

Q6

Why does each Cancel button on My holds carry an aria-label that names its book?

```
<button type="submit" class="wp-element-button" aria-label="Cancel the hold on 1984
<button type="submit" class="wp-element-button" aria-label="Cancel the hold on Dune
```

A) Search engines index aria-labels

B) A screen reader would otherwise read "Cancel, Cancel, Cancel" with no way to tell which hold each one cancels

C) WordPress requires an aria-label on every button

D) It makes the button work without JavaScript

> **Why:** Every control needs a name that makes sense out of context (Lesson 8.3). The visible text stays short; the accessible name says what it acts on.
> **A** is wrong: Accessibility names are for assistive technology, not search.
> **C** is wrong: Nothing in core enforces it.
> **D** is wrong: The form works without JavaScript because it posts to admin-post.php.

9.1 · PREDICT, THEN RUN

P1

Predict the output, then run it.

```
wp eval 'echo count( PagesCo\Books\Holds\HoldsTable::waiting( "2000-01-01" ) ), "\n
```

```
2
```

> **Why:** `waiting()` returns the holds that are waiting and whose pickup date is on or after the date given; from 2000-01-01 nothing counts as expired, so both waiting holds of the saved site (Jordan's Dune and 1984) count. From today the answer depends on the day: that is the computed "expired".

P2

Predict the output, then run it.

```
wp option get pagesco_books_holds_db_version
```

```
1
```

Why: The schema version the code wrote after creating the table; `maybe_upgrade()` compares it with `HoldsTable::DB_VERSION` on every init and installs again only when the code is newer.

9.2 · SELF-CHECK

Q7

A front end on another origin asks the shop's API. What does this print, and does it make the holds routes public?

```
curl -s -I -H 'Origin: https://shop.example' http://localhost:8080/wp-json/pagesco/
  | tr -d '\r' | grep -i '^access-control-allow-origin'
```

`Access-Control-Allow-Origin: https://shop.example` **; no: WordPress echoes any origin, so a page anywhere may READ the answers, but the permission callbacks still refuse (a visitor gets 401 from /holds)**

Why: CORS decides what a browser lets a page read, not who may act. Safety comes from the permission callbacks and nonces, never from CORS (Lesson 9.2's key takeaway).

Q8

A headless storefront shows the heading "Pages & Co." (the entity, as text). What is the right fix?

A) Switch the heading to innerHTML

B) Change the site title in Settings to "Pages and Co."

C) Know which fields are HTML: decode the stored-escaped name once to text (DOMParser), then put it on the page with textContent

D) Escape it again with encodeURIComponent()

Why: blogname is stored HTML-escaped and the REST index returns it that way (Lesson 1.2, 9.2's Common Mistake 1). textContent is safe and shows the entity literally; innerHTML would show the ampersand and run any HTML in the field.
A is wrong: It fixes the ampersand and opens the page to any HTML that reaches the field.
B is wrong: Changes the shop's name to hide a bug.
D is wrong: URL encoding is for addresses, not text on the page.

Q9

What does WordPress 7.1 ship for AI?

A) Plumbing: the AI client (wp_ai_client_prompt()), the Connectors screen and a switch to turn AI off; no model, no provider and no core feature

B) A small built-in model for alt text

C) A paid OpenAI integration enabled on every site

D) Nothing: AI is only in the separate AI plugin

Why: On the course site wp_supports_ai() is true, but no installed provider offers a model, so prompts return a WP_Error; core never calls the client itself (Q10).
B is wrong: Core ships no model.
C is wrong: Providers are separate plugins, inactive until installed, and need a key.
D is wrong: The client and Connectors ARE in core since 7.0; the features are in plugins.

Q10

What does this print on the course site, and what does it tell you?

```
grep -rl 'wp_ai_client_prompt *(' wp-admin wp-includes
```

`wp-includes/ai-client.php` : only the file that defines the function; no core screen, block or feature calls the AI client

> **Why:** Core provides one shared, provider-neutral client; plugins (the WordPress project's AI plugin first) build the features.

Q11

Lesson 9.2's model-check.php lists the pagesco/check-stock ability as a tool for a model. Which name does the model see?

A) `pagesco/check-stock`

B) `check_stock`

C) `pagesco-check-stock`

D) `wpab__pagesco__check-stock`

> **Why:** The ability's name made safe for function-calling APIs: the prefix wpab__ and the slash as a double underscore. The resolver then runs the ability as the current user: data for Maria, ability_invalid_permissions for Devon.
>
> **A** is wrong: Function-calling APIs do not allow the slash.
>
> **B** is wrong: The namespace stays in the name.
>
> **C** is wrong: A double underscore replaces the slash, after the wpab__ prefix.

Q12

Before the repository goes public, Lesson 9.2 searches with `git log --all -S` , not only the current files. Why?

a secret that was committed once and deleted later is still in the history, and anyone who clones the repository can read it; -S lists every commit that added or removed the string, on every branch

> **Why:** The kit's publish_scan lab checks the files on disk (what you are about to commit); the history search checks what you already committed. Both, before publishing.

9.2 · PREDICT, THEN RUN

P3

Predict the output, then run it.

```
curl -s 'http://localhost:8080/wp-json/?_fields=name'
```

```
{"name":"Pages &amp;amp; Co."}
```

Why: The site title as WordPress stores it, HTML-escaped. A front end decodes it once to text (Lesson 9.2's Common Mistake 1).

P4

Predict the output, then run it.

```
wp eval 'var_dump( function_exists( "wp_ai_client_prompt" ), wp_supports_ai() );'
```

```
bool(true)
bool(true)
```

Why: The client exists (7.0+) and nothing switched AI off. That says nothing about a model: `is_supported_for_text_generation()` is false on the course site.

STUDIO REFERENCE OUTPUTS (MILESTONES 5 TO 9)

Lesson 9.1's own files and checks run in page order on a site at the end of Lesson 8.4: every output equals the lesson's card (lines the lesson prints as `...` are tool timings). Milestone 9 and the wp-env run are the lesson's RECORDED cards, labelled.

MILESTONE 5 (HOMEWORK): THE FOUR STATES AND THE SMOKE TEST

```
<p class="wp-block-pagesco-reserve"><a href="http://localhost:8080/wp-login.php?redire
```

```
Log in to see your holds
```

```
<button type="submit" class="wp-element-button" data-wp-bind--disabled="context.busy">
```

```
<p class="wp-block-pagesco-reserve__message" role="status" data-wp-text="context.messag
```

```
ok      visitor sees the login link
ok      REST refuses a visitor (status 401)
ok      jordan can log in
ok      a form without a nonce is refused (status 403)
ok      the form reserves the book
ok      My holds shows the waiting hold
ok      the customer cancels the hold
ok      a customer cannot mark holds collected (status 403)
exit status 0
```

MILESTONE 6 (HOMEWORK): LIVE PRICES, AND WITHOUT THE PLUGIN

```
<p class="has-small-font-size wp-block-paragraph">$14.99</p>
<p class="has-small-font-size wp-block-paragraph">$10.99</p>
<p class="has-small-font-size wp-block-paragraph">$10.49</p>
Success: Updated custom field 'pagesco_price'.
<p class="has-small-font-size wp-block-paragraph">$12.99</p>
Success: Updated custom field 'pagesco_price'.
```

```
<p class="has-small-font-size wp-block-paragraph">Ask us for the price</p>
+---------------+--------+
| name          | status |
+---------------+--------+
| pagesco-books | active |
+---------------+--------+
```

MILESTONE 7: LINT, TESTS (RECORDED), SPEED, ACCESSIBILITY

```
PHP CODE SNIFFER REPORT SUMMARY
----------------------------------------------------------------------
FILE                                                  ERRORS  WARNINGS
----------------------------------------------------------------------
src/Holds/HoldsTable.php                              0       11
----------------------------------------------------------------------
A TOTAL OF 0 ERRORS AND 11 WARNINGS WERE FOUND IN 1 FILE
----------------------------------------------------------------------


...

Script phpcs handling the lint event returned with error code 1
exit status 1
```

```
exit status 0
```

Recorded in wp-env:

```
Installing...
Running as single site... To run multisite, use -c tests/phpunit/multisite.xml
Not running ajax tests. To execute these, use --group ajax.
Not running ms-files tests. To execute these, use --group ms-files.
Not running external-http tests. To execute these, use --group external-http.
PHPUnit 9.6.38 by Sebastian Bergmann and contributors.

Book Shortcode (PagesCo\Books\Tests\BookShortcode)
 ✔ Renders a published book
 ✔ Prints nothing for drafts and pages
 ✔ Escapes the author

Books Endpoint (PagesCo\Books\Tests\BooksEndpoint)
 ✔ Filters by genre and sorts by price
 ✔ Rejects an unknown genre
 ✔ Every book matches the schema

Daily Sync (PagesCo\Books\Tests\DailySync)
 ✔ Schedules one daily event at three
 ✔ Run removes only past events

Holds (PagesCo\Books\Tests\Holds)
 ✔ Place keeps the book for the set days and sends an email
 ✔ Refuses duplicates sold out books and too many holds
 ✔ An expired hold is computed and frees the book
 ✔ Only the customer can cancel their hold
 ✔ The rest route needs a logged in customer

Supplier Client (PagesCo\Books\Tests\SupplierClient)
 ✔ Reads availability by isbn
 ✔ Turns an error status into a wp error
 ✔ Rejects an answer without items
 ✔ Passes on a network failure

Time: 00:00.214, Memory: 46.50 MB

OK (17 tests, 47 assertions)
```

```
PHPUnit 9.6.38 by Sebastian Bergmann and contributors.

F                                                                   1 / 1 (100%)

Time: 00:00.041, Memory: 44.50 MB

There was 1 failure:

1) PagesCo\Books\Tests\HoldsTest::test_only_the_customer_can_cancel_their_hold
Failed asserting that stdClass Object (...) is an instance of class "WP_Error".

/wordpress-phpunit/includes/abstract-testcase.php:879
/var/www/html/wp-content/plugins/pagesco-books/tests/HoldsTest.php:117

FAILURES!
Tests: 1, Assertions: 1, Failures: 1.
```

```
Success: Added the constant 'SAVEQUERIES' to the 'wp-config.php' file with the raw valu
[26-Oct-2026 13:00:04 UTC] /: 31 queries; missing options: can_compress_scripts, site_t
[26-Oct-2026 13:00:04 UTC] /books/dune/: 25 queries; missing options: can_compress_scri
[26-Oct-2026 13:00:04 UTC] /find-a-book/: 25 queries; missing options: can_compress_scr
Success: Deleted the constant 'SAVEQUERIES' from the 'wp-config.php' file.
```

```
<button type="submit" class="wp-element-button" aria-label="Cancel the hold on 1984">Ca
<button type="submit" class="wp-element-button" aria-label="Cancel the hold on Dune">Ca
```

MILESTONE 8: POT, TAG, BUILD, THE ZIP CHECK

```
Plugin file detected.
Success: POT file successfully generated.
191
2
```

```
 33 files changed, 1648 insertions(+), 50 deletions(-)
```

```
Composer could not detect the root package (pagesco/pagesco-books) version, defaulting
Installing dependencies from lock file
Verifying lock file contents can be installed on current platform.
Nothing to install, update or remove
Generating optimized autoload files
```

```
4
8
bool(true)
bool(true)
Version: 1.2.0
pagesco-books.zip  pagesco.zip
```

MILESTONE 9 (RECORDED ON THE REHEARSAL SERVER)

```
holds-smoke.sh
pagesco-1.1.1.zip
pagesco-1.2.0.zip
pagesco-books-1.1.1.zip
pagesco-books-1.2.0.zip
pagesco-live.sql
pagesco-site.php
```

```
Success: Exported to '/backups/pagesco-before-1.2.0.sql'.
-rw-r--r--    1 1000     1000        109925 Oct 26 07:43 /backups/pagesco-before-1.2.0
```

```
Unpacking the package...
Installing the plugin...
Removing the old version of the plugin...
Plugin updated successfully.
Success: Installed 1 of 1 plugins.
Unpacking the package...
Installing the theme...
Removing the old version of the theme...
Theme updated successfully.
Success: Installed 1 of 1 themes.
Tables_in_pagesco (wp_pagesco_holds)
wp_pagesco_holds

1
+---------------+--------+---------+
| name          | status | version |
+---------------+--------+---------+
| pagesco-books | active | 1.2.0   |
+---------------+--------+---------+
+---------+--------+---------+
| name    | status | version |
+---------+--------+---------+
| pagesco | active | 1.2.0   |
+---------+--------+---------+
```

```
404
```

```
57
200
4
+------------+--------------+
| user_login | display_name |
+------------+--------------+
| shoptest   | Shop Test    |
+------------+--------------+
```

```
ok      visitor sees the login link
ok      REST refuses a visitor (status 401)
ok      shoptest can log in
ok      a form without a nonce is refused (status 403)
ok      the form reserves the book
ok      My holds shows the waiting hold
ok      the customer cancels the hold
ok      a customer cannot mark holds collected (status 403)
exit status 0
```

The development site's history afterwards:

```
0
Pages & Co. Books 1.2.0 and theme 1.2.0: Reserve & Pick Up
Reserve & Pick Up blocks, live prices in the staff picks
Reserve & Pick Up: holds table, settings, rules, email, REST routes, form handlers, sta
Pages & Co. Books 1.1.1 and theme 1.1.1: HTTPS for the supplier, Update URI
Pages & Co. release 1: theme 1.1.0, Pages & Co. Books 1.1.0, site plugin 1.2.0
release-1       First launch
release-2       Plugin 1.1.1, theme 1.1.1
release-3       Plugin 1.2.0, theme 1.2.0: Reserve & Pick Up
```

LAB REFERENCE SOLUTIONS

Every output below is real: the lab files run exactly as `labs/check_labs.py` runs them, `l9_1-final-project/` on a site at the end of Lesson 8.4, `l9_2-next-steps/` at the end of Lesson 9.1.

- LAB 9.1-A: LABS/MODULE-18/L9_1-FINAL-PROJECT/VERSION_AUDIT_STARTER.PHP → VERSION_AUDIT_SOLUTION.PHP

As shipped:

```
pagesco-books.php header   1.1.1
PAGESCO_BOOKS_VERSION      TODO
readme.txt Stable tag      TODO
package.json version       TODO
theme style.css Version    TODO
Verdict: TODO
```

The solution prints:

```
pagesco-books.php header   1.1.1
PAGESCO_BOOKS_VERSION      1.1.1
readme.txt Stable tag      1.1.1
package.json version       1.1.1
theme style.css Version    1.1.1
All five agree: 1.1.1
```

On a learner's site after Milestone 8 every line says 1.2.0. `get_plugin_data()` with `false, false` neither marks up nor translates (and so does not load the text domain too early); `get_file_data()` reads any `Name: value` header line, the readme's Stable tag included; theme header names are case-sensitive (`Version`).

- LAB 9.1-B: LABS/MODULE-18/L9_1-FINAL-PROJECT/FILES_LIST_STARTER.PHP → FILES_LIST_SOLUTION.PHP

As shipped:

```
src/Holds (planned)        TODO
src/*.php: TODO
```

The solution prints:

```
src/Admin                  shipped
src/Rest                   shipped
src/Shortcodes             shipped
src/Supplier               shipped
src/blocks                 not shipped, on purpose: build/ ships the blocks
src/Holds (planned)        MISSING from "files": add it in the same commit
src/*.php: 13 files, shipped by the pattern "src/*.php"
```

src/blocks is left out on purpose (the built blocks ship from build/); src/Holds is the folder Milestone 2 creates, missing from the list until Milestone 8 adds it. After the project, it says shipped.

- LAB 9.1-C: LABS/MODULE-18/L9_1-FINAL-PROJECT/SMOKE_REST_STARTER.SH → SMOKE_REST_SOLUTION.SH

As shipped:

```
ok      Dune's page answers (status 200)
TODO    the REST API lists 11 books
TODO    an unknown genre is refused
TODO    a book that does not exist is a 404
TODO    the settings screen sends visitors to the login
```

The solution prints (exit status 0):

```
ok      Dune's page answers (status 200)
ok      the REST API lists 11 books
ok      an unknown genre is refused (status 400)
ok      a book that does not exist is a 404 (status 404)
ok      the settings screen sends visitors to the login
```

Each check uses the response, never PHP: status codes with `-w '%{http_code}'`, a header with `-I`, a redirect with `%{redirect_url}`. On the rehearsal server the same script takes `http://localhost:8081`.

- HANDOUT: LABS/MODULE-18/L9_1-FINAL-PROJECT/DEBUG_RELEASE_CHECK.PHP (DELIBERATELY BROKEN)

As shipped:

```
Warning: Undefined array key "verison" in /var/www/html/debug_release_check.php on line
plugin header    1.1.1  -> 1.2.0 ok
Deprecated: version_compare(): Passing null to parameter #1 ($version1) of type string
package.json            -> 1.2.0 ok
theme style.css         -> 1.2.0 ok
```

1. **Warning, line 23:** the key is `version`, not `verison`; the null it returns is the cause of the next message.
2. **Deprecated (in the printf):** `version_compare()` received null. Fixing the key removes both.
3. **Silent, line 24:** `get( 'version' )` returns false (header names are case-sensitive: `Version`); `version_compare()` turns false into an empty string, which is "older" than 1.2.0, so the line says ok for the wrong reason.

Fixed, it prints:

```
plugin header    1.1.1  -> 1.2.0 ok
package.json     1.1.1  -> 1.2.0 ok
theme style.css  1.1.1  -> 1.2.0 ok
```

- LAB 9.2-A (GUIDED): LABS/MODULE-18/L9_2-NEXT-STEPS/CORS_HEADERS_STARTER.SH → CORS_HEADERS_SOLUTION.SH

As shipped:

```
== GET /pagesco/v1/books, from http://localhost:3000 ==
Allow-Origin: http://localhost:3000
Allow-Credentials: TODO
Expose-Headers: TODO
Books: TODO
holds as a visitor: TODO
```

The solution prints:

```
== GET /pagesco/v1/books, from http://localhost:3000 ==
Allow-Origin: http://localhost:3000
Allow-Credentials: true
Expose-Headers: X-WP-Total, X-WP-TotalPages, Link
Books: 11 on 2 pages
holds as a visitor: 401, Allow-Origin: http://localhost:3000
```

WordPress repeats any origin and allows credentials, so the browser sends cookies with the page's requests: a nonce is what stops a forged change, as in Lesson 9.1. The 401 on /holds is the permission callback; CORS only makes the refusal readable.

- LAB 9.2-B: LABS/MODULE-18/L9_2-NEXT-STEPS/AI_READINESS_STARTER.PHP → AI_READINESS_SOLUTION.PHP

As shipped:

```
AI client in this WordPress: yes
AI allowed on this site: yes
AI providers listed:  (active: 0)
A text model available: TODO
Answer: TODO
With AI switched off: allowed yes, answer: TODO
```

The solution prints:

```
AI client in this WordPress: yes
AI allowed on this site: yes
AI providers listed: anthropic, google, openai (active: 0)
A text model available: no
Answer: Ask us at the counter about Dune.
With AI switched off: allowed no, answer: Ask us at the counter about Dune.
```

With a provider and a key the answer would be the model's sentence, through sanitize_text_field(). The filter switches AI off for one request; the constant `WP_AI_SUPPORT` (Lesson 9.2's Exercise 3) does it for the whole site.

- LAB 9.2-C: LABS/MODULE-18/L9_2-NEXT-STEPS/PUBLISH_SCAN_STARTER.SH → PUBLISH_SCAN_SOLUTION.SH

As shipped:

```
Files scanned: 109
== Secrets ==
sandbox-7f3a9c   TODO
maria-pass       TODO
devon-pass       TODO
jordan-pass      TODO
sk-ant-          TODO
== Never-publish files ==
Never-publish files in the shop's folders: TODO
```

The solution prints:

```
Files scanned: 109
== Secrets ==
sandbox-7f3a9c   0 file(s)
maria-pass       0 file(s)
devon-pass       0 file(s)
jordan-pass      0 file(s)
sk-ant-          0 file(s)
== Never-publish files ==
Never-publish files in the shop's folders: 0
```

vendor/, node_modules/ and build/ are left out: generated, ignored by Git, and full of strings that match nothing of the shop's. A learner's site with a test password pasted into a file shows it here before it is committed; the history search of step 4 finds what was committed already.

HANDOUT: LABS/MODULE-18/L9_2-NEXT-STEPS/DEBUG_HEADLESS_CLIENT.PHP (DELIBERATELY BROKEN)

As shipped:

```
Warning: Undefined array key "title" in /var/www/html/debug_headless_client.php on line
Deprecated: html_entity_decode(): Passing null to parameter #1 ($string) of type string
: 6 books on 2 pages
Last in the list: Lord of the Rings
```

1. **Warning, line 19:** the REST index's field is `name` (the request asks for `_fields=name`); `title` does not exist.
2. **Deprecated, the same line:** `html_entity_decode()` received the null. Fixing the key removes both.
3. **Silent, line 22:** one request is one page of six books, while X-WP-TotalPages says 2: loop until the last page (Lesson 9.2's Common Mistake 3).

Fixed, it prints:

```
Pages & Co.: 11 books on 2 pages
Last in the list: To Kill a Mockingbird
```

LESSON 9.2 REFERENCE OUTPUTS

The lesson's own blocks on a site at the end of Lesson 9.1 (all equal to the lesson's cards):

```
bool(true)
bool(false)
bool(true)
prompt_invalid_argument: No models found that support text_generation for this prompt.
Array
(
    [status] => 400
    [exception_class] => WordPress\AiClient\Common\Exception\InvalidArgumentException
)
```

```
anthropic Anthropic         ai_provider    ai-provider-for-anthropic/plugin.php, not ac
google    Google            ai_provider    ai-provider-for-google/plugin.php, not activ
openai    OpenAI            ai_provider    ai-provider-for-openai/plugin.php, not activ
akismet   Akismet Anti-spam spam_filtering akismet/akismet.php, not active
```

```
prompt_invalid_argument
```

```
Tool: wpab__pagesco__check-stock
Description: Says whether the shop has a book in stock, how many copies the supplier ha
Input schema: {"type":"object","properties":{"slug":{"type":"string","description":"The
Answer for maria: {"title":"The Great Gatsby","in_stock":false,"supplier_available":0,"
```

```
Answer for devon: {"error":"Ability \"pagesco\/check-stock\" does not have necessary pe
```

```
WordPress version: 7.1.3
Database revision: 61833
TinyMCE version:   4.9110 (49110-20250317)
Package language:  en_US
Success: WordPress is at the latest version.
+------+---------+----------------+
| name | version | update_version |
+------+---------+----------------+
+------+---------+----------------+
+------+---------+----------------+
| name | version | update_version |
+------+---------+----------------+
+------+---------+----------------+
+----------+--------+
| language | update |
+----------+--------+
| es_ES    | none   |
+----------+--------+
```

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 9.1 · FINAL PROJECT: LAUNCH THE PAGES & CO. WEBSITE

1. **B** 404 pagesco_hold_not_found: for her, the hold does not exist: Cancelling puts her user ID in the WHERE clause, so no row matches and the rules class answers pagesco_hold_not_found with status 404, the same answer as for an ID that does not exist. Jordan's hold stays waiting.

2. **A** In Holds::place(), which every door calls: Every door, the REST route, the form handler and the tests, calls Holds::place(), so a rule there holds for all of them, and changes in one place.

3. **C** HoldsTable::maybe_upgrade() runs on init and finds no version option, so it creates the table: HoldsTable registers maybe_upgrade() on init; the version option is missing (0), lower than DB_VERSION 1, so it runs dbDelta() and records the version. Activation hooks never run on an update.

4. **C** The binding found no published book with that slug, so the paragraph kept its own text: The paragraph keeps its own text when the binding source returns null, which BookBindings does when the slug matches no published book: a renamed slug, a draft, or a deleted book.

5. **B** Listing the ZIP and loading a new class with the ZIP's own autoloader: Only a check of the built ZIP sees what ships: counting its src/Holds files, and loading a new class with the ZIP's own autoloader, as Milestone 8 does.

LESSON 9.2 · NEXT STEPS: HEADLESS, AI IN CORE & YOUR CAREER

1. **B** Read X-WP-TotalPages and fetch every page: The route pages its results (six per page, the shop's Books per list setting). X-WP-Total says 11 and X-WP-TotalPages says 2; a front end loops until the last page.

2. **C** A WP_Error: no models support text generation: No provider plugin is installed, so no model supports text generation: generate_text() returns a WP_Error with the code prompt_invalid_argument and status 400.

3. **A** An ANTHROPIC_API_KEY environment variable or constant on the server: WordPress looks for ANTHROPIC_API_KEY as an environment variable, then as a PHP constant, then in the option the Connectors screen saves. On a server, the environment or wp-config.php keeps it out of Git and out of the database dumps you copy around.

4. **C** The ability's permission callback refuses Devon, and the model gets an error back: The resolver executes the ability as the current user, so its permission_callback (edit_others_posts) refuses Devon, and the model receives an error response instead of the shop's data.

5. **B** Search Trac and GitHub for a ticket, then report it with exact steps to reproduce: Someone may already have reported it; if not, a ticket with the version, the exact steps, what you expected and what happened is what lets contributors confirm and fix it. That is the evidence habit of the whole course.

EXIT TICKET

1. **Proves it:** HoldsTest's `test_only_the_customer_can_cancel_their_hold` (the recorded failing run shows it catches the bug) and Milestone 3's holds-check.php (Priya's "not found"). **Cannot:** holds-smoke.sh, which plays one customer only.

2. **Up:** pagesco-books.zip and pagesco.zip 1.2.0 (code). **Stays / created on live:** the My holds page and the shoptest account; the holds table is created by the new code itself (maybe_upgrade).

3. **Next:** their own; a good answer names a feature and a lesson (custom post types 5.2, blocks 6.x, REST 7.2 ...).

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 B.** Silencing the sniff plugin-wide: the habit to break before a client codebase.
- **Q3 D.** Believes only activation creates tables; ask what happens on every update of a live plugin.
- **Q4 A.** The most dangerous answer of the session: importing a development database overwrites live customers. Spend a minute on it.
- **Q8 A.** innerHTML fixes the symptom and opens an XSS hole.
- **Q9 B or C.** The headlines; show the lesson's var_dump on their own site.
- **Q11 A.** Thinks the model calls the ability by its own name; the function-calling name has no slash.

Complete Answer Key · all 18 sessions · v1.0 · © 2026 Ray de la Paz